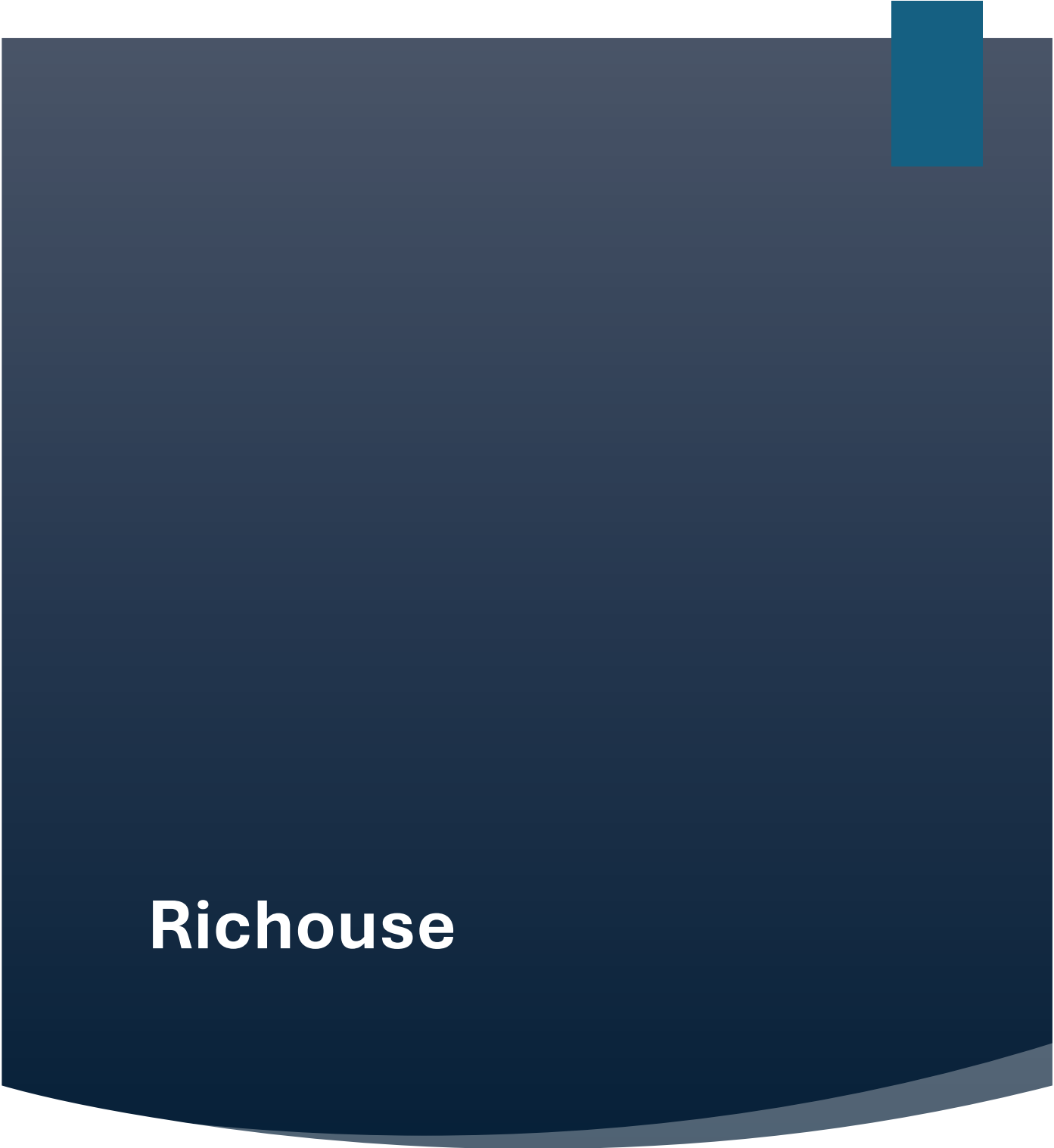




Richouse

CONTENTS

ANALYSIS.....	4
AN OUTLINE OF THE PROBLEM	4
STAKEHOLDERS.....	4
HOW THE COMPUTATIONAL METHODS CAN SOLVE THE PROBLEM	5
THINKING ABSTRACTLY	5
THINKING AHEAD	6
THINKING PROCEDURALLY	7
THINKING LOGICALLY	9
THINKING CONCURRENTLY	9
COMPUTATIONAL METHODS JUSTIFICATION	9
RESEARCH.....	10
INTERVIEWS PLANNING	10
INTERVIEW SCRIPT - ENZO GAROFANI (TRANSLATED FROM PORTUGUESE)	11
INTERVIEW SCRIPT - ALEX MICHALSKI (ENGLISH TAKEN)	12
INTERVIEW SCRIPT - ZACH WARD (ENGLISH TAKEN)	14
INVESTIGATING THE “BOB THE ROBBER 2”	16
FEATURE OPTIONS	21
FEATURES OF THE SOLUTION	23
HARDWARE AND SOFTWARE REQUIREMENTS	26
HARDWARE REQUIREMENTS.....	26
SOFTWARE REQUIREMENTS.....	27
SUCCESS CRITERIA	27
DESIGN	30
DISTRIBUTION CODE	33
CLASSES RELATIONSHIPS	34
CLASSES, VARIABLES AND DATA STRUCTURES.....	35
MAIN()	41
GAME_LOOP()	42
INPUT()	42
PLAYER CLASS METHODS	45
BUTTON CLASS METHODS	46
SYSTEM CLASS METHODS	48
PREFERENCES CLASS METHOD	50

GAMEPLAY CLASS METHODS	51
MINIGAME CLASS METHOD	52
FINGERPRINT CLASS METHOD	52
HACKING CLASS METHOD	53
LOCKPICK CLASS METHOD	53
GUARD CLASS METHOD	53
LOCAL DATABASE	54
API CODE	54
CODE	54
SERVER DATABASE	56
GRAPHIC DESIGN	57
SCREENS	57
SPRITES	65
STAKEHOLDERS' FEEDBACK ABOUT THE LAYOUTS	66
TESTING	66
WHITE BOX TESTING	67
BLACK BOX TESTING	67
ALPHA TESTING	73
BETA TESTING	77
PROJECT MANAGEMENT	78
ETHICAL, LEGAL AND ENVIRONMENTAL CONSIDERATIONS	81
ETHICAL CONSIDERATIONS	81
LEGAL CONSIDERATIONS	81
ENVIRONMENTAL CONSIDERATIONS	82
DEVELOPMENT	82
TIMELINE	82
FILES	83
GRAPHICAL DEVELOPMENT	83
WEEK 1 - USERNAME, MENU, OPTIONS, RANK AND MAP SCREENS	85
WEEK 2 - MINIGAME AND END GAME SCREENS	105
CODE DEVELOPMENT AND TESTING	122
WEEK 3 – GAME INITIALIZATION AND FIRST SCREEN	122
WEEK 4 – MENU AND USERNAME SCREEN INPUT HANDLING	137
WEEK 5 – USERNAME REGISTER, RANK AND OPTIONS SCREENS	153
WEEK 6 - PLAYER, INTERACTIONS, STATUSES AND END GAME	169

WEEK 7 - MINIGAMES, LASER, GUARD AND GAME OVER.....	191
EVALUATION	206
TESTING.....	206
WEEK 8 ALPHA AND BETA TESTINGS	206
GAME IDEA AND IMPACT	213
SUCCESS CRITERIA	213
PARTIALLY ACHIEVED.....	219
NOT ACHIEVED.....	220
STAKEHOLDERS NEEDS	220
NOT IMPLEMENTED FEATURES.....	223
SYSTEM PERFORMANCE	223
TESTING RESULTS.....	224
BLACK BOX TESTING	225
ALPHA TESTING	232
BETA TESTING.....	236
RELIABILITY	237
USER EXPERIENCE.....	238
USABILITY FEATURES	239
MAINTAINABILITY.....	250
LIMITATIONS.....	251
POSSIBLE EXTENSIONS.....	252
REFLECTION	253
PROJECT APPENDIXES	254
DISTRIBUTION-CODE	254
CONFIG.PY	254
FUNCTIONS.PY.....	256
MAIN.PY	266
LOCALUSERINFO.JSON.....	268
SERVER-CODE	268
APP.PY.....	268
GLOBALDATABASE.DB	269
MEDIA.....	269

ANALYSIS

AN OUTLINE OF THE PROBLEM

For my project I will develop a game inspired by “Bob The Robber 2”, this is a 2D game created by two small developers, Flazm and Meow Beast, and distributed by websites, such as Friv. that the main objective is steal something from a house or another build protected by guarders, cameras and lasers. My game will be a top down with objective and features like the “Bob The Robber 2”, but implementing new puzzle features to improve the immersion of the game. As examples of what It will demand:

- Registration screen with a textbox to input the username.
- Menu screen allowing to access the other screens, including gameplay and options.
- Sprites for interactions during the gameplay.
- Hitboxes for the collisions.
- Player statuses bar.
- Game over and mission completed screens.

STAKEHOLDERS

The game won't have any restriction for ages, all of elements will be “family friendly”. It won't also have limitation for audio, given that it won't have dialogues, and the music won't interfere in the gameplay. As it's inspired by “Bob The Robber 2”, the main public will be persons aged between 14- 18 that probably played this game before. Consequently, my stakeholders are:

- Enzo Garofani, a 17 years-old student from Brazil. He's a casual player enjoying a range of short duration games, therefore he will be able to give feedback about the missions features to make the game more dynamic. Moreover, I can contact him weekly, which is essential to finish the next sections faster.
- Alex Michalski, a 17 years-old student from United Kingdom. As a casual player who likes First Person Shooters (FPSs) and stealth games, he can add to the game a mix of both worlds. Furthermore, he can contact almost daily.
- Zach Ward, a 17 years-old student from United Kingdom. Because of his passion for 2D games, he has a good repertory of features used in them, thereby suggesting possible improvements in UI and playability. He can also be contacted daily.

HOW THE COMPUTATIONAL METHODS CAN SOLVE THE PROBLEM

THINKING ABSTRACTLY

I need to identify and decide what will be necessary for the game, for this I should think in an abstract way, describing the game in a simple way. The abstraction divides the “Richouse” into 4 essential parts:

➤ Menu

The initial screen of the game, contain 4 buttons: the button to start the mission, after chose one map; the button to open the configurations; the button to open the ranking and the button to quit the game.

1. User finish registration.
2. Use the keyboard to select the buttons.
3. Confirm using ENTER.

➤ The configuration

User can change basic configurations, such as audio volume and the controls.

1. Users confirm the configurations button.
2. All the preferences are printed.
3. User can use the keyboard to select each preference.
4. They can change the preference.
5. They can press a key to return to menu.
6. All changes are saved.

➤ Ranking

Two tables showing the best performances in each of the two maps, the top 10.

1. Users confirm the ranking button.
2. Usernames of the players with the best performance are printed.
3. User can return to the menu screen pressing a key.

➤ Gameplay

The game start in the front door of the house. The house will be saw in the top, so users can't see the roof, they see the walls dividing the square rooms. The house has security systems such as cameras, guards, lasers and locks. The status bar will be a single bar with time taken and player's life. If guards and cameras bar is yellow, they are identifying you, if red, they already saw you and you lost a life.

1. Gameplay starts when player confirm the start button.
2. Sprites and background are rendered.
3. Using the keyboard the player can move and interact.

4. Hitboxes allow the collisions.
5. Certain locations can give items or start minigames.
6. Game finish when player is in the main door after completing the objective.

THINKING AHEAD

I should be prepared for possible complications and choose the tools to be used. The first step is complete the gameplay, after this I can complete the things with lower priority, the menu screen followed by ranking and finally the configurations.

Tools:

➤ Python with Pygame

I choose it because the python is a widely used language, it allows the use of object-oriented programming to abstract the code and its “sqlite3” library can connect the application with SQLite databases. The “Pygame” is the library that will allow the creation of the GUI including sprites, collision and sounds. With this library, for example, I can handle inputs with “pygame.event.get()”, detect the collisions with “pygame.sprite.collide_rect()”, add music using “pygame.mixer” and add other features without the need of an external tool. The use of external tools, such as engines, can difficult the development, so the ideal is using the Pygame because it’s a library of Python easily installable.

➤ SQLite

It allows the creation of multiple tables into a single file, it’s better than use plain files. Its operations allow an efficient handle of database data, such as “SELECT ... FROM ... WHERE ...” to fetch the data quickly. It’s better than JSON and CSV files for large data sets because the values can be indexed instead of look for all of them. Furthermore, these commands are easily executable with sqlite3 module in Python.

Inputs:

➤ Text.

User should be able to enter a text to register a username in the registration screen.

➤ Selection keyboard keys.

Keys used to change the selected button, it can be changed in the configurations screen.

➤ Movement and interaction keyboard keys.

Keys used to control the player’s actions during the gameplay, it can be change in the configurations screen.

Output:

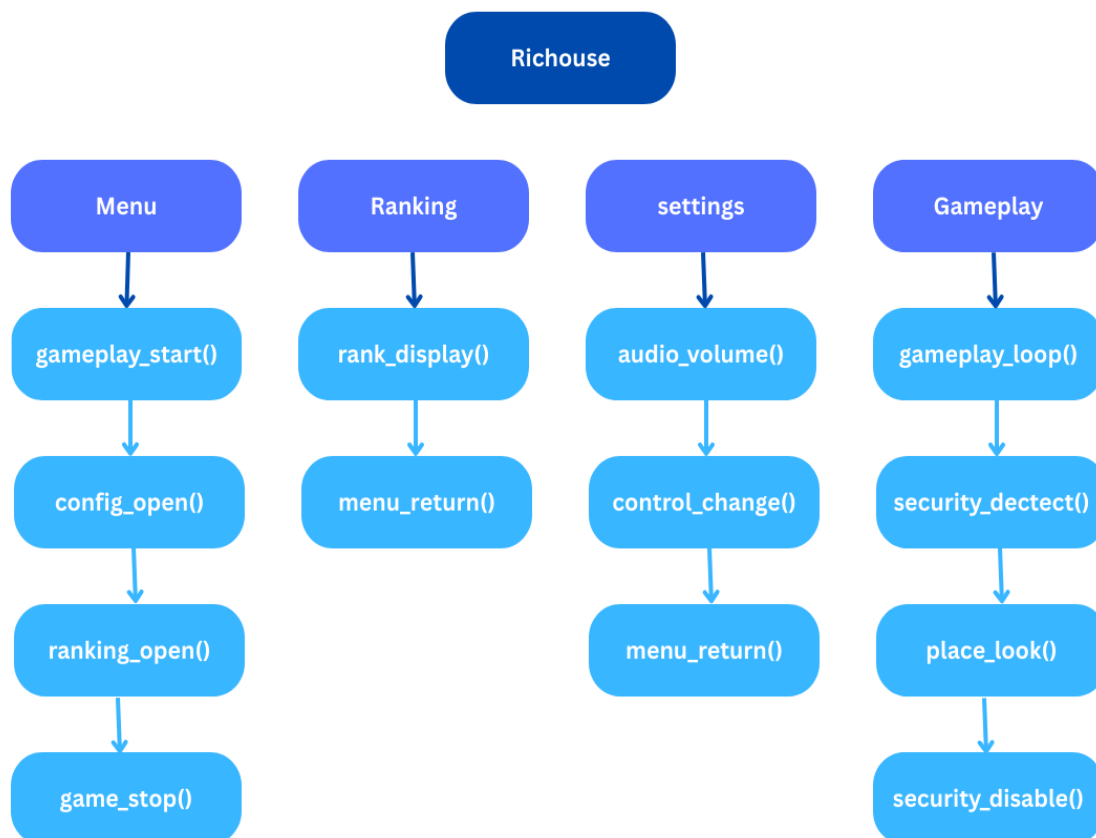
- Music.
Background music is present in the whole game, no specific one per level or game state.
- Graphics.
The background and sprites will be printed each frame, if it's printed more times per second, the game will be more fluid.

Storage:

- Relational Database.
When large data sets are needed. It makes faster to access data through indexes and easier to sort the contents. On the other hand, can be harder to implement and normalize.
- Flat-file Database.
When few data are stored. Although it may be slightly slower, it is simpler to implement and the performance almost doesn't change in small sets.

THINKING PROCEDURALLY

Look at the structure of game, break it into blocks to clarify the steps and how all of it will work together.



➤ **Richouse**

The name of the 2D top-down game being developed. It will be stealth, user should try to escape from the house with the golden statue, it will define the success of the mission.

➤ **Menu**

Initial screen of the game that use functions: `gameplay_start()` to control the start button starting the mission in the map that user chose; `config_open()` to control the button that open the settings screen; `ranking_open()` to control the button that open the ranking screen; and the `game_stop()` to control the quit button. The functions work when the ENTER is pressed in the button wanted.

➤ **Ranking**

The objective is displaying the ranks so it will use around 2 functions. The `rank_display()` is responsible to display the rank, and the `menu_return()` will be used again here.

➤ **Settings**

It has the function to control the change in the game settings. The `audio_volume()` is responsible for controlling the volume bar, pressing A to decrease and D to increase; the `control_change()` allows the change of gameplay control by pressing the button that you would like to use and pressing ENTER to confirm. Finally, the function `menu_return()` return the user to the menu when the button is pressed

➤ **Gameplay**

The gameplay features will be controlled by a series of functions. The `gameplay_loop()` control the movement, collisions, and all other things that should be updated constantly, responsible by the keyboard inputs, Pygame's main loop; `security_detect()` will control the sprites perception; `place_look()` will control the search feature; and `security_disabled`.

THINKING LOGICALLY

There are some aspects to be considered in the execution to avoid problems.

- The same keyboard buttons should be handled differently when used to move or select button. The condition “if button == ‘W’ and screen == ‘gameplay’” will move forward, already the “if button == ‘W’ and screen == ‘menu’” will select the button on top. It will be handled by `gameplay_loop()`.
- The area where sprites will identify you, guards, dogs and cameras can’t see you if they aren’t looking to you, it can be solved with a variable “wide == true”, the condition “if security_detect() and wide != false” will make the security detect you, but it won’t detect if “wide == true”.
- Make sure that any of the solid parts of map could trap the character or the other sprites. An example is when enter in a room with two doors with “status == ‘locked’”

THINKING CONCURRENTLY

I should consider that some features of the game work at the same time of another ones. Some of them are constantly executed during the game.

- The timer will work at same time of the other features, it will be controlled by `gameplay_loop()`, Pygame’s main loop, using the feature “`pygame.time.Clock`” that allow the concurrency of features.
- Other feature controlled by the `gameplay_loop()` will be the inputs, using the feature “`pygame.key.get_pressed`” to detect the input and handle it considering the variables, such as the “screen” that show which screen is open. It happen while other features are working.
- The music and the audio effects are reproduced by `gameplay_loop()` constantly during the duration of the game. It’s possible because of the feature “`pygame.mixer`” that execute it.

COMPUTATIONAL METHODS JUSTIFICATION

Computational methods give the basis of thinking for the project. In the abstraction we can break the problem in steps to be solved, with this I decided to finish the gameplay first, because in this way the stakeholders will be able to test the game features, while it happens, I can finish other screens and add design increasing the productivity.

Already the thinking ahead I could prepare for possible tools used, inputs and outputs being handled, and methods to store data. In this way I’m ready to use more efficient

approaches, such as Object-Oriented Programming for a more structures code and Sqlite3 relational databases for large sets, because of SELECT operator. Furthermore, the game will be input and output resistant.

With this information I could divided the game in blocks of functions, it facilitates to think how to implement in Python, each screen has its own specific, `rank_display()`, or general functions, `gameplay_loop()`. The `gameplay_loop()` will control the concurrent features using the “`pygame.time.Clock`” that allow this concurrency, it will handle considering variables created to avoid problems, such as the “screen” variable.

RESEARCH

INTERVIEWS PLANNING

For the success of the game, it's important to ask for the stakeholders some questions about the ideas given until now. I should know their opinion to know what features are important, besides it I need to know the suitability and acceptance of my planning. I divided into three categories and take some questions for each one.

Question layout:

Gameplay

- Do you like stealth games? If yes, what features make you like it? If no, what make the game boring in your opinion?
- In general games, which buttons do you like to use for the movement? And for the interactions?
- In a robber stealth game, what status should be included? Example: player's lives, or time taken.
- Do you like the audiovisual effects in the stealth games? What is their importance?
- What configurations usually do you change when you play a game?

Ranking System

- What do you think about the games that have a ranking system? How it impacts on the gameplay?
- Which of the status that you wrote above would be ranked?

Design

- Do you like to play 2D top-down games or you prefer other types? What kind of camera position do you like?
- What art styles do you like for stealth games? What is your favorite art style?
- What should be included in a menu screen?

Extra (Optional)

- Are there some other ideas that you want to tell?

INTERVIEW SCRIPT - ENZO GAROFANI (TRANSLATED FROM PORTUGUESE)

Gameplay

- Do you like stealth games? If yes, what features make you like it? If no, what make the game boring in your opinion?

Yes, the suspense creates is good and explore good mechanics in games.

- In general games, which buttons do you like to use for the movement? And for the interactions?

W A S D, E F C.

- In a robber stealth game, what status should be included? Example: player's lives, or time taken.

Player's life, something like visibility, how much you were detected.

- Do you like the audiovisual effects in the stealth games? What is their importance?

Yes, the immersion is very important.

- What configurations usually do you change when you play a game?

None, I play the game in the way that it was programed to be played.

Ranking System

- What do you think about the games that have a ranking system? How it impacts on the gameplay?

I think it's useless depending on game style, unless it's a game against the time or stimulate competitiveness.

- Which of the status that you wrote above would be ranked?

Any of them, I don't see necessity to implement ranking in this game

Design

- Do you like to play 2D top-down games or you prefer other types? What kind of camera position do you like?

They are too good, 2D horizontal games can be good for furtivity games.

- What art styles do you like for stealth games? What is your favorite art style?

Could be more industrial, or more cartoon-styled, pixeled graphics could be strange for this kind of game.

- What should be included in a menu screen?

Something more interactive and dynamic, don't have a static menu values the game, as well as a beautiful art.

Extra (Optional)

- Are there some other ideas that you want to tell?

All games that look for a sensation of identity in their proposal always become an interesting indie to play. Thus, for example, I don't believe that beautiful arts make sense in this kind of game, however a variety of mechanics is always welcome.

INTERVIEW SCRIPT - ALEX MICHALSKI (ENGLISH TAKEN)

Gameplay

- Do you like stealth games? If yes, what features make you like it? If not, what makes the game boring in your opinion?

Yes, I think stealth-based games are very interesting to play. Some of the best features is the strategic side and "ghosting" (going unnoticed)

- In general games, which buttons do you like to use for the movement? And for the interactions?

In my opinion, I like to use WASD for movement and F or E for interactions.

- In a robber stealth game, what status should be included? Example: player's lives, or time taken.

I think you should include player life, amount stolen/taken, time left and how close you are to being detected: for instance if you get seen by an NPC a bar turns yellow and the closer you are it turns to orange and finally red once spotted.

- Do you like the audio-visual effects in stealth games? What is their importance?

Yes, because it gives you one more sense which makes the game feel more real and helps players immerse themselves into the game.

- What configurations usually do you change when you play a game?

Normal, (depending on what game) I would change the button to change tool or weapon to mouse button 4 and 5.

Ranking system

- What do you think about games that have a ranking system? How does it impact on the gameplay?

I think games can't go wrong with a ranking system as it adds an additional achievement to reach. It makes the player more competitive.

- Which of the statuses that you wrote above would be ranked?

Amount stolen: greediness rank

Time left: swiftness rank

Detection: stealthiness

Design

- Do you like to play 2D top-down games or do you prefer other types? What kind of camera position do you like?

My personal favorite is first person, but I do enjoy a good 2D top-down game as they feel retro. I like one where the camera is close and follows the character as if you could see the whole map and whereabouts of enemies it could make the game too easy and less enjoyable.

- What art styles do you like for stealth games? What is your favorite art style?

For a stealth game I think you could use most styles. But it depends if you want your game to be scary go for dystopian realistic style, but you want a relaxing casual game go for a cartoon style. My favorite is realistic.

- What should be included in the menu screen?

Start button, quit button, options button: where you can reconfigure key binds and change volume of sound effects or music, you could also add a credits button.

Extra (optional)

- Are there some other ideas that you want to tell?

You could add vanity where you can change how your robber looks and make him unique.

You could also have a story line for the robber about his past to make the player feel sympathy, so they enjoy the game more.

INTERVIEW SCRIPT - ZACH WARD (ENGLISH TAKEN)

Gameplay

- Do you like stealth games? If yes, what features make you like it? If not, what makes the game boring in your opinion?

I enjoy stealth games that create tension and constant fear of being caught, the game can become boring without this added fear. There must be a variety of ways to attempt each objective to create a sense of freedom that comes with the stealth genre

- In general games, which buttons do you like to use for the movement? And for the interactions?

WASD, E for interact, I for inventory, numbers 1 - 9 for equipment or items

- In a robber stealth game, what status should be included? Example: player's lives, or time taken.

Time Taken should be included at the end of the mission, maybe items stolen, value of items, most valuable item stolen should be included at the end of the mission, along with a letter rank or a score from 0 - 100 based on the success of the mission

- Do you like the audiovisual effects in stealth games? What is their importance?

Simple effects, notifies that user which objects and interactions make noise/could alert people nearby. Too complicated HUD icons or sound effects could break the user's immersion into the stealth game.

- What configurations usually do you change when you play a game?

Mouse Sensitivity, Graphics quality e.g. High, Medium, Low. In-game resolution.
Option to change your controls.

Ranking System

- What do you think about games that have a ranking system? How does it impact on the gameplay?

If rewards are offered, it creates a strong incentive to play and a competitive nature in the game, however, the use of bugs and cheats by other users could inflate the ranking system.

- Which of the statuses that you wrote above would be ranked?

Individual mission performance, total performance ranked across every mission.
Potential for an online mode where users compete to rob the same area and can attempt to stop each other through e.g. weapons and gadgets

Design

- Do you like to play 2D top-down games or do you prefer other types? What kind of camera position do you like?

For stealth games, I've usually only seen 3D first person or third person, so a 2D top-down stealth game would be really innovative and potentially easier to code.

Thinking of something similar to Hotline

Miami: <https://youtu.be/Kqr0yUuSiTs> Hotline Miami is much more fast paced, but the camera and art style would be unique for a stealth game.

- What art styles do you like for stealth games?

I've only seen more realistic, 3D stealth games made before, see Thief Simulators and Hitman:

https://store.steampowered.com/app/704850/Thief_Simulator/

https://store.steampowered.com/app/1659040/HITMAN_World_of_Assassination/

- What is your favourite art style?

I am a fan of the Hotline Miami style; it would be easier to create art for and code while being unique and differentiated itself from other stealth games.

- What should be included in the menu screen?

- Start/Play/Missions

- Ranked

- Options
- Achievements
- Extras e.g. credits

Extra (Optional)

- Are there some other ideas that you want to tell?

Hope the game development planning goes well, take some good inspiration from Thief Simulators on steam, they have nailed the robber stealth game mechanics e.g. lock picking, safe cracking, detection.

INVESTIGATING THE “BOB THE ROBBER 2”

I played the “Bob The Robber 2” in *Friv*, with this I could remember the mechanics of the game. I will use adaptative versions of these features and I will implement new features, using puzzles. I took some screenshots to analyze the feature and plan some adaptation.

Menu Screen:



A simple menu screen, it just has the “play game” button to start the game. It doesn’t have a specific screen for configurations, it just shown the audio intensity of top right and the option to mute the music. It has an attractive art that shown intuitively the main objective of the game. I chose to use this style with all options and the logo in same column, I will just add the configurations and ranking button.

Player status bar:



It includes the time taken in phase, represented by clock. There is also the money amount stole, represented by the money icon. Furthermore, it uses alarm icons to show how many times the player was got by security system. From this status, I will use the alarm (lives) and the time, I won't include the money because the main objective will be stealing a golden statue as quick as possible. The house furniture will have items to be used in puzzles, instead of money.

Map:



It has a map with different rooms, each one with different widths and same height. It also has the arrows showing where the main objective is. There are different kind of mobilities, lifts and stairs. For my game, the rooms will be all squares or rectangles, but with different sizes, that will fit on map. Furthermore, I prefer to use a red circle to show the objective, like as a house map demarcation. I won't include lift and stairs, because of the game will be top-down.

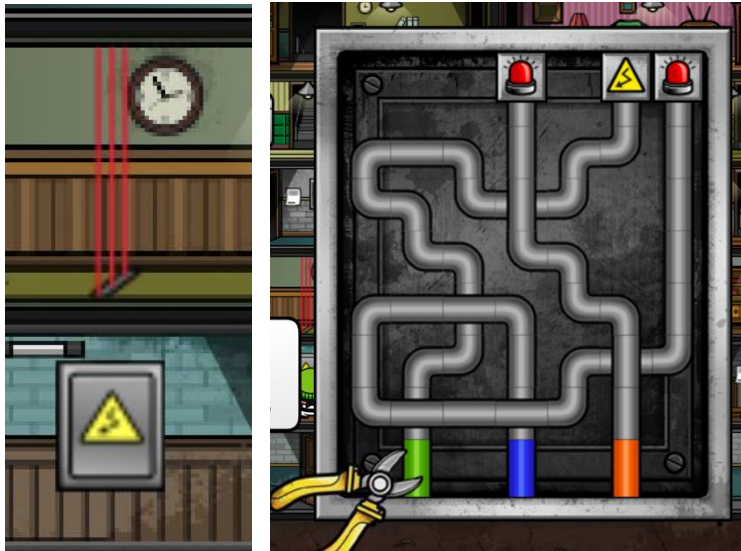
Security System:



The game has locked doors, I used a lockpick to open, I align the middle metal using the left and right arrows and press the up arrow to unlock, it makes me use more the agility to press the button in the right moment. In my game, the lockpick could be used on horizontal, pressing “F”, align the lockpick with pines using “A” and “D”, and press “W” to press each pine until all of them be pressed.



It uses doors with passwords, I pressed “W” to open the panel, it makes me think about where the password is, in this way the game becomes more challenging. In my game, it could be a digital panel, and I could implement a system to put some flour in it to reveal the password used.



It uses lasers with a system of lasers that can be disabled using its panel, it uses “W” to open the panel, it makes me feel like an eclecticist. In my game, the system of lasers and cameras could be controlled online, so the user could press a specific button to open the character hacker tool to crack the house system, as result the security system is disable during a time.

Lives, game over and mission completed



It counts the lives using alarms, each time that the security system got me, the alarm turns on. When this happen, I lost one live and appear some audiovisual effects connected with alarm, increasing the tension when I was got. I will remain this idea for my game.



When I was got 4 times, a game over screen appears, but instead of a write of “Game Over”, an image showing the destiny of my character appear. It is important to remain the user in the immersion even when they lose.



When I finished a phase, a journal screen appears, like as my character was seeing its own crime on TV, it increases the feel of success on me. Therefore, I will inspire on these screens to make my own game over and mission completed screens without just writing phrases.

FEATURE OPTIONS

After the research I should group all ideas from interviews and game research. It supports in the decision of features.

Feature idea	From
Gameplay	
Stealth game	Developer, all stakeholders agree
Security systems	
Guards, dogs or robots	Game research
Locks	Game research
Cameras	Game research
Lasers	Game research
Door with code	Game research
Buttons	
Movement: W, A, S and D	All stakeholders; Game research
Interactions: E and F	All stakeholders; Game research
Statuses	
Player's life	Alex and Enzo; Game research
Stealthiness	Enzo and Alex
Amount stolen	Alex and Zach; Game research
Time taken	Zach; Game research
Most valuable items	Zach
Audiovisual effects	
Notify what kind of object was got	Zach
Interactions have effects and affect sprites	Zach
Ghosting, hide in places	Alex
NPC bar turn yellow when start to see you	Alex; Game research.
Configurations	
Option to change graphics	Zach
Option to change controls	Zach
Option to change sound effects	Alex

Ranking	
Should be included	Alex and Zach
Status to be ranked	
Amount stolen	Alex
Time taken	Alex
Stealthiness	Alex
Design	
Symbols representing statuses	Game research
Camera	
Top-down	Enzo, Alex and Zach
Horizontal	Enzo
Art style	
Realistic	Alex and Zach
Cartoon	Enzo and Alex
Pixel	Zach
Screens	
Game over/mission complete	
Unique art	Game research
Menu button	Game research
Replay button	Game research
Menu screen	
Dynamic	Enzo
Start button	Alex, Game Research
Quit button	Alex
Options button	Alex
Credits button	Alex; Game research
Change your character	Alex
Attractive art	Enzo; Game research
Map	
Rooms with similar size	Game research
Difference mobilities	Game research

FEATURES OF THE SOLUTION

After I see all suggestion from the stakeholders' interview, I should select the most interesting features ideas and find a way to implement it.

Feature	Justification	Limitation
Screens		
Menu	A screen before the game start, it can't just start.	Can only appear only when user is already registered
Start button	Start to play one of the 2 maps, chose by player	Must be selected and confirmed using predefined keys.
Rank button	Access the ranking with the best scores of the game	Must be selected and confirmed using predefined keys.
Options button	Where user press to access the configuration of the game	Must be selected and confirmed using predefined keys.
Quit button	It will stop the program when pressed.	Must be selected and confirmed using predefined keys.
Dynamic	More interactive menu makes the game more interesting for the user. Each button is selected using W for up and S for down, press ENTER to select	It won't have complex animations because it will user too much development time and processing power.
Select character	User will be able to select the pre-designed sprite that they most like, press A to change to the left character and D for to the right	It's a complex feature that won't change much in the game. Therefore, it will have low priority during development.
Configuration	The user would be able to personalize some game features to their liking	It must change the preferences in the database affecting the game directly.
Option to change volume	Each user has a different preference for the volume of audio, increase using D and decrease using A	It will use a bar instead of a number, so user won't see the exact volume but the proportion.

Option to change controls	Each user has a different preference for the buttons used; it includes the selection buttons listed above.	User might choose the same button for more than 1 preference that may unable the playability.
Ranking	A table showing the best scores of the game, all store in an SQLite database. It increases the game competitiveness. 2 ranks, one for each map	It won't show the user position in the ranking because it would use too much server time to find it every time.
First criteria: Player' life	How many times the player was found, if they weren't found, the life will stay the same. Most important of a stealth game.	Many players will finish the mission with the same number of lives.
Second criteria: Time taken	Fewer is the time taken per mission, better is the thief.	If player take longer than the maximum time supported by the format, it will overflow.
Game over/mission complete	A way to show to the user that mission was finished. Use A and D to select the button to be pressed.	It will appear only in specific states of the game, such as when user return to initial door with the statue.
Continue button	It will redirect the user to the menu screen.	It should return to the same gameplay state.
Restart button	It will restart the mission in the same map.	It should restart the gameplay state losing the progress.
Gameplay		
Control buttons	The game should have few buttons as possible and should be common, with this the user gameplay becomes easier.	Only keyboard buttons because it will make a possible adaptation for console easier.
Movement: W, A, S and D	Most common buttons for the movements, W to go forward, S to go backward, A to go left and D. to go right. Users often already know it.	It will stop to work after change the preference in the options screen. So, the user must remember later.
Interactions: E and F	The buttons E and F are close to the movement buttons; it makes easier to press.	It will stop to work after change the preference in the options screen. So, the user must remember later.
Stealth	The game will have the focus on stealth; the player shouldn't be found by security systems. It	Stealth elements tend to make the game slower, but it can't happen because of competitiveness.

	creates a tension resulting in a higher immersion for the user.	
Security Systems	They have the objective to search and found any criminal. If they found the user, one live is lost.	They must work at same time as the other gameplay elements run.
Guards and dogs	The house will have sprites locomoting and searching for any criminal. Inside will have guards and outside will have dogs.	It won't use AI because in small maps, as this game, it will make the gameplay too difficult.
Locks	It will lock some doors. The users will be able to open using a lockpick by pressing the pines inside it.	Creation of a whole minigame is needed, therefore making the development slower.
Cameras	It will be a fixed and rotating sprite. It will relate to house system; user will be able to hack the system to disable it.	Rotating hitboxes can bug when using rectangles, so a more complex shape is needed.
Lasers	It will block some places, including the main vault. It will also be linked with house system.	A hitbox is needed for it, so it can't have shapes other than rectangular ones.
Doors with code	The door will have a digital panel. User will be able to use a UV light to see the fingerprint, they will press in crescent order of intensity.	Creation of a whole minigame is needed, therefore making the development slower.
Statuses bar	A bar showing information about the user during the mission.	It must be showed in a way that doesn't bother the user gameplay.
Player's life	The number of times that user can be found until lose. Represented by an alarm symbol.	It must be handled to avoid negative numbers.
Time taken	The time taken in the mission. Represented by an alarm symbol.	In case of user taking longer than the maximum number an overflow can happen.
Stealthiness perception	The guards and cameras will have an indicator up their heads; it will change the color when the player is precepted or found.	It's a vague quantity that will be difficult to be measured making the game harder to develop.
Places to hide	The player will be able to enter inside some places to hide. Guards, dogs and cameras won't be able to see.	Complex mechanic that demands the stealthiness perception to work.

Audiovisual effects		
Audio	The game will have a music and an alarm sound, when player is found, to increase the immersion.	Alarm sound can bug the music or restart it.
Visual	The game will have effects when perform an interaction and when the player is found.	Effects can be imperceptible if printed in few frames.
Design		
Symbol for statuses	The game will have symbols to represent the statuses in the bar. The time will be a clock and the player's life will be an alarm.	It must be integrated as a sprite or part of the background to be printed every frame.
Top-down camera	The camera will be top-down because it's an uncommon camera style for stealth game, an innovation for the it.	It can lose details about the gameplay ecosystem.
Cartoon art style	A common art style for the 2D games and it will be easier to implement.	It will take long times to produce our own designs.
Map with square rooms	The map will be constituted by square rooms of different sizes; it will support on the implementation of features.	Limit the creation of places and objects with different shapes.

HARDWARE AND SOFTWARE REQUIREMENTS

HARDWARE REQUIREMENTS

Hardware	Reason
Keyboard	For buttons to move, interact with map and to access the menu.
Monitor	To display the graphics of the game, it includes the special effects. The recommended is 1280x720
Speakers/headphones	To play the music and the audio effects of the game

500MB of hard drive storage	For the Visual Studio Code and the files with the game code
2GB of RAM DDR3	Minimum requirements to install Visual Studio Code
1.6GHz processor	Minimum requirements to install Visual Studio Code

SOFTWARE REQUIREMENTS

Software	Reason
Windows 10/ macOS 10.12/Linux	Minimum requirements for the VS Code
Visual Studio Code	The coding editor used
Python 3.6 or higher	The computer language used, I used 3.6 version but can be higher. It's installed with VS Code
Pygame 2.0 or higher	The Python library used to develop the graphics and sound of the game, the version used was the 2.0, can be higher. It's installed on python with command pip.

SUCCESS CRITERIA

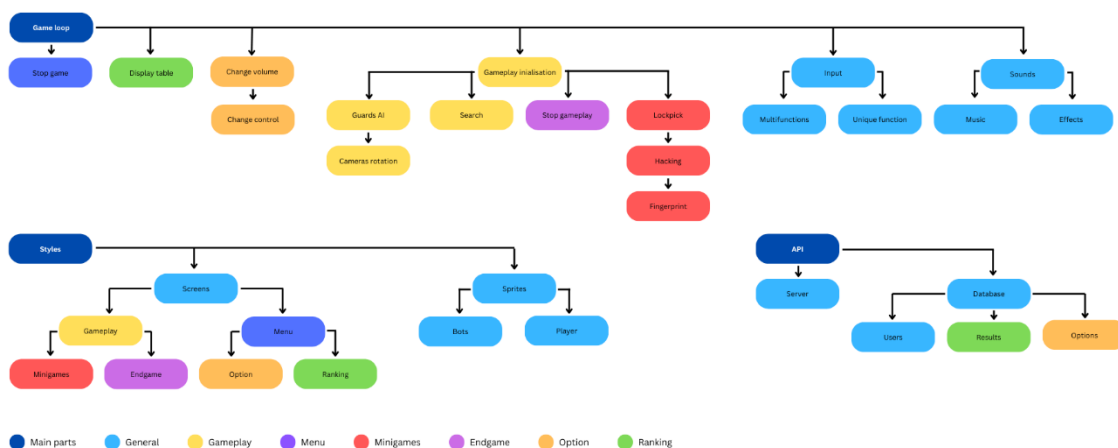
Number	Requirement	Justification
1	Menu screen is rendered after initializing the game and complete the registration.	We need a screen to access the game resources such as maps selection and ranking table.
2	We can selected buttons using W to select up and S to select down	Buttons are aligned in a column, so the game code can be reused, and the keys serve as general selection. Besides it, these keys are intuitive because are commonly used for it in other games.
3	The option button opens the option screen when pressed	In this way user can access the preferences of controls and change it.
4	The ranking button opens the ranking when pressed	It allows the user opens the ranking screen and sees the highest scores

5	The start game button starts the mission when pressed	It will enable the user to choose one of the two maps and start it.
6	The quit button stops the program when pressed	It enables the user to leave the game.
7	Ranking screen display two tables with top 10 scores in each map	It makes the best 10 results in each map available for every user can see.
8	The players life after mission is the status with highest impact during ranking. Time used to break the tie between users.	The ranking should have a main criterion to be ranked, and the player's life is the most important in a game. Time will break the tie, so player will try to finish as quick as possible playing more times the same map.
9	Return to menu when ESC is pressed	Allows the user to return to menu screen, it's intuitive because it's used in multiple games.
10	Configurations screen show the preferences for each action.	Show all game preferences, allow user to see and change it
11	Configuration selection with W to select up and S to selected down	It allows the code of button selection be reused. Furthermore, it makes intuitive to the user.
12	Volume bar increase when selected and D is pressed, and decrease when selected and A is pressed	User is allowed to change the volume intuitively using buttons close to general selection, W and S that are commonly used in games.
13	Selected preferences are changed when SHIFT + (alphabetic/numeric key) is pressed, the key is defined for that action.	It enables the selection of different buttons from keyboard for the gameplay.
14	Return to menu screen when ESC is pressed.	Allows user to return to menu screen using an intuitive key, because it's commonly used for it in other games.
15	Use only keyboard keys for any game action.	It makes easier to adapt the game for consoles in future given that they don't have mouse but a multiple static buttons.
16	Press W character moves forward, S it moves backward, A it moves left, and D it moves right.	It's the standard button for movement in many games. It can be personalized at configurations

17	Press F searches the searchable places, and E uses the item during minigame interaction.	Makes easier to press because they are close to the movement buttons.
18	Get 3 items in total after search all searchable places	Drawers and cabinets will hide the items necessary to start minigames. It will be 3 because they're 3 minigames. Some of the places will be empty.
19	Security system decreases player's life by 1 when find them.	It's used to make the game harder, so user will take more time.
20	Sprites of guards and dogs constantly moves across the map during gameplay screen.	Forces the user to move around the map, if they are caught, they lose 1 live
21	Guards and dogs disable temporarily after kicking them from behind, using the key F	It disables the sprite for few seconds, so player can pass without care about them.
22	Sprites of cameras rotating in same place.	It covers the maximum angle possible, so it makes harder to pass.
23	Lasers blocking some places.	Forces user to play the hacking minigame to disable it.
24	After player finish the hacking minigame, the sprites of cameras and lasers can't get them no more.	Turns the game dynamic because the user should find an item to hack it with the tablet, need to be found.
25	Locks blocking some rooms.	Forces user to find the lockpick and play the minigame to enter.
26	Users complete the lockpick minigame if press the 3 arrow keys correctly	It's a challenge that demand attention, but it isn't that hard.
27	When user press E in fingerprint minigame while the UV light is in inventory, circles with different intensities shows the correct order, less intense to more intense.	Common feature shown in movies. Turn harder because the user should find the UV light.
28	Player statues bar at top.	Allows user to keep track of their performance.
29	Player's life	Shows how many times that user can be caught before losing. Limit of tries.
30	Shown the time taken.	Allows the user to know how long they're tanking.

31	Game finish when the golden statue arrives in the door	It gives a clear and predefined objective; money amount couldn't be clear enough about how much is the maximum.
32	Game over screen appear when user lost all lives.	It makes the game finish; users can't play the same mission infinitely.
33	Cartoon graphics	Common graphics style for 2D games. Easier to develop than realistic or pixelated.

DESIGN



I use the diagram to divide the design in three main parts. The first part is the game loop, here is where the actual game code work, it includes anything that is executed in the user computer. The second part is the styles; all the style of screens and sprites will be design here. Finally, the last main part is the API, it's all code that don't run in user computer, it because we should have the access to some information in real time and it's dangerous to give the users information in the distribution code. The other parts are separated in colors; it supports the link of parts between the main sections.

➤ Game loop

It's the main loop used in the game. Its role is updating the current frame based on game states changed using the functions and classes.

➤ Stop game

As the game need to be stopped, it's a condition in that the game loop will break, and everything will be correctly closed

➤ Display table

The system used to get the username of the best performers in each map and print it into different tables of the rank screen.

➤ **Change volume**

The volume of audio can be altered using this process that will find the current value in the database, change and save it.

➤ **Change control**

The pre-defined keys for the game actions can be changed according to the user preferences. It's responsible to change and save it locally.

➤ **Gameplay initialization**

When the game starts, the sprites, statuses and hitboxes must be initialized. Therefore, it will run every time that a new gameplay is created.

➤ **Guard AI**

The algorithm controlling the next guard moves based on player's movement. It can be implemented using Dijkstra algorithm.

➤ **Camera's rotation**

Cameras can't be static because it would block permanently some locations. Consequently, an algorithm is used to make it dynamic.

➤ **Search**

Player's action to look for items in specific locations. They're used to start minigames later.

➤ **Stop gameplay**

Interface allowing user return to the menu at any time of the gameplay. However, they lose all the progress in the mission.

➤ **Lockpick**

The minigame used to open doors with locks by completing a sequence correctly.

➤ **Fingerprint**

The minigame used to open doors with code locks by completing a sequence correctly.

➤ **Hacking**

The minigame used to disable the security systems by completing a sequence correctly.

➤ **Input**

It handles any user input using conditions to decide which functions are going to be executed.

➤ **Sounds**

It includes any audio output of the game. They must be handled that they can be overlapped without bugs.

➤ **Music**

The same song is used for the whole game, so it must be replayed when finish. Therefore, a song with a loop is the ideal.

➤ **Effects**

Even they aren't essential for the gameplay, the inclusion of effects increase the immersion of actions, such as losing a life with an alarm sound.

➤ **Styles**

The game will use its own designs, even if simple, to give a completely original product to users.

➤ **Screens**

Multiple screens are used to separate the game in states for user, thereby creating a structured UI.

➤ **Gameplay**

It includes all maps screens with different walls, searchable places and minigames locations.

➤ **Menu**

Give access to all the main screens, including the rank, options and gameplay ones.

➤ **Minigames**

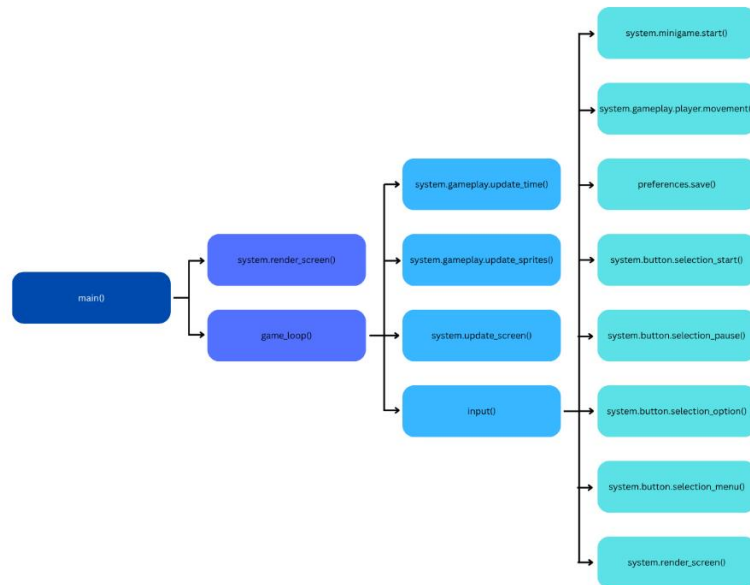
All the minigames have different screens according to their idea.

➤ **Endgame**

The mission success or failure should be clearly expressed to the user through these screens.

- **Option**
Gives user the access to the preferences about keys and volume allowing them to modify it.
- **Ranking**
Shows the best performances for the users, consequently increasing the competitiveness to “register” their usernames in the history of the game.
- **API**
It’s the part of the code that runs in a remote server instead of users’ machine. It’s used to store the best results.
- **Server**
It will run the algorithms that connect the information into the server to the user’s machine.
- **Database**
It stores the best performance of each user enabling the global ranking.
- **Users**
It’s a register of all taken usernames, as a result the new users can’t choose them.
- **Results**
It’s a register of all users’ best performance. It’s used to rank them.
- **Options**
It’s a local database that store the user preferences. It doesn’t need to be stored globally because it’s only one device per username, no logins.

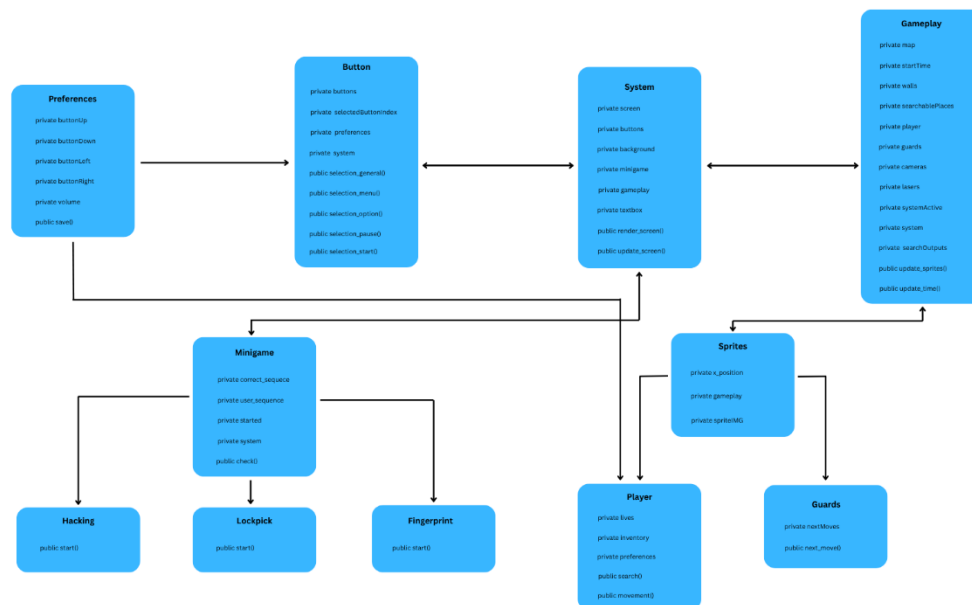
DISTRIBUTION CODE



The distribution code is the game code that run on user's machine. I chose to divide the explanation in parts of pseudocode; it makes easier for the development part given that the computational logic is already applied. The diagram below shows how the functions, procedures and methods link together. It shows each layer of the code; it doesn't include any function inside the methods used. Each layer has a color.

CLASSES RELATIONSHIPS

I divide the code in multiple classes, it will support me to divide the features code making easy to implement, reuse and test the code. The diagram below shows the private attributes and public methods of each one. The arrow shows the relationships that could be inheritance, I use because it is useful when no existent class attributes are used, or composition, I use because it is useful when the attribute to be used is from an existent class.



Inheritance relationship:

- Minigame -> Lockpick, Hacking and Fingerprint
- Sprites -> Player and Guards

Composition relationship:

- Gameplay, Button and Minigame -> System
- Sprites -> Gameplay

Aggregation relationship:

- Preferences -> Button and Player
- System -> Gameplay, Minigame and Button
- Gameplay -> Player

CLASSES, VARIABLES AND DATA STRUCTURES

It's important to understand the variables and data structures used to understand the work of the pseudocode. They can be present inside a class, so they are attributes.

Name	type	Scope	Explanation	Justification
isrunning	Boolean	Global	It defines if the main loop should continue.	I can handle the loop safely avoiding infinite execution.
system	Object	Local	Object with attributes used to	I need an object to be able to access the

			store states of the game and create relationships.	System class methods.
preferences	Object	Local	Store the object with the user preferences got in API	I need an object to access methods and attributes from Preferences class.
System class	Class	Global	Class responsible to handle any change in the screen state.	It's easier to store background and use functions to update screen when it all inside the same class.
screen	String	Class	Store the current screen state	Pygame need a screen object to print anything.
background	Object	Class	Store the background to be rendered. It's a file.	The game prints the background every frame, so it must remember it.
button	Object	Class	Create a composition relationship between System and Button	I need to be able to create and access buttons to determine the screen state.
gameplay	Object	Class	Create a composition relationship between System and Button	The gameplay actions affect directly the screen state.
minigame	Object	Class	Create a composition relationship between System and Minigame	I need to create the minigame while the gameplay run.
textbox	String	Class	Store the text input	I need to check and save the username later.
Preferences class	Class	Global	It the class that handles the interactions with the local database and allow to access it faster.	I need to user the preferences to handle input. Furthermore, I need to save options changes in the file.

buttonUp	String	Class	Store the preferred button to select up or move upwards	It allows users to use different keys for the up selection/movement.
buttonDown	String	Class	Store the preferred button to select down or move downwards	It allows users to use different keys for the down selection/movement.
buttonLeft	String	Class	Store the preferred button to move left	It allows users to use different keys for the left selection/movement.
buttonRight	String	Class	Store the preferred button to move right	It allows users to use different keys for the right selection/movement.
volume	Integer	Class	Store the preferred volume of sound	It allows the users to change the preferred volume.
Button class	Class	Global	Store and handle the buttons in the screen with selection.	It's easier to create a button behavior without need to pass them as parameter.
buttons	List	Class	A list of names that will be print as buttons, the order of buttons is essential	The buttons are organized in sequence, so it's easier to print in the same order.
selectedButtonIndex	Integer	Class	The index of the button selected it show the position of it in buttons	It's more efficient to store a list index than store the same data again.
preferences	Object	Class	Create an aggregation relationship between Preferences and Button	I need to know which key defined must be pressed to select the button.
system	Object	Class	Create an aggregation relationship between System and Button	I need to use this class to print the buttons.

Gameplay	Class	Global	The gameplay actions are stored and handled by it.	I can use together multiple methods using the same variables during the gameplay.
map	String	Class	Store the current map being played	I need to keep track when pausing or during minigames.
startTime	String	Class	Store the start time of the mission	I can calculate the total time used in mission using it.
walls	List of Tuples	Class	Store the coordinates of all the walls in map. Tuple store the coordinate.	I need to check collision with all the maps walls to don't let the player pass through them.
SearchablePlaces	List of Tuples	Class	Store the coordinates of all searchable places. Tuple store the coordinate	It allows to find items only when search is used while touching specific places
player	Object	Class	Create a player sprite with the composition relationship between Player and Gameplay	I need to create the player object inside the gameplay class.
searchOutputs	List	Class	The list of possible outputs when player search	It allows user to get each item only once.
guards	List	Class	List of objects, the guards' sprites. Use the composition relationship between Minigame and Gameplay, relationship passed across by inheritance.	I need to create the guard class inside the gameplay class. Therefore, the list keeps track of the existing ones.
cameras	List	Class	List of objects, the cameras' sprites. Use the composition relationship	I need to create the cameras sprites inside the gameplay class. Therefore, the

			between Minigame and Gameplay, relationship passed across by inheritance.	list keeps track of the existing ones.
lasers	List	Class	List of objects, the lasers' sprites. Use the composition relationship between Minigame and Gameplay, relationship passed across by inheritance.	I need to create the lasers sprites inside the gameplay class. Therefore, the list keeps track of the existing ones.
systemActive	Boolean	Class	Store the current state of the alarm	Keep track of the system is essential to disable lasers and cameras.
system	Object	Class	Create an aggregation relationship between Gameplay and System	I need to user this object to print elements on the screen.
Sprites	Class	Global	It handles any action of the sprites and store their info.	It's easier to create a general class to allow the creation of child classes.
coordinate	Tuple	Class	Store the current sprite coordinate	I need to know its position for collisions and printing.
gameplay	Object	Class	Create an aggregation relationship between Gameplay and Sprites	I need to know information about the current mission to choose some of the sprite's attributes.
spriteIMG	Object	Class	Stores the file with the image of the sprite	The sprite image is printed every gameplay frame, so it must be recorded.
Guards	Class	Global	A specific class to create guards inheriting the Sprites class	The actions of the guards can be handled at same time of the game by using

				threading in its methods.
nextMoves	Queue	Class	It stores the next 5 moves for the guards	The algorithm is limited to the next 5 moves to don't delay the guard movement.
Player	Class	Global	A specific class to create the player inheriting the Sprites class	Player information as lives can be handled easily by its methods.
Lives	Integer	Class	Store the number of remaining lives	Limits the number of times that player can wrong.
Inventory	List	Class	Store the items got from searchable places	Keep track of the items found.
preferences	Object	Class	Create an aggregation relationship between Player and Preferences	Allows to know what the preferred button is for searching.
Minigames	Class	Global	It contains all the information needed to complete the minigames.	The checking of the minigame sequences is the same, so they can inherit it.
correct_sequence	List	Class	Store the correct sequence of the minigame	The sequence to be printed and checked must be recorded.
user_sequence	List	Class	Store the user input sequence	Allows to compare with the correct one.
started	Boolean	Class	Store the minigame state	Don't allow the sequence to be printed more than once.
system	Object	Class	Create an aggregation relationship with System	Allows to print the sequence on the screen.
Lockpick	Class	Global	It will handle the lockpick minigame. It inherits the minigame class to	The start part of the minigames is different.

			check the sequences.	
Fingerprint	Class	Global	It will handle the fingerprint minigame. It inherits the minigame class to check the sequences.	The start part of the minigames is different.
Hacking	Class	Global	It will handle the hacking minigame. It inherits the minigame class to check the sequences.	The start part of the minigames is different.

I chose to use most of the variable to be attributes because it allows multiple function to use them, we just pass the object as parameter turning the code easier. The “isrunning” is global because is the main control of the game state, so we can stop the game anywhere. The other local variables are used to create the objects. I use list in the big part of data structures because it’s a build-in Python tool, so the development is easier. The only data structure that isn’t a list is the queue, but it will be implemented using lists.

MAIN()

It’s the initialization of the code. It initializes the tool, create the screen and the title, and start the background music that will stays the same for the whole game. Here the first interaction with database happens, it’s a local database that store the user preferences. The creation of system variable is essential to control the screen states. Preferences will store the user preferences about buttons and send it to API.

```

procedure main()

    initialize pygame

    create screen 600 x 600

    create title "Richouse"

    play music forever "music.mp3"

    select user data from database

    if user data is null then

        system = System()

        system.render_screen("account")

        preferences = Preferences(standard preferences)

```

```

        else

            preferences = Preferences(data from database)

            system = System()

            system.render_screen("menu")

        endif

        game_loop(system, preferences)

endprocedure

```

I separated the system and preferences objects because the preferences would relate with multiple classes, so it's better to create an own class to make these relationships easier. The game loop is separated also because in this way I can test different parts of code inside the main() function without need to use the game_loop(). The screen will be 600 x 600 because will be easier to develop the pathfinding algorithm. The audio file will be mp3 because I will use some existent music and it's the most common file type for it. There is a local database beside the API one because it doesn't depend on server, using less server processing. We can see that using Pygame it can be easily executed with few commands making more reliable to use it.

GAME_LOOP()

This is the main loop of the game, so it controls the things that should be constantly updated. It includes the current screen, the position of the sprites, the time taken during the phase and the inputs entered.

```

procedure game_loop(system: object, preferences: object)

    while isrunning

        system.update_screen()

        if system.screen in ["gameplay", "hacking", "lockpick", "fingerprint"] then

            system.gameplay.update_sprites()

            system.gameplay.update_time()

        endif

        input(system, preferences)

    endwhile

endprocedure

```

I divide the loop in four parts that update things that should be constantly updated such as the screen, the sprites, the time and input handle. Sprites and time taken are only update during the mission.

INPUT()

It will handle the inputs entered by the user. The input is gotten by Pygame, another valuable feature of this tool. I divide the input into screens because the input will be

difference for each screen. The classes methods are used because the code become reusable.

```
procedure input(system: object, preferences: object)

    for key in keysPressed

        switch screen:

            case "menu":

                system.buttons.selection_menu(key)

            case "rank":

                if key == "ESC":

                    system.render_screen("menu")

                endif

            case "option":

                system.buttons.selection_option(key)

                if key == "ESC" then

                    preferences.save()

                    system.render_screen("menu")

                endif

            case "gameplay":

                system.gameplay.player.movement(key)

                if key == "F" then

                    interact with object searching it or open a minigame

                else if key = "ESC" then

                    system.render_screen("pause")

                endif

            case "hacking":

                switch key:

                    case "E":

                        If "chip" in system.gameplay.player.inventory then

                            system.minigame.start()

                        endif

                    case "ESC":

                        system.minigame.started = False

                        system.render_screen("gameplay")

                    default:

                        add key to system.minigame.user_sequence

            case "lockpick":

                switch key:

                    case "E":
```

```

        If "lockpick" in system.gameplay.player.inventory then
            system.minigame.start()
        endif

    case "ESC":
        system.minigame.started = False
        system.render_screen("gameplay")

    default:
        add key to system.minigame.user_sequence

case "fingerprint":
    switch key:
        case "E":
            If "UV light" in system.gameplay.player.inventory then
                system.minigame.start()
            endif

        case "ESC":
            system.minigame.started = False
            system.render_screen("gameplay")

        default:
            add key to system.minigame.user_sequence

case "pause":
    system.button.selection_pause(key)

case "account":
    if key == "Enter" then
        send username to API

        if username don't exist then
            save in local database
        else
            print("Username already exist")
        endif
    else:
        system.textbox = system.textbox + key
    endif

case "start":
    system.button.selection_start(key)

next key

endprocedure

```

The menu, option, pause and account handle inputs only for selection of buttons, all buttons are in vertical because in this way the code of selection can be reused. The

gameplay handle inputs related to player sprite movement and interactions; there is a button to open the pause screen. The fingerprint, hacking and lockpick have inputs that are used to complete the minigame by completing sequences. Account stores the key in a textbox that will be the username. Given that multiple function I divide it into methods of classes that link one with another, so one of motifs to use Python is because of it's a Object Oriented programming language. Something to consider is the authentication of user input in username textbox, some attacks such as SQL injection or XSS attack could be a problem.

PLAYER CLASS METHODS

It controls the player sprite actions, it includes the movement and the search actions.

```
public procedure movement(key: string)

    Switch key:

        case preference.buttonRight:

            position[0] = position[0] + 5

        case preference.buttonLeft:

            position[0] = position[0] - 5

        case preference.buttonDown:

            position[1] = position[1] - 5

        case preference.buttonUp:

            position[1] = position[1] + 5

endprocedure
```

The first method is the movement() that handle the movement of player based on the input entered. The Pygame lead the screen as a cartesian graph, so it's easy to control using coordinates. I use tuples to represent the coordinates because it easier if consider a 2d cartesian graph.

```
public procedure search()

    get at random an element of gameplay.searchOutputs

    add item to inventory

    remove item from gameplay.searchOutputs

    remove place from gameplay.searchablePlaces

endprocedure
```

The other function that control the player actions is the search() that is executed when we search one of the searchable places, so the function use the relationship with the Gameplay to see the possible outputs of the search.

BUTTON CLASS METHODS

This class has methods related to buttons selection. The selection is based on a list and the selected button is an element of the list with certain index, it makes easier to lead with button logic.

```
public procedure selection_general(key: string)

    switch key:

        case preferences.upButton:

            selectedButtonIndex = (selectedButtonIndex + 1) MOD LENGTH(buttons)

        case preferences.downButton:

            selectedButtonIndex = (selectedButtonIndex - 1) MOD LENGTH(buttons)

endprocedure
```

The selection can be general, used to select buttons up and down, or exclusive, that extend the general. All buttons are vertical because I can reuse this general code in all exclusive selections.

```
public procedure selection_menu(key: string)

    selection_general(key)

    if key == "ENTER" then

        switch buttons[selectedButtonIndex]:

            case "start":

                system.render_screen("start")

            case "rank":

                system.render_screen("rank")

            case "option":

                system.render_screen("option")

            case "quit":

                isrunning = False

        endif

    endif

endprocedure
```

The next we can see the extension of menu selection, 4 possible buttons that do different actions. The isrunning is used again without be passed as parameter because it's global, in this way user can stop the game.

```
public procedure selection_option(key: string)

    selection_general()
```

```

if SHIFT is pressed then

    switch buttons[selectedButtonIndex]:

        case "forward":

            preferences.upButton = key

        case "left":

            preferences.leftButton = key

        case "down":

            preferences.downButton = key

        case "right":

            preferences.rightButton = key

        case "use":

            preferences.useButton = key

        case "interact":

            preferences.interactButton = key

    endif

if buttons[selectedButtonIndex] == "volume" then

    switch key:

        case "A":

            preferences.volume = preferences.volume - 5

        case "D":

            preferences.volume = preferences.volume + 5

    endif

endif

endprocedure

```

The selection in option is used to change volume and change the preferred controls. When user press SHIFT + key in one of the buttons controlling the action button preferences, this pressed button will be placed as preferred. The volume button will change the volume settings by increasing the volume by 5 when pressed right and decrease 5 when pressed left. The preferences is not a parameter because the attribute used to make the aggregation relationship.

```

public procedure selection_pause(key: string)

    selection_general(key)

    If key == "ENTER" then

        switch buttons[selectedButtonIndex]:

            case "replay":

                system.render_screen("gameplay")

            case "menu":

                system.render_screen("menu")

        endif

    endif

```

```
endprocedure
```

Selection of buttons when you pause the screen, it allows the user return to menu or just stop the gameplay temporally.

```
public procedure selection_start(key: string, system)

    selection_general(key)

    if key == "ENTER" then

        switch buttons[selectedButtonIndex]:

            case "map1":

                start the map 1

            case "map2":

                start the map 2

        else if key == "ESC":

            system.render_screen("menu")

        endif

    endif

endprocedure
```

Control the selection of maps in menu, it makes easier to extend the number of maps in future. We see that ENTER was used as the confirmation of selection, it's the standard button of selection in many software, so it's intuitive. For the same motif, the ESC was chose as the return button. W, A, S and D are used both for selection and movement to minimize the number of buttons and make it intuitive, easy for a possible update to consoles in future.

SYSTEM CLASS METHODS

This class control screen state, so its methods are related to screen updates and rendering. Screen should be constantly update because users input change its state. I will use PNG files instead of JPEG because it uses lossless instead of lossy, so the graphic quality will be improved and won't be a problem given that images won't be transferred over internet; another motif is because I'm using Canva because of its simplicity and it's a free tool to make graphic elements.

```
public procedure render_screen(nextScreen: string, userPreferences: object)

    switch nextScreen:

        case "menu":

            buttons = Button(buttons=["start", "rank", "option",
            "quit"],preferences=userPreferences, system=self)

            background = menu.png

        case "rank":

            buttons = NULL

            access API
```

```

        background = rank.png

case "option":

    buttons = Button(buttons=["volume", "forward", "backward", "left", "right", "use",
"interation"], preferences=userPreferences, system=self)

    background = option.png

case "gameplay":

    if screen == "start" then

        map = buttons[selectedButtonIndex]

        buttons = Null

        if map = "map1" then

            gameplay = Gameplay(map=map, startTime= current time , walls= walls
of map 1, searchablePlaces= serachable places in map 1)

            player = Player(add initial place for map 1)

            create guards sprites with Guards(add initial place for map 1)

            create lasers and cameras using Sprites(add places for map 1)

            background = map1.png

        else

            gameplay = Gameplay(map=map, startTime= current time , walls= walls
of map 2, searchablePlaces= serachable places in map 2)

            player = Player(add initial place for map 2)

            create guards sprites with Guards(add initial place for map 2)

            create lasers and cameras using Sprites(add places for map 2)

            background = map2.png

        endif

        add sprites to gameplay.sprites

        start the threading of method next_move() in guards

    endif

case "fingerprint":

    buttons = Null

    minigame = Fingerprint(random sequence in range 0 to 9)

    background = fingerprint.png

case "lockpick":

    buttons = Null

    minigame = Lockpick(random sequence of arrows)

    background = lockpick.png

case "hacking":

    buttons = Null

    minigame = Hacking(random sequence of letters)

    background = hacking.png

case "pause":

```

```

        buttons = Button(buttons=["replay", "menu"], preferences=userPreferences, screen=self)

        background = pause.png

    case "start":

        buttons = Button(buttons=["map1", "map2"], preferences=userPreferences, screen=self)

        background = start.png

    case "account":

        background = account.png

        textbox = ""

    screen = nextScreen

    print background

    print rank scores if in rank screen

    print sprites and status bar if in gameplay screen

    print textbox if in account screen

    print the buttons, different color for the selected

endprocedure

```

Control the initialization or resume of the screens. The menu, option, account, start, pause, hacking, fingerprint and lockpick will always initialize the same. Already the rank will depend on API results to make the rank, results should be in API because we should know the highest score including all player in the world, because it the username is unique. The gameplay will depend on the map choice or if the game was started or just resumed, if resumed, the sprites and time will continue in the same state as before the start screen. I will use the request library to access the API because it's a build-in Python tool, makes the development easier.

```

public procedure update_screen()

    print background

    print the buttons, different color for the selected

    print textbox if in account screen

    print the status bar if in gameplay or any minigame screen

endprocedure

```

I just update the current states using the same thing of render_screen() but changing the selection because the button selected constantly change. It doesn't lead with sprites and time printing, other functions are responsible for it. Using background as attribute allow us to update without use parameters. The button selected is know by System because the composition relationship with Button.

PREFERENCES CLASS METHOD

It has only one method that is responsible to update the local database with the preferred controls.

```

public procedure save()

    update the database preferences with attributes

endprocedure

```

The local database store only the preferences and the username. To access it the sqlite3 library will be the best option because of its simplicity and it's build-in Python tool. SQLITE is better for it because it uses few space and work well for the simple specs of this application.

GAMEPLAY CLASS METHODS

It's responsible to update the elements that only happen while the gameplay in running.

```

public procedure update_sprites()

    print the position player

    print the next position of guards

    print the cameras position but rotate

    if systemActive == true then

        print lasers positions

    endif

    if player touch laser or player within 50px in front of guards or active cameras front then

        sprites return to initial position

        player lost a live

        alarm sound effect happen

        if player.lives == 0 then

            print game over screen

            wait few seconds

            system.render_screen("menu")

        endif

    endif

    if player is in door and have the golden statue then

        print mission completed screen

        send results to API, if it's the higher

        wait few seconds

        system.render_screen("menu")

    endif

endprocedure

```

The player, guards and cameras are always print because they can disappear, already the laser only appear if system is active. We should check if the player position can be seen by security system, if yes the player lost a live. If player lost all lives then the game will finish with game over and no results are stored. If the player arrive at front door with

the golden statue, the mission completed screen appear and the results are stored case they are the higher one. I will use the Python time library to wait these few time.

```
public procedure update_time()

    timeTaken = current time - startTime

    print timeTaken

endprocedure
```

It will update and print the current time taken in the mission, can be calculated using the Python datetime library. The time is only printed during the gameplay or minigames.

MINIGAME CLASS METHOD

It contains only one method that will be inherited by Fingerprint, Hacking and Lockpick.

```
public procedure check()

    length = LENGTH(user_sequence)

    for x=0 to length

        if user_sequence[x] != correct_sequence[x] then

            wrong sound effect is executed

            system.render_screen("gameplay")

        else if length == 4 then

            success sound effect is executed

            system.render_screen("gameplay")

        endif

    next x

endprocedure
```

It uses the list of inputs entered by the user, while they are trying to complete the minigame, and compare with the list containing the right sequence. If the sequence is correct when its size 4, so user complete the minigame and something happen in the gameplay, it depends on extension of this method in the development. If the sequence entered is wrong, the minigame quit without any change.

FINGERPRINT CLASS METHOD

It is the method used to start the minigame when the player press E with the required item.

```
public procedure start()

    system.background = fingersolution.png

    started = True

endprocedure
```

It just change the background to the background with the solution. The started control the state of the minigame, so the user can repeat multiple times to change the sequence. It will have also a extension of check().

HACKING CLASS METHOD

It will start the hacking minigame when the key E is pressed when the chip is in the player inventory.

```
public procedure start()

    system.background = hackingsolution.png

    started = True

endprocedure
```

It change the background to the background with a hint of the solution. In development section it will have an extension of check().

LOCKPICK CLASS METHOD

It will start the lockpick minigame when the E is pressed while the lockpick is in inventory.

```
public procedure start()

    for element in correct_sequence

        print element

        wait few time

        print screen.background

    next element

endprocedure
```

Temporally print the solution sequence and user should input it in same order. It will extend the check().

GUARD CLASS METHOD

It will have on method that will be used to find the next moves of the guards automatically, so it will run concurrently.

```
public procedure next_move()

    Dijkstra algorithm using the screen size and the wall and sprites position

    if LENGTH(nextMoves) < 5 then

        add the next move found by algorithm to nextMoves queue

    endif

endprocedure
```

The Dijkstra algorithm will find the shortest path to player to calculate the better next move; it will constantly change given that player is moving. I chose this algorithm because it's widely used in networking given its efficiency to find paths, furthermore it's covered in A level making easy to connect with our school content practice. The queue nextMoves have a limit of 5 because as the player move we need to find shortest paths to them, so few next moves are predicted to prevent delay.

LOCAL DATABASE

It's a global database used to store the username and user preferences; it's stored locally to decrease the number of server requests decreasing the server processing that could slow down the server.

```
CREATE TABLE user {  
  
    username TEXT,  
  
    upButton TEXT,  
  
    downButton TEXT,  
  
    leftButton TEXT,  
  
    rightButton TEXT,  
  
    useButton TEXT,  
  
    interactButton TEXT  
  
};
```

It will use only one table for all because it's just accessed once per session, so no needed of normalization to decrease the processing. One table make simple to introduce without worry about rules.

API CODE

CODE

The API server will have few code, only the necessary to get the information and check the database, so I won't use diagram or separate the code in parts. I will use Flask for this part because it's a simple tool that is implemented in Python, so no external tool needed. Flask allows the communication using POST methods by using the request Python library.

```
route the request of new user
```

```

function new_user()

    get the message

    decrypt the message

    check if username exist

    if username exist then

        return error

    else

        add username to database

        return success

    endif

endfunction

```

It is used to add the new username to the database if it doesn't exist, this request happens when we are in the account screen, and we confirm the username that we want. The encryption of data transfer should happen because man-in-the-middle attacks can happen, they could add malicious code to the legitimate packet sent by user and the server. The code should return something to tell the distribution code if it was successful or failed. A server-side authentication should also happen to check possible SQL injection or XSS attack.

```

route the request of new highest score

function set_score()

    get the message

    decrypt it

    update the database with the score

    if error then

        return error

    endif

    return success

endfunction

```

It's used to set the best score of the user when it's sent to server. In this way all users in the world could see it if part of the 10 best scores in world.

```

route the request of the best 10 scores

function best_scores()

    find the best 10 scores from database

    return the best 10 scores as a list

endfunction

```

It's used to get the 10 best scores and return it to the distribution code, with this the ranking can be displayed. In all the functions above, the Python sqlite3 library is used to

execute the commands in the database, and all data returned is encrypted before send to prevent man-in-the-middle attacks.

SERVER DATABASE

This database will have the data from diverse users worldwide. Given that multiple operations happen on it, we should normalize it in maximum as possible.

```
CREATE TABLE users {  
  
    username TEXT PRIMARY KEY  
  
};
```

Store all usernames used by people around the world, it's checked every time that a new username wants to be created. The username is a primary key because it's unique. Stored in different table case we need more user information in future, if it happens then we just increase the table.

```
CREATE TABLE maps {  
  
    ID INTEGER PRIMARY KEY,  
  
    name TEXT  
  
};
```

It stores the maps used in the game. There is an ID to identify them, it's used in normalization to do not store the same map name repeatedly.

```
CREATE TABLE results {  
  
    username TEXT,  
  
    mapID INTEGER,  
  
    lives INTEGER,  
  
    timeTaken TEXT,  
  
    FOREIGN KEY username REFERENCES users (username),  
  
    FOREIGN KEY mapID REFERENCES maps (ID)  
  
};
```

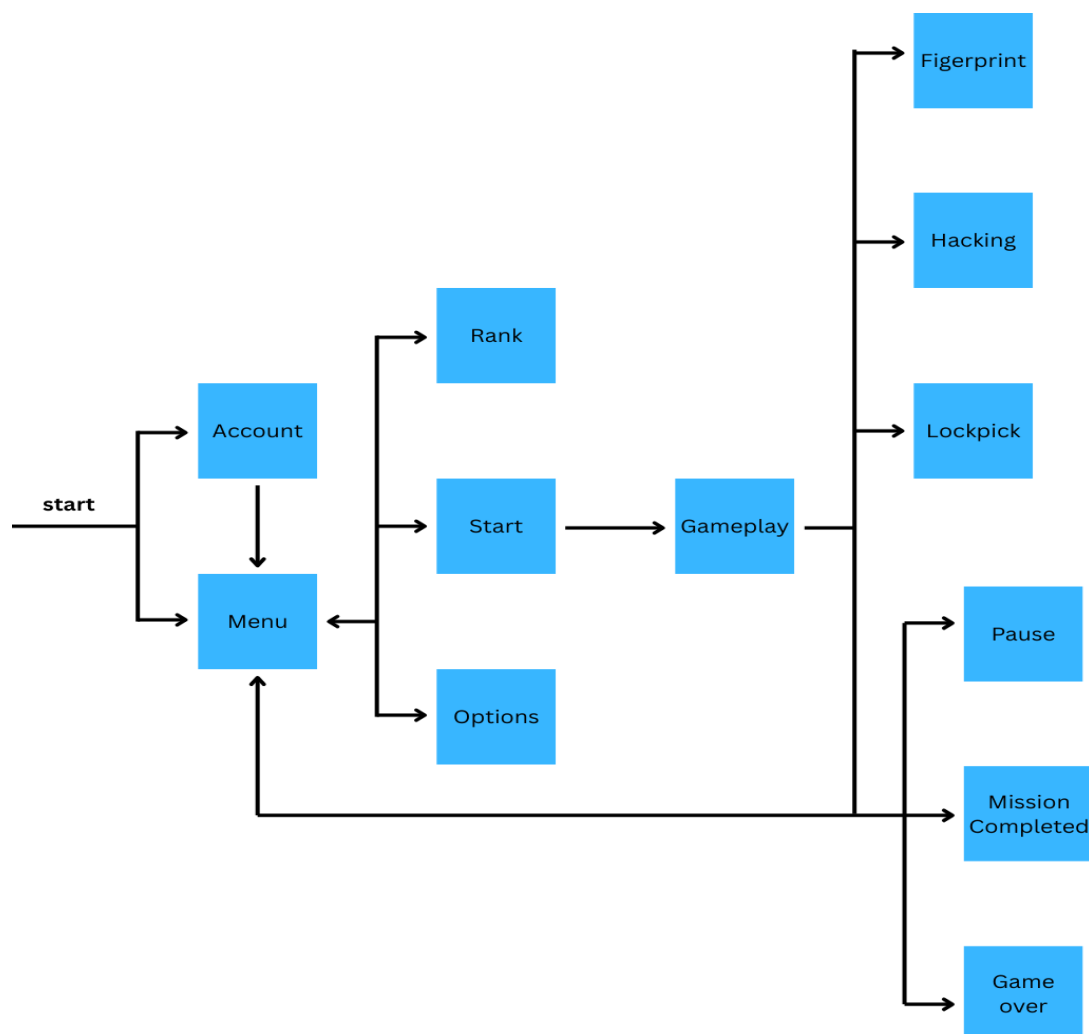
It stores the number of lives and the time taken by the user in each map, the main definition of best score is the number of lives, as higher as possible, after is the time

taken, as lower as possible. The tables are separated because it improve the operations time by using normalization.

GRAPHIC DESIGN

SCREENS

The screens will constantly change and each of them have its own elements to allow the user to comprehend what is happening in the code. The diagram shows how the screen are linked together. As said before, the screens will be squared, 600x600, because it makes simpler to implement Dijkstra algorithm during the gameplay.



The relationship is show by arrows, the point shows the next screen, so:

- Account goes to menu
- Menu go to Rank, Start or Options
- Rank, Start or Options go to Menu
- Start go to Gameplay

- Gameplay go to Hacking, Fingerprint, Lockpick, Pause, Mission completed or Game over
- Lockpick, Hacking, Fingerprint and pause go to Gameplay
- Pause, Mission completed and Game over go to Menu

MENU SCREEN



- Usability features:
1. Buttons are centralized because it fixes the user attention to the middle of the screen while the side spaces can be fill by a beautiful art, in this way user can feel more immersion.
 2. The game title is the bigger text because in this way user can have sure that it's the title furthermore it's more important to identify the game than the buttons.
 3. Buttons are in vertical; it was a decision for all screens to allow me to reuse the general selection code in all screens with buttons.
 4. The box around the selected button is used to show the user what they are currently selecting, connection between computer logic and user understand.

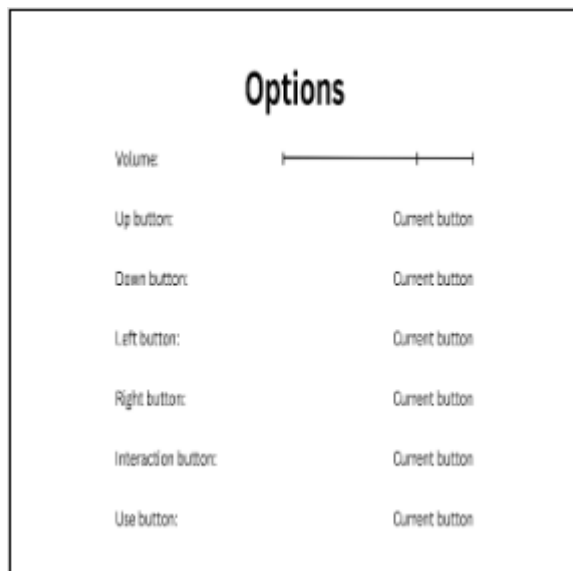
RANKING SCREEN



➤ Usability features:

1. There are 2 rankings because there are 2 different maps that take different amount of time, it's unfair to compare the time using only 1 rank.
2. The top 10 was chosen because it's more attractive than top 50 for example, the exclusivity can make the users play and effort more to participate in the ranking.

OPTIONS



➤ Usability features:

1. The screen used to change the buttons and volume preferences.
2. The volume is shown by a bar because in this way we can visualize the audio intensity.

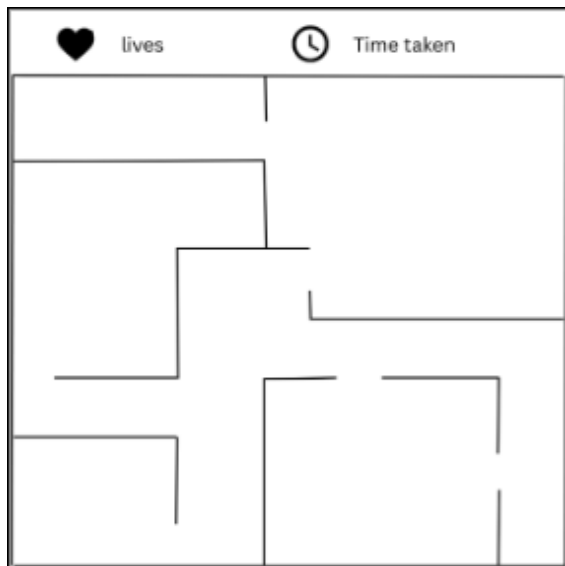
3. The other options are the button preferences; it show what button it refers and in the right is the current button registered.
4. The screen name is always print on top; it supports the user identify where they entered.

START SCREEN



- Usability features:
1. The screen used to choose the map before starting the gameplay.
 2. It will have 2 maps, and the background will be divided in 2, one showing a part of each map, in this way the user can see and choose the map.
 3. Again, the button selected has a box and the title showing the screen.

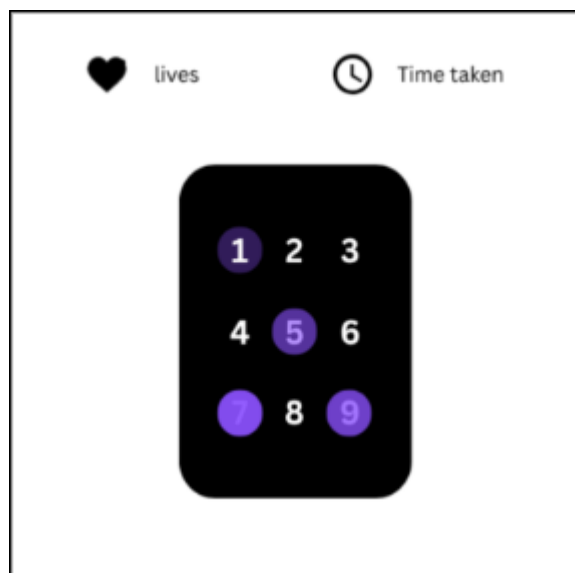
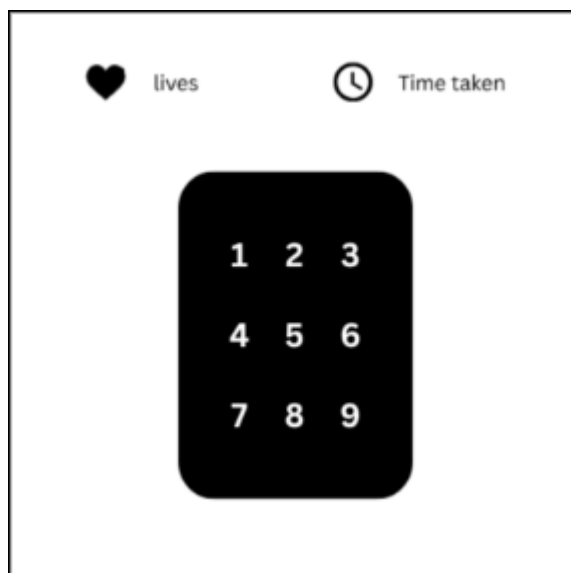
GAMEPLAY SCREEN



➤ Usability features:

1. It's the structure of the gameplay screen; the maps will be different.
2. The screen has the statuses on top, it shows the number of lives and the time taken, it supports the user to see its performance in the mission.
3. The map has lines showing the walls, it connects the logic of walls in code with users' vision.
4. The map is square to support the implementation of Dijkstra algorithm.

FINGERPRINT SCREEN

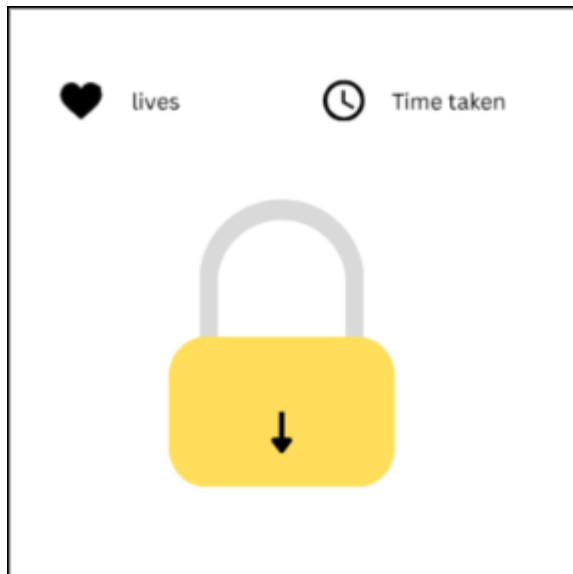


➤ Usability features:

1. The first screen is when you enter in the minigame, it gives a background to user understand what's going on.

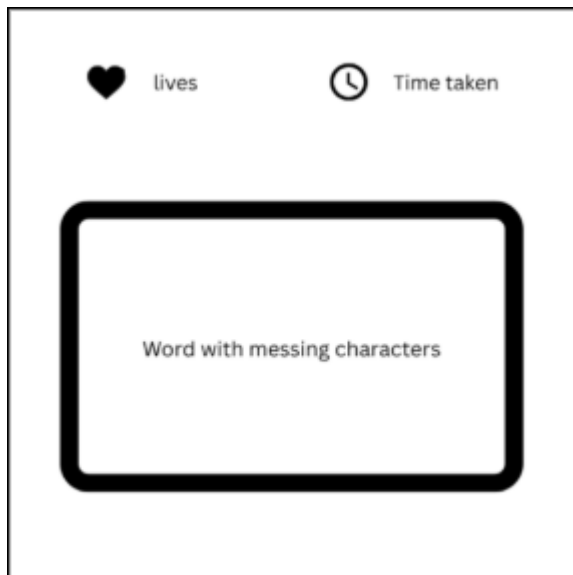
2. The statuses still being printed because the guards will continue to look for player and the time still running.
3. The second is when you start it with the necessary tool.
4. The screen shows the sequence by increasing order of intensity, it looks like a fingerprint.

LOCKPICK SCREEN



- Usability features:
1. It's the screen opened when the lockpick game is entered.
 2. In beginning the lock doesn't have anything inside, after starts it show a sequence of 4 arrow.
 3. The user should input the correct arrow sequence.
 4. The statuses continue to be printed because the gameplay is happening.

HACKING SCREEN



- Usability features:
1. The screen when you enter in the hacking minigame.
 2. The screen when user enter in the minigame there is no word in it, after starting a word with missing characters is printed.
 3. User should input the missing characters in order. The statuses continue to be printed.

PAUSE SCREEN



- Usability features:
1. When you press ESC during the gameplay.
 2. There are 2 options, the return to continue gameplay or menu to return to menu screen.
 3. Big title to know what's happening.

4. The selected button is marked by a box.

GAME OVER SCREEN



- Usability features:
 1. The screen opened when you lost all the lives, and the mission fail.
 2. It's just a screen with an art as background and a big "Game over" to tell user what happened.

MISSION COMPLETED SCREEN



- Usability features:
 1. Appear when the player arrives in the front door with the golden statue.

2. It will have an art showing the glory of success and a big mission completed to give a feel of success for the user.
- 3.

SPRITES

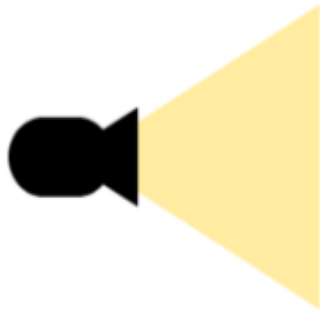
GUARDS AND PLAYER SPRITES



➤ Usability feature:

1. A simple character that will be used as base of player and guards' sprites.
2. It has a cartoon and doesn't have complex details, it makes the user focus on gameplay instead of graphics, furthermore it's a good minimalist design.
3. There's no mouth because the game doesn't have dialogues.

CAMERAS SPRITES



➤ Usability features:

1. It's a simple sprite that will rotate during the game.
2. The yellow part shows the range that camera can see the user, a way to connect the code logic and the user game view.
3. It's simpler to combine with another map elements.

STAKEHOLDERS' FEEDBACK ABOUT THE LAYOUTS

- [Enzo Garofani \(translated from Portuguese\):](#)

The GDD is good, it has an identity, the screens seem to make sense with this minimalist and well-executed gameplay noir. I hope the game follows a good identity like the one presented here.

- [Zach Ward:](#)

I really like the concept of geometric forms to build the Sprites. However, the map showed could have less walls, as a result the gameplay would be freer to move and the screen clearer.

- [Alex Michalski:](#)

The screens and sprites look good in general. On the other hand, believe that the statuses would be better positioned if it's showed inside a bar, the same for all screens, in the top rather than let it use this much space.

TESTING

WHITE BOX TESTING

It's the testing of the code structure, so it will optimize the code and its functionality. Useless lines can slow down the code, and poorly written conditions and Boolean expression can bug the game in some cases.

For this test I can check each module manually by getting each function and putting it in the main() to run individually. When it runs, I can check the lines being executed using debuggers that shows the current line being executed. This test won't test any possible input, only the structure.

I can also test using an automatic tool. In this case I would use the PyTest, it's a library that can be installed to check your code. It will automatize the process of white box testing using less time, in this way I could focus on another tests.

BLACK BOX TESTING

It's the testing for possible inputs entered by the user during the play. It checks the basic function ability of the game before distributing it in the Alpha and Beta testing. The inputs tested in each screen will be listed below.

Number	Data	State	Expected result	Justification
Menu				
1	W	Valid	Up button selected, return to last button if top arrived	It's the default button for many games but can be changed in the options.
2	S	Valid	Down button selected, return to first button if bottom arrived	It's the default button for many games but can be changed in the options.
3	ENTER	Valid	Confirm the selected button, open the screen	It's an intuitive button to confirm in many systems.
4	Any other key	Invalid	The key doesn't do anything	The useless keys can't affect the functionality of the others.
Rank				

15	ESC	Valid	Return to the menu screen	It's an intuitive button to return in many systems.
6	Any other key	Invalid	The key doesn't do anything	The useless keys can't affect the functionality of the others.
Options				
7	W	Valid	Up button selected, return to last button if top arrived	It's the default button for many games but can be changed in the options.
8	S	Valid	Down button selected, return to first button if bottom arrived	It's the default button for many games but can be changed in the options.
9	A	Valid	It's valid if volume button selected, decrease volume by 5 if higher than 5, else doesn't do anything	It's the default button for many games but can be changed in the options.
10	D	Valid	It's valid if volume button selected, increase volume by 5 if lower than 100, else doesn't do anything	It's the default button for many games but can be changed in the options.
11	SHIFT + Letters A - Z	Valid	It's valid if the button selected is used to update preferences, it will be the button for the action chose	It can differentiate the preference selection from random clicks by adding the SHIFT layer.
12	SHIFT + Numbers 0 - 9	Valid	It's valid if the button selected is used to update preferences, it will be the button for the action chose	It can differentiate the preference selection from random clicks by adding the SHIFT layer.
13	ESC	Valid	Return to the menu screen	It's an intuitive button to return in many systems.
14	Any other key	Invalid	The key doesn't do anything	The useless keys can't affect the functionality of the others.
Start				

15	W	Valid	Up button selected, return to last button if top arrived	It's the default button for many games but can be changed in the options.
16	S	Valid	Down button selected, return to first button if bottom arrived	It's the default button for many games but can be changed in the options.
17	ENTER	Valid	Confirm the selected button, open the screen	It's an intuitive button to confirm in many systems.
18	ESC	Valid	Return to the menu screen	It's an intuitive button to return in many systems.
19	Any other key	Invalid	The key doesn't do anything	The useless keys can't affect the functionality of the others.
Fingerprint				
20	E pressed once	Valid	Start the minigame by changing the background screen	It will allow the current sequence to be printed when user is ready.
21	E pressed more than once	Invalid	The key E pressed more than once should do anything more	It can't allow the user to restart when they want.
22	Input of 4 numbers 0-9	Valid	The result will be checked by comparing with correct sequence	The correct sequence is made of 4 numeric characters.
23	Input of more than 4 numbers 0-9	Invalid	The extra keys inputted will be ignored.	It would cause an index error during the check with the correct sequence if more numbers were inputted.
24	Correct sequence of numbers	Valid	The minigame is completed and return to gameplay changing some sprite state	The minigame must contain a sequence in which it will finish.
25	Incorrect sequence of numbers	Valid	The minigame is completed and return to gameplay without change sprites state	The minigame must block any wrong attempt to make the game harder.

26	ESC	Valid	Return to the gameplay screen without change any sprite state	It's an intuitive button to return in many systems.
27	Any other inputs	Invalid	The key doesn't do anything	The useless keys can't affect the functionality of the others.
Hacking				
28	E pressed once	Valid	Start the minigame by printing the word with missing characters	It will allow the current sequence to be printed when user is ready.
29	E pressed more than once	Invalid	The key E pressed more than once should do anything more	It can't allow the user to restart when they want.
30	Input of 4 letters A-Z	Valid	The result will be checked by comparing with correct sequence	The correct sequence is made of 4 alphabetic characters.
31	Input of more than 4 letters A-Z	Invalid	The extra keys inputted will be ignored.	It would cause an index error during the check with the correct sequence if more characters were inputted.
32	Correct sequence of letters	Valid	The minigame is completed and return to gameplay changing some sprite state	The minigame must contain a sequence in which it will finish.
33	Incorrect sequence of letters	Valid	The minigame is completed and return to gameplay without change sprites state	The minigame must block any wrong attempt to make the game harder.
34	ESC	Valid	Return to the gameplay screen without change any sprite state	It's an intuitive button to return in many systems.
35	Any other inputs	Invalid	The key doesn't do anything	The useless keys can't affect the functionality of the others.
Lockpick				

36	E pressed once	Valid	Start the minigame by printing the arrows sequence	It will allow the current sequence to be printed when user is ready.
37	E pressed more than once	Invalid	The key E pressed more than once should do anything more	It can't allow the user to restart when they want.
38	Input of 4 arrows	Valid	The result will be checked by comparing with correct sequence	The correct sequence is made of 4 different arrows.
39	Input of more than 4 arrows	Invalid	The extra keys inputted will be ignored.	It would cause an index error during the check with the correct sequence if more arrows were inputted.
40	Correct sequence of arrows	Valid	The minigame is completed and return to gameplay changing some sprite state	The minigame must contain a sequence in which it will finish.
41	Incorrect sequence of arrows	Valid	The minigame is completed and return to gameplay without change sprites state	The minigame must block any wrong attempt to make the game harder.
42	ESC	Valid	Return to the gameplay screen without change any sprite state	It's an intuitive button to return in many systems.
43	Any other inputs	Invalid	The key doesn't do anything	The useless keys can't affect the functionality of the others.
Pause				
44	W	Valid	Up button selected, return to last button if top arrived	It's the default button for many games but can be changed in the options.
45	S	Valid	Down button selected, return to first button if bottom arrived	It's the default button for many games but can be changed in the options.
46	ENTER	Valid	Confirm the selected button and render the screen	It's an intuitive button to confirm in many systems.

47	Any other key	Invalid	The key doesn't do anything	The useless keys can't affect the functionality of the others.
Gameplay				
48	W	Valid	Move character five pixels up if there is no wall, else it is move to one pixel before the wall.	It's the default button for many games but can be changed in the options.
49	A	Valid	Move character five pixels left if there is no wall, else it is move to one pixel before the wall.	It's the default button for many games but can be changed in the options.
50	S	Valid	Move character five pixels down if there is no wall, else it is move to one pixel before the wall.	It's the default button for many games but can be changed in the options.
51	D	Valid	Move character five pixels right if there is no wall, else it is move to one pixel before the wall.	It's the default button for many games but can be changed in the options.
52	F	Valid	It will interact with minigame or searchable place if next to it, else do nothing	It's close to the movement default keys, so it was chosen as default for interactions. However, can be changed in the options.
53	ESC	Valid	Open the pause screen	It's easier to use the same key of the return action to avoid the use of many different keys.
54	Open game over screen if lost all lives	Valid	Print the game over screen and return to menu	The game must have a finish state even if the user didn't stop yet.
55	Open mission completed screen if arrive at door with statue	Valid	Print the mission completed screen and return to the menu	The game must finish when the final objective is achieved.

56	Lost a live if detected by security system	Valid	Lost one live and teleport user to front door	Live systems make the game harder.
57	Any other input	Invalid	Do nothing	The useless keys can't affect the functionality of the others.
Account				
58	Enter a username smaller than 10 characters	Valid	The username is sent to API to check if it's available	More than 10 characters won't fit in the ranking screen.
59	Enter a username bigger than 10 characters	Invalid	The keys after 10 th character are ignored	If not ignored can cause overflow in the system.
60	Number 0-9	Valid	It enters in the username	Give more variation to the username.
61	Letters A-Z	Valid	It enters in the username	Give more variation to the username.
62	_ key	Valid	It enters in the username	Give more variation to the username.
63	Any other key	Invalid	It won't enter in the username	The other keys are considered dangerous for the system integrity.

When I did this I consider security methods, when the input is checked in database or executed it can't have programming symbols to avoid SQL injections and XSS attacks.

ALPHA TESTING

In this stage I will play the game as a user looking for bugs and possible improvements in the gameplay.

First, I will try to open all screens to check if all screens load correctly. After I will try to play both maps, I will use the movement, interaction and use button to check the searchable places, enter and start in the minigames, and open the pause screen. I will check the guard's detection, and the mission completed and the game over screens.

After I will check if my results were stored and sent to API. To do it, I will check the rank because I will be the only player in the world, if it appears there it's working. Finally, I will try to change the preferences and play with the new preferences.

Check if in the account screen I can't use a username that has some coding character such as <, - and / that could be used in SQL injection or XSS attack.

This testing has some expectations about the general game functionality:

Number	Feature	Validity	Justification
1	The username register screen is opened right after game start.	Only if it's the first time opening the game, or no username is registered.	A username is needed to rank the user performance.
2	A username can be registered and confirmed pressing ENTER.	Only if the username has a maximum of 10 characters (alphanumeric only).	It will confirm that the text wasn't entered wrong.
3	Menu screen is rendered after register username.	Only if the username isn't in use. Otherwise, it will return "Unavailable".	Allows users to access all the other screens.
Menu screen			
4	Menu allows to selected buttons in a column. It acts as a loop when the top or bottom arrive.	Only if the buttons are the ones used for the selection. It's defined in the options screen.	It's in a column to reuse the same general selection code.
5	The rank screen is rendered when the rank button is confirmed.	Only if ENTER is pressed and the server is opened.	It's needed to print the best performers in each map.
6	The options screen is rendered when the buttons is confirmed.	Only if the ENTER is pressed.	It's needed to change the predefinitions of the configurations
7	The maps selection screen is rendered when the buttons is confirmed.	Only if the ENTER is pressed.	It allows users to select the map before the gameplay start.
8	The game is stopped if the quit button is confirmed.	Only If the ENTER is pressed.	It will give a stopping condition for the game.
Maps selection screen			
9	It allows to selected buttons in a column. It acts as a loop when the top or bottom arrive.	Only if the buttons are the ones used for the selection. It's defined in the options screen.	It's in a column to reuse the same general selection code.

10	The buttons have name of abbreviate the map.	Only an abbreviation of the name.	A way to show the user what map they're selecting.
11	User can return to the menu if ESC is pressed.	The buttons in the screen changes, but the background stays the same.	It connects the screen allowing a fluid game flow.
Rank screen			
12	The top 10 results in each map are printed in different tables.	The order is from the higher amount of life and lowest time. Only one result per user.	Usually persons rank the top 10, so I choose it as commonsense.
13	User can return to the menu if ESC is pressed.	The background changes and buttons appear.	It connects the screen allowing a fluid game flow.
Options screen			
14	The current preferences are printed.	They are printed in the correct places.	Shows the users the current preferences before they change it.
15	It allows to selected buttons in a column. It acts as a loop when the top or bottom arrive.	Only if the buttons are the ones used for the selection. It's defined in the options screen.	It's in a column to reuse the same general selection code.
16	The volume can be adjusted by using D to increase and A to decrease.	Only if the buttons are the ones defined for it as preference.	Reuse the button of left and right movement while make the action intuitive for the user.
17	The button preference can be changed by pressing SHIF + wanted key.	Only if the key is an alphanumeric character.	It distinguishes the key definition from random presses.
18	User can return to the menu if ESC 2is pressed.	The background changes and buttons appear.	It connects the screen allowing a fluid game flow.
Baker Avenue and Usbourne Way screens			
19	Player starts in the main door.	It will depend on the map.	It shows to the user where to go when finish the objective.

20	Player collisions with the walls are handled correctly.	The player can't pass through the wall or be teleported.	No teleports or player passing through walls are allowed.
21	Guard moving in loops.	It's constantly moving around something.	An AI algorithm would make the game too difficult because it's small.
22	Player loses one live when collide with the guard.	Even if it's below the inventory.	Don't allow user to attempt infinitely.
23	An item is appended to the inventory after searching.	Fridge – Lockpick. TV cabinet – UV light. Desk – Chip. Sink - Nothing	Extra layer of work before the minigame make it take longer without annoy the user.
24	A minigame starts if player interact with its sprite with the correct item.	Fingerprint – code door with the UV light. Lockpick – lock door with the lockpick Hacking – server with the chip.	User is forced to look for them, so the gameplay becomes more challenging.
25	A sprite is removed when the minigame is completed successfully.	Lockpick – remove the lock door. Fingerprint – remove the code door. Hacking – remove the laser.	New spaces can be explored exponentially, so the player can discover the map.
26	Player loses one live when collide with the guard or fail in the minigame.	Only if its lives are higher than 0.	It makes the game harder by adding obstacles.
27	Game over screen is rendered when lives become 0.	It will print no final time taken.	The stopping condition is needed to don't let the user play infinitely.
28	The time is updated in every gameplay related screen.	It includes the maps, minigames and pause screens.	Shows the users their performance so far.
29	The statue is appended to the inventory when	The sprite is no longer printed after collision	It's simpler than creating more searchable places.

	the player collides with its sprite.	and only one statue is appended.	
30	The mission is successfully completed when player arrive in the main door with the statue.	The mission completed screen is rendered and the time taken is showed.	The game must finish in some way.
End game screens			
31	The menu screen is rendered when the ENTER is pressed.	The screen shows the user what to do.	Allow user stay as longer as they want.

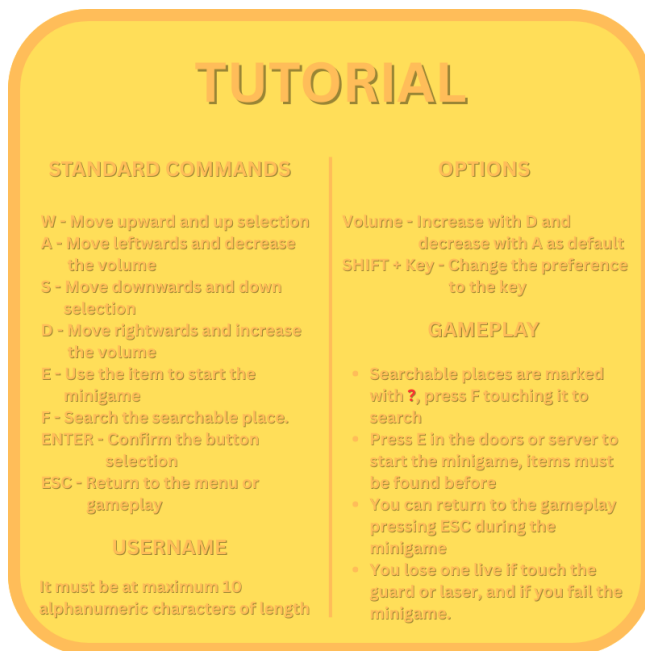
BETA TESTING

The stage in that stakeholders will have access to the game. They will play and give the feedback about the game, it's essential to find bugs and improve the features to make the game attractive. They should install the VS Code and the Pygame to be able to play the game, they don't need Flask because it is used in the server not in user machine. There are some files to be sent.

The first type of file will be the .py, it stores all Python code with containing the actual game logic. It will use Python version 3.12.0 to support features such as the match case and the `__main__` variable.

The other kind of file will be the .png, it will store the game screens and should be distributed with the code for Pygame be able to render it. Another file related with audiovisual effects is the .mp3 that will be files found on the Internet to use a music and effects without copyright.

The last type is a file .db, it's used to store the tables of the SQLITE. It's in binary, so if user can't use SQLITE they won't be able to access the tables. Even if they change the table, they won't be able to change their score on server, just locally. The following image will be sent as tutorial:



PROJECT MANAGEMENT

I should divide the design section in parts for the development; it will turn the development steps easier and more organized.

To manage the files, I will use a VS Code environment in GitHub, it will allow me to access the code anywhere by accessing VS Code remotely. Another tool that will help me is the Git, it allows to save project versions, so I could access the past versions of the code with I think they're better than the current one.

Furthermore, put my code on web can benefit me to show the code for public, it can attract more users and support in feedback integration. I don't need to care about wide the code because it will be open source.

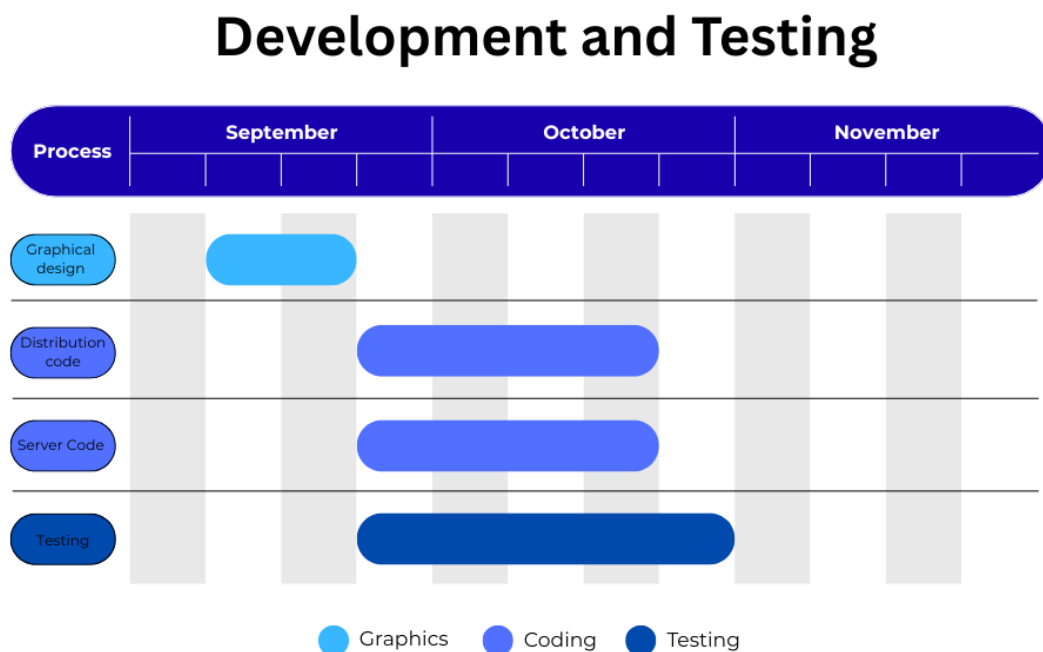
Here is the development plan:

Number	Design part	Status
Distribution code		
1	main() function	Not completed
2	game_loop() function	Not completed
3	input() function	Not completed
4	System class	Not completed
5	Button class	Not completed
6	Preferences class	Not completed

7	Gameplay class	Not completed
8	Minigame class	Not completed
9	Hacking class	Not completed
10	Fingerprint class	Not completed
11	Lockpick class	Not completed
12	Sprites class	Not completed
13	Player class	Not completed
14	Guards class	Not completed
15	Local database	Not completed
Server code		
16	new_user()	Not completed
17	set_score()	Not completed
18	best_score()	Not completed
19	Complete server database	Not completed
20	Put the code in some free Internet server	Not completed
Graphic design		
21	Account screen	Not completed
22	Menu screen	Not completed
23	Rank screen	Not completed
24	Option screen	Not completed
25	Start screen	Not completed
26	Map 1 gameplay screen	Not completed
27	Map 2 gameplay screen	Not completed
28	Lockpick screen	Not completed
29	Hacking screen	Not completed
30	Fingerprint screen	Not completed
31	Pause screen	Not completed
32	Mission completed screen	Not completed
33	Game over screen	Not completed
Testing		
34	White box testing	Not completed
35	Black box testing	Not completed
36	Alpha testing	Not completed

37	Beta testing	Not completed
----	--------------	---------------

I should thing in a way to divide efficiently the work, below I put a diagram showing the expected time for each part.



Now let's break out the weeks of each step. The diagram uses more weeks to handle possible lates in each part of development.,

Week 1	Username, Menu, Options, Rank and Map Screens	Graphics
Week 2	Minigame and End Game Screens	Graphics
Week 3	Game Initialization and First Screen	Coding/Testing
Week 4	Menu and Username Screen Input Handling	Coding/Testing
Week 5	Username Register, Rank and Options	Coding/Testing
Week 6	Player, Interactions, Statuses and End Game	Coding/Testing
Week 7	Minigames, Laser, Guard and Game Over	Coding/Testing
Week 8	Alpha and Beta Testing	Testing

There are some things that could late the project, during the coding I could use more time given that It's the first time using the Pygame tool. I can have problems to solve problems if I don't learn how to use the most important Pygame features. Another possible late is in stakeholders feedback, may be the stakeholders can use long times to give the feedback slowing down the process. Finally, in the resting the lates are more

present given that I can find bugs and change features, it would take long times depending on the problem to be solved.

ETHICAL, LEGAL AND ENVIRONMENTAL CONSIDERATIONS

ETHICAL CONSIDERATIONS

The main consideration is about the accessibility, it's a fundamental part to make every user happy with my game, I can't just think in my style of gameplay.

The first thing to consider is the accessibility for the deaf persons. The game will have audios just to increase the immersion, so it won't effect on difficult or playability of the game. I can add more signal for specific actions, such as an alarm symbol in the middle of the screen when the alarm starts, it shows to use what happen before teleport the player. Unfortunately, I would not be able to adapt the game for blind persons, as I said the audio don't affect much in the play and the graphics are essential, in a stealth game the adaptivity for blind is a big issue.

Another accessibility option is the adaption of the game for special controllers, these controlled support persons with paralysis to play the game. I can adapt the option inputs, in this way special controllers or even normal controllers can be used, in this way I can expand the game for other devices too.

There are no considerations about age, the game won't have anything that could be prejudicial for children. It will be a family friendly game.

LEGAL CONSIDERATIONS

Let's start with cautions about the Data Protection Act. It's important to use encryption to transfer the users' data from server to user machine and vice versa, man-in-the-middle attacks can access their data or implement malicious code inside the files to access their computers. I should constantly update their data; an up-to-date data is essential for the accuracy of their information. I should handle just when it needed, I can't use their data for another purpose beside the purpose in that it was collected. When the user no longer wants to use my game, all it's data should be removed from server. Finally, I should use hashing if I use some password method, it improves the security inside the database.

Other legal consideration is the Copyright, Designs and Patents Act, it ensures that any intellectual property won't be copied. I should consider it when I use the .mp3 files that

I will get in Google. It's crime to use any file with copyright in my game. I should see all terms and conditions of the resources used; some resources can't be used for commercial purpose.

ENVIRONMENTAL CONSIDERATIONS

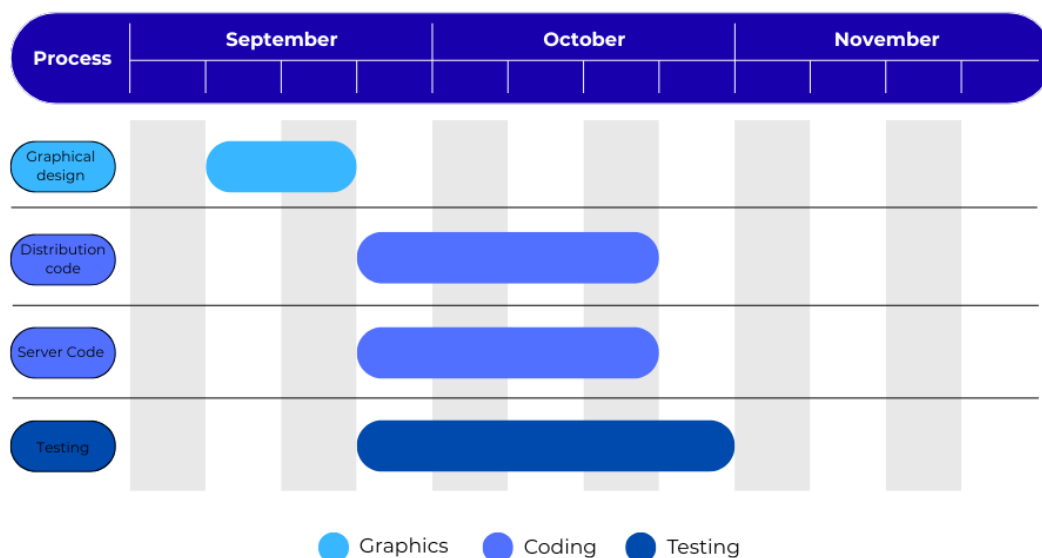
The game will use a server, so I should be informed about the environmental effect of it even if the server isn't mine. In the server website, I should look for the environmental precautions, it will show the initiative of the company. Server uses loads of energy, so look for places that uses solar panels is a good thing for example. It's essential to improve the company image in world.

DEVELOPMENT

TIMELINE

For the development I divided the work in three main blocks, the Graphical Development, Distribution Code Development and Server Code Development. This is linked to diagram showed in design session.

Development and Testing



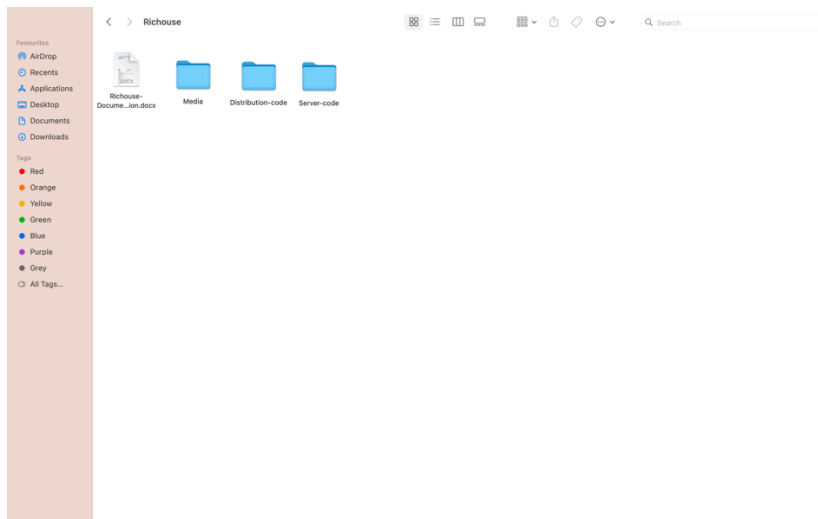
I decided that I going to start with Graphical Development because it will allow me to use the actual elements during code development. It will bring benefits in creation of hit boxes, using the actual shape I can adapt it better. Furthermore, I can adapt the code in

case it doesn't work well with some graphical element. Graphical Designs are essential to attract users, so early feedback from stakeholder is crucial as well.

After I develop the distribution code, it will allow me finish game on time because this code has a higher priority than server code. The priority is given because it's vital role in game functionality, if I don't finish server code, I can just delete some functionalities from the game while it's being developed. The functionalities from server code can be added after, on time case I have enough time or in future updates.

FILES

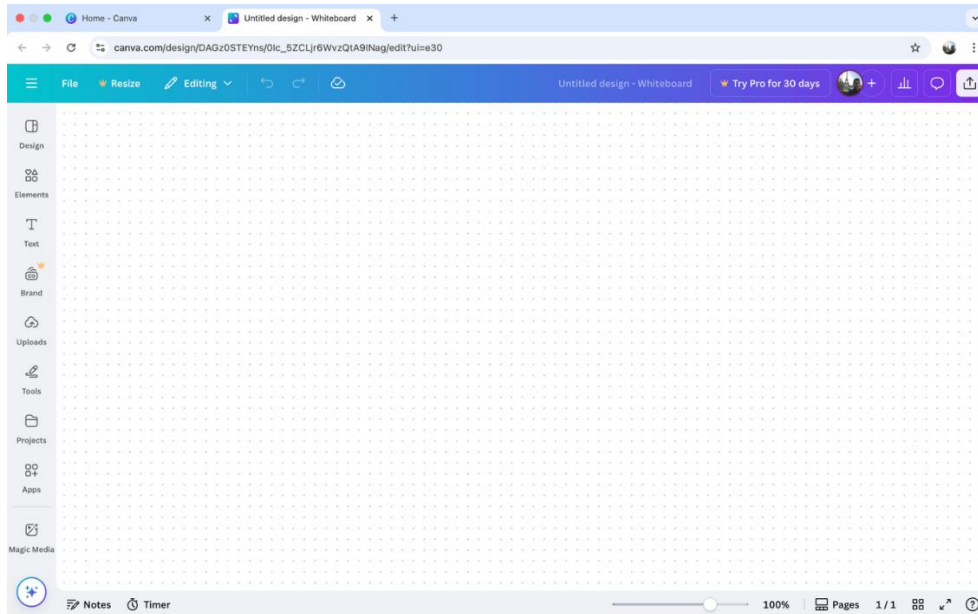
The last thing before start is the place where all the development will be store in computer. I will divide it in a file called "Richouse" that will store everything related to the project.



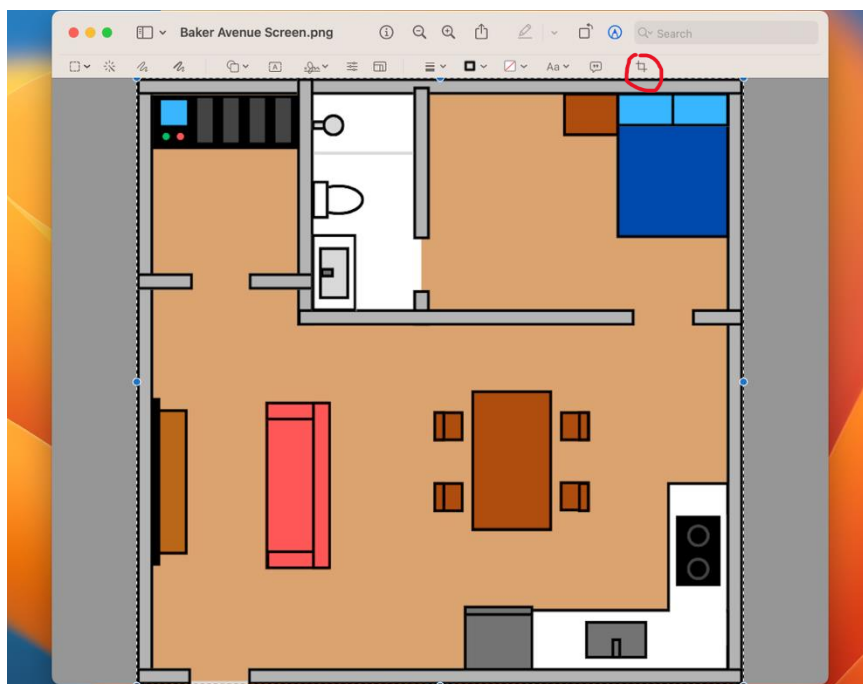
It has three folders separated in Media, it includes the screens as well as the sprites; Distribution Code, all Python code used to develop the actual game; and Server Code, it's the code that users won't be able to see. I saved a copy of my writeup as well.

GRAPHICAL DEVELOPMENT

The first thing to be decided is the graphical editor used to develop the elements. The tool chose was the Canva because it's a free, reduce costs, can be accessed on browser, enable me to develop in places where I'm using another computer instead of my main laptop, and it has a variety of elements to be used, so it will be enough for my simple design.



It has everything needed for the design, as show in left bar of image. However, I found a problem to download in the actual game screen size, so I used a build-in feature of Apple image file management system.



In the crop button, circled in red, I can crop the selected area, so I can crop to fit in 800x800, the screen size.

It's everything used in design session, so now I can start to develop the game graphical items. It will be saved in images folder inside game folder as showed before.

WEEK 1 - USERNAME, MENU, OPTIONS, RANK AND MAP SCREENS

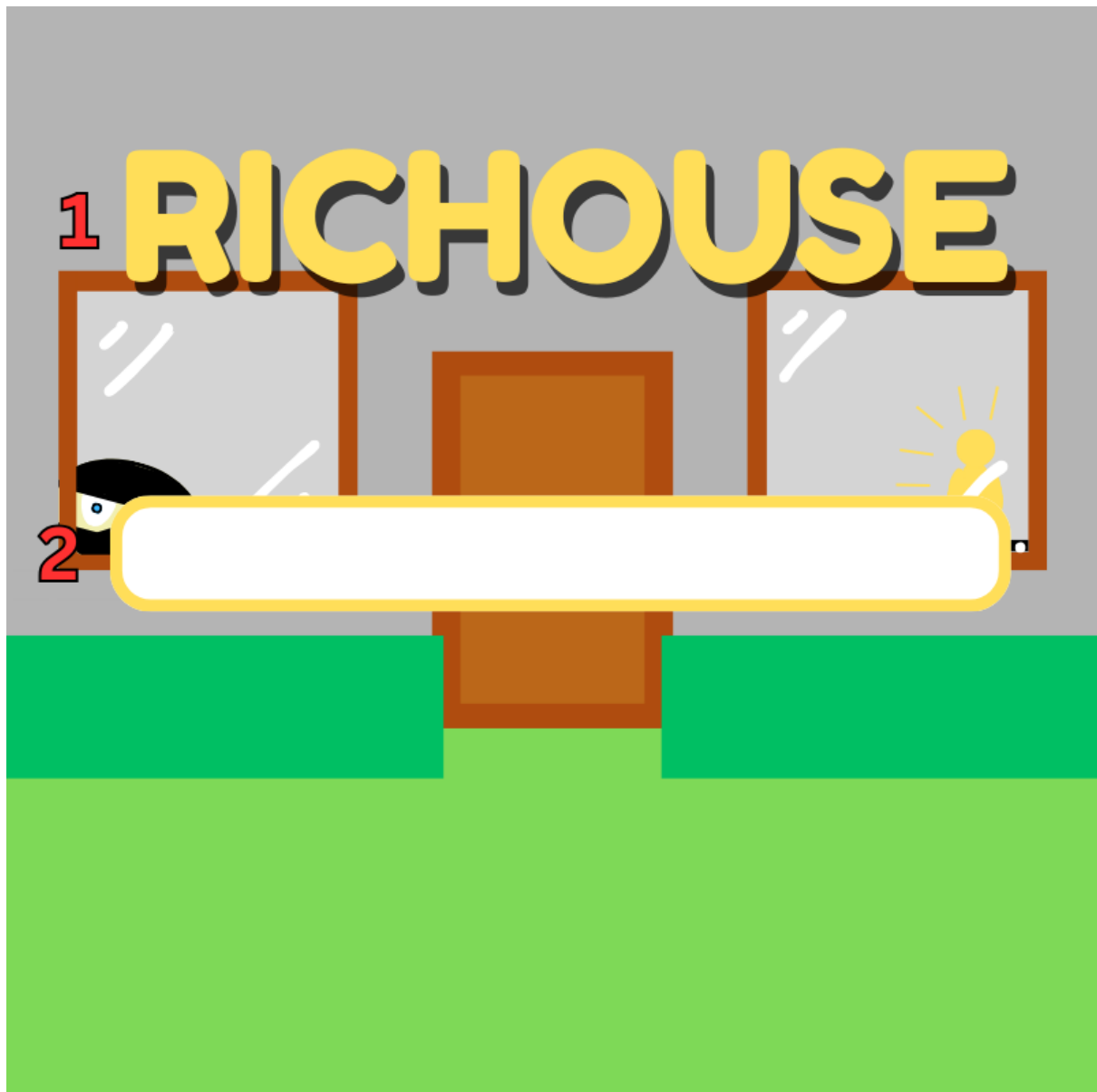
- **Objective:** Complete the design of the user registration, menu, options, rank and map screens and elements.
- **Success Criteria:**
 - **1 – Design of the menu screen is finished**
 - **7 – Design of the rank screen is finished**
 - **10 - Design of the options screen is finished**
 - **28 – Design of the gameplay includes a statuses bar**
 - **29 – Statuses bar includes a place for the lives**
 - **30 – Statuses bar includes a place for the time taken**
 - **33 – Cartoon geometric graphics were used**
- **Directories and files:**
 - **Media**
 - Username-screen.png
 - Menu-screen.png
 - Start-button.png
 - Rank-button.png
 - Options-button.png
 - Quit-button.png
 - Options-screen.png
 - Rank-screen.png
 - Baker-avenue-screen.png
 - Usbourne-way-screen.png
 - Main-character.png
 - Guard-sprite.png
 - Lock-door.png
 - Code-door.png
 - Golden-statue.png
- **Progress Management:**
 - **21 – Completed**
 - **22 – Completed**
 - **23 – Completed**
 - **24 – Completed**
 - **25 – Completed**
 - **26 - Completed**
 - **27 - Completed**
- **Videos:** N/A

- **Tests Results:** N/A
- **Stakeholders' Approval:** **PASS**

INTERACTION 1.1 – USERNAME SCREEN

- **Objective:** Complete the design of the user registration screen.
- **Success Criteria:**
 - **33 – Cartoon geometric graphics were used**
 -
- **Directories and files:**
 - **Media**
 - Username-screen.png
- **Videos:** N/A
- **Tests Results:** N/A
- **Stakeholders:** Add elements to facilitate the navigation

SCREEN DESIGN



The next screen is the first screen that you see in game, used to input your username completing the register. Afterwards, it will no longer be rendered.

1. The game title, it shows user that they are in the correct software.
 - Yellow, colour connected to gold and money
 - Shadow to contrast with background
2. Bar to input the username.
 - White, to highlight the text.
 - Circular edges to make the box less geometric
 - Yellow border to connect with game vibe of gold.
3. The other elements are from menu screen, which will be explained in the next interaction.

CHANGES FROM DESIGN

1. The screen had not existed until now, it's a development solution for username input, it should be filled only once.

FEEDBACK

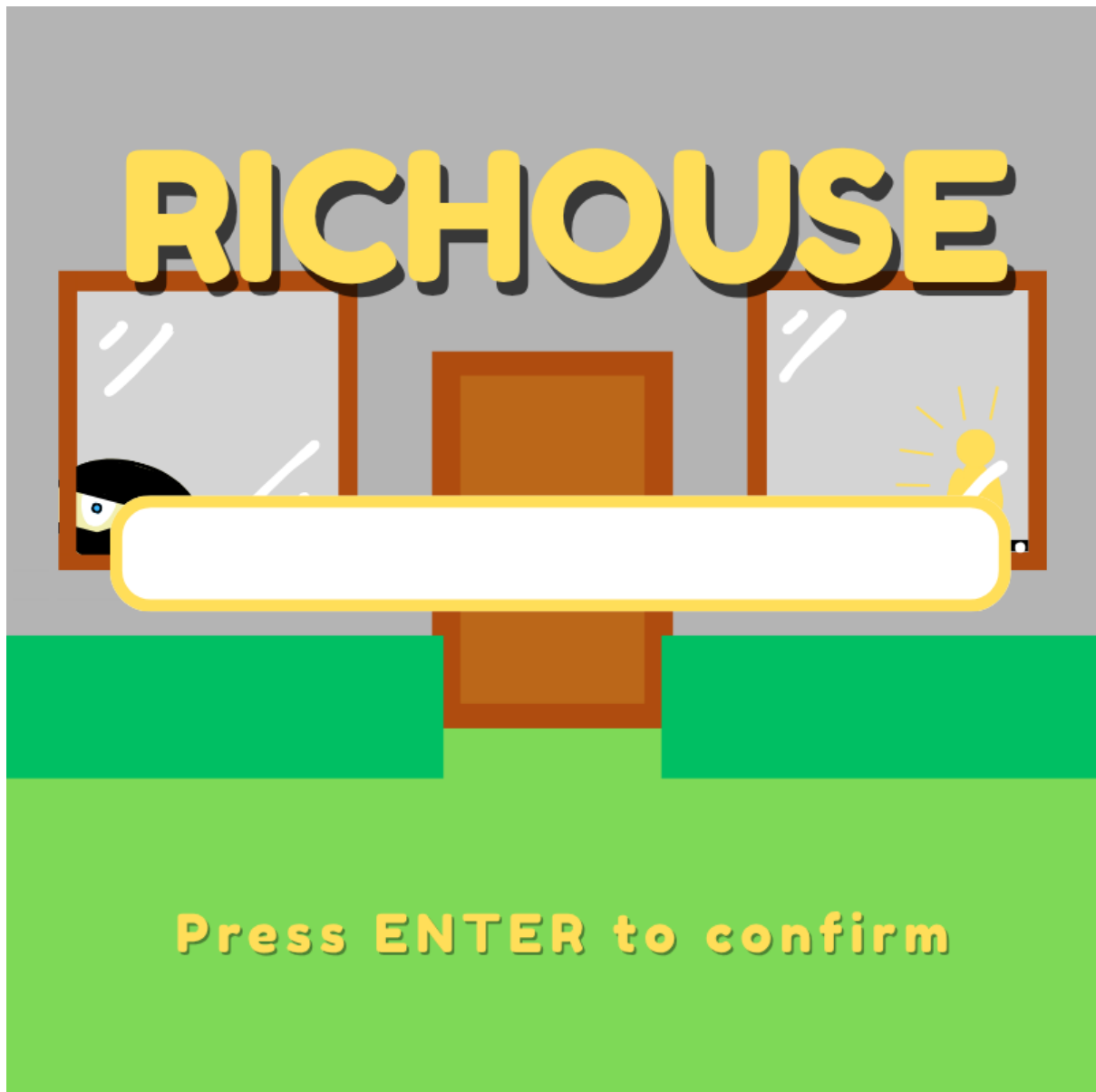
- Enzo Garofani (translated from Portuguese):

I believe there is a lack of visual elements to instruct the user about the features.

INTERACTION 1.2 – USERNAME SCREEN

- **Objective:** Complete the design of the user registration screen.
- **Success Criteria:**
 - **33 – Cartoon geometric graphics were used**
- **Directories and files:**
 - **Media**
 - Username-screen.png
- **Videos:** N/A
- **Tests Results:** N/A
- **Stakeholders:** **Approved**

SCREEN DESIGN



The screen is the same but adding a feature to facilitate the navigation.

1. Message below the text bar to teach user how to user it.
 - Now they can be sure about what to press when they choose the name.

INTERACTION 2 – MENU SCREEN

- **Objective:** Complete the design of the menu screen and buttons.
- **Success Criteria:**
 - 1 – Design of the menu screen is finished
 - 33 – Cartoon geometric graphics were used
- **Directories and files:**

- **Media**
 - Menu-screen.png
 - Start-button.png
 - Rank-button.png
 - Options-button.png
 - Quit-button.png

➤ **Videos:** N/A

➤ **Tests Results:** N/A

➤ **Stakeholders:** **Approved**

SCREEN DESIGN



1. A big title showing the game name.
 - Yellow, it related to gold that is something that you directly link to money.
 - Big font and centralized, attract user look to the game name, it will make them record its name.
 - Shadow, it gives a contrast between title and the other elements making it clearer.
2. The main character inside the house.
 - He wears black mask, it gives a felling of hidden and mysterious, perfect for a stealth game.
 - Greater part of body isn't showed, it emphasizes the hidden and stealth vibe of the game.
 - Opposite side of statue, showing that player need to play to conquest it.
3. Golden statue that is the game main objective.

- Yellow with black base, I used the Oscar as reference for the statue because it's extremely famous worldwide.
- Yellow lines to provide an effect of brightness.

BUTTONS DESIGN

START
RANK
OPTIONS
QUIT

They will be the responsible to access most of the other screens.

1. Yellow connecting to gold.
2. Less contrast and smaller than title to highlight the Title, making it clear
3. The start button opens the selection screen.
4. The rank button opens the ranking screen.
5. The options button opens the options screen.
6. The quit button stops the program.

CHANGES FROM DESIGN

1. The title is now entirely uppercase; it will highlight the title more.
2. The Selected button will have different size instead of a border around it.

INTERACTION 3 – RANKING SCREEN

- **Objective:** Complete the design of the ranking screen.
- **Success Criteria:**

- 7 – Design of the rank screen is finished
- 33 – Cartoon geometric graphics were used
- Directories and files:
 - Media
 - Rank-screen.png
- Videos: N/A
- Tests Results: N/A
- Stakeholders: Approved

SCREEN DESIGN



1. Name of the maps, it identifies each map ranking.
 - Use same font of game's title to standardize the texts.
 - Yellow still being used to connect with gold and money.
 - Shadow to create a contrast between text and background.
2. Table showing the top 10 scores of each map.
 - Yellow as main colour to represent gold and money.
 - Orange edges to make a clear separation between table and background.
 - Numeration from 1 to 10.
3. The symbol that shows the scorer position in ranking.
 - Orange to highlight the text.
 - Use a \$ symbol instead of #, that is the standard for rankings, to connect with money idea provided by game.
4. ESC is present in the top right to show how to return to the menu.
 - A home symbol is used to represent the menu/home screen.

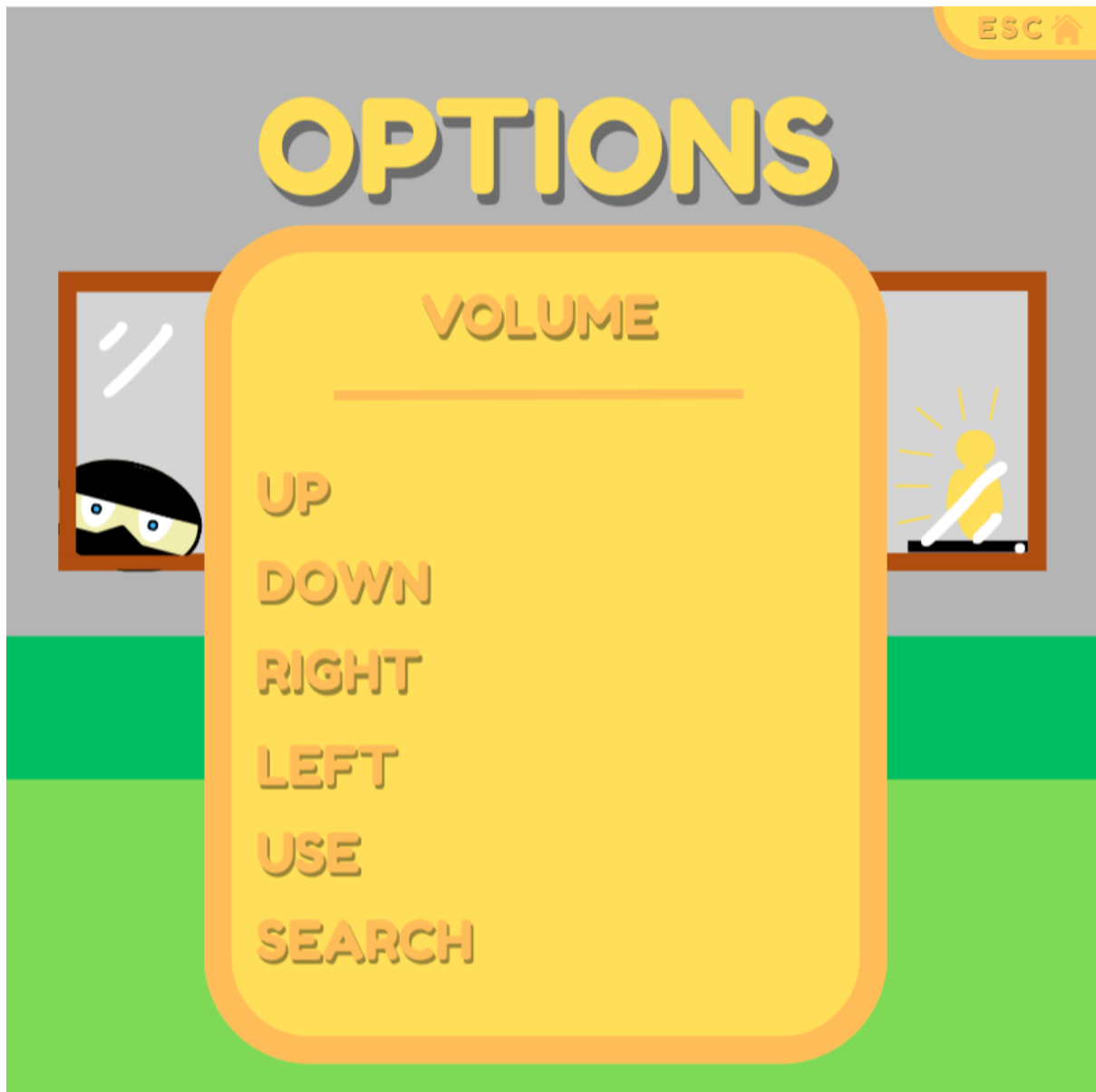
CHANGES FROM DESIGN

1. The maps now have names, Usbourne Way and Baker Avenue, they're randomly chosen.
2. The #s in ranking positions were substituted by \$, it makes an aesthetic of money and richness.

INTERACTION 4 – OPTIONS SCREEN

- **Objective:** Complete the design of the options screen.
- **Success Criteria:**
 - **10 - Design of the options screen is finished**
 - **33 – Cartoon geometric graphics were used**
- **Directories and files:**
 - **Media**
 - Options-screen.png
- **Videos:** N/A
- **Tests Results:** N/A
- **Stakeholders:** **Approved**

SCREEN DESIGN



1. The title of the screen to show user that they are in options.
 - It uses the standard font chose in the game, yellow connect to gold.
 - Shadow to contrast with background.
2. Volume preference selection.
 - Uses an orange text to identify the preference, highlight the text.
 - A bar to select the volume, it is the standard for much software.
 - The line showing the currently volume will be draw by Pygame because it's dynamic.
3. Selection for action buttons, it will allow user to change the buttons of movement and interactions.
 - Orange text to highlight from background, it shows which action is being selected.

- In the left to move the text away from actual button selected for that action, it will be displayed in right by Pygame, because it's dynamic, so we can see it clearly.
- 4. ESC is present in the top right to show how to return to the menu.
 - A home symbol is used to represent the menu/home screen.

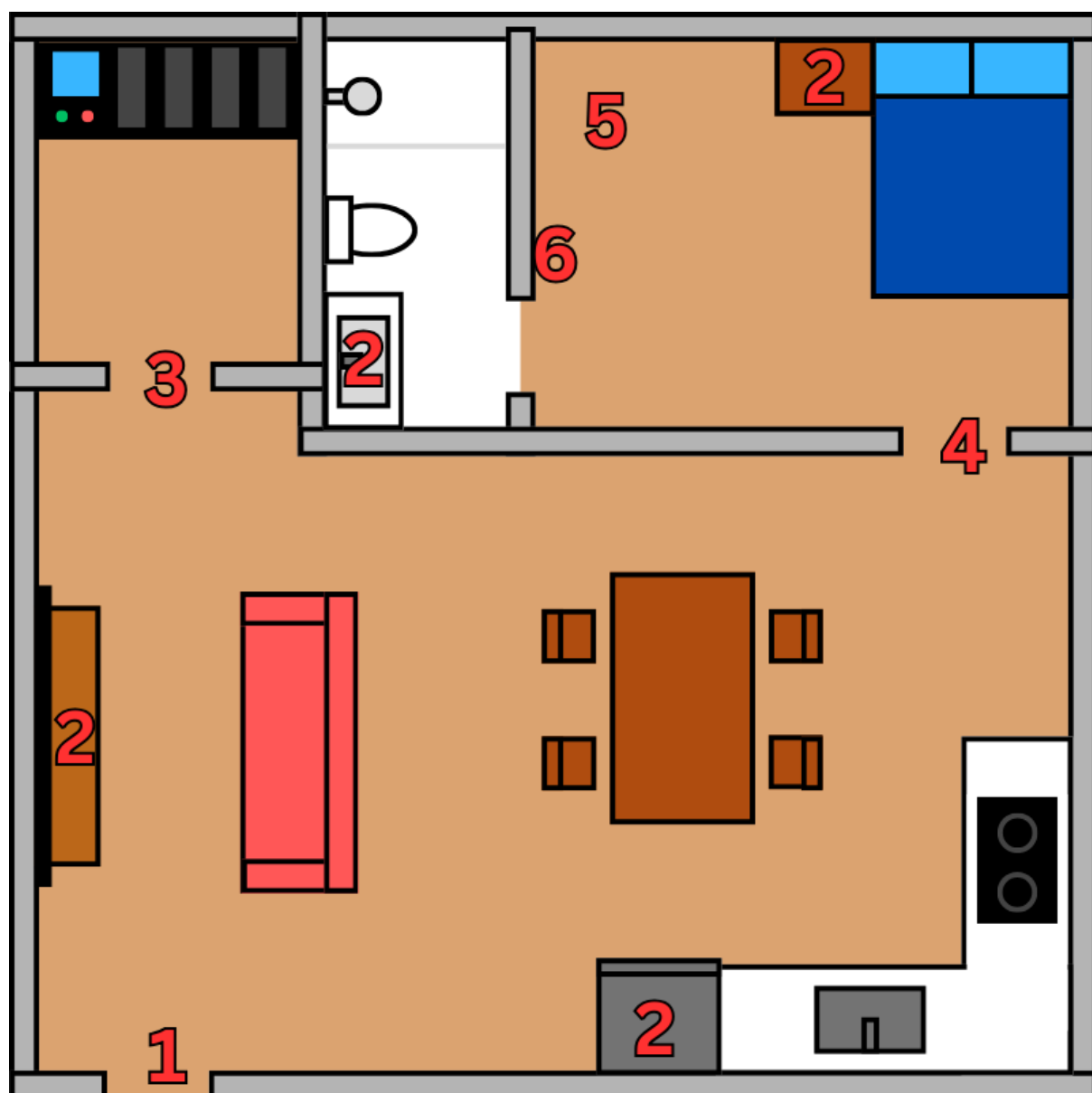
CHANGES FROM DESIGN

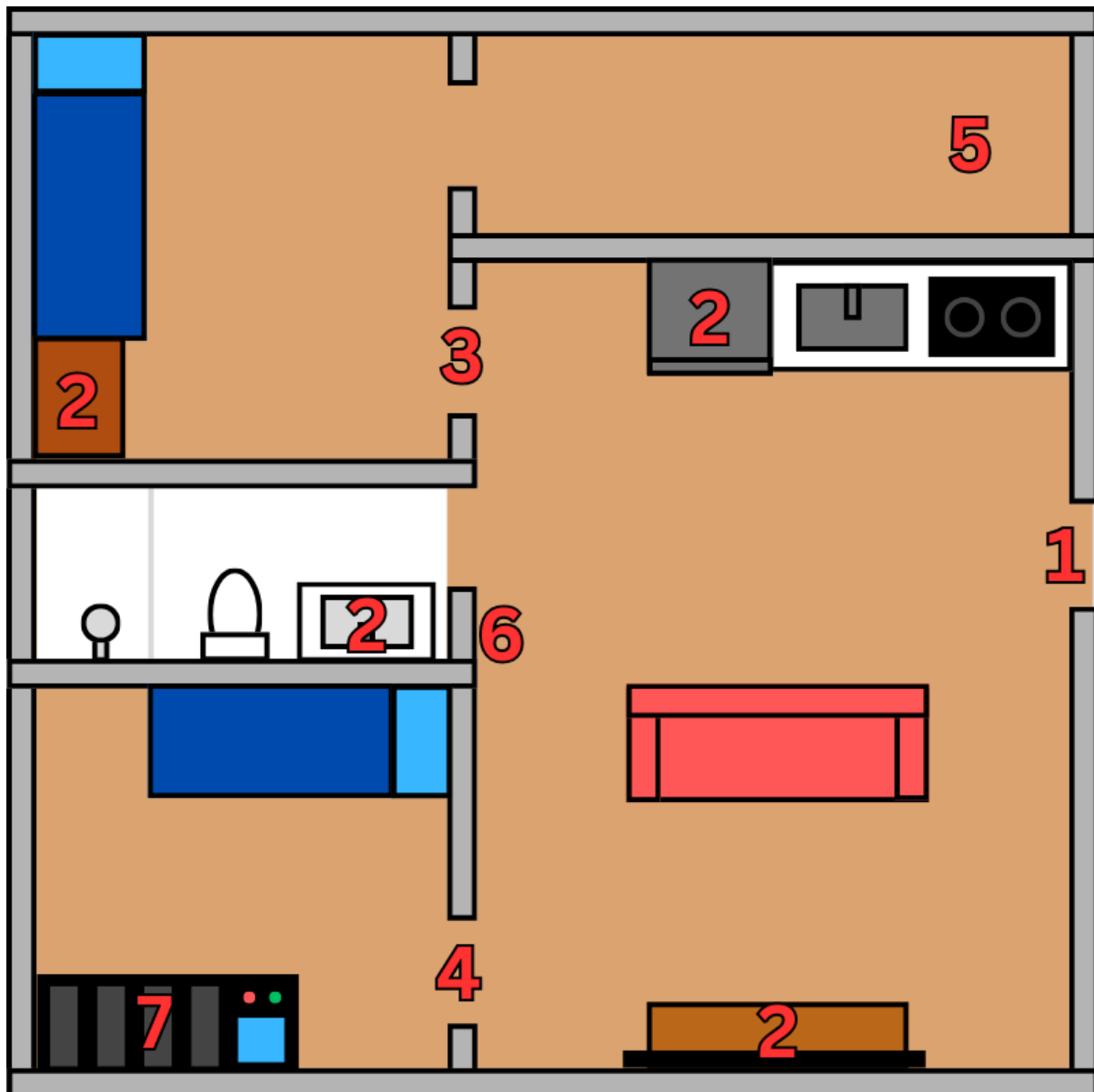
1. Volume bar was centralized, and the word "volume" is now at top of bar, it differentiates the volume from the others.
2. The options were rounded by a frame, and the background is the same from menu, its standards the features accessed through menu.

INTERACTION 5.1 – MAP SCREENS

- **Objective:** Complete the design of the user map screens and sprites.
- **Success Criteria:**
 - **33 – Cartoon geometric graphics were used**
- **Directories and files:**
 - **Media**
 - Baker-avenue-screen.png
 - Usbourne-way-screen.png
 - Main-character.png
 - Guard-sprite.png
 - Lock-door.png
 - Code-door.png
 - Golden-statue.png
- **Videos:** N/A
- **Tests Results:** N/A
- **Stakeholders:** Remove the cameras, add statuses bar, add button to pause and an inventory.

SCREENS DESIGN





1. It's the front door, where the player will start the mission.
 - It has the size of character width because its hitbox.
2. The searchable places, where you can get items.
 - The options are the TV cabinet, the fridge, the desk and bathroom cabinet because they're places where you can hide objects.
 - Distributed in map to make harder to find the items.
3. Location of a door with a lock.
 - It will be blocked by a sprite printed by Pygame, because it's dynamic given that disappear when lockpick minigame is completed.
4. Location of a door with code.
 - It will be blocked by a sprite printed by Pygame, because it's dynamic given that disappear when fingerprint minigame is completed.
5. Location of golden statue and lasers blocking it.

- The statue sprite is printed by Pygame, because it's dynamic, it disappears when the player collects it.
- Laser is printed by Pygame, because it's dynamic, from one wall to another, and disappears when hacking minigame is completed.
- 6. Location of camera, it changes its looking to one side to another.
 - It's printed by Pygame because it's dynamic and change the place where it's looking. It stops to work after hacking minigame be completed.
- 7. The sever where the hacking minigame can be initiated.
 - Multiple grey blocks representing server components.
 - Blue block representing a graphical interface.
 - Red and green circles representing power buttons.

SPRITES DESIGN



1. Main character uses mask to hide the face, directly link with stealth.
2. Main character wear black, it gives a feeling of hidden.
3. Guard uses a hat like a police officer in USA, multiple movies show it so everybody link it to law.
4. Guard wears blues and has moustache, it's again because of USA stereotype of police.



1. Camera is grey because it's the most common color of big security cameras.
2. Door with lock has a key to show that it has the lockpick minigame.
3. Door with code lock has a code to show that it has the fingerprint minigame.



1. It yellow to link with gold

2. It has a shape like Oscar award because many people are familiar with its reputation and status.

CHANGES FROM DESIGN

1. Creation of a username register screen to allow users to register their usernames, as a result it can appear in the ranking as an arcade. The register is one time only, so it can't be changed after.
2. Development of sprites to increment the gameplay, including buttons, minigame elements and the statue.
3. The maps selection screen is now the same of menu, but with different buttons. It makes simpler given that it's only used to start the gameplay.
4. As asked in stakeholders' feedback, I made maps with less walls and bigger areas to allow the player to move freely.
5. The volume bar is now centralized in the top of the options box, consequently the volume can be easily distinguished from button preferences.

FEEDBACK

➤ **Zach Ward:**

There are some elements missing in the gameplay, such as the menu button, the inventory and the statuses.

➤ **Alex Michalski:**

I'm still missing the presence of a structured statuses bar. Furthermore, I believe the map is too small to include both guard and camera.

INTERACTION 5.2 – MAP SCREEN

- **Objective:** Complete the design of the user map screens and sprites.
- **Success Criteria:**
 - **28 – Design of the gameplay includes a statuses bar**
 - **29 – Statuses bar includes a place for the lives**
 - **30 – Statuses bar includes a place for the time taken**
 - **33 – Cartoon geometric graphics were used**
- **Directories and files:**
 - **Media**

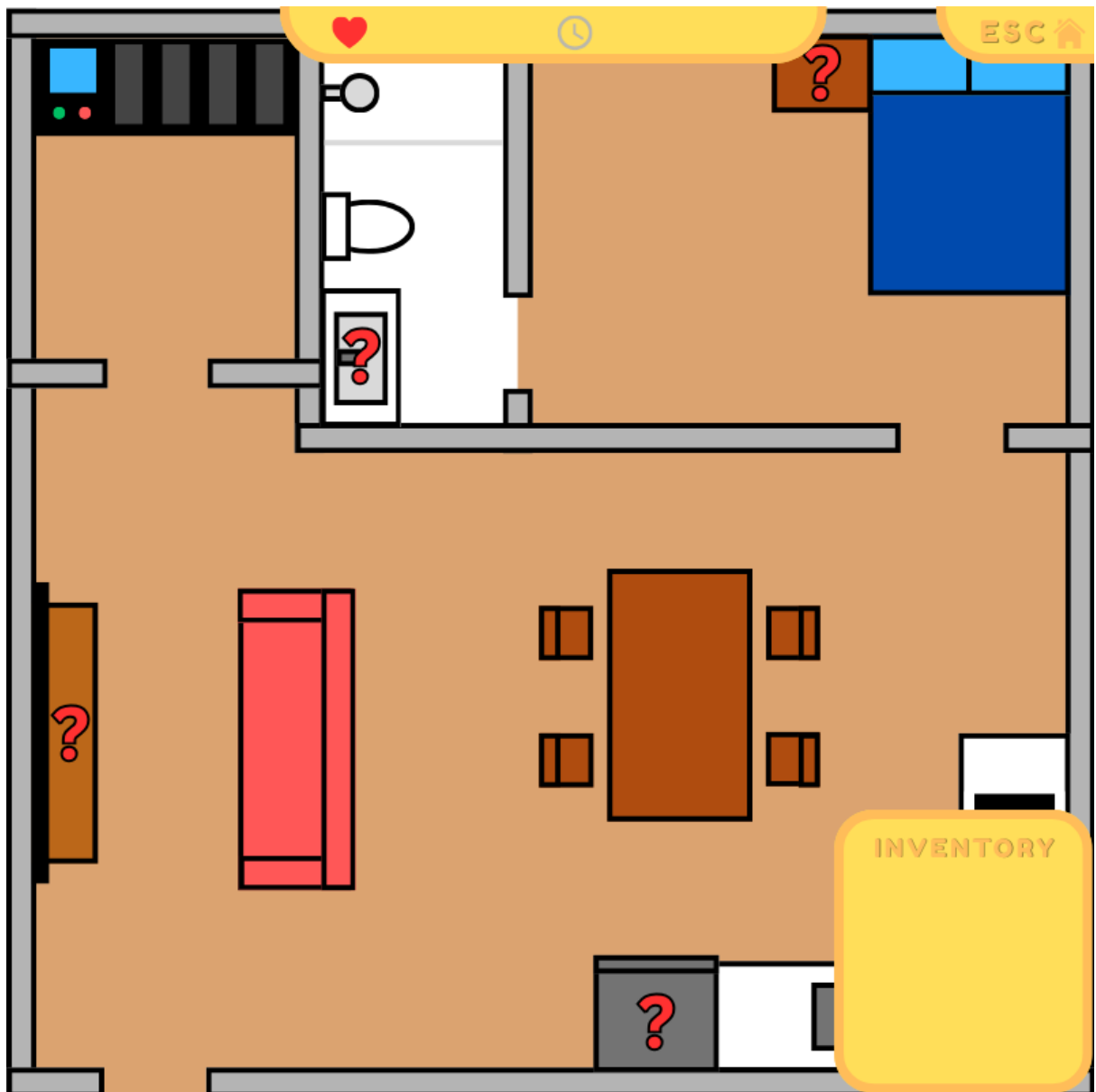
- Baker-avenue-screen.png
- Usbourne-way-screen.png
- Main-character.png
- Guard-sprite.png
- Lock-door.png
- Code-door.png
- Golden-statue.png

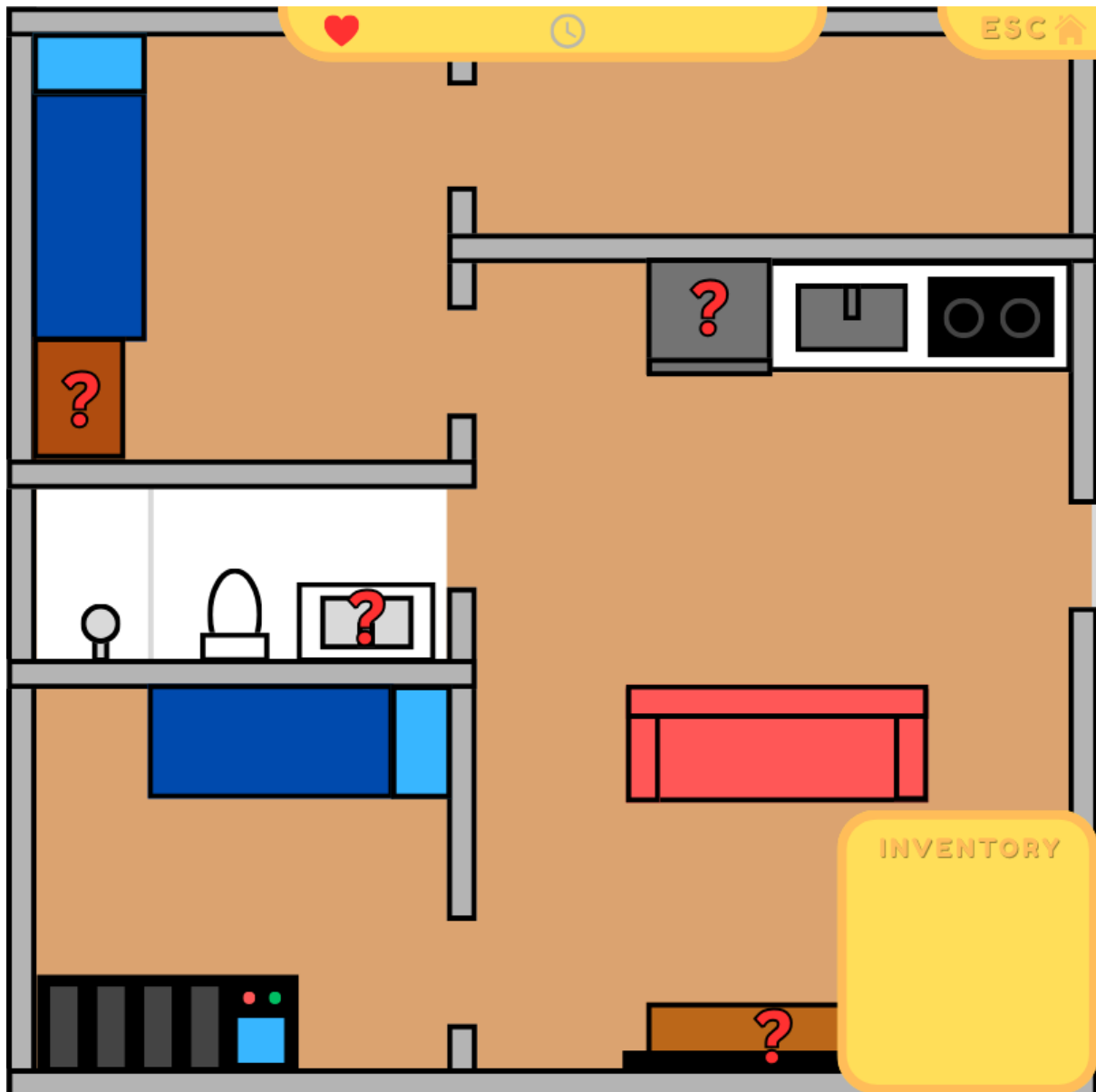
➤ **Videos:** N/A

➤ **Tests Results:** N/A

➤ **Stakeholders:** **Approved**

SCREENS DESIGN





1. Statuses bar in the top of the screen.
 - It will inform user about remaining lives and time taken without annoy them during the gameplay.
2. Inventory is printed in the bottom right.
 - Items in the player inventory are printed in a position that won't affect much the gameplay.
3. ESC is present in the top right to show how to return to the menu.
 - A home symbol is used to represent the menu/home screen.
4. Cameras won't be present in the map.
 - The game would be too polluted including both guard and camera, so the camera was discarded because it won't interact much with the user.

WEEK 2 - MINIGAME AND END GAME SCREENS

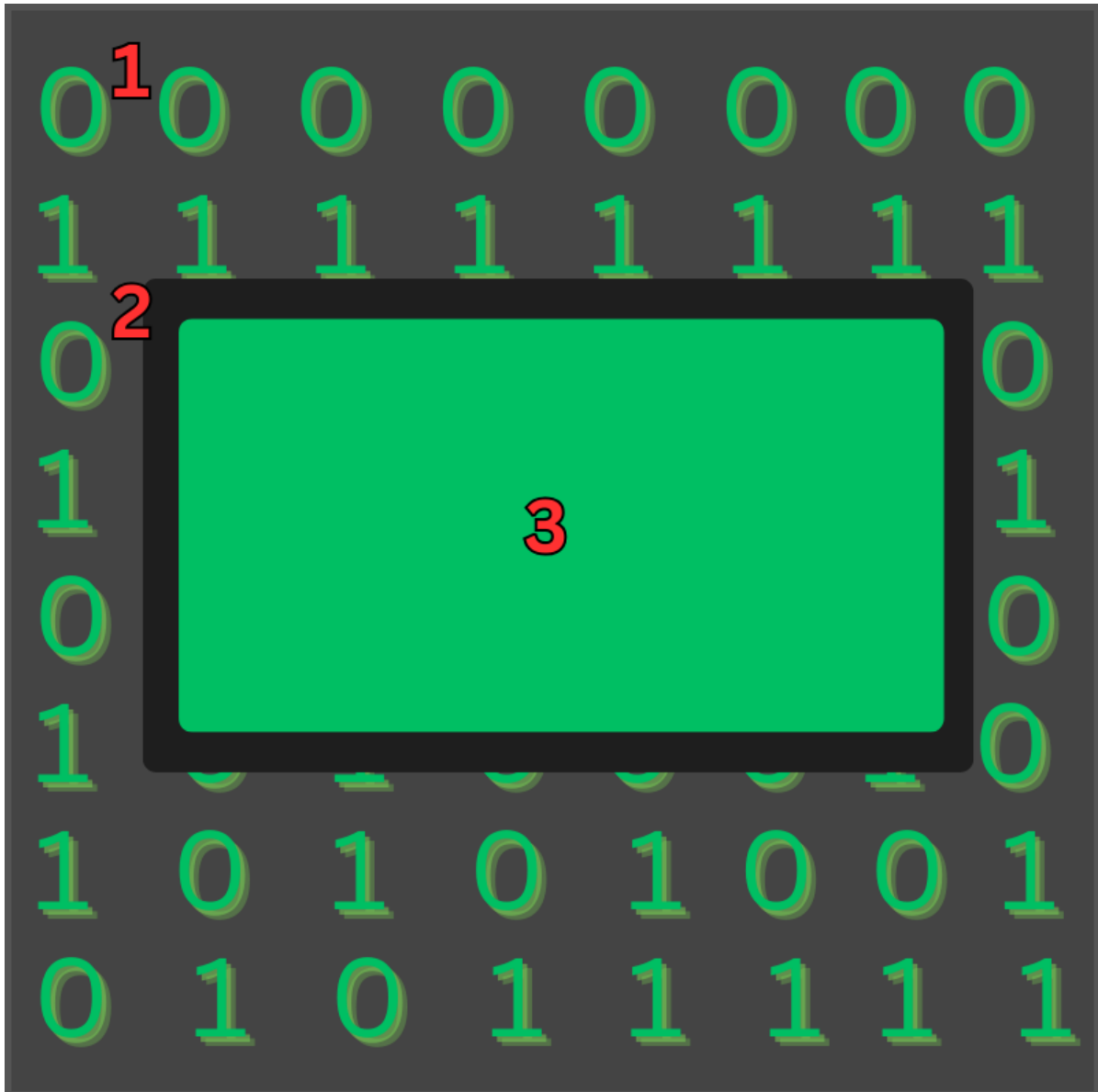
- **Objective:** Complete the design of the hacking, fingerprint and lockpick minigames; and the mission completed and game over screens and elements.
- **Success Criteria:**
 - **24 – The hacking minigame screen design is finished.**
 - **26 - The lockpick minigame screen design is finished.**
 - **27 - The fingerprint minigame screen design is finished.**
 - **32 – The gamer over and mission completed screen designs are finished.**
 - **33 – Cartoon geometric graphics were used**
- **Directories and files:**
 - **Media**
 - Hacking-screen.png
 - Lockpick-screen.png
 - Up-arrow.png
 - Down-arrow.png
 - Left-arrow.png
 - Right-arrow.png
 - Fingerprint-screen.png
 - Weak-finger.png
 - Mid-finger.png
 - Strong-finger.png
 - Pause-screen.png
 - Continue-button.png
 - Menu-pause.button.png
 - Mission-completed-screen.png
 - Game-over-screen.png
- **Progress Management:**
 - **28 - Completed**
 - **29 - Completed**
 - **30 - Completed**
 - **31 - Completed**
 - **32 - Completed**
 - **33 – Completed**
- **Videos:** N/A
- **Tests Results:** N/A

- **Stakeholders' Approval:** **PASS**

INTERACTION 1.1 – HACKING SCREEN

- **Objective:** Complete the design of the user hacking minigame screen.
- **Success Criteria:**
 - **24 – The hacking minigame screen design is finished.**
 - **33 – Cartoon geometric graphics were used**
- **Directories and files:**
 - **Media**
 - Hacking-screen.png
- **Videos:** N/A
- **Tests Results:** N/A
- **Stakeholders:** Add the statuses bar to the minigame screens.

SCREEN DESIGN



8-bit binary code.

- It gives the screen a hacker feeling, binary is connected to hacking in movies and series.
- Green with black background connect with some systems where the CLI use this design.

A tablet that is used to hack the server.

- Green screen to match with the binary numbers.
- Black borders to highlight where screen starts.

Location of the uncompleted word.

- The word is printed is printed by Pygame because it's dynamic, change each time.

FEEDBACK

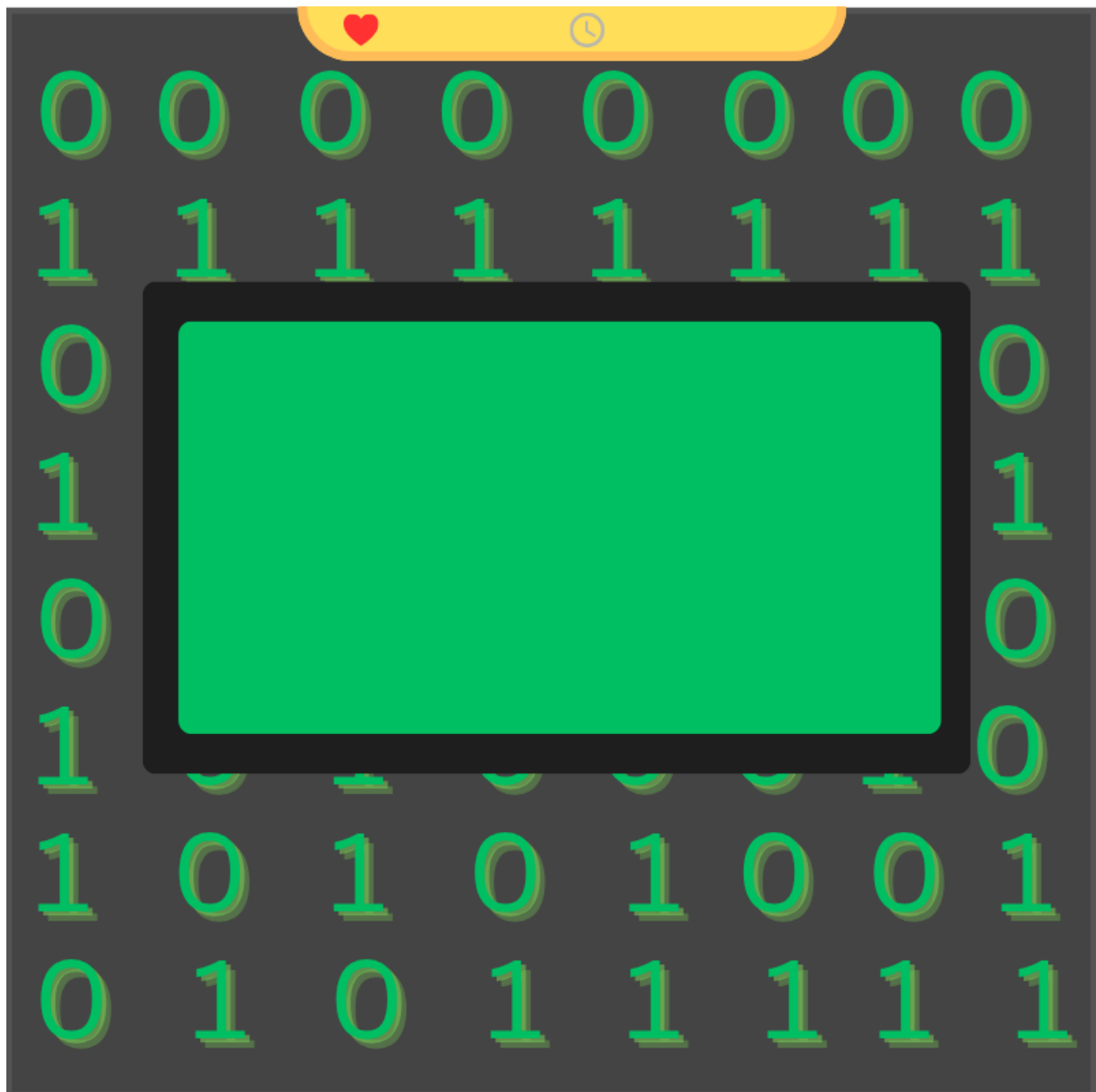
- Enzo Garofani (translated from Portuguese):

A statuses bar during the minigames would be better to identify the current performance.

INTERACTION 1.2 – HACKING SCREEN

- **Objective:** Complete the design of the hacking minigame screen.
- **Success Criteria:**
 - 24 – The hacking minigame screen design is finished.
 - 33 – Cartoon geometric graphics were used
 -
- **Directories and files:**
 - **Media**
 - Hacking-screen.png
- **Videos:** N/A
- **Tests Results:** N/A
- **Stakeholders:** Approved

SCREEN DESIGN



1. Statuses bar on top.
 - User can be informed about the performance without the bar annoy them during the minigame.

INTERACTION 2 – LOCKPICK SCREEN

- **Objective:** Complete the design of the lockpick minigame screen.
- **Success Criteria:**
 - **26 - The lockpick minigame screen design is finished.**
 - **33 – Cartoon geometric graphics were used**
- **Directories and files:**

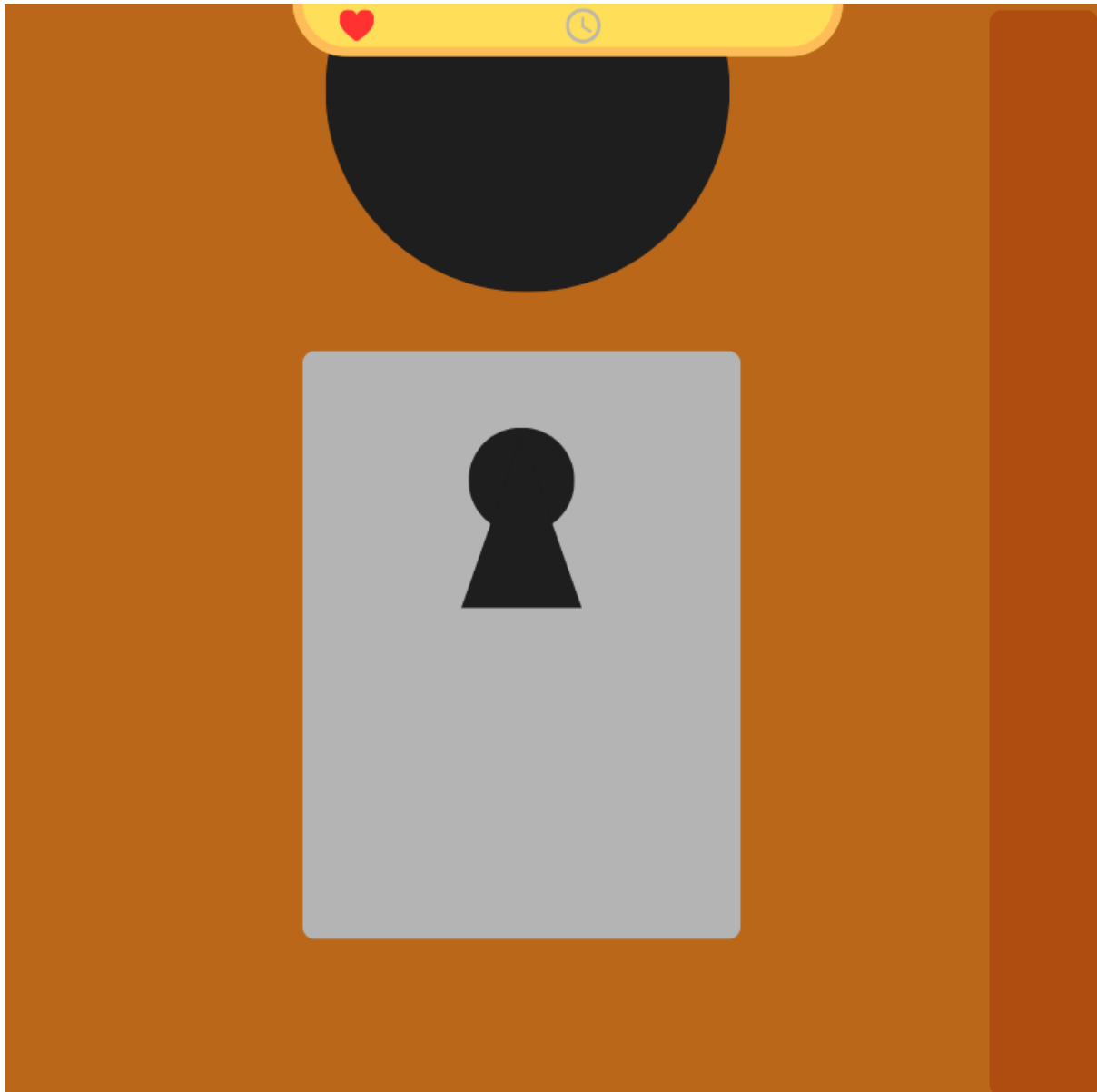
- **Media**
 - Lockpick-screen.png
 - Up-arrow.png
 - Down-arrow.png
 - Left-arrow.png
 - Right-arrow.png

➤ **Videos:** N/A

➤ **Tests Results:** N/A

➤ **Stakeholders:** **Approved**

SCREEN DESIGN



1. Circular door handle.
 - It's black to contrast with the door color.
2. It's the door frame.
 - The handle is close to it, so make more sense if it appears as well.
3. The lock.
 - Visual representation that door is blocked.
 - It's grey to represent metal.
 - The Circle and triangle together look like a lock hole.
4. The arrow will be printed below the lock inside the gray box.
 - Pygame will dynamically print the correct sequence.
5. Statuses bar on top.
 - User can be informed about the performance without the bar annoy them during the minigame.

ARROWS DESIGN



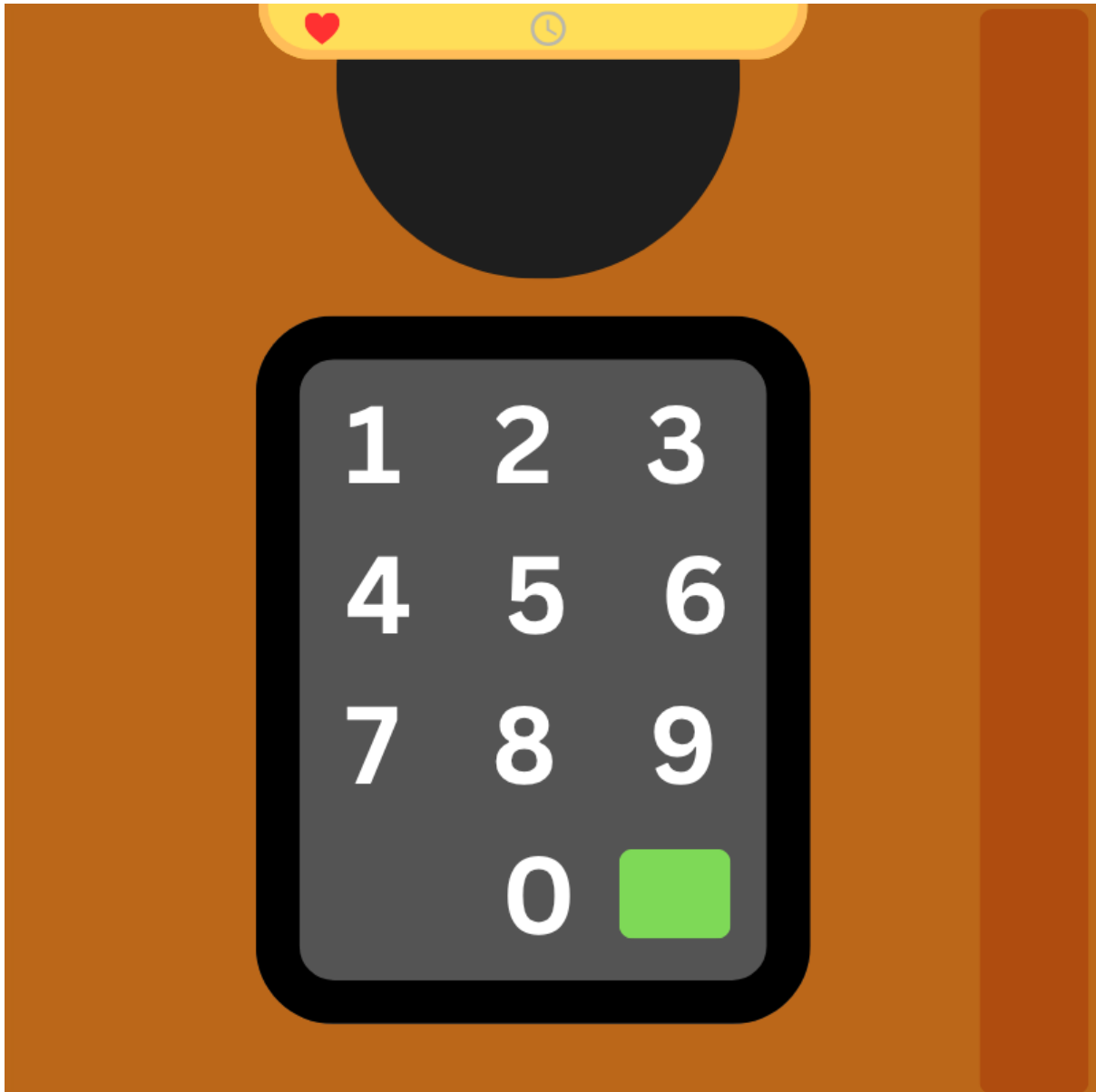
Arrows used to print the sequence.

1. Dark grey to contrast with light grey of the lock.
2. Not too wide to be clear for user.

INTERACTION 3 – FINGERPRINT SCREEN

- **Objective:** Complete the design of the fingerprint screen.
- **Success Criteria:**
 - **27 - The fingerprint minigame screen design is finished.**
 - **33 – Cartoon geometric graphics were used**
- **Directories and files:**
 - **Media**
 - Fingerprint-screen.png
 - Weak-finger.png
 - Mid-finger.png
 - Strong-finger.png
- **Videos:** N/A
- **Tests Results:** N/A
- **Stakeholders:** **Approved**

SCREEN DESIGN



1. Circular handle.
 - Show that it's a door
 - Black contrast with door colour
2. Door frame.
 - Show that it's close to the right end of the door.
 - Darker to highlight from door.
3. Code lock.
 - It's dark to match with handle
 - It has a black circular border; it contrasts with door and give an aesthetic of modern.
 - Numbers in white to highlight from dark grey.
 - Numbers grouped in order like a real code lock.
 - Green rectangle to represent the confirmation of code

- Pygame will print the fingerprint, because it's dynamic given that it changes each time.
- 4. Statuses bar on top.
 - User can be informed about the performance without the bar annoy them during the minigame.

FINGERPRINTS DESIGN



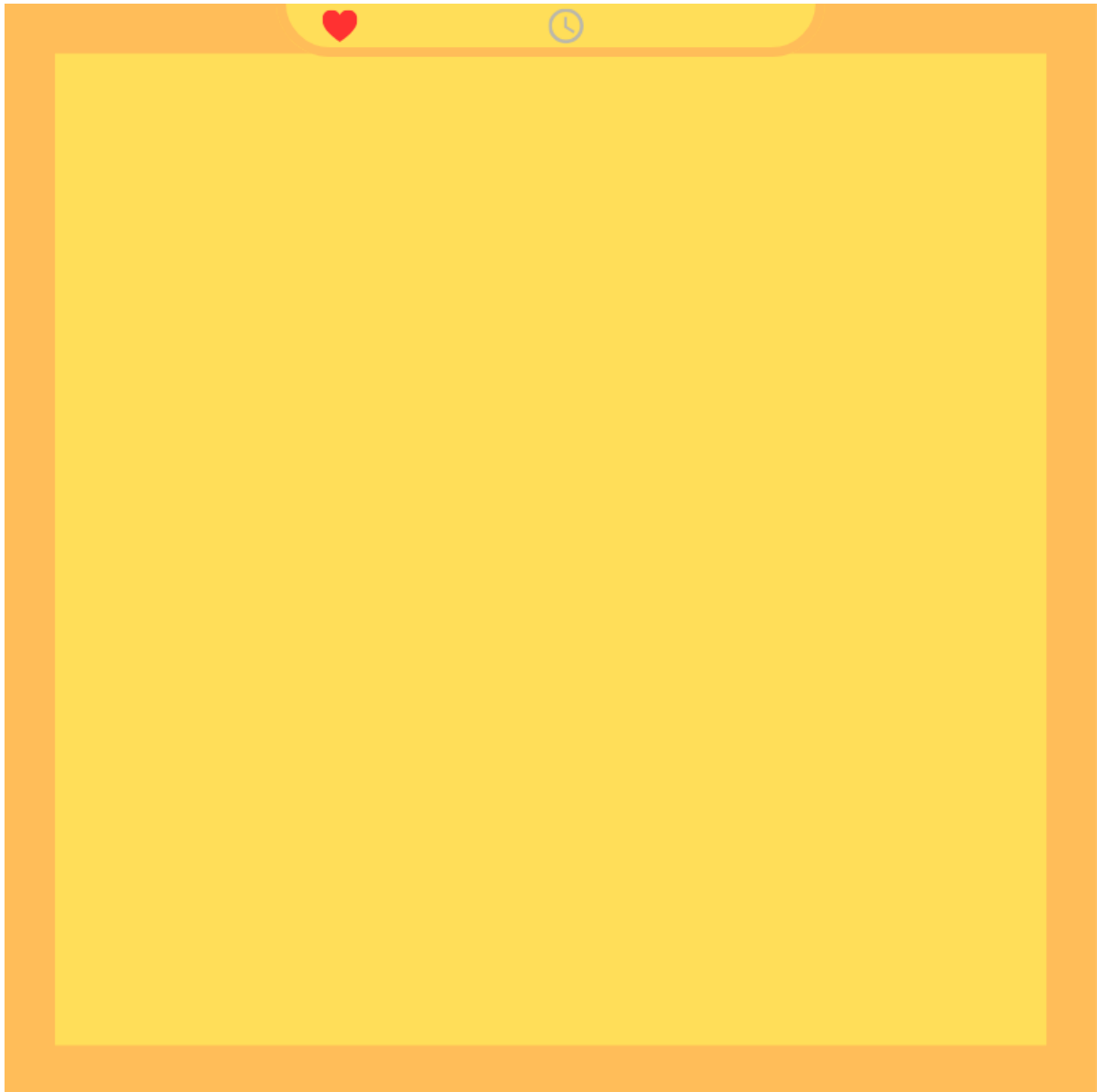
The circles used to represent the fingerprints.

1. Circular because it's a similar shape of a finger
2. Purple because we use UV light to identify the fingerprints.
3. Different intensities to show the correct sequence, the lighter first because it was pressed earlier than the darker.

INTERACTION 4 – PAUSE SCREEN

- **Objective:** Complete the design of the pause screen.
- **Success Criteria:**
 - **33 – Cartoon geometric graphics were used**
- **Directories and files:**
 - **Media**
 - Pause-screen.png
 - Continue-button.png
 - Menu-pause.button.png
- **Videos:** N/A
- **Tests Results:** N/A
- **Stakeholders:** **Approved**

SCREEN DESIGN



Screen created to allow users to return to menu screen before the mission be completed.

1. Screen border.
 - Gives depth to screen, it becomes less flat
2. Where the buttons will be displayed.
 - It's dynamic because the selected button change, so Pygame should print it.
3. Statuses bar on top.
 - User can be informed about the performance without the bar annoy them during the minigame.

BUTTONS DESIGN

CONTINUE

MENU

Buttons that can be printed by Pygame in pause screen.

1. Continue return to game screen
2. Menu return to menu scree
3. Orange to contrasts with pause screen background.
4. Shadow strength the contrast

INTERACTION 5.1 – MISSION COMPLETED SCREEN

- **Objective:** Complete the design of the mission completed screen.
- **Success Criteria:**
 - **32 – The gamer over and mission completed screen designs are finished.**
 - **33 – Cartoon geometric graphics were used**
- **Directories and files:**
 - **Media**
 - Mission-completed-screen.png
- **Videos:** N/A
- **Tests Results:** N/A
- **Stakeholders:** Add guidance to return to menu, show the final result.



1. Mission completed text.
 - In the centre of screen because it's the first place where user will look.
 - Yellow is linked to gold.
 - Shadow contrast with background, mainly with moon that is yellow as well.
2. Night background in a balcony perspective.
 - I got inspiration from "Grand Thief Auto" mission completed screen for balcony.
 - It's night because the game is stealth and night looks quiet.
Moon makes evident that is night.
3. Main character relaxing with money.

- Leaning on the balcony show that he is relaxing, inspiration from “Grand Thief Auto” mission completed screens.
- Money packet shows that mission was a success.

FEEDBACK

➤ **Zach Ward:**

Extra guidance would be ideal in the endgame screens, like including a text saying what to do.

➤ **Alex Michalski:**

The mission completed screen could show the result.

INTERACTION 5.2 – MISSION COMPLETED SCREEN

- **Objective:** Complete the design of the mission completed screen.
- **Success Criteria:**
 - **32 – The gamer over and mission completed screen designs are finished.**
 - **33 – Cartoon geometric graphics were used**
- **Directories and files:**
 - **Media**
 - Mission-completed-screen.png
- **Videos:** N/A
- **Tests Results:** N/A
- **Stakeholders:** **Approved**

SCREEN DESIGN



1. Result table in the left.
 - The final time taken is clearly printed for the user, but it hides some part of the screen.
2. Message in the bottom right for guidance.
 - It allows the user to remain in the screen as long as they want, so they just need to press ENTER later.

INTERACTION 6 – GAME OVER SCREEN

- **Objective:** Complete the design of the game over screen.
- **Success Criteria:**

- **32 – The gamer over and mission completed screen designs are finished.**
- **33 – Cartoon geometric graphics were used**

- **Directories and files:**
 - **Media**
 - Game-over-screen.png

- **Videos:** N/A

- **Tests Results:** N/A

- **Stakeholders:** **Approved**

SCREEN DESIGN



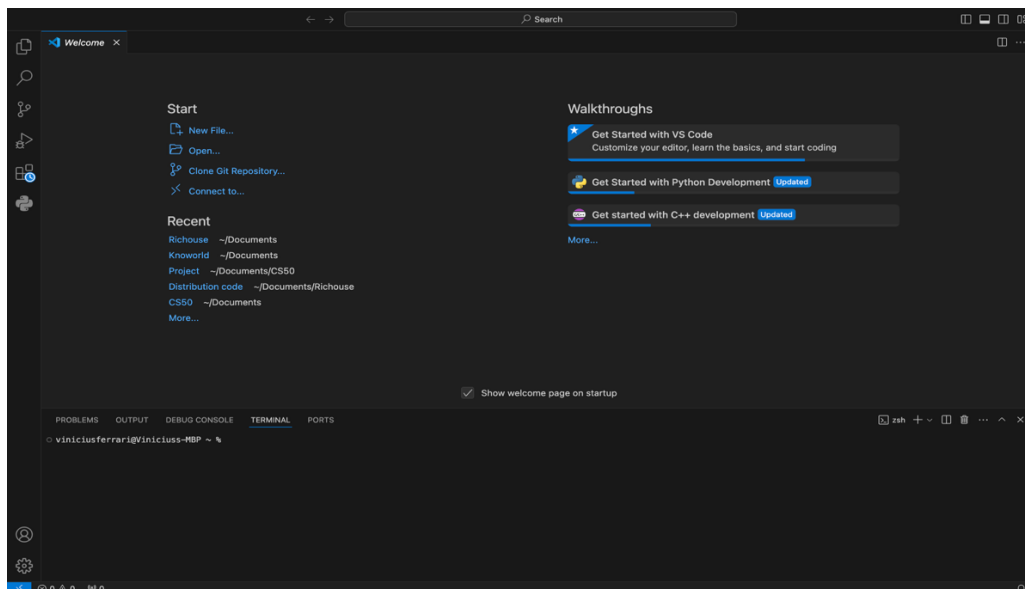
1. Screen title to identify the result of mission when you lose.
 - Yellow connects with gold
 - Shadow give contrast with background
2. The guard character.
 - It's outside the jail to show that he was the responsible to catch the robber.
 - Blue uniform to link with police uniform that is blue in USA, so multiple movies already show police as blue, and persons already have this perception.
 - Moustache make the face less flat, give a visual highlight.
3. The main character jailed.
 - Shows that user fail the mission resulting in their prison.
 - Behind jail show that he's locked.
4. Message in the centre for guidance.

- It allows the user to remain in the screen as long as they want, so they just need to press ENTER later.

CODE DEVELOPMENT AND TESTING

With the designs finished I can start to implement the code. I can use the actual images, so it's easier to visualize how it will look like and control the elements proportions.

I have decided to use Visual Studio Code to develop the codes used both for server and distribution code. It's the best option given that I can use it for free and it enable me to use multiple tools in the same place, I can install everything directly from its terminal and it provide isolated areas to store these external libraries, so it won't affect any other project using the IDE.



WEEK 3 – GAME INITIALIZATION AND FIRST SCREEN

- **Objective:** Create the code for screen and audio initialization, and develop the code to render the first screen, which are the register or menu screens depending on the game state.
- **Success Criteria:**
 - **1 – Menu screen becomes the first screen if username is in the local database**
- **Directories and files:**

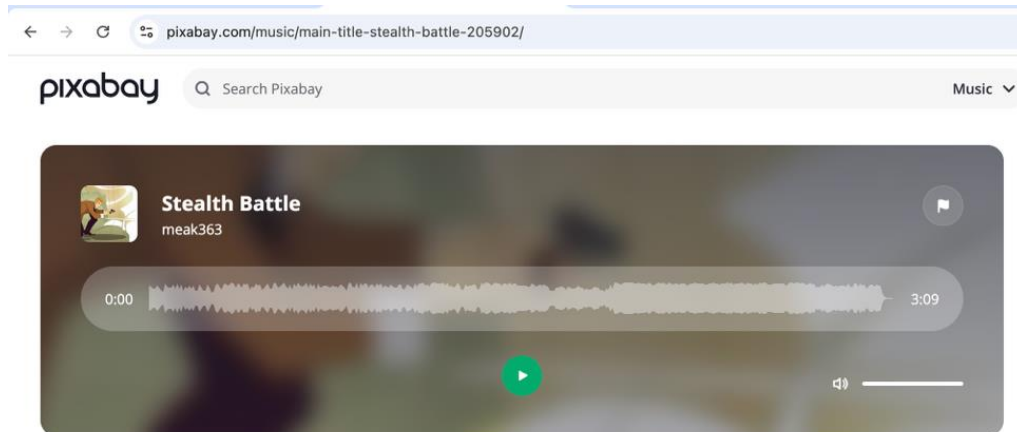
- **Media**
 - Stealth-battle.mp3
- **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
 - LocalUserInfo.json
- **Progress Management:**
 - **1 - Completed**
 - **2 - In progress**
 - **3 - In progress**
 - **4 - In progress**
 - **6 - In progress**
 - **15 - In progress**
 - **35 - In progress**
- **Videos:**
 - Initialization-initialization
 - Rendering-username-screen
 - Rendering-menu-screen
- **Tests Results: PASS**
- **Stakeholders' Approval: PASS**

INTERACTION 1 – MUSIC CHOICE

- **Objective:** Find a music without copyright to use it as background music for the whole game.
- **Success Criteria:** N/A
- **Directories and files:**
 - **Media**
 - Stealth-battle.mp3
- **Videos:** N/A
- **Tests Results:** N/A

- **Stakeholders:** **Approved**

MUSIC



Fight, Battle, Stealth music. Free for use.

<https://pixabay.com/music/main-title-stealth-battle-205902/>

1. The music is from a website called pixabay.com
 - It provides a range of no copyright music, so it won't have any legal implication.
2. The music is Stealth Battle, it has a vibe of action and hidden
 - It doesn't have a controlled intensity, so it provides a certain pressure on user
3. The file is mp3
 - Pygame won't have problem to run it, and its name is stealth-battle.mp3.

INTERACTION 2 – INITIALIZATION

- **Objective:** Initialization of the game is completed successfully.
- **Success Criteria:** N/A
- **Directories and files:**
 - **Distribution code**
 - Main.py
- **Videos:**
 - Initialization
- **Tests Results:** **PASS**

➤ **Stakeholders:** N/A

CODE

```
8 # Main module excuted when game starts
9 def main():
10     # Initialization of pygame and its features
11     pygame.init()
12     pygame.mixer.init()
13
14     # Create the title
15     pygame.display.set_caption("Richouse")
16
17     # Start the music
18     pygame.mixer.music.load("/Users/viniciusferrari/Documents/Richouse/Media/stealth-battle.mp3")
19     pygame.mixer.music.play(loops=-1)
20
21     #Set the mouse cursor invisible in screen
22     pygame.mouse.set_visible(False)
```

1. Pygame initializers.
 - o `pygame.init()` initialize the pygame enabling to use its features.
 - o `pygame.mixer.init()` initialize the mixer that enables to run music.
2. `pygame.display.set_caption(<title>)` create the game title, Richouse.
3. Load and run the music.
 - o `pygame.mixer.music.load(<file>)` will load the music of the game, located in Richouse directory with name stealth-battle.mp3.
 - o `pygame.mixer.music.play(loops)` will set how many times the music will be played, I set to -1 to run infinitely while the game is running.
4. `pygame.mouse.set_visible(<bool>)` will state with mouse cursor will be showed or not, so I set False to make it invisible.

TESTING

- **Objective:** Start the game and the music will start.
- **Blackbox Testing Data:** N/A
- **Video:**
 - o Initialization
- **Syntax Errors:** N/A
- **Status:** **PASS**

INTERACTION 3 – USERNAME REGISTER SCREEN

- **Objective:** The username register screen is rendered when there's no username in the local database.
- **Success Criteria:** N/A
- **Directories and files:**
 - **Distribution code**
 - Main.py
 - LocalUserInfo.json
- **Videos:**
 - Rendering-username-screen
- **Tests Results:** PASS
- **Stakeholders:** Approved

CODE

```

8  # Main module excuted when game starts
9  def main():
10     # Initialization of pygame and its features
11     pygame.init()
12     pygame.mixer.init()
13
14     # Create the title
15     pygame.display.set_caption("Richouse")
16
17     # Start the music
18     pygame.mixer.music.load("/Users/viniciusferrari/Documents/Richouse/Media/stealth-battle.mp3")
19     pygame.mixer.music.play(loops=-1)
20
21     #Set the mouse cursor invisible in screen
22     pygame.mouse.set_visible(False)
23
24     # Create the system object, it will be responsible for screens
25     system = System()
26
27     # Get the user data from local flatfile database, JSON type
28     with open("/Users/viniciusferrari/Documents/Richouse/Distribution-code/LocalUserInfo.json") as f:
29         userData = json.load(f)
30
31     # Check for registered username
32     username = userData[0]["username"]
33     if username == "":
34         preferences = Preference() # Give default preferences
35         system.render_screen(userPreferences=preferences, nextScreen="username") # Render the screen to register an username
36
37     game_loop(system=system, preferences=preferences) # Redirect to the function with the game loop
38
39
40
41
42

```

1. Open database and get user data, it's store in a JSON file.
 - `open(<file>)` is used to open a flat file.
 - `json.load()` will load the element of database.
2. Create preferences object and render next screen based on username data.
 - `userData[0]["username"]` is used to access the username field in the first row of the file, it will be the only row because only one user is registered locally.

- o `Preference(<parameters>)` is used to create a object to store the user preferences, it uses the default preferences when no parameter is used. I used an object because it will have a method to change preferences in database as well, a list won't enable it.
 - o `System()` is used to create a system object responsible to create and lead with backgrounds of screen object
 - o `system.render_screen()` is responsible to change the screen and print it in first time.
3. `game_loop()` function is responsible to call the game main loop.

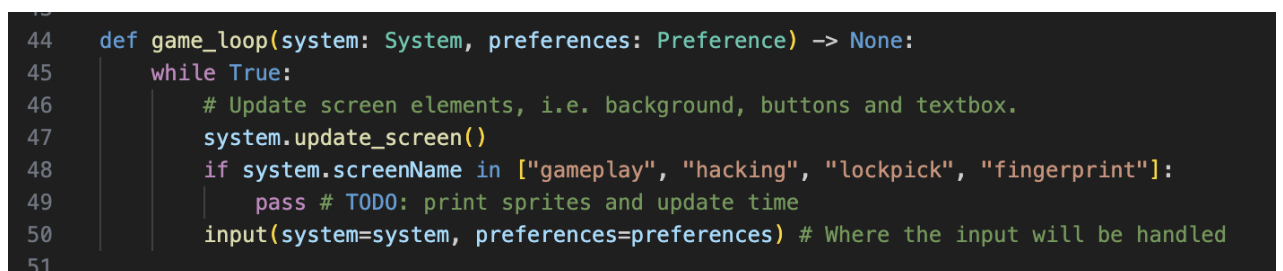


```

{} LocalUserInfo.json > ...
1  [
2      {
3          "username": "",
4          "controls": {
5              "up": null,
6              "down": null,
7              "left": null,
8              "right": null
9          }
10     }
11 ]

```

1. The database type used locally was a flat file, JSON type, because it's simpler than relational database, and the problem of redundancy and difficult search of data won't happen given that only one record will be stored on it.
2. Username starts with "" because there isn't a username registered and it makes easier to use selection, `if userData[0]["username"] == ""`.
3. Controls are stored in a dictionary, it makes simpler to access it through key works, i.e. "up" for the up-selection button.



```

44 def game_loop(system: System, preferences: Preference) -> None:
45     while True:
46         # Update screen elements, i.e. background, buttons and textbox.
47         system.update_screen()
48         if system.screenName in ["gameplay", "hacking", "lockpick", "fingerprint"]:
49             pass # TODO: print sprites and update time
50         input(system=system, preferences=preferences) # Where the input will be handled
51

```

The game main loop, here is where activities of constant change will be repeated until game be stopped.

1. `system.update_screen()` will update the screen rendered before by `system.render_screen()`, all screen related task are in system object.
2. If statement that will be used in the future to update elements present in gameplay screen, it's separated from `update_screen()` because they are more specific and complex, i.e. where the guard will be next.
3. `input()` is called to handle the inputs entered by user.

```
1
2 def input(system: System, preferences: Preference) -> None:
3     # Get the keys to be handled
4     for event in pygame.event.get():
5         # Essential to maintain pygame working
6         if event.type == pygame.QUIT:
7             pygame.quit()
8             sys.exit()
```

The input handling happens here; at this stage it contains only the essential to maintain the pygame running.

1. `pygame.event.get()` gets all events that happened in the time of frame changing.
2. The only selection is the one used to identify when game is quitted, it's essential for pygame workflow, if not present, it will freeze.

```
110 class System:
111     def __init__(self):
112         self.__screenObj = pygame.display.set_mode((800, 800))
113         self.__screenName = ""
114         self.__background = None
115         self.__button = None
116         self.__gameplay = None
117         self.__minigame = None
118         self.__textbox = ""
119
120     # Enable the access of the private attribute screenName
121     @property
122     def screenName(self) -> str:
123         return self.__screenName
124
125     def render_screen(self, nextScreen: str, userPreferences: object) -> None:
126         # Uses the next screen actions
127         match nextScreen:
128
129             case "username":
130                 # Load the username-screen that is the username registred screen, textbox starts empty
131                 self.__background = pygame.image.load("/Users/viniciusferrari/Documents/Richouse/Media/username-screen.png").convert_alpha()
132                 self.__textbox = ""
133
134             case "rank":
135                 self.__screenName = nextScreen
136                 self.__screenObj.blit(self.__background, (0, 0))
137                 if self.__screenName == "rank":
138                     # TODO: make it print the top 10 scores in each map table
139                     pass
140                 self.display_elements()
141                 pygame.display.flip()
```

1. The constructor method storing the attributes.
 - o `self.__screenObj` stores the screen object to be handled, store inside the class enable access it without passes as parameter.
 - o `self.__screenName` maintain a record of the currently screen.

- `self.__background` will store the background to be printed every frame update.
 - `self.__button` stores the button objects created inside the methods of `System`.
 - `self.__gameplay` stores the gameplay object created inside the methods of `System`, it isn't currently being used.
 - `self.__minigame` stores the minigame object created inside the methods of `System`, it isn't currently being used.
 - `self.__textbox` store the currently word being typed in username register screen.
2. `screenName()` is used with decorator `@property` to work as a getter method of the encapsulate attribute `self.__screenName`.
 3. Use a match statement to select the next instruction based on screen chosen.
 4. Username screen rendering.
 - Load the `username-screen.png` that is the background, Pygame must load every image before print using `pygame.image.load(<file>)`.
 - Set the textbox to "" meaning that no input was made yet.
 5. `self.__screenName` maintain a record of the current screen, used to control future actions.
 6. `self.__screenObject.blit()` uses the screen object to print the background, it happens before elements because Pygame print the images in layers.
 7. A selection is used to make a field where the code of rank screen elements will be printed in the future.
 8. `self.display_elements()` will display the buttons and textbox, as this code is used both in `render_screen()` and `update_screen()`, so I can reuse this part.
 9. `pygame.display.flip()` change the frame, so the game won't stay static in the first frame.

```

196
197     def update_screen(self) -> None:
198         self.__screenObj.blit(self.__background, (0,0)) # Print background
199         self.display_elements() # Display buttons and textbox
200         pygame.display.flip() # Change the frame
201

```

The function that will update the screen by printing the background, buttons and textbox in every frame that they're needed.

1. `self.__screenObj.blit()` print the background every frame.
2. `Self.display_elements()` method is used to reuse the code of selection of elements to be printed, the buttons and the textbox.
3. `pygame.display.flip()` is used to change the frame.

```

71 class Preference:
72     def __init__(self, username: str="Guest", up: int=pygame.K_w, down: int=pygame.K_w, left: int=pygame.K_w,
73                 right: int=pygame.K_w, volume: int=50):
74         self.__username = username
75         self.__buttonUp = up
76         self.__buttonDown = down
77         self.__buttonLeft = left
78         self.__buttonRight = right
79         self.__volume = volume

```

The used to store the user preferences, it includes all buttons and the volume but also the username that is needed to send the results to server in the future.

1. Constructor method.
 1. `self.__username` store the username.
 2. `self.__buttonUp` is the predefined button to select up or move forward.
 3. `self.__buttonDown` is the predefined button to select down or move downward.
 4. `self.__buttonLeft` is the predefined button to select move left.
 5. `self.__buttonRight` is the predefined button to move right.
 6. `self.__volume` store the music volume.
2. A getter method with decorator `@property` is used to return the encapsuled attribute volume that is used in main function to change volume intensity.

CHANGES FROM DESIGN

1. Screen object is created inside system class to make easier to access it inside the methods.
2. A flat file database was used instead of relational database because it proved to be easier to be handled without any performance loss in this case, only one record stored won't cause redundancy.
3. Username is store in preferences to make easier to send request to the server in the future.

TESTING

- **Objective:** The register screen starts when the database stores no username.
- **Blackbox Testing Data:** N/A
- **Video:**
 - Rendering-username-screen
- **Syntax Errors:**

- **TypeError: list indices must be integers or slices, not str.**
 - I used `userData["username"]` to access the username in JSON record, but the records are hold in lists.
 - Solved by using `userData[0]["username"]`, access the record and after use string to access the field wanted.
- **AttributeError: 'Preference' object has no attribute '__volume'.**
 - I tried to access the private attribute `self.__volume`, but there weren't a getter method.
 - I solved by adding a getter method `volume` using `@property` decorator.
- **AttributeError: type object 'system' has no attribute 'render_screen'.**
 - I passed the screen object in the wrong way; I used the parameter `system: object`.
 - The solution is pass in the format `system: System`.
- **AttributeError: type object 'button' has no attribute 'buttons'.**
 - It happened to attributes `buttons`, `selectedButtonIndex` and `backgrounds`.
 - The solution is creating a getter method using `@property`, it because of attributes are private.

➤ **Status: PASS**

INTERACTION 4.1 – MENU SCREEN

- **Objective:** When there is a username in the database, the menu screen is rendered.
- **Success Criteria:**
 - **1 – Menu screen becomes the first screen if username is in the local database**
- **Directories and files:**
 - **Distribution code**
 - `Main.py`
 - `Functions.py`
 - `Config.py`
 - `LocalUserInfo.json`

- **Videos:**
 - Rendering-menu-screen-fail
- **Tests Results: FAIL**
- **Stakeholders: N/A**

CODE

```
110 class System:
111     def __init__(self):
112         self.__screenObj = pygame.display.set_mode((800, 800))
113         self.__screenName = ""
114         self.__background = None
115         self.__button = None
116         self.__gameplay = None
117         self.__minigame = None
118         self.__textbox = ""
119
120     # Enable the access of the private attribute screenName
121     @property
122     def screenName(self) -> str:
123         return self.__screenName
124
125     def render_screen(self, nextScreen: str, userPreferences: object) -> None:
126         # Uses the next screen actions
127         match nextScreen:
128             case "menu":
129                 # Create the buttons: start, rank, option and quit, the format is [name, coordinate]
130                 self.__button = Button(buttons=[["start", (239, 275)], ["rank", (267, 400)], ["options", (175, 525)], ["quit", (276, 650)]],
131                                         preferences=userPreferences, system=self)
132                 # Load the menu background, menu-screen.png
133                 self.__background = pygame.image.load("/Users/viniciusferrari/Documents/Richouse/Media/menu-screen.png").convert_alpha()
134                 self.__background = pygame.transform.scale(self.__background, (800, 800))
135             case "username":
136                 # Load the username-screen that is the username registred screen, textbox starts empty
137                 self.__background = pygame.image.load("/Users/viniciusferrari/Documents/Richouse/Media/username-screen.png").convert_alpha()
138                 self.__textbox = ""
139
140         self.__screenName = nextScreen
141         self.__screenObj.blit(self.__background, (0, 0))
142         if self.__screenName == "rank":
143             # TODO: make it print the top 10 scores in each map table
144             pass
145         self.display_elements()
146         pygame.display.flip()
```

1. Menu screen rendering.

- Create a button object to create and control four buttons behavior, they are the start, rank, options and quit buttons. Using an object, I can create action for the buttons, something that a list won't enable me. The tuples are the position (x, y) in screen.
- Load the menu-screen.png that is the background, Pygame must load every image before print using `pygame.image.load(<file>)`.
- I had to use `pygame.transform.scale()` to change the size of menu-screen.png that I mistake to save in the wrong size, so it just turns it 800x800.

```

202 def display_elements(self):
203     #Check if there is some button
204     if self.__button is not None:
205         buttons = self.__button.buttons # List of all buttons
206         selectedButtonIndex = self.__button.selectedButtonIndex # The position in the list of the selected button
207         backgrounds = self.__button.backgrounds # Buttons backgrounds
208
209         #Select the buttons in list, each one has a format [name, (x-coordinate, y-coordinate)]
210         for button in buttons:
211             img = pygame.image.load(backgrounds[button[0]]).convert_alpha() # Load button background
212
213             # Check if it's the selected button
214             if button == buttons[selectedButtonIndex]:
215                 width = img.get_width()
216                 height = img.get_height()
217                 # Change the sizes by a facto 1.3
218                 newWidth = width * 1.3
219                 newHeight = height * 1.3
220                 img = pygame.transform.scale(img, (newWidth, newHeight)) # Change the button scale
221                 x = button[1][0]
222                 y = button[1][1]
223
224             self.__screenObj.blit(img, button[1]) # Print button in normal scale and position
225
226     # Check if there is textbox
227     if self.__textbox is not None:
228         font = pygame.font.Font(None, 48) # Uses the standar font and size 48
229         text_surface = font.render(self.__textbox, True, (0, 0, 0)) # Render the font with white colour
230         text_rect = text_surface.get_rect(center=(400, 400)) # Create a surface for textbox
231         self.__screenObj.blit(text_surface, text_rect) # Print the surface
232
233
234
235

```

Ln 235, Col 13 Spaces: 4 UTF-8 LF {} Python 3.12.0

The code of selection of the elements that isn't printed in every screen. It's used in more than one method, so this code was developed separated to be reused.

1. Check if there are buttons in the screen.

- Use the variables to store the buttons info without the need to access the other class every time. `buttons` is list of lists, each one is a button; `selectedButtonIndex` is the position of the selected button in `buttons`; and `backgrounds` is the dictionary of backgrounds.
- `Interaction` is used to access and print every button in the list.
- Get the button background and load it, use `button[0]` because each button is list in format `[name, (x-coordinate, y-coordinate)]`.
- Get the height and width of background and change by a scale 1.3 if it's selected

2. Check if there is textbox.

- Create the font with standard style and size 48.
- Create the text with the font and white color, rgb of (0, 0, 0).
- Create a surface for the text, (400, 400) is the position in screen.
- Print the surface and the text appear.

```

9 class Button:
10     def __init__(self, buttons: list, preferences: object, system: object):
11         self.__buttons = buttons
12         self.__selectedButtonIndex = 1
13         self.__preferences = preferences
14         self.__system = system
15         self.__backgrounds = {
16             "start": "/Users/viniciusferrari/Documents/Richouse/Media/start-button.png",
17             "rank": "/Users/viniciusferrari/Documents/Richouse/Media/rank-button.png",
18             "options": "/Users/viniciusferrari/Documents/Richouse/Media/options-button.png",
19             "quit": "/Users/viniciusferrari/Documents/Richouse/Media/quit-button.png",
20             "again": "/Users/viniciusferrari/Documents/Richouse/Media/again-button.png",
21             "menu-pause": "/Users/viniciusferrari/Documents/Richouse/Media/menu-pause-button.png",
22             "menu-menu": "/Users/viniciusferrari/Documents/Richouse/Media/menu-menu-button.png",
23             "continue": "/Users/viniciusferrari/Documents/Richouse/Media/continue-button.png"
24         }
25
26     # buttons getter method
27     @property
28     def buttons(self):
29         return self.__buttons
30
31     # selectedButtonIndex getter method
32     @property
33     def selectedButtonIndex(self):
34         return self.__selectedButtonIndex
35
36     # backgrounds getter method
37     @property
38     def backgrounds(self):
39         return self.__backgrounds
40

```

The class used to store buttons and control their actions.

1. Constructor method.

- `self.__buttons` is the list of the buttons present on the screen.
- `self.__selectedButtonIndex` store the position of selected button in `self.__buttons`, avoid storing data twice.
- `self.__preferences` store the preferences object, it will store the user buttons and volume preferences, I chose an object instead of a list because I can implement a method to update the database.
- `self.__system` is the object that handle the screen, so it's needed in the future method of `Button`.
- `self.__backgrounds` is a dictionary that stores all button backgrounds, it makes easier to access it, the developer doesn't need to pass all the long path as parameter every time.

2. The methods with the decorator `@property` are the getter methods of the encapsuled attributes, it's needed to access it into methods from other classes.

CHANGES FROM DESIGN

1. Selected buttons change the size instead of the color, it's easier to implement and the results is similar.

TESTING

- **Objective:** The menu screen and buttons are rendered properly
- **Blackbox Testing Data:** N/A
- **Video:**
 - Rendering-menu-screen-fail
- **Syntax Errors:** N/A
- **Status:** **FAIL, buttons aren't in the correct position**

INTERACTION 4,2 – MENU SCREEN

- **Objective:** When there is a username in the database, the menu screen is rendered.
- **Success Criteria:**
 - **1 – Menu screen becomes the first screen if username is in the local database**
- **Directories and files:**
 - **Media:**
 - Stealth-battle.mp3
 - **Distribution code**
 - Main.py
 - Functions.py
 - Config.py
- **Videos:**
 - Rendering-menu-screen-success
- **Tests Results:** **PASS**
- **Stakeholders:** **Approved**

CODE

```

202 def display_elements(self):
203     #Check if there is some button
204     if self.__button is not None:
205         buttons = self.__button.buttons # List of all buttons
206         selectedButtonIndex = self.__button.selectedButtonIndex # The position in the list of the selected button
207         backgrounds = self.__button.backgrounds # Buttons backgrounds
208
209         #Select the buttons in list, each one has a format [name, (x-coordinate, y-coordinate)]
210         for button in buttons:
211             img = pygame.image.load(backgrounds[button[0]]).convert_alpha() # Load button background
212
213             # Check if it's the selected button
214             if button == buttons[selectedButtonIndex]:
215                 width = img.get_width()
216                 height = img.get_height()
217                 # Change the sizes by a facto 1.3
218                 newWidth = width * 1.3
219                 newHeight = height * 1.3
220                 img = pygame.transform.scale(img, (newWidth, newHeight)) # Change the button scale
221                 x = button[1][0]
222                 y = button[1][1]
223                 self.__screenObj.blit(img, (x - ((newWidth - width)/2), y - ((newHeight - height)/2))) # Recalculate the position
224                                                         # in screen and print it
225                 continue
226             self.__screenObj.blit(img, button[1]) # Print button in normal scale and position
227
228     # Check if there is textbox
229     if self.__textbox is not None:
230         font = pygame.font.Font(None, 48) # Uses the standar font and size 48
231         text_surface = font.render(self.__textbox, True, (0, 0, 0)) # Render the font with white colour
232         text_rect = text_surface.get_rect(center=(400, 400)) # Create a surface for textbox
233         self.__screenObj.blit(text_surface, text_rect) # Print the surface

```

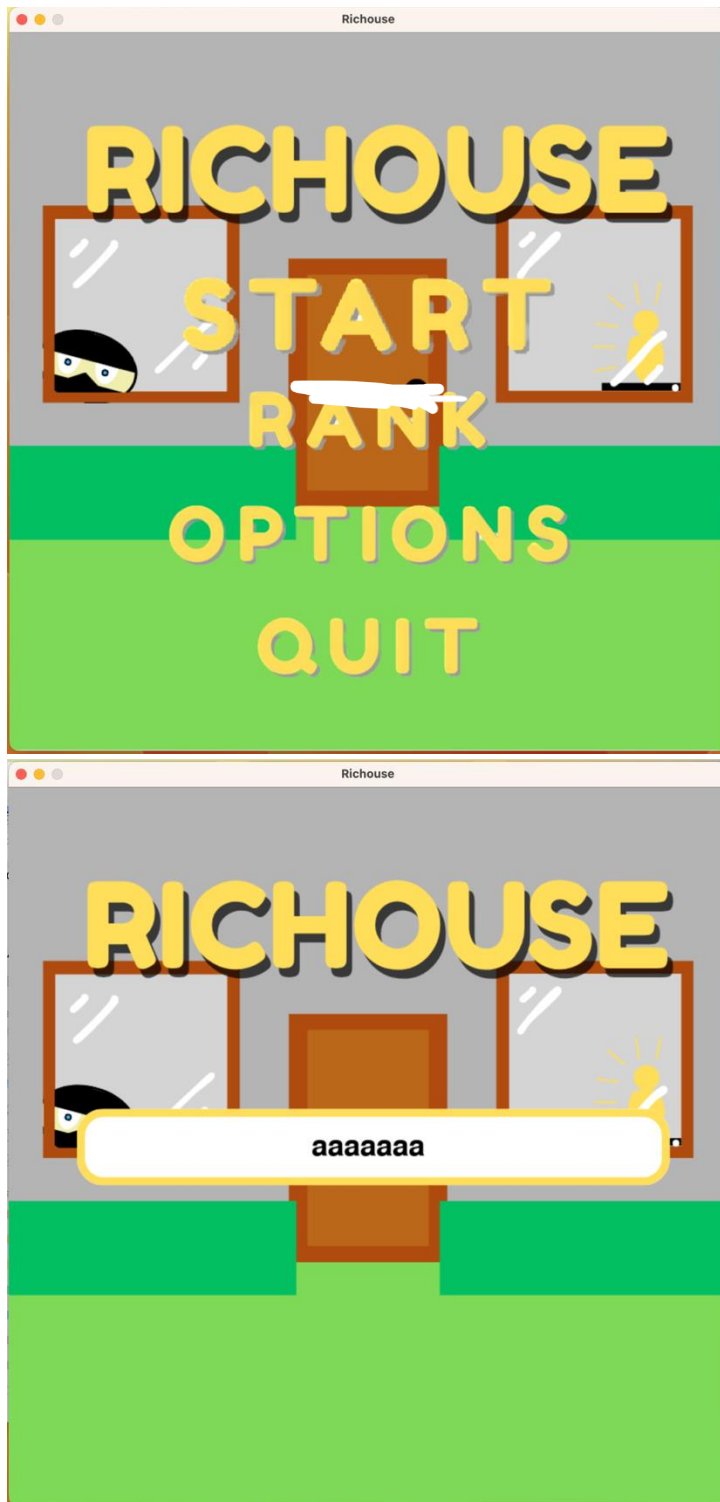
1. Use a calculation to print it in the correct position, $(newWidth - width)$ is the number of pixels extra in x, so I got it and divided by 2 because it's the increase in width in each side, finally I subtracted from x because in this way the button will be printed in the same position of original scale.
2. I did the same with height.

TESTING

- **Objective:** The menu screen and buttons are rendered properly
- **Blackbox Testing Data:** N/A
- **Video:**
 - Rendering-menu-screen-success
- **Syntax Errors:** N/A
- **Status:** **PASS**

PROTOTYPE

- **Stakeholders' feedback:** **Approved**
- **Video:** N/A
- **Images:**



WEEK 4 – MENU AND USERNAME SCREEN INPUT HANDLING

- **Objective:** Develop the input handling for the username register and menu screens, including the rendering of the next screens.
- **Success Criteria:**

- 1 – The menu screen is rendering after username register screen, both in same and different sessions.
 - 2 – W and S are selecting up and down buttons in the menu screen.
 - 3 – The options screen is rendered after the button be confirmed.
 - 4 – The rank screen is rendered after the button be confirmed.
 - 5 – The start buttons no longer start the game directly, instead it opens the map selection screen.
 - 6 – The program stops when the quit button is confirmed.
 - 15 – Only keyboard keys are used for selection and text input.
- **Directories and files:**
- **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
 - LocalUserInfo.json
- **Progress Management:**
- 3 - In progress
 - 4 - In progress
 - 5 - In progress
 - 35 - In progress
- **Videos:**
- Username-screen-input
 - Menu-screen-input-fail
 - Menu-screen-input-success
 - Prototype-week-4
- **Tests Results: PASS**
- **Stakeholders' Approval: Approved**

INTERACTION 1 – REGISTER SCREEN INPUT HANDLING

- **Objective:** I can input a text of at maximum 10 alphanumeric characters, which will be saved as my username. Afterwards, the menu screen is rendered.
- **Success Criteria:**
- 1 – The menu screen is rendering after username register screen, both in same and different sessions.

- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
 - LocalDatabase.json
- **Videos:** N/A
- **Tests Results:** PASS
- **Stakeholders:** Approved

CODE

```

104 class System:
105     def __init__(self, preferences: Preference):
106         self.__screenObj = pygame.display.set_mode((800, 800))
107         self.__screenName = ""
108         self.__background = None
109         self.__button = None
110         self.__gameplay = None
111         self.__minigame = None
112         self.__textbox = ""
113         self.__preferences = preferences

```

I added a preferences attribute to `System` class, it enables me to access it inside the `systems` method without pass as parameter making easier to develop and avoiding parameter errors.

1. It uses a `Preference` type.
2. All `system.render_screen()` now only uses the `nextScreen` parameter
3. `Button()` will get only the `system` as object parameter, as `preferences` is already `System` parameter we don't need to pass it.
4. A getter method using `@property` was implemented to access the `preferences` object inside other classes methods

```

53 def input(system: System) -> None:
54     # Get the events to be handled
55     for event in pygame.event.get():
56         # Essential to maintain pygame working
57         if event.type == pygame.QUIT:
58             pygame.quit()
59             sys.exit()
60         # Handle any key pressed, no matters if key still pressed, it will handle once per press
61         elif event.type == pygame.KEYDOWN:
62             # Different handling according to the screen
63             match system.screenName:
64                 case "username":
65                     textbox_handling(system=system, key=event.key) # Uses a function to handle the textbox

```

I added more code to `input()`, they are responsible to handle once the key inputs when pressed, no matters if it still pressed.

1. The new selection identifies the `KEYDOWN` event, it will handle the key pressed only once per press, even if it still pressed.
2. The way that the key will be handled depends on screen, so now we can see why is useful to store the screen state in the attribute `system.__screenName`.
3. Username register screen uses method/function to handle the key in that screen, it makes the code clearer and structured in reusable modules. All classes/function are stored in `functions.py`.

```

136     @property
137     def textbox(self):
138         return self.__textbox
139
140     # Setter method for self.__textbox
141     @textbox.setter
142     def textbox(self, newText: str):
143         if newText is not None:
144             self.__textbox = newText

```

1. I gave to other classes methods/functions the access to `self.__textbox` and I enable them to change its value.
 - o I used the decorator `@property` to create the getter method.
 - o I used the decorator `@textbox.setter` to create the setter method
 - o A setter method enables me to sanitize the value to be defined, in this case I won't allow any method outside `System` to set `textbox` to `None`. I can set to `None` inside `System` using `self.__textbox`.

```

146     def render_screen(self, nextScreen: str) -> None:
147         #Reset the textbox and buttons
148         self.__button = None
149         self.__textbox = None
150
151         # Uses the next screen actions
152         match nextScreen:
153             case "menu":
154                 # Create the buttons: start, rank, option and quit, the format is [name, (x-coordinate, y-coordinate)]
155                 self.__button = Button(buttons=[["start", (239, 275)], ["rank", (267, 400)], ["options", (175, 525)], ["quit", (276, 650)]],
156                                         system=self)
157                 # Load the menu background, menu-screen.png
158                 self.__background = pygame.image.load("/Users/viniciusferrari/Documents/Richouse/Media/menu-screen.png").convert_alpha()
159             case "username":
160                 # Load the username-screen that is the username registered screen, textbox starts empty
161                 self.__background = pygame.image.load("/Users/viniciusferrari/Documents/Richouse/Media/username-screen.png").convert_alpha()
162                 self.__textbox = ""

```

The method `render_screen()` experienced changes given the need of the other screens to check if the selection happened correctly.

1. I added a piece of code in the beginning to set `textbox` to `None`.
 - o Avoids them to be printed in the new screen unless the screen has some `textbox`.

```

234 def display_buttons(self) -> None:
235     # Check if there is some button
236     if self.__button is not None:
237         buttons = self.__button.buttons # List of all buttons
238         selectedIndex = self.__button.selectedIndex # The position in the list of the selected button
239         backgrounds = self.__button.backgrounds # Buttons backgrounds
240
241         # Select the buttons in list, each one has a format [name, (x-coordinate, y-coordinate)]
242         for button in buttons:
243             img = pygame.image.load(backgrounds[button[0]]).convert_alpha() # Load button background
244
245             # Check if it's the selected button
246             if button == buttons[selectedIndex]:
247                 width = img.get_width()
248                 height = img.get_height()
249                 # Change the sizes by a factor 1.3
250                 newWidth = width * 1.3
251                 newHeight = height * 1.3
252                 img = pygame.transform.scale(img, (newWidth, newHeight)) # Change the button scale
253                 x = button[1][0]
254                 y = button[1][1]
255                 self.__screenObj.blit(img, (x - ((newWidth - width)/2), y - ((newHeight - height)/2))) # Recalculate the position
256                                                         # in screen and print it
257                 continue
258             self.__screenObj.blit(img, button[1]) # Print button in normal scale and position
259
260
261 def display_textbox(self) -> None:
262     # Check if there is textbox
263     if self.__textbox is not None:
264         font = pygame.font.Font(None, 48) # Uses the standar font and size 48
265         text_surface = font.render(self.__textbox, True, (0, 0, 0)) # Render the font with white colour
266         text_rect = text_surface.get_rect(center=(400, 400)) # Create a surface for textbox
267         self.__screenObj.blit(text_surface, text_rect) # Print the surface

```

It's the same code of `display_elements()` showed in Game Initialization and First Screen section, however now it's divided into `display_buttons()` and `display_textbox()` making easier to read and understand the code.

```

330 def textbox_handling(system: System, key: int):
331     # Use the key int constant to choose the correct action
332     match key:
333         # RETURN key confirm the username
334         case pygame.K_RETURN:
335             if system.textbox != "":
336                 # Get the user data from database
337                 with open("Users/viniciusferrari/Documents/Richouse/Distribution-code/LocalUserInfo.json", "r") as f:
338                     userData = json.load(f)
339
340                 # Modify database to register the username
341                 userData[0]["username"] = system.textbox
342
343                 # Write the changes in database
344                 with open("Users/viniciusferrari/Documents/Richouse/Distribution-code/LocalUserInfo.json", "w") as f:
345                     json.dump(userData, f)
346                 # Render the menu screen
347                 system.render_screen(nextScreen="menu")
348             # BACKSPACE delete one character
349             case pygame.K_BACKSPACE:
350                 system.textbox = system.textbox[:-1] # Create a new string cutting off the last character of textbox
351             # Any other character
352             case _:
353                 if pygame.key.name(key).isalnum() and len(system.textbox) <= 16: # Limit the username to 16 alphanumeric characters
354                     system.textbox = system.textbox + pygame.key.name(key) # append the new character to the textbox

```

Finally, I developed a function to handle the textbox, as the game has only one textbox it isn't worthy to create a class to do it.

1. Match statement uses the integer constant of the key to decide the action.
2. The RETURN key is responsible to confirm the register.
 - o The user data is opened and read using `open()` and `json.load()`.
 - o The data extracted is changed to set the username to the registered one.
 - o The data is written into the database using `open()` and `json.dump()`.
3. The BACKSPACE key will delete the last character.
 - o Strings don't have the `pop()` method, so the solution was creating a new string slicing the `system.textbox` to cut off the last character and set the result as the value of `system.textbox`.
4. Any other key is sent to be checked and may be appended to textbox.

- It will limit the string to alphanumeric values and maximum size of 16 characters.
- It will use the value of `system.textbox` to add the key and after use the result as its value, it's needed because strings don't have `append()` method.

MAIN CHANGES FROM DESIGN

1. Now the preferences object is stored as an attribute of `System` class instead of being passed as parameter every time, it allows an easier access inside `System` and avoids parameter errors.
2. The `render_screen()` now reset the buttons and textbox before next screen rendering; it avoids the wrong elements to be printed.
3. The elements aren't printed directly inside `render_screen()`, they're printed using `display_buttons()` and `display_textbox()` methods, it makes the methods shorter and more structured, as results it becomes more readable.

TESTING

- **Objective:** I can input a text of at maximum 10 alphanumeric characters, which will be saved as my username. Afterwards, the menu screen is rendered.
- **Blackbox Testing Data:**
 - **58 – I can enter any username smaller than 10 characters**
 - **59 – The game ignores the extra characters.**
 - **60 - I can enter numbers.**
 - **61 – I can enter letters.**
 - **62 – I'm no longer using _ as a valid character because there is already a fair number of possibilities.**
 - **63 – All the other keys are ignored.**
- **Video:**
 - Username-screen-input
- **Syntax Errors:**
 - **TypeError: unsupported operand type(s) for +=: 'property' and 'str'.**
 - I tried to set a value to textbox without a setter method.

- I used a `@textbox.setter` decorator in a method to solve it. I developed it to use a maximum length of 10 character, it won't enable an overflow in Pygame or sever database.
- **The keys aren't being printed, only numbers**
 - `event.key` return only an integer constant instead of the key
 - I just need to use `pygame.key.name(event.key)` to the the actual key

➤ **Status:** PASS

INTERACTION 2.1 – MENU SCREEN INPUT HANDLING

- **Objective:** I can use the W and S to select buttons, and the ENTER to confirm it. Afterwards, the appropriate screen is rendered.
- **Success Criteria:**
 - **2 – W and S are selecting up and down buttons in the menu screen.**
 - **3 – The options screen is rendered after the button be confirmed.**
 - **4 – The rank screen is rendered after the button be confirmed.**
 - **5 – The start buttons no longer start the game directly, instead it opens the map selection screen.**
 - **6 – The program stops when the quit button is confirmed.**
 - **15 – Only keyboard keys are used for selection and text input.**
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
- **Videos:**
 - Menu-screen-input-fail
- **Tests Results:** FAIL
- **Stakeholders:** N/A

CODE

```

53 def input(system: System) -> None:
54     # Get the events to be handled
55     for event in pygame.event.get():
56         # Essential to maintain pygame working
57         if event.type == pygame.QUIT:
58             pygame.quit()
59             sys.exit()
60         # Handle any key pressed, no matters if key still pressed, it will handle once per press
61         elif event.type == pygame.KEYDOWN:
62             # Different handling according to the screen
63             match system.screenName:
64                 case "menu":
65                     system.button.selection_menu(key=event.key) # Uses a method to handle the buttons
66                 case "username":
67                     textbox_handling(system=system, key=event.key) # Uses a function to handle the textbox
68             # Just to enable me to return to menu screen at this stage
69             case "start" | "rank" | "options":
70                 if event.key == pygame.K_ESCAPE:
71                     system.render_screen(nextScreen="menu")

```

1. Menu and username register screens uses method/function to handle the key in that screen, it makes the code clearer and structured in reusable modules. All classes/function are stored in `functions.py`.
2. The last selection uses 3 screens and it's temporally, I'm using it for now to enable me to return menu screen during the testing, in this way I don't need to restart the code. It will render the menu screen again when ESC is pressed.

```

51     # Getter method for self.__volume
52     @property
53     def volume(self):
54         return self.__volume
55
56     # Getter method for self.__buttonUp
57     @property
58     def buttonUp(self):
59         return self.__buttonUp
60
61     # Getter method for self.__buttonDown
62     @property
63     def buttonDown(self):
64         return self.__buttonDown
65
66     # Getter method for self.__buttonLeft
67     @property
68     def buttonLeft(self):
69         return self.__buttonLeft
70
71     # Getter method for self.__buttonRight
72     @property
73     def buttonRight(self):
74         return self.__buttonRight

```

I developed getter methods for Preference class attributes using `@property`, it enables me to access the encapsulated attributes in methods from other classes.

```

109 class System:
110     def __init__(self, preferences: Preference):
111         self.__screenObj = pygame.display.set_mode((800, 800))
112         self.__screenName = ""
113         self.__background = None
114         self.__button = None
115         self.__gameplay = None
116         self.__minigame = None
117         self.__textbox = ""
118         self.__preferences = preferences
119
120     # Enable the access of the private attribute screenName
121     @property
122     def screenName(self):
123         return self.__screenName
124
125     # Getter method for self.__button
126     @property
127     def button(self):
128         return self.__button
129
130     # Getter method for self.__preferences
131     @property
132     def preferences(self):
133         return self.__preferences
134
135     # Getter method for self.__textbox
136     @property
137     def textbox(self):
138         return self.__textbox
139
140     # Setter method for self.__textbox
141     @textbox.setter
142     def textbox(self, newText: str):
143         if newText is not None:
144             self.__textbox = newText

```

Here are the changes in `System` class attributes and method to handle them.

1. I added the `preferences` object as an attribute of `System` creating an aggregation relationship.
 - o It makes easier to access the `preferences` object inside the methods of `System`, furthermore it avoids errors in parameters.
 - o `system.render_screen()` method now use only `nextScreen` as parameter.
 - o `Button` class doesn't need `preferences` as attribute no more, it because of `system` object that is already one of its attributes.
 - o A getter method with `@property` is needed to access the `self.__preferences` in other classes methods.
2. I create a getter method using `@property` to enable the outside class access of `self.__buttons`, it is accessed at `input()` function.

```

146 def render_screen(self, nextScreen: str) -> None:
147     #Reset the textbox and buttons
148     self.__button = None
149     self.__textbox = None
150
151     # Uses the next screen actions
152     match nextScreen:
153         case "menu":
154             # Create the buttons: start, rank, option and quit, the format is [name, (x-coordinate, y-coordinate)]
155             self.__button = Button(buttons=[("start", (239, 275)), ("rank", (267, 400)), ("options", (175, 525)), ("quit", (276, 650))],
156                                     system=self)
157             # Load the menu background, menu-screen.png
158             self.__background = pygame.image.load("/Users/viniciusferrari/Documents/Richouse/Media/menu-screen.png").convert_alpha()
159         case "username":
160             # Load the username-screen that is the username registered screen, textbox starts empty
161             self.__background = pygame.image.load("/Users/viniciusferrari/Documents/Richouse/Media/username-screen.png").convert_alpha()
162             self.__textbox = ""
163         case "rank":
164             # Load the rank background, rank-screen.png
165             self.__background = pygame.image.load("/Users/viniciusferrari/Documents/Richouse/Media/rank-screen.png")
166         case "options":
167             # Load the username-screen that is the options screen, here user can change the predefined keys and volume
168             self.__background = pygame.image.load("/Users/viniciusferrari/Documents/Richouse/Media/options-screen.png").convert_alpha()
169         case "start":
170             # Create the buttons: map 1 and map 2, the format is [name, (x-coordinate, y-coordinate)]
171             self.__button = Button(buttons=[("map-1", (250, 350)), ("map-2", (250, 500))], system=self)
172             # Loads the map selection background, it's the same of menu but with different buttons
173             self.__background = pygame.image.load("/Users/viniciusferrari/Documents/Richouse/Media/menu-screen.png").convert_alpha()

```

The method `render_screen()` experienced changes given the need of the other screens to check if the selection happened correctly.

1. I added a piece of code in the beginning to set button to `None`.
 - avoids them to be printed in the new screen unless the screen has some button.
2. Rank selection initializes the rank screen.
 - Now I have only the background because server data is needed to print the top 10 scorers of each map.
3. Options selection initializes the screen to change the predefinitions.
 - It doesn't have the element to select each configuration because their backgrounds will be handled different from normal buttons, so extra code is needed.
4. Start selection initializes the map selection screen.
 - It already has buttons because it uses normal buttons, however they aren't working yet given that selection code wasn't implemented.
 - The loaded background is the same of menu; I decided to use the same background with different buttons because the game doesn't need a complex screen for a simple action as this one.

```

269 class Button:
270     def __init__(self, buttons: list, system: System):
271         self.__buttons = buttons
272         self.__selectedButtonIndex = 0
273         self.__system = system
274         self.__backgrounds = {
275             "start": "/Users/viniciusferrari/Documents/Richouse/Media/start-button.png",
276             "rank": "/Users/viniciusferrari/Documents/Richouse/Media/rank-button.png",
277             "options": "/Users/viniciusferrari/Documents/Richouse/Media/options-button.png",
278             "quit": "/Users/viniciusferrari/Documents/Richouse/Media/quit-button.png",
279             "again": "/Users/viniciusferrari/Documents/Richouse/Media/again-button.png",
280             "menu-pause": "/Users/viniciusferrari/Documents/Richouse/Media/menu-pause-button.png",
281             "menu-menu": "/Users/viniciusferrari/Documents/Richouse/Media/menu-menu-button.png",
282             "continue": "/Users/viniciusferrari/Documents/Richouse/Media/continue-button.png",
283             "map-1": "/Users/viniciusferrari/Documents/Richouse/Media/map-1-button.png",
284             "map-2": "/Users/viniciusferrari/Documents/Richouse/Media/map-2-button.png"
285         }

```

I added two more backgrounds in `self.__backgrounds`, they are the start screen buttons, the map 1 and map 2 buttons. I used to already implement the buttons when start screen be loaded after menu selection.

```
302 def selection_general(self, key: int) -> None:
303     # Selection based on constant value of the key pressed
304     match key:
305         # Access the system object to access the preferences and after get the constant of up button
306         case self._system.preferences.buttonUp:
307             self.__selectedIndex = (self.__selectedIndex - 1) % len(self.__buttons) # Decrease the index
308             # without exceed the index
309         # Access the system object to access the preferences and after get the constant of down button
310         case self._system.preferences.buttonDown:
311             self.__selectedIndex = (self.__selectedIndex + 1) % len(self.__buttons) # Increase the index
```

This selection was developed as general because it's a reusable piece of code in more specific selections, such as the menu screen.

1. It gets only the constant of the key as parameter, not the whole event object.
2. A match statement selects the action based on the key integer constant
3. Subtract 1 to select up button
 - The button order in the list is upper first, so to access upper buttons we must decrease the index
 - The modulus ensure that the index limit of the list is met.
4. Add 1 to select down button
 - The button order in the list is upper first, so to access lower buttons we must increase the index
 - The modulus ensure that the index limit of the list is met.

```
314 def selection_menu(self, key: int) -> None:
315     # Uses the general selection for W and S buttons
316     self.selection_general(key=key)
317     # The RETURN key is responsible to confirm the selection
318     if key == pygame.K_RETURN:
319         # Selection based on the button selected
320         match self.__buttons[self.__selectedIndex][0]:
321             case "start":
322                 self._system.render_screen(nextScreen="start") # Render start screen
323             case "rank":
324                 self._system.render_screen(nextScreen="rank") # Render rank screen
325             case "options":
326                 self._system.render_screen(nextScreen="options") # Render rank screen
327             case "quit":
328                 pygame.quit() # Stop the game
```

After finishing the general selection, I was able to develop selections specific by screen. This is the menu selection, handling the access to other screen and game quit.

1. It uses the `selection_general()` to handle the W and S keys, up and down selection respectively.
2. I added the selection for RETURN key, it's essential to confirm the selection.
 - If start button is confirmed, it will render the start screen.
 - If rank button is confirmed, it will render the rank screen.
 - If options button is confirmed, it will render the options screen.
 - If quit button is confirmed, it will stop the game.

MAIN CHANGES FROM DESIGN

1. Two more buttons were added, the map 1 and map 2 that weren't designed before, they're essential to choose the map.

TESTING

- **Objective:** I can use the W and S to select buttons, and the ENTER to confirm it. Afterwards, the appropriate screen is rendered.
- **Blackbox Testing Data:**
 - **1 – W doesn't do anything.**
 - **2 – S doesn't do anything.**
 - **3 – ENTER confirms the button.**
 - **4 – All the other keys are ignored.**
- **Video:**
 - Menu-screen-input-fail
- **Syntax Errors:** N/A
- **Status:** **Fail, W and S aren't selecting because the default values were None.**

INTERACTION 2.2 – MENU SCREEN INPUT HANDLING

- **Objective:** I can use the W and S to select buttons, and the ENTER to confirm it. Afterwards, the appropriate screen is rendered.
- **Success Criteria:**
 - **2 – W and S are selecting up and down buttons in the menu screen.**
 - **3 – The options screen is rendered after the button be confirmed.**
 - **4 – The rank screen is rendered after the button be confirmed.**
 - **5 – The start buttons no longer start the game directly, instead it opens the map selection screen.**
 - **6 – The program stops when the quit button is confirmed.**
 - **15 – Only keyboard keys are used for selection and text input.**
- **Directories and files:**
 - **Distribution-code**

- Main.py
- Functions.py
- Config.py

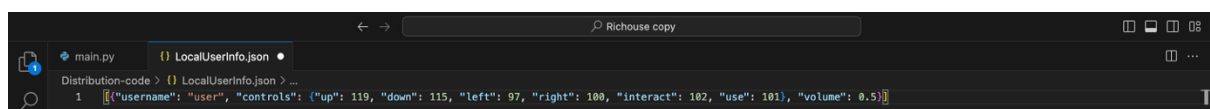
➤ **Videos:**

- Menu-screen-input-success

➤ **Tests Results: PASS**

➤ **Stakeholders:** add guidance in the map selection screen.

CODE



I inserted the integer value of the default keys to the local database.

1. W is represented by 119 and it's the up selection.
2. S is represented by 115 and it's the down selection.
3. A is represented by 97.
4. D is represented by 100.
5. F is represented by 102.
6. E is represented by 101.

TESTING

➤ **Objective:** I can use the W and S to select buttons, and the ENTER to confirm it. Afterwards, the appropriate screen is rendered.

➤ **Blackbox Testing Data:**

- **1 – W selects the up button, returning to the bottom when the top arrives.**
- **2 – S selects the down button, returning to the top when the bottom arrives.**
- **3 – ENTER confirms the button.**
- **4 – All the other keys are ignored.**

➤ **Video:**

- Menu-screen-input-success

➤ **Syntax Errors:** N/A

➤ **Status:** **PASS**

STAKEHOLDERS' FEEDBACK

- **Zach Ward:**

I recommend the addition of some visual feature to indicate how to return to the menu screen when you are in the map selection screen.

INTERACTION 3 – MAP SELECTION SCREEN

➤ **Objective:** Design a screen to select a map that contains guidance.

➤ **Success Criteria:** N/A

➤ **Directories and files:**

- **Media**
 - Map-screen.png

➤ **Videos:** N/A

➤ **Tests Results:** N/A

➤ **Stakeholders:** **Approved**

SCREEN DESIGN

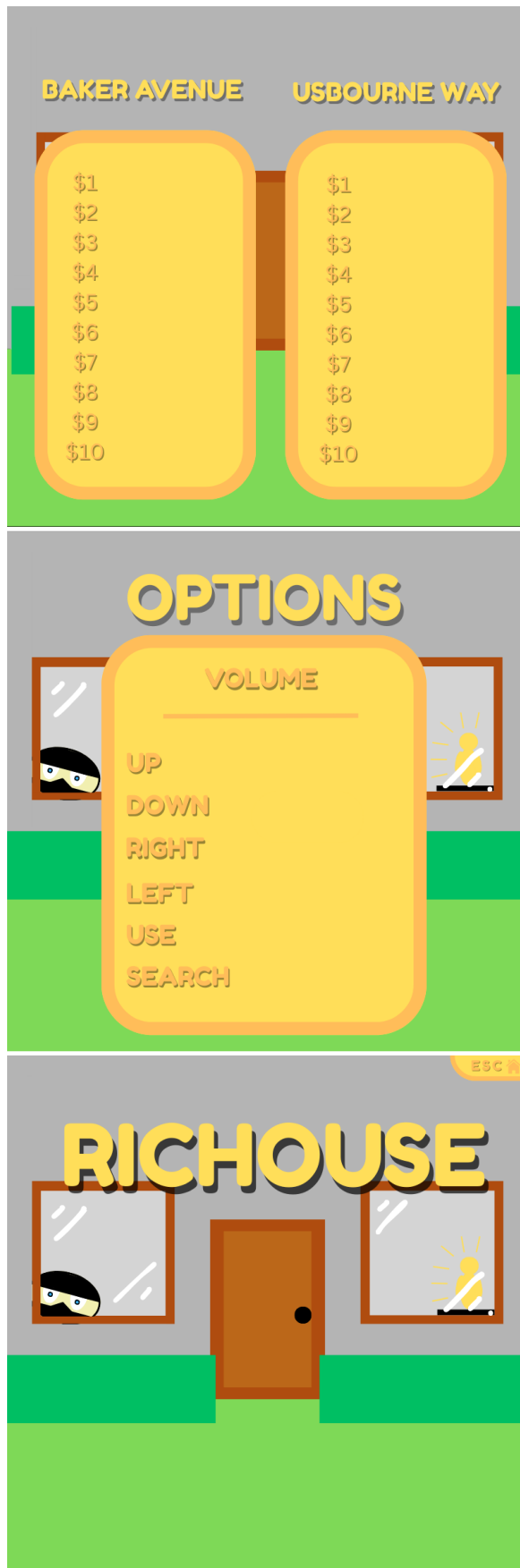


1. Same background of menu.
 - There is no need of a new art for a map selection screen because it's simple.
2. Included the ESC guidance in the top right.
 - I used a house to show that it returns to the menu/home screen.

PROTOTYPE

- **Stakeholders' feedback:** **Approved**
- **Video:**
 - Prototype-week-4

➤ Images:



WEEK 5 – USERNAME REGISTER, RANK AND OPTIONS SCREENS

- **Objective:** Develop the input handling for the username register and menu screens, including the rendering of the next screens.
- **Success Criteria:**
 - **7 – The rank screen opens and display the names.**
 - **8 – The player's life is no longer considered because many users finish with the same number.**
 - **9 - It returns to the menu**
 - **10 – The options screen is rendered and print all the preferences.**
 - **11 – W and S selects up and down, respectively.**
 - **12 – The volume bar increases with D and decreases with A.**
 - **14 – It returns to the menu.**
 - **15 – Only keyboard keys are being used so far.**
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
 - **Server-code**
 - App.py
 - GlobalDatabase.db
- **Progress Management:**
 - **3 - In progress**
 - **4 - In progress**
 - **5 - In progress**
 - **6 - In progress**
 - **18 - Completed**
 - **19 - Completed**
 - **35 - In progress**
- **Videos:**
 - Username-screen-server-1
 - Username-screen-server-2
 - Rank-screen-fail
 - Rank-screen-success
 - Options-screen-fail
 - Options-screen-success
 - Prototype-week-5

- **Tests Results: PASS**
- **Stakeholders' Approval: Approved**

INTERACTION 1.1 – USERNAME SERVER CHECK

- **Objective:** The register screen will only allow to take username that wasn't already taken.
- **Success Criteria:**
 - **15 – Only keyboard keys are being used so far.**
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
 - LocalUserInfo.json
 - **Server-code**
 - App.py
 - GlobalDatabase.db
- **Videos:**
 - Username-screen-server-1
- **Tests Results: PASS**
- **Stakeholders:** Change the colour of the message “Unavailable” to red.

CODE

```
522 def textbox_handling(system: System, key: int) -> None:
523     # Use the key int constant to choose the correct action
524     match key:
525         # RETURN key confirm the username
526         case pygame.K_RETURN:
527             if system.textbox != "":
528                 request = requests.post("http://127.0.0.1:5000/register", json={"username": system.textbox})
529                 status = request.json()["success"]
530                 if status:
531                     system.preferences.username = system.textbox # store username into preferences attributes
532                     system.preferences.save() # Save current preferences attributes into local database
533                     # Render the menu screen
534                     system.render_screen(nextScreen="menu")
535                 else:
536                     system.textbox = "Unavailable"
537             # BACKSPACE delete one character
538         case pygame.K_BACKSPACE:
539             system.textbox = system.textbox[:-1] # Create a new string cutting off the last character of textbox
540         # Any other character
541         case _:
542             if pygame.key.name(key).isalnum() and len(system.textbox) <= 16: # Limit the username to 16 alphanumeric characters
543                 system.textbox = system.textbox + pygame.key.name(key) # append the new character to the textbox
```

I just changed the part that saves the username into the database. I used the method `system.preferences.save()` instead because it will do basically the same of the piece of code present before, it also makes easier to debug.

```
6 @app.route("/register", methods=["POST"])
7 def new_user() -> Response:
8     # Get the username sent inside a dictionary, json style
9     data = request.get_json()
10    username = data["username"] # Access the username given that data is a dictionary
11    conn = sqlite3.connect("/Users/viniciusferrari/Documents/Richouse/Server-code/GlobalDatabase.db") # Connect with database
12    cursor = conn.cursor()
13    try:
14        cursor.execute('INSERT INTO user (username) VALUES (?)', (username,)) # Check with username already exist, as username is primary
15        # key it will return error if already exist
16        conn.commit()
17        conn.close()
18        return jsonify(success=True) # Return successful status
19    except:
20        conn.close()
21        return jsonify(success=False) # Return unsuccessful status
22
```

It will handle the `/register` GET request sent by distribution code.

1. `request.json()` will get the preferences in a JSON format.
2. `data["username"]` gets the username in the field of JSON format (dictionary)
3. `sqlite3.connect()` open the server database.
4. `cursor.execute()` tries to insert the new username into database, if it already exists it will return error because it's a primary key in `user` table.
 7. Return successful status if the username is inserted correctly.
 8. If the error rise, the exception will return the unsuccessful status.
 9. Remember that database must be closed before return anything.

TESTING

- **Objective:** The register screen will only allow to take username that wasn't already taken.
- **Blackbox Testing Data:** N/A
- **Video:**
 - Username-screen-server-1
- **Syntax Errors:** N/A
- **Status:** PASS

STAKEHOLDERS' FEEDBACK

- **Alex Michalski:**

The unavailability of the username would be clearer if the text is printed in a red colour.

INTERACTION 1.2 – USERNAME SERVER CHECK

- **Objective:** The register screen will only allow to take username that wasn't already taken.
- **Success Criteria:**
 - **15 – Only keyboard keys are being used so far.**
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
 - LocalUserInfo.json
 - **Server-code**
 - App.py
 - GlobalDatabase.db
- **Videos:**
 - Username-screen-server-2
- **Tests Results: PASS**
- **Stakeholders: Approved**

CODE

```
362 def display_textbox(self) -> None:
363     # Check if there is textbox
364     if self.__textbox is not None:
365         font = pygame.font.Font(None, 48) # Uses the standar font and size 48
366         if self.textbox == "Unavailable":
367             text_surface = font.render(self.__textbox, True, (255, 0, 0)) # Render the font with red colour
368         else:
369             text_surface = font.render(self.__textbox, True, (0, 0, 0)) # Render the font with black colour
370         text_rect = text_surface.get_rect(center=(400, 400)) # Create a surface for textbox
371         self.__screenObj.blit(text_surface, text_rect) # Print the surface
```

1. If statement check if the message is “Unavailable”:
 - It will be printed as red.
2. If it's not “Unavailable”, the text will be printed in black as before.

TESTING

- **Objective:** The register screen will only allow to take username that wasn't already taken.

- **Blackbox Testing Data:** N/A
- **Video:**
 - Username-screen-server-2
- **Syntax Errors:** N/A
- **Status:** PASS

INTERACTION 2.1 – RANK SCREEN

- **Objective:** The rank screen is rendered, and the username of the best performers are printed.
- **Success Criteria:**
 - **7 – The rank screen opens and display the names.**
 - **8 – The player's life is no longer considered because many users finish with the same number.**
 - **9 -It returns to the menu**
 - **15 – Only keyboard keys are being used so far.**
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
 - LocalUserInfo.json
 - **Server-code**
 - App.py
 - GlobalDatabase.db
- **Videos:**
 - Rank-screen-fail
- **Tests Results:** FAIL
- **Stakeholders:** N/A

CODE

```

72         case "rank":
73             if event.key == pygame.K_ESCAPE:
74                 system.render_screen(nextScreen="menu")

```

The `input ()` implementation for rank screen.

1. The rank screen uses a single selection of ESC key to allow user to return to menu screen.
 - o `system.render_screen ()` will render the menu screen.

```

357 def display_rank(self) -> None:
358     if self.__screenName == "rank":
359         # Request the JSON file to the server and extract the content from response
360         response = requests.get("http://127.0.0.1:5000/rank")
361         rank = response.json()
362
363         font = pygame.font.Font(None, 48) # Uses the standard font and size 48
364
365         # It will print the Baker Avenue ranking
366         baker = rank["Baker"] # Get the list for Baker Avenue rank
367         for x in range(len(baker)): # It will pass through all possible indexes
368             userB = baker[x] # Get element in that index, it has the format [username, time in seconds]
369             text_surface = font.render(f"{userB[0]} {str(userB[1])}", True, (0, 0, 0)) # Render the font with black colour
370             text_rect = text_surface.get_rect(center=(230, 270 + 50 * x)) # Create a surface for textbox, we can adjust the y-coordinate
371                                     # of each y-coordinate of each buttons based on its index
372             self.__screenObj.blit(text_surface, text_rect) # Print the surface
373
374         # It will print the Usbourne Way ranking
375         usbourne = rank["Usbourne"] # Get the list for Usbourne Way rank
376         for x in range(len(usbourne)): # It will pass through all possible indexes
377             userU = usbourne[x] # Get element in that index, it has the format [username, time in seconds]
378             text_surface = font.render(f"{userU[0]} {str(userU[1])}", True, (0, 0, 0)) # Render the font with black colour
379             text_rect = text_surface.get_rect(center=(630, 270 + 50 * x)) # Create a surface for textbox, we can adjust the y-coordinate
380                                     # of each y-coordinate of each buttons based on its index
381             self.__screenObj.blit(text_surface, text_rect) # Print the surface
382

```

The method that requests the ranking from server and prints the ranking of the best player performances in each make.

1. It sends a GET request to the server and store the response in rank.
 - o `request.get ()` send the request, in URL we can see the server IP (127.0.0.1), the port used (5000) and the service requested (/rank)
 - o `response.json ()` get the response data in JSON format.
2. Print the rankings separately.
 - o `rank []` access the list (rank) wanted.
 - o For loop passes through all indexes.
 - o `usbourne [x]` and `baker [x]` access the result in that index, smaller is the index higher is in ranking.
 - o `font.render ()` creates the text.
 - o `text_surface.rect ()` creates the place for text to be printed, here is why I used a for loop with indexes instead to uses the items directly, x works as part of the calculation of y-position, each button preference is separated with 50 pixels from another.
 - o `self.__screenObj.blit ()` print the text.

```

23 @app.route("/rank")
24 def best_score() -> Response:
25     conn = sqlite3.connect("/Users/viniciusferrari/Documents/Richouse/Server-code/GlobalDatabase.db") # connect with database
26     cursor = conn.cursor()
27     cursor.execute("SELECT user, time FROM results WHERE map = 'Baker Avenue' ORDER BY time LIMIT 10") # Get the best 10 scorers in
28                                                         # Baker Avenue
29     resultsBaker = cursor.fetchall()
30     cursor.execute("SELECT user, time FROM results WHERE map = 'Usbourne Way' ORDER BY time LIMIT 10") # Get the best 10 scorers in
31                                                         # Usbourne Way
32     resultsUsbourne = cursor.fetchall()
33     return jsonify({"Baker": resultsBaker, "Usbourne": resultsUsbourne}) # return both ranks in a json format

```

The function that handles the `/rank` POST request.

1. `sqlite3.connect()` opens the database.
2. `cursor.execute()` first get the ranking for Baker and after Usbourne limiting the output to 10.
3. `fetchall()` fetch the data from execution, it returns a list of tuples where each one is a row.
4. Return `jsonify()` make the data be returned as JSON.

MAIN CHANGES FROM DESIGN

1. I implemented the ranking and options printing code in separated methods to make the debugging easier.

TESTING

- **Objective:** The rank screen is rendered, and the username of the best performers are printed.
- **Blackbox Testing Data:**
 - **5 – Screen doesn't open**
 - **6 – Screen doesn't open**
- **Video:**
 - Rank-screen-fail
- **Syntax Errors:**
 - **IndexError: list index out of range.**
 - I used the object returned by `cursor.execute()` instead of the actual result.
 - To solve it, we just must use the `fetchall()` commands that fetches all the content.
- **Status: FAIL, the server returned a message 405.**

INTERACTION 2.2 – RANK SCREEN

- **Objective:** The rank screen is rendered, and the username of the best performers are printed.
- **Success Criteria:**
 - **7 – The rank screen opens and display the names.**
 - **8 – The player's life is no longer considered because many users finish with the same number.**
 - **9 -It returns to the menu**
 - **15 – Only keyboard keys are being used so far.**
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
 - LocalUserInfo.json
 - **Server-code**
 - App.py
 - GlobalDatabase.db
- **Videos:**
 - Rank-screen-success
- **Tests Results: PASS**
- **Stakeholders: Approved**

CODE

```
25 @app.route("/rank", methods=["GET"])
26 def ranking() -> Response:
27     conn = sqlite3.connect("/Users/viniciusferrari/Documents/Richouse/Server-code/GlobalDatabase.db") # connect with database
28     cursor = conn.cursor()
29     cursor.execute("SELECT user, time FROM results WHERE map = 'baker' ORDER BY time LIMIT 10") # Get the best 10 scorers in
30                                                     # Baker Avenue
31     resultsBaker = cursor.fetchall()
32     cursor.execute("SELECT user, time FROM results WHERE map = 'usbourne' ORDER BY time LIMIT 10") # Get the best 10 scorers in
33                                                     # Usbourne Way
34     resultsUsbourne = cursor.fetchall()
35     return jsonify({"Baker": resultsBaker, "Usbourne": resultsUsbourne}) # return both ranks in a json format
```

1. I added GET as an allowed method.
 - Now, when the distribution code asks for the rank, it can use the GET to request it from the server.

TESTING

- **Objective:** The rank screen is rendered, and the username of the best performers are printed.
- **Blackbox Testing Data:**
 - **5 – I can return to the menu screen when I press ESC.**
 - **6 – Any other input is ignored.**
- **Video:**
 - Rank-screen-success
- **Syntax Errors:** N/A
- **Status:** **PASS**

INTERACTION 3.1 – OPTIONS SCREEN

- **Objective:** The options screen is rendered, and the preferences can be seen and modified.
- **Success Criteria:**
 - **10 – The options screen is rendered and print all the preferences.**
 - **11 – W and S selects up and down, respectively.**
 - **12 – The volume bar increases with D and decreases with A.**
 - **14 – It returns to the menu.**
 - **15 – Only keyboard keys are being used so far.**
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
 - LocalUserInfo.json
- **Videos:**
 - Options-screen-fail
- **Tests Results:** **FAIL**
- **Stakeholders:** N/A

CODE

```
72         case "rank":
73             if event.key == pygame.K_ESCAPE:
74                 system.render_screen(nextScreen="menu")
75         case "options":
76             system.button.selection_options(event=event)
```

The `input()` implementation for the option screen.

1. Options screen uses specific selection.
 - o I implemented the selection as a method because it makes easier to debug than writing the code directly inside `input()` function.

```
42 class Preference:
43     def __init__(self, username: str="Guest", up: int=pygame.K_w, down: int=pygame.K_s, left: int=pygame.K_a,
44                 right: int=pygame.K_d, interact: int=pygame.K_f, use: int=pygame.K_e, volume: int=0.5):
45         self.__buttonUp = up
46         self.__buttonDown = down
47         self.__buttonLeft = left
48         self.__buttonRight = right
49         self.__buttonInteract = interact
50         self.__buttonUse = use
51         self.__volume = volume
52         self.__username = username
53
54     def __dir__(self):
55         return (pygame.key.name(self.__buttonUp), pygame.key.name(self.__buttonDown),
56                 pygame.key.name(self.__buttonRight), pygame.key.name(self.__buttonLeft),
57                 pygame.key.name(self.__buttonUse), pygame.key.name(self.__buttonInteract))
```

I added new attributes into `Preference` and created a method to return the buttons preferences.

1. 2 new attributes:
 - o `self.__buttonUse` store the buttons preferences to use items, it allows user to choose their own button.
 - o `self.__buttonInteract` store the buttons preferences to interact with map, it allows user to choose their own button.
2. `__dir__` method is the personalized method to return the buttons preferences.
 - o Use a list to return every button preference.
 - o It can be accessed trough `preferences.__dir__` that was essential to print the buttons preferences in options screen.

```

94 # Getter method for self.__username
95 @property
96 def username(self):
97     return self.__username
98
99 # Setter method for self.__buttonUp
100 @username.setter
101 def username(self, username: str):
102     if username is not None:
103         self.__username = username
104
105 # Setter method for self.__buttonUp
106 @buttonUp.setter
107 def buttonUp(self, key: int):
108     if key is not None:
109         if pygame.key.name(key).isalpha():
110             self.__buttonUp = key
111
112 # Setter method for self.__buttonDown
113 @buttonDown.setter
114 def buttonDown(self, key: int):
115     if key is not None:
116         if pygame.key.name(key).isalpha():
117             self.__buttonDown = key
118
119 # Setter method for self.__buttonRight
120 @buttonRight.setter
121 def buttonRight(self, key: int):
122     if key is not None:
123         if pygame.key.name(key).isalpha():
124             self.__buttonRight = key
125
126 # Setter method for self.__buttonLeft
127 @buttonLeft.setter
128 def buttonLeft(self, key: int):
129     if key is not None:
130         if pygame.key.name(key).isalpha():
131             self.__buttonLeft = key
132
133 # Setter method for self.__buttonUse
134 @buttonUse.setter
135 def buttonUse(self, key: int):
136     if key is not None:
137         if pygame.key.name(key).isalpha():
138             self.__buttonUse = key
139
140 # Setter method for self.__buttonInteract
141 @buttonInteract.setter
142 def buttonInteract(self, key: int):
143     if key is not None:
144         if pygame.key.name(key).isalpha():
145             self.__buttonInteract = key
146
147 # Setter method for self.__volume
148 @volume.setter
149 def volume(self, value: int):
150     if value is not None:
151         if value >= 0 and value <= 1:
152             self.__volume = value

```

I added getter and setter methods needed to access and manipulate the variables outside the class Preferences.

1. Getter method for `self.__username` using `@property` decorator.
2. Setter methods for all attributes using `@name.setter` where name is the name of method with `@property` decorator.
 - The key preferences are settled only if it's not null and it's an alphabetic character, it avoids the use of dangerous characters, i.e. ``` and ``` that are used in programming to create strings.
 - The `self.__volume` setter method allows only volumes different of null and between 0 and 1, it because it must be a factor of the original audio that is already loud.

```

154     def save(self) -> None:
155         # Get the user data from database
156         with open("/Users/viniciusferrari/Documents/Richouse/Distribution-code/LocalUserInfo.json", "r") as f:
157             userData = json.load(f)
158
159         # Modify database to register the username
160         userData[0]["username"] = self.__username
161         userData[0]["controls"]["up"] = self.__buttonUp
162         userData[0]["controls"]["down"] = self.__buttonDown
163         userData[0]["controls"]["left"] = self.__buttonLeft
164         userData[0]["controls"]["right"] = self.__buttonRight
165         userData[0]["controls"]["use"] = self.__buttonUse
166         userData[0]["controls"]["interact"] = self.__buttonInteract
167         userData[0]["volume"] = self.__volume
168
169         # Write the changes in database
170         with open("/Users/viniciusferrari/Documents/Richouse/Distribution-code/LocalUserInfo.json", "w") as f:
171             json.dump(userData, f)

```

The method that saves the current preferences into the database to allow it to be remembered during next sessions.

1. Get the data from the database.
 - o `open()` opens the file with the reading method.
 - o `json.load()` load the content of the opened file.
2. Update all preferences of the file with the current ones.
 - o It's why I created the setter methods in `Preference`.
3. Write the data to the database again.
 - o `open()` now uses the writing method.
 - o `json.dump()` write the data in the JSON file.

```

269         case "options":
270             # Create the arrow for options selection, the format is [name, [x-coordinate, y-coordinate]]
271             self.__button = Button(buttons=[["arrow", [590, 275]]], system=self)
272             # Load the username-screen that is the options screen, here user can change the predefined keys and volume
273             self.__background = pygame.image.load("/Users/viniciusferrari/Documents/Richouse/Media/options-screen.png").convert_alpha()

```

The options screen selection in `render_screen()` method in `System` class, I used the `Button` class to create an arrow, as I will use only once it's better to just adapt the functionality with existent class.

1. Use an arrow instead of buttons for each preference.
 - o It would be harder to create an image for each element because the button preferences can be any of the 26 letters in the alphabet.
2. The coordinate is now passed as a list instead of tuple, the reason is that the adaptation uses the arrow y-coordinate to determine where it will be printed and tuples aren't mutable.

```

393     def display_options(self) -> None:
394         if self.__screenName == "options":
395             # Create a rectangle 10x20 and print it in the middle of volume bar
396             rect = pygame.Rect(242 + 300 * self.__preferences.volume, 275, 10, 20) # The position in bar is defined by volume,
397             # that is a factor of original volume
398             pygame.draw.rect(self.__screenObj, (235, 165, 59), rect) # Print with peach colour
399
400             # Print each button current preference in the correct place
401             keys = self.__preferences.__dir__() # Return the lists of button preferences (up, down, right, left, use, interaction)
402             font = pygame.font.Font(None, 48) # Uses the standard text and size 48
403             for x in range(6): # Pass through the index of all buttons preferences in the list
404                 key = keys[x] # Get the button preference in index x
405                 text_surface = font.render(key, True, (235, 165, 59)) # Render the font with peach colour
406                 text_rect = text_surface.get_rect(center=(570, 350 + 70 * x)) # Create a surface for textbox
407                 self.__screenObj.blit(text_surface, text_rect) # Print the text

```

The method to display the current preferences in options screen.

1. Check if the contest should be printed.

2. Draw a 10x20 rectangle to mark the current volume in the volume bar.
 - o `pygame.Rect()` create the rectangle, I used the `self.__preferences.volume` as the fraction of the line that rectangle should be printed, it ranges from 0 to 1.
 - o `pygame.draw.rect()` draw the rectangle with peach color to contrast with the background.
3. Get the current button preferences and print them.
 - o `self.__preferences.__dir__` uses the personalized method to return the current button preferences.
 - o A for loop pass through all indexes of the list returned.
 - o `keys[x]` access the one the preferences.
 - o In `text_surface.get_rect()` the index `x` is used to find the position that preference should be printed each one is horizontally separated by 70 pixels.

```

388 class Button:
389     def __init__(self, buttons: list, system: System):
390         self.__buttons = buttons
391         self.__selectedButtonIndex = 0
392         self.__system = system
393         self.__backgrounds = {
394             "start": "/Users/viniciusferrari/Documents/Richouse/Media/start-button.png",
395             "rank": "/Users/viniciusferrari/Documents/Richouse/Media/rank-button.png",
396             "options": "/Users/viniciusferrari/Documents/Richouse/Media/options-button.png",
397             "quit": "/Users/viniciusferrari/Documents/Richouse/Media/quit-button.png",
398             "again": "/Users/viniciusferrari/Documents/Richouse/Media/again-button.png",
399             "menu-pause": "/Users/viniciusferrari/Documents/Richouse/Media/menu-pause-button.png",
400             "menu-menu": "/Users/viniciusferrari/Documents/Richouse/Media/menu-menu-button.png",
401             "continue": "/Users/viniciusferrari/Documents/Richouse/Media/continue-button.png",
402             "baker": "/Users/viniciusferrari/Documents/Richouse/Media/baker-button.png",
403             "map-2": "/Users/viniciusferrari/Documents/Richouse/Media/map-2-button.png",
404             "arrow": "/Users/viniciusferrari/Documents/Richouse/Media/arrow.png"
405         }

```

The arrow background was added to Button, as we see it's times easier than implement a way to make each preference as a button because pygame use 2 functions to create the text and 1 to print, therefore it would need changes in function such as print buttons making it unreliable.

```

483 def selection_options(self, event: pygame.event.Event) -> None:
484     # Selection case the SHIFT is held
485     if event.mod & pygame.KMOD_SHIFT:
486         # Selection with the y coordinate, buttons has the format [[button name, [x, y]]] in this case
487         match self.__buttons[0][1][1]:
488             case 345: # y-position of up button preference
489                 self.__system.preferences.buttonUp = event.key # Change the up button preference
490             case 415: # y-position of down button preference
491                 self.__system.preferences.buttonDown = event.key # Change the down button preference
492             case 485: # y-position of right button preference
493                 self.__system.preferences.buttonRight = event.key # Change the right button preference
494             case 555: # y-position of left button preference
495                 self.__system.preferences.buttonLeft = event.key # Change the left button preference
496             case 625: # y-position of use button preference
497                 self.__system.preferences.buttonUse = event.key # Change the use button preference
498             case 695: # y-position of interaction button preference
499                 self.__system.preferences.buttonInteract = event.key # Change the interact button preference
500         else:
501             match event.key: # Use the integer constant of the key to select
502                 case self.__system.preferences.buttonUp: # up selection
503                     new = self.__buttons[0][1][1] - 70 # find the new y position for the arrow
504                     self.__buttons[0][1][1] = new # set the new preference
505                 case self.__system.preferences.buttonDown: # down selection
506                     new = self.__buttons[0][1][1] + 70 # find the new y position for the arrow
507                     self.__buttons[0][1][1] = new # set the new preference
508                 case self.__system.preferences.buttonLeft: # left selection
509                     if self.__buttons[0][1][1] == 275: # It will restrict it to volume preference
510                         self.__system.preferences.volume -= 0.05 # decrease the volume by 0.05, min of 0
511                 case self.__system.preferences.buttonRight: # right selection
512                     if self.__buttons[0][1][1] == 275: # restrict it to volume preference
513                         self.__system.preferences.volume += 0.05 # increase the volume by 0.05, max of 1
514                 case pygame.K_ESCAPE: # ESC selection
515                     self.__system.preferences.save() # Method to save the current preferences in the local database
516                     pygame.mixer.music.set_volume(self.__system.preferences.volume) # Use the new volume
517                     self.__system.render_screen(nextScreen="menu") # Return to menu

```

As the arrow is an adaption of a button, it's selection is very different from the other selections that used the `selection_general()` method.

1. The first case is if the SHIFT is held, it has a different selection because SHIF + KEY is the command to change a key preference.
 - It selects based on y-coordinate of arrow, because of it the `self.__buttons[0][1][1]` is used given that `self.__buttons` format would be `[["arrow", [x, y]]]`.
 - The setter methods of different buttons are use depending on the y-coordinate.
2. The second case is if the SHIF isn't held.
 - Now selection is based on the key pressed.
 - If it's the up-selection button, it will decrement the y-coordinate by 70, but only if it's not equal the first preference y-coordinate. Remember that pygame count pixels from top left.
 - If it's the down-selection button, it will increment the y-coordinate by 70, but only if it's not equal the first preference y-coordinate. Remember that pygame count pixels from top left.
 - The left- and right-button selection are limited to volume preference, that has y-coordinate 275, and decrease and increase volume by 0.05, respectively.
 - The ESC is used to return to menu screen, but first the preferences are saved using `self.__system.preferences.save()` that save everything in database for next sessions.

TESTING

- **Objective:** The options screen is rendered, and the preferences can be seen and modified.
- **Blackbox Testing Data:**
 - **7 – W selects up but using an arrow instead of buttons because it's unreliable to develop one button per possible preference. However, it doesn't stop when the top arrives.**
 - **8 – S selects down but using an arrow instead of buttons. However, it doesn't stop when the bottom arrives.**
 - **9 – A decreases the volume properly.**
 - **10 – D increases the volume properly.**
- **Video:**
 - Options-screen-fail

➤ **Syntax Errors:**

- **TypeError: text must be a unicode or bytes.**
 - I used the integers values to store the keys, so they are integers representing a letter.
 - It's solved by using `pygame.key.name()` that return the key based on the integer value inputted.
- **TypeError: can only concatenate tuple (not "int") to tuple.**
 - I used the wrong syntax to access the y-coordinate of button, it's an 3D collection instead of 2D.
 - I solved by adding a third index. Remembering that the coordinate of arrow in this case should be stored in a list instead of tuple because its coordinate must be mutable.

➤ **Status: FAIL, the arrow continues to go up/down even when there is no preference there.**

INTERACTION 3.2 – OPTIONS SCREEN

➤ **Objective:** The options screen is rendered, and the preferences can be seen and modified.

➤ **Success Criteria:**

- **10 – The options screen is rendered and print all the preferences.**
- **11 – W and S selects up and down, respectively.**
- **12 – The volume bar increases with D and decreases with A.**
- **14 – It returns to the menu.**
- **15 – Only keyboard keys are being used so far.**

➤ **Directories and files:**

- **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
 - LocalUserInfo.json

➤ **Videos:**

- Options-screen-success

➤ **Tests Results: PASS**

➤ **Stakeholders: Approved**

CODE

```
483 def selection_options(self, event: pygame.event.Event) -> None:
484     # Selection case the SHIFT is held
485     if event.mod & pygame.KMOD_SHIFT:
486         # Selection with the y coordinate, buttons has the format [[button name, [x, y]] in this case
487         match self.__buttons[0][1][1]:
488             case 345: # y-position of up button preference
489                 self.__system.preferences.buttonUp = event.key # Change the up button preference
490             case 415: # y-position of down button preference
491                 self.__system.preferences.buttonDown = event.key # Change the down button preference
492             case 485: # y-position of right button preference
493                 self.__system.preferences.buttonRight = event.key # Change the right button preference
494             case 555: # y-position of left button preference
495                 self.__system.preferences.buttonLeft = event.key # Change the left button preference
496             case 625: # y-position of use button preference
497                 self.__system.preferences.buttonUse = event.key # Change the use button preference
498             case 695: # y-position of interaction button preference
499                 self.__system.preferences.buttonInteract = event.key # Change the interact button preference
500         else:
501             match event.key: # Use the integer constant of the key to select
502                 case self.__system.preferences.buttonUp: # up selection
503                     new = self.__buttons[0][1][1] - 70 # find the new y position for the arrow
504                     if new >= 275: # check if it isn't below the y position of the first preference
505                         self.__buttons[0][1][1] = new # set the new preference
506                 case self.__system.preferences.buttonDown: # down selection
507                     new = self.__buttons[0][1][1] + 70 # find the new y position for the arrow
508                     if new <= 695: # check if it isn't above the y position of the last preference
509                         self.__buttons[0][1][1] = new # set the new preference
510                 case self.__system.preferences.buttonLeft: # left selection
511                     if self.__buttons[0][1][1] == 275: # It will restrict it to volume preference
512                         self.__system.preferences.volume -= 0.05 # decrease the volume by 0.05, min of 0
513                 case self.__system.preferences.buttonRight: # right selection
514                     if self.__buttons[0][1][1] == 275: # restrict it to volume preference
515                         self.__system.preferences.volume += 0.05 # increase the volume by 0.05, max of 1
516                 case pygame.K_ESCAPE: # ESC selection
517                     self.__system.preferences.save() # Method to save the current preferences in the local database
518                     pygame.mixer.music.set_volume(self.__system.preferences.volume) # Use the new volume
519                     self.__system.render_screen(nextScreen="menu") # Return to menu
```

Using the Pygame count of pixels, starting on top left, I used if statements to block the arrow movement in certain points.

1. Added a limitation to the pixel 275.
 - The arrow can't be moved higher than it.
2. Added a limitation to the pixel 695.
 - The arrow can't be moved lower than it.

TESTING

- **Objective:** The options screen is rendered, and the preferences can be seen and modified.
- **Blackbox Testing Data:**
- **7 – W selects up but using an arrow instead of buttons because it's unreliable to develop one button per possible preference.**
 - **8 – S selects down but using an arrow instead of buttons.**
 - **9 – A decreases the volume properly.**
 - **10 – D increases the volume properly.**
- **Video:**
- Options-screen-success

- **Syntax Errors:** N/A

- **Status:** **PASS**

PROTOTYPE

- **Stakeholders' feedback:** **Approved**
- **Video:**
 - Prototype-week-5
- **Images:** N/A

WEEK 6 - PLAYER, INTERACTIONS, STATUSES AND END GAME

- **Objective:** Develop the player movement and interactions while the statuses are printed. Moreover, when the statue in the inventory and player touch the front door, the game finishes.
- **Success Criteria:**
 - **15 – Only keyboard keys are being used so far.**
 - **16 – The movement is working correctly.**
 - **17 – F searches the places.**
 - **18 – The three items are being appended to the inventory.**
 - **28 – Bar is at the top of the screen.**
 - **29 – Player's life is printed.**
 - **30 – Time taken is updated and printed.**
 - **31 – The game finishes when the player touches the front door with the statue in the inventory.**
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
 - LocalUserInfo.json
 - **Server-code**
 - App.py
 - GlobalDatabase.db

- **Progress Management:**
 - 2 - Completed
 - 3 - In progress
 - 4 - In progress
 - 5 - Completed
 - 6 - Completed
 - 7 - In progress
 - 12 - Completed
 - 13 - Completed
 - 15 - Completed
 - 17 - Completed
 - 35 - In progress
- **Videos:**
 - Player-movement
 - Searching-fail
 - Searching-success-1
 - Searching-success-2
 - Mission-completed-fail
 - Mission-completed-success
 - Prototype-week-6
- **Tests Results: PASS**
- **Stakeholders' Approval: Approved**

INTERACTION 1 – PLAYER MOVEMENT

- **Objective:** Player can move around the map and collide with the walls.
- **Success Criteria:**
 - 15 – Only keyboard keys are being used so far.
 - 16 – The movement is working correctly.
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py

- **Videos:**
 - Player-movement
- **Tests Results: PASS**
- **Stakeholders: Approved**

CODE

```
124     if system.screenName in ["baker", "usbourne"]: # Limit the action to gameplay screens
125         system.gameplay.player.movement() # Player Method
```

Handling for the movement added to the input function.

1. Player movement is handled only when it's one of the maps' screens.
 - It can't be inside the KEYDOWN if statement because it must be continuous.
 - It must be limited to the gameplay screen because the player doesn't exist in the other ones.

```
209     # Getter method for self.__gameplay
210     @property
211     def gameplay(self):
212         return self.__gameplay
```

I created a method to enable the access to the gameplay private attribute of `System` class, it will be used in `Player` methods, for example.

```
256         case "baker":
257             if self.__gameplay is None: # Only creates a new gameplay if there isn't already one
258                 timeNow = datetime.datetime.now() # Get the current time
259                 player = Player(system=self, coordinate=config.bakerInitialPosition) # Create the player sprite
268         case "usbourne":
269             if self.__gameplay is None: # Only creates a new gameplay if there isn't already one
270                 timeNow = datetime.datetime.now() # Get the current time
271                 player = Player(system=self, coordinate=config.usbourneInitialPosition) # Create the guard sprite
322     self.display_sprites() # Display the sprites (Baker Avenue and Usbourne Way screens)
```

I developed the initialization of the map screens inside the `render_screen()` method of `System`. Moreover, the methods to display the sprites.

1. The time of the start of the mission is recorded using `datetime.datetime.now()`.
2. The player sprite object is created using `Player` class, it has exclusive attributes for the main character.
3. A method is used to display the sprite of the player.

```

411     def display_sprites(self) -> None:
412         if self.__screenName in ["baker", "usbourne"]: # Only displays sprites during gameplay screens
413             # ...
414             self.__screenObj.blit(self.__gameplay.player.image, self.__gameplay.player.rect) # Print the main character

```

I added the `display_sprites()` method inside the `System` class to display the player and statue sprites on the screen.

1. Only displayed in the maps screen.
2. Player sprite is displayed.
 - It must include both image and hitbox of the sprite (created in the `Player` class).

```

624 class Player(Sprite):
625     def __init__(self, system: System, coordinate: tuple):
626         super().__init__(image=config.mainSpritePath, coordinate=coordinate)
627         self.__lives = 3
628         self.__inventory = []
629         self.__system = system
630
631         # Getter method for self.__inventory
632         @property
633         def inventory(self):
634             return self.__inventory
635
636         # Getter method for self.__lives
637         @property
638         def lives(self):
639             return self.__lives
640
641         @lives.setter
642         def lives(self, value: int):
643             if value is not None:
644                 self.__lives = value

```

The `Player` class inherits the `Sprite` class and extends its attributes to fit in the needs for the player sprite. It adds the attributes `lives`, `inventory` and `system`.

1. `self.__lives` stores the total player lives, allow to know when they lose the mission.
2. `self.__inventory` stores the items that player found, they are essential to start the future minigames.
3. `self.__system` stores the system object, it allows us to use the `System` class methods inside `Player` class methods.
4. A getter and a setter method is used to access and securely modify the private attribute `lives` into other functions or other classes methods, it doesn't allow to set it to `None`.

```

646 def movement(self) -> None:
647     keys = pygame.key.get_pressed() # Provide a dict that says if key is pressed or not, uses True and False values
648     if keys[self.__system.preferences.buttonUp]: # If up button is being pressed
649         self.rect.y -= 4 # Velocity of 4 pixels
650         wall = self.check_collision() # Check if collided with something
651         if wall is not None: # Only if collided
652             self.rect.top = wall.bottom # The player will be blocked to pass through the wall
653     if keys[self.__system.preferences.buttonDown]: # If down button is being pressed
654         self.rect.y += 4 # Velocity of 4 pixels
655         wall = self.check_collision() # Check if collided with something
656         if wall is not None: # Only if collided
657             self.rect.bottom = wall.top # The player will be blocked to pass through the wall
658     if keys[self.__system.preferences.buttonRight]: # If right button is being pressed
659         self.rect.x += 4 # Velocity of 4 pixels
660         wall = self.check_collision() # Check if collided with something
661         if wall is not None: # Only if collided
662             self.rect.right = wall.left # The player will be blocked to pass through the wall
663     if keys[self.__system.preferences.buttonLeft]: # If left button is being pressed
664         self.rect.x -= 4 # Velocity of 4 pixels
665         wall = self.check_collision() # Check if collided with something
666         if wall is not None: # Only if collided
667             self.rect.left = wall.right # The player will be blocked to pass through the wall

```

This method inside Player class handles the keys used for movement during the maps screen. Any collisions are checked in each movement.

1. This function is used outside the event loop because it considers the key being hold, not only clicked. So, `pygame.key.get_pressed()` return a list of the states of the keys.
2. If the up button is pressed, then the player y position decreases by 4 because the Pygame start to count the pixels in the top left of the screen.
3. If the down button is pressed, then the player y position increases by 4 because the Pygame start to count the pixels in the top left of the screen.
4. If the right button is pressed, then the player x position increases by 4 because the Pygame start to count the pixels in the top left of the screen.
5. If the up button is pressed, then the player x position decrease by 4 because the Pygame start to count the pixels in the top left of the screen.
6. I used 4 pixels in each movement above because it was the best speed for the map.
7. Each movement the collision is checked, if it happens the player coordinate, x or y depending on the command above, is set to the border of the collided object.

```

741 class Gameplay:
742     def __init__(self, map: str, startTime: datetime.timedelta, walls: list, searchablePlaces: list, player: Player,
743                 guard: Guard, laser: Sprite, system: System, statueCoordinate: tuple, door: pygame.Rect,
744                 codeDoor: Sprite, lockDoor: Sprite, server: pygame.rect.Rect):
745         self.__map = map
746         self.__startTime = startTime
747         self.__walls = walls.copy()
748
749         self.__player = player
750
751
752
753
754
755         self.__system = system
756
757
758
759     # Getter method for self.__walls
760     @property
761     def walls(self):
762         return self.__walls
763
764     # Getter method for self.__player
765     @property
766     def player(self):
767         return self.__player
768
769     # Getter method for self.__map
770     @property
771     def map(self):
772         return self.__map
773
774     # Getter method for self.__startTime
775     @property
776     def startTime(self):
777         return self.__startTime

```

The `Gameplay` class is the responsible to store the states of the gameplay. As the `Sprite` and `System` classes needs their attributes, multiple getter and setter method were implemented for a secure access and modification of the attribute.

1. `self.__map` store the map name, used to identify the correct action according to the map name.
2. `self.__startTime` stores the beginning time, it's used to find the time taken.
3. `self.__player` stores the main character sprite, it's controlled by user.
4. `self.__system` stores the system object, it's used to access `System` methods inside the `Gameplay` methods.

```

669     def check_collision(self) -> pygame.Rect:
670         for wall in self.__system.gameplay.walls: # Access each wall in the list of the map walls
671             if self.rect.colliderect(wall): # Check if collided with some wall in the list
672                 return wall # Return the wall collided

```

The method inside `Player` class responsible to check for any collision between the player sprite and the map components.

1. For loop is used to access all the map's walls, as consequence the collision with walls can be checked individually and it can be returned. It returns the wall because the player position will be set to the edge of wall when collide.

MAIN CHANGES FROM DESIGN

1. There is a collision checker method.

TESTING

- **Objective:** Player can move around the map and collide with the walls.
- **Blackbox Testing Data:**
 - 48 – W moves the character up, but a speed of 4 pixels suits better than 5.
 - 49 – A moves the character left.
 - 50 – S moves the character down.
 - 51 – D moves the character right
 - 54 – The other keys are ignored.
- **Video:**
 - Player-movement
- **Syntax Errors:**
 - **AttributeError: 'NoneType' object has no attribute 'player'**
 - Player movement method was in the loop of the other screens, so the errors arise because it doesn't exist outside the gameplay.
 - I fixed by adding it to the if statement for gameplay screens in the input function.
 - **AttributeError: 'Player' object has no attribute '_Player__rect'.**
 - I tried to access the inherited private attribute without a getter method.
 - I created a getter method using the @property decorator.
- **Status: PASS**

INTERACTION 2.1 – SEARCHING AND INVENTORY

- **Objective:** Searchable place is searched and item appended to the inventory.
- **Success Criteria:**
 - 17 – F doesn't do anything
 - 18 – The items aren't being appended.
- **Directories and files:**
 - Distribution-code

- Main.py
- Functions.py
- Config.py

➤ **Videos:**

- Searching-fail

➤ **Tests Results: FAIL**

➤ **Stakeholders: N/A**

CODE

```

702 def interact(self) -> None:
703     searchablePlaces = self.__system.gameplay.searchablePlaces # The places where there are items
704     for searchable in searchablePlaces:
705         if self.rect.colliderect(searchablePlaces[searchable]): # Check collision with the searchable place
706             match searchable: # The item to be added in the inventory depends on the place
707                 # The lockpick is inside the fridge
708                 case "fridge":
709                     if "Lockpick" not in self.__inventory:
710                         self.__inventory.append("Lockpick")
711                     return
712                 # The UV light is inside the tv support
713                 case "tv":
714                     if "UV light" not in self.__inventory:
715                         self.__inventory.append("UV light")
716                     return
717                 # The chip is inside the desk
718                 case "desk":
719                     if "Chip" not in self.__inventory:
720                         self.__inventory.append("Chip")

```

It's inside `Player` class and it's responsible for interactions with the searchable places. It's only executed when the interaction key is pressed in the `input()` function.

1. For loop is used to access all the searchable places and individually check the collisions with user. As a dictionary, each item is the key, so we need to use `searchablePlaces[searchable]` to access the actual hitbox.
2. A selection is used to decide the item to be appended in the inventory by using the place's name. In all maps, the lockpick is inside the fridge, the UV light is in the tv hack and the chip in the desk; the standardization makes the development easier, no random choice algorithm needed.

```

47 def game_loop(system: System, clock: pygame.time.Clock) -> None:
48     while True:
49         # Update screen elements, i.e. background, buttons and textbox.
50         system.update_screen()
51         if system.screenName in ["baker", "usbourne", "hacking", "lockpick", "fingerprint", "pause"]: # Only happen in gameplay screens
52             # Update inventory
53             system.gameplay.player.display_inventory() # Display the inventory
54             # Update the inventory
55             # ...
56         input(system=system) # Where the input will be handled
57         pygame.display.flip() # Change the frame
58         clock.tick(60) # Limit the fps to 60

```

I added the `display_inventory()` methods from `Gameplay` class inside the `game_loop()` function.

1. `display_inventory()` displays the items in the player's inventory.
 - o It allows the items to be shown inside the inventory box.

```
87         case "baker" | "usbourne":
88             match event.key:
89                 case system.preferences.buttonInteract: # Button used to search
90                     system.gameplay.player.interact() # Player method
```

I added the commands during maps screen.

1. `interact()` method is used when the interaction button is pressed to search.
 - o It will try to search the place.

```
733 def display_inventory(self) -> None:
734     if self.__system.screenName in ["baker", "usbourne"]: # Only prints on gameplay screens
735         font = pygame.font.Font(None, 25)
736         for index in range(len(self.__inventory)): # Range of all indexes of the inventory
737             text = font.render(self.__inventory[index], True, (235, 165, 59))
738             rect = text.get_rect(topleft=(633, 644 + 35 * index)) # The index is used to create a spacing between printed items
739             self.__system.screenObj.blit(text, rect)
```

It's the only display method that isn't inside `System` class, but in `Player` class. The inventory is only printed in the maps screen, so it's easier to implement directly as a player attribute.

1. It's only printed in the maps screen.
2. It uses the standard font with size 25.
3. The index of all items are accessed by the for loop, each time the item is printed in peach colored text and with a spacing of 35 of each other, because of it the index multiplied by 35 is added to the y position.

```
741 class Gameplay:
742     def __init__(self, map: str, startTime: datetime.timedelta, walls: list, searchablePlaces: list, player: Player,
743                 guard: Guard, laser: Sprite, system: System, statueCoordinate: tuple, door: pygame.Rect,
744                 codeDoor: Sprite, lockDoor: Sprite, server: pygame.rect.Rect):
745         self.__map = map
746         self.__startTime = startTime
747         self.__walls = walls.copy()
748         self.__searchablePlaces = searchablePlaces
749         self.__player = player
750
751         # ...
752
753         --
754
755         self.__system = system
756
757     # Getter method for self.__searchablePlaces
758     @property
759     def searchablePlaces(self):
760         return self.__searchablePlaces
```

1. `self.__searchablePlaces` stores all the places that can have items, used to use the search to find the minigame items.

- It can be access using a getter method developed using the @property decorator.

MAIN CHANGES FROM DESIGN

2. Implementation of a method, inside Player class, to display the inventory.

TESTING

- **Objective:** Searchable place is searched and item appended to the inventory.
- **Blackbox Testing Data:**
 - **52 – F is pressed in a searchable place, but nothing happens.**
 - **54 – The other keys are ignored.**
- **Video:**
 - Searching-fail
- **Syntax Errors:** N/A
- **Status:** **FAIL, the hitbox of searchable is in the same size of the walls, therefore player won't touch it.**

INTERACTION 2.2 – SEARCHING AND INVENTORY

- **Objective:** Searchable place is searched and item appended to the inventory.
- **Success Criteria:**
 - **17 – F searches the places.**
 - **18 – The three items are being appended to the inventory.**
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
- **Videos:**
 - Searching-success-1

- **Tests Results: PASS**
- **Stakeholders:** Don't print the player above the inventory.

CODE

```
app.py server code
main.py Distribution-code
73 bakerSearchable = {"desk": pygame.Rect(562, 5, 69, 77),
74                  "sink": pygame.Rect(238, 206, 60, 102),
75                  "tv": pygame.Rect(5, 437, 66, 192),
76                  "fridge": pygame.Rect(431, 695, 89, 85)}

usbourneSearchable = {"desk": pygame.Rect(5, 241, 85, 94),
93                  "fridge": pygame.Rect(469, 193, 92, 86),
94                  "tv": pygame.Rect(469, 727, 192, 52),
95                  "sink": pygame.Rect(213, 418, 102, 60)}
```

1. Create specific hitboxes for each of the searchable places.
 - They are slightly larger than the walls.
 - Divided per type.

TESTING

- **Objective:** Searchable place is searched and item appended to the inventory.
- **Blackbox Testing Data:**
 - **52 – F searches the place, and the item is appended to the inventory.**
 - **54 – The other keys are ignored.**
- **Video:**
 - Searching-success-1
- **Syntax Errors:** N/A
- **Status: PASS**

STAKEHOLDERS' FEEDBACK

- **Enzo Garofani:**

I would add a feature to don't allow the player be printed above the inventory.

INTERACTION 2.3 – SEARCHING AND INVENTORY

- **Objective:** Searchable place is searched and item appended to the inventory.
- **Success Criteria:**

- **17 – F searches the places.**
- **18 – The three items are being appended to the inventory.**
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
- **Videos:**
 - Searching-success-2
- **Tests Results: PASS**
- **Stakeholders: Approved**

CODE

```

411 def display_sprites(self) -> None:
412     if self.__screenName in ["baker", "usbourne"]: # Only displays sprites during gameplay screens
413         if not self.__gameplay.player.rect.colliderect(config.inventoryArea): # Won't display the main character above the inventory
414             self.__screenObj.blit(self.__gameplay.player.image, self.__gameplay.player.rect) # Print the main character

```

1. The player isn't printed in inventory area.
 - It will cause a little delay in the edges of the area, but nothing that will annoy the user.

TESTING

- **Objective:** Searchable place is searched and item appended to the inventory.
- **Blackbox Testing Data:**
 - **52 – F searches the place, and the item is appended to the inventory.**
 - **54 – The other keys are ignored.**
- **Video:**
 - Searching-success-2
- **Syntax Errors:** N/A
- **Status: PASS**

INTERACTION 3 – STATUSES, GOLDEN STATUE AND PAUSE SCREEN

- **Objective:** The time is updated and printed with the life in the statuses bar. Furthermore, the pause screen is opened when ESC is pressed during the gameplay screen.
- **Success Criteria:**
 - **15 – Only keyboard keys are being used so far.**
 - **28 – Bar is at the top of the screen.**
 - **29 – Player's life is printed.**
 - **30 – Time taken is updated and printed.**
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
- **Videos:**
 - Statuses-statue
- **Tests Results: PASS**
- **Stakeholders: Approved**

CODE

```
47 def game_loop(system: System, clock: pygame.time.Clock) -> None:
48     while True:
49         # Update screen elements, i.e. background, buttons and textbox.
50         system.update_screen()
51         if system.screenName in ["baker", "usbourne", "hacking", "lockpick", "fingerprint", "pause"]: # Only happen in gameplay screens
52             system.gameplay.update_status() # Update and print the time and lives
53             system.gameplay.player.display_inventory() # Display the inventory
54             if system.gameplay.player.lives == 0: # Finish the mission if player lose all lives
55                 system.render_screen(nextScreen="game-over") # Fail screen
56         input(system=system) # Where the input will be handled
57         pygame.display.flip() # Change the frame
58         clock.tick(60) # Limit the fps to 60
```

I added the `update_status()` methods from `Gameplay` class inside the `game_loop()` function, as well as a checker for player's lives.

1. `update_status()` updates the time and print it with the lives.
2. I use an if statement to check each frame if the user didn't lose, if they lost it will render the game-over screen. This screen will be used in the next development stage.

```

87         case "baker" | "usbourne":
88             match event.key:
89                 case system.preferences.buttonInteract: # Button used to search
90                     system.gameplay.player.interact() # Player method
91                 case system.preferences.buttonUse: # Button used to start the minigames
92                     system.gameplay.player.use() # Player method
93                 case pygame.K_ESCAPE: # Button to open the pause screen
94                     system.render_screen(nextScreen="pause") # System method
95             case "pause":
96                 system.button.selection_pause(key=event.key) # Button method
97         case "mission-completed":
98             if event.key == pygame.K_RETURN: # Button used to continue to menu

```

I added the commands during maps screen.

1. `use()` method is used when the use button is pressed to start the minigame.
2. Pause screen is rendered when ENTER is pressed in the maps screen.
3. `selection_pause()` is the exclusive button selection for pause screen.
4. Menu screen is rendered when the ENTER is pressed in mission completed screen.

```

71     # Getter method for self.__lastTime
72     @property
73     def lastTime(self):
74         return self.__lastTime
75
76     # Setter method for self.__lastTime
77     @lastTime.setter
78     def lastTime(self, lastTime: str):
79         if lastTime is not None:
80             self.__lastTime = lastTime

```

The getter and setter methods for the `self.__lastTime`, which stores the last result of the user mission. I should be accessed in other methods outside the Preference class.

```

256         case "baker":
257             if self.__gameplay is None: # Only creates a new gameplay if there isn't already one
258                 timeNow = datetime.datetime.now() # Get the current time
259                 player = Player(system=self, coordinate=config.bakerInitialPosition) # Create the player sprite
260                 codeDoor = Sprite(image=config.codeDoorPath, coordinate=(73, 256)) # Create the sprite of the door with code lock
261                 lockDoor = Sprite(image=config.lockDoorPath, coordinate=(657, 303)) # Create the sprite of the door with normal lock
262                 # Store and control the information about the current gameplay session
263                 self.__gameplay = Gameplay(map="baker", startTime=timeNow, walls=config.bakerWalls, searchablePlaces=config.bakerSearchablePlaces,
264                                             player=player, guard=None, laser=None, system=self, statueCoordinate=(425, 60),
265                                             door=config.bakerDoor, codeDoor=codeDoor, lockDoor=lockDoor, server=None)
266                 # Load the map 1 background, Baker Avenue
267                 self.__background = pygame.image.load(config.bakerScreenPath).convert_alpha()
268         case "usbourne":
269             if self.__gameplay is None: # Only creates a new gameplay if there isn't already one
270                 timeNow = datetime.datetime.now() # Get the current time
271                 player = Player(system=self, coordinate=config.usbourneInitialPosition) # Create the guard sprite
272                 codeDoor = Sprite(image=config.codeDoorPath, coordinate=(322, 223)) # Create the sprite of the door with code lock
273                 lockDoor = Sprite(image=config.lockDoorPath, coordinate=(322, 670)) # Create the sprite of the door with normal lock
274                 # Store and control the information about the current gameplay session
275                 self.__gameplay = Gameplay(map="usbourne", startTime=timeNow, walls=config.usbourneWalls,
276                                             searchablePlaces=config.usbourneSearchablePlaces, player=player, guard=None,
277                                             laser=None, system=self, statueCoordinate=(690, 80), door=config.usbourneDoor,
278                                             codeDoor=codeDoor, lockDoor=lockDoor, server=None)
279                 # Load the map 2 background, Usbourne Way
280                 self.__background = pygame.image.load(config.usbourneScreenPath).convert_alpha()
281         case "pause":
282             # Create the buttons: continue and menu, the format is [name, (x-coordinate, y-coordinate)]
283             self.__button = Button(buttons=[["continue", (135, 300)], ["menu-pause", (260, 475)]], system=self)
284             # Load pause screen, here player choose to return to menu or continue the mission
285             self.__background = pygame.image.load(config.pauseScreenPath).convert_alpha()

```

```

322     self.display_sprites() # Display the sprites (Baker Avenue and Usbourne Way screens)

```

I developed the initialization of the the pause and mission completed screen inside the `render_screen()` method of `System`. Moreover, the methods to display the sprites and the final time of the mission were also included.

1. The code and lock doors sprites are created using `Sprite` class, it uses sprites general attributes.
2. The gameplay is created and controlled by the class `Gameplay`, currently with no guard, laser and server.
3. The pause screen buttons are created using `Button` class, it uses the format `[name, (x, y)]` for each button, continue and menu in this case.
4. `display_sprites()` display the player sprite.

```
741 class Gameplay:
742     def __init__(self, map: str, startTime: datetime.timedelta, walls: list, searchablePlaces: list, player: Player,
743                 guard: Guard, laser: Sprite, system: System, statueCoordinate: tuple, door: pygame.Rect,
744                 codeDoor: Sprite, lockDoor: Sprite, server: pygame.rect.Rect):
745         self.__map = map
746         self.__startTime = startTime
747         self.__walls = walls.copy()
748         self.__searchablePlaces = searchablePlaces
749         self.__player = player
750         self.__guard = guard
751         self.__laser = laser
752         self.__door = door
753         self.__server = server
754         self.__statue = Sprite(image=config.statueSpritePath, coordinate=statueCoordinate)
755         self.__system = system
756         self.__codeDoor = codeDoor
757         self.__lockDoor = lockDoor
758
759         # Getter method for self.__guard
760         @property
761         def guard(self):
762             return self.__guard
763
764         # Getter method for self.__server
765         @property
766         def server(self):
767             return self.__server
768
769         # Getter method for self.__walls
770         @property
771         def walls(self):
772             return self.__walls
773
774         # Getter method for self.__player
775         @property
776         def player(self):
777             return self.__player
778
779         # Getter method for self.__map
780         @property
781         def map(self):
782             return self.__map
```

```

784 # Getter method for self.__searchablePlaces
785 @property
786 def searchablePlaces(self):
787     return self.__searchablePlaces
788
789 # Getter method for self.__codeDoor
790 @property
791 def codeDoor(self):
792     return self.__codeDoor
793
794 # Getter method for self.__lockDoor
795 @property
796 def lockDoor(self):
797     return self.__lockDoor
798
799 # Getter method for self.__door
800 @property
801 def door(self):
802     return self.__door
803
804 # Getter method for self.__startTime
805 @property
806 def startTime(self):
807     return self.__startTime
808
809 # Getter method for self.__statue
810 @property
811 def statue(self):
812     return self.__statue
813
814 # Getter method for self.__laser
815 @property
816 def laser(self):
817     return self.__laser

```

```

819 # Setter method for self.__laser
820 @laser.setter
821 def laser(self, value):
822     self.__laser = value
823
824 # Setter method for self.__statue
825 @statue.setter
826 def statue(self, value):
827     self.__statue = value
828
829 # Setter method for self.__codeDoor
830 @codeDoor.setter
831 def codeDoor(self, value):
832     self.__codeDoor = value
833
834 # Setter method for self.__lockDoor
835 @lockDoor.setter
836 def lockDoor(self, value):
837     self.__lockDoor = value

```

The Gameplay class is the responsible to store the states of the gameplay. As the Sprite and System classes needs their attributes, multiple getter and setter method were implemented for a secure access and modification of the attribute.

2. `self.__guard` stores the guard character sprite, it has a pre-defined path.
3. `self.__laser` stores the laser sprite, it's used to block the player to get the statue.
4. `self.__door` stores the hitbox of the main door, used to finish the mission successfully.
5. `self.__server` stores the hitbox for the server, it allows the start of the hacking minigame.
6. `self.__statue` stores the statue sprite, it allows the user to collide with the statue to collect it.
7. `self.__codeDoor` stores the hitbox for the code door, it allows the start of the fingerprint minigame.
8. `self.__lockDoor` stores the hitbox for the lock door, it allows the start of the lockpick minigame.

```

839     def update_status(self) -> None:
840         currentTime = datetime.datetime.now() # Get the current time
841         timeUsed = str(currentTime - self.__startTime) # Find the time used so far by subtracting the current time from the start one
842         timeUsed = timeUsed[:-4] # Limit the seconds to 2 decimal places
843         font = pygame.font.Font(None, 40)
844         # Print the time
845         text = font.render(timeUsed, True, (235, 165, 59))
846         rect = text.get_rect(topleft=(440, 5))
847         self.__system.screenObj.blit(text, rect)
848         # Print the lives
849         text = font.render(str(self.__player.lives), True, (235, 165, 59))
850         rect = text.get_rect(topleft=(275, 5))
851         self.__system.screenObj.blit(text, rect)

```

The method inside the `Gameplay` class that updates the time taken and print it with the number of lives inside the statuses bar.

1. The current time is get using `datetime.datetime.now()`.
2. The start time is subtracted from the current one to find the time taken so far. After, the result is transformed to a string that is sliced to remove the four decimal places, it results in the format 0:00:00.00
3. It uses peach colored text to print the time string and the lives inside their correct location in the statuses bar.

```

669     def check_collision(self) -> pygame.Rect:
670         for wall in self.__system.gameplay.walls: # Access each wall in the list of the map walls
671             if self.rect.colliderect(wall): # Check if collided with some wall in the list
672                 return wall # Return the wall collided
673         if self.__system.gameplay.statue is not None: # Check if statue wasn't taken yet
674             if self.rect.colliderect(self.__system.gameplay.statue.rect): # Check if player collided with statue
675                 self.__inventory.append("Statue") # Add statue to the inventory if collision happened
676                 self.__system.gameplay.statue = None # Reset the statue sprite
677         elif self.rect.colliderect(self.__system.gameplay.door) and "Statue" in self.__inventory: # Check if the player touched the main door
678                                                     # with the statue in its inventory
679             currentTime = datetime.datetime.now() # Get the time that mission finished
680             timeUsed = str(currentTime - self.__system.gameplay.startTime) # Find the time used in the mission by subtracting the final time
681                                                     # from the start time
682             self.__system.preferences.lastTime = timeUsed[:-4] # Transform the result in the string h:mm:ss.msms limiting the time decimal
683                                                     # to 2
684             self.__system.preferences.check_best_score(map=self.__system.gameplay.map) # Check if it was the best time
685             self.__system.render_screen(nextScreen="mission-completed") # Render the mission completed screen

```

The method inside `Player` class responsible to check for any collision between the player sprite and the map components.

1. For loop is used to access all the map's walls, as consequence the collision with walls can be checked individually and it can be returned. It returns the wall because the player position will be set to the edge of wall when collide.
2. If the statue sprite still existing, the collision with it is checked, if it collides with player its sprite will be set to `None`, so it's no longer printed, and the statue item is appended to the player's inventory.
3. If the player collides with the main entrance with the statue in the inventory, the final time is recorded and the time taken is found using the start and final times. The slicing is used to remove 4 decimal places, returning the format 0:00:00.00, and it's set as last score. Finally, it uses the `check_best_score()` method and render the mission-completed screen.

MAIN CHANGES FROM DESIGN

1. The `update_time()` method now not just update the time but display it and the number of lives. Therefore, its name is now `display_status()`.
2. Introduction an attribute to store the statue sprite inside Gameplay class.
3. There is a collision checker method.

TESTING

- **Objective:** The time is updated and printed with the life in the statuses bar. Furthermore, the pause screen is opened when ESC is pressed during the gameplay screen.
- **Blackbox Testing Data:**
 - 44 – W selects up.
 - 45 – S selects down.
 - 46 – ENTER confirms the selection.
 - 47 – The other keys are ignored
- **Video:**
 - Statuses-statue
- **Syntax Errors:**
 - **AttributeError: 'NoneType' object has no attribute 'rect'.**
 - The code tries to check the collision with statue even when it does not exist no more.
 - It's solved by adding an if statement to limit it to when the statue sprite isn't null.
 - **TypeError: text must be a unicode or bytes**
 - I tried to print the integer lives using `blit()`
 - I solved it by using the Pygame functions to print text, so I can print the number after transform it to a string.
- **Status: PASS**

INTERACTION 4.1 – MISSION COMPLETED SCREEN AND BEST SCORE

- **Objective:** The mission completed screen is rendered and the best score recorded locally and globally.

- **Success Criteria:**
 - **31 – The game finishes when the player touches the front door with the statue in the inventory.**
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
 - **Server-code**
 - App.py
 - GlobalDatabase.db
- **Videos:**
 - Mission-completed-screen-fail
- **Tests Results: FAIL**
- **Stakeholders: N/A**

CODE

```
323 self.display_result() # Display the final time (mission completed only)
```

1. `display_results()` display the final time into the box of the game over screen.

```
294 case "mission-completed":
295     # Load the menu background, menu-screen.png
296     self.__background = pygame.image.load(config.missionCompletedScreenPath).convert_alpha()
```

It will render the mission completed screen.

```
156 def check_best_score(self, map: str) -> None:
157     # Open local database and get the information stored in JSON format
158     with open(config.localDatabasePath, "r") as f:
159         localData = json.load(f)
160
161     bestTime = localData[0]["bestTime"][map] # Access the best time registered in the database
162     lastTime = self.__lastTime # Access the last time
163     # Get the number of seconds based on the string h:mm:ss.msms
164     lastTimeSeconds = int(lastTime[0]) * 3600 + int(lastTime[2:4]) * 60 + int(lastTime[5:7]) + int(lastTime[8:]) / 100
165     bestTimeSeconds = int(bestTime[0]) * 3600 + int(bestTime[2:4]) * 60 + int(bestTime[5:7]) + int(bestTime[8:]) / 100
166     if lastTimeSeconds < bestTimeSeconds: # Compare the amonuts of seconds
167         bestTime = self.__lastTime # If it's lower time it store as the best time
168         # Send the best result to the server
169         request = requests.post("http://127.0.0.1:5000/bestTime", json={"bestTime": bestTime, "map": map, "username": self.__username})
170     localData[0]["bestTime"][map] = bestTime # Update the local data
171
172     # Open and write the updated data
173     with open(config.localDatabasePath, "w") as f:
174         json.dump(localData, f)
```

This method inside `Preference` class gets the locally stored best time and compared with the time in the end of mission. I used this check before send to the server because it will use the client processing instead of the server one, so it decreases its workload.

1. Open the local file using the `open()` with the read mode and get the user info using `json.load()`
2. It accesses the correct best time using `localData[0]["bestTime"][map]`. So, it accesses the first and only record, after the best time field. Finally, it chooses the correct map time based on the map passed as parameter.
3. It converts both last and best times to their float number of seconds, hours have 3600s, minutes 60s and the decimal places uses 1/100 seconds.
4. If the last time is better than the best stored, then the new time is sent, with the map and username, to the server in a JSON format.
5. It uses the `open()` in the write format and the `json.dump()` to write the data into the local database again.

```
440     def display_result(self) -> None:
441         if self.__screenName == "mission-completed": # Display the final result only on the mission completed screen
442             font = pygame.font.Font(None, 80) # Uses the standar font and size 48
443             text_surface = font.render(self.__preferences.lastTime, True, (235, 165, 59)) # Render the font with peach colour
444             text_rect = text_surface.get_rect(topleft=(95, 640)) # Create a surface for textbox
445             self.__screenObj.blit(text_surface, text_rect) # Print the result
```

It prints the time taken in the mission when the player finishes it, only if the mission is successful, it uses the format 0:00:00.00.

1. It limits the function to the mission completed screen.
2. It uses standard font with size 80.
3. It uses peach colored text.
4. Printed In the bottom-left part of the screen.

```
571 class Sprite:
572     def __init__(self, image: str, coordinate: tuple):
573         self.__image = pygame.image.load(image).convert_alpha()
574         self.__rect = self.__image.get_rect(topleft=coordinate)
575
576     # Getter method for self.__image
577     @property
578     def image(self):
579         return self.__image
580
581     # Getter method for self.__rect
582     @property
583     def rect(self):
584         return self.__rect
```

I created the class `Sprite` to handle the general sprites attributes, including their image and their hitbox.

1. `self.__image` is the sprite image, it's static because the game won't use animation to simplify it.
2. `self.__rect` is the hitbox of the sprite, allow to check the collisions later.

3. The two getter methods are needed to access these private attributes into another classes.

TESTING

- **Objective:** The mission completed screen is rendered and the best score recorded locally and globally.
- **Blackbox Testing Data:** N/A
- **Video:**
 - Mission-completed-screen-fail
- **Syntax Errors:**
 - **TypeError: 'str' object is not callable**
 - I used an attribute setter method as a function, but it isn't.
 - I just need to set the attribute as a variable.
 - **TypeError: list indices must be integers or slices, not str.**
 - I tried to access the JSON content directly without use an index to select the first element.
 - I need to add a [0] to access the JSON record and after user other number to access the field.
- **Status: FAIL, the function in the server doesn't allow the POST method to add the best score to the database.**

INTERACTION 4.2 – MISSION COMPLETED SCREEN AND BEST SCORE

- **Objective:** The mission completed screen is rendered and the best score recorded locally and globally.
- **Success Criteria:**
 - **31 – The game finishes when the player touches the front door with the statue in the inventory.**
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py

- **Server-code**
 - App.py
 - GlobalDatabase.db
- **Videos:**
 - Mission-completed-screen-success
- **Tests Results: PASS**
- **Stakeholders: Approved**

CODE

```
37 @app.route("/bestTime", methods=["POST"])
38 def best_time() -> None:
39     data = request.get_json()
40     bestTime = data["bestTime"]
41     map = data["map"]
42     user = data["username"]
43     conn = sqlite3.connect("/Users/viniciusferrari/Documents/Richouse/Server-code/GlobalDatabase.db") # Connect with database
44     cursor = conn.cursor()
45     try:
46         cursor.execute("INSERT INTO results(time, map, user) VALUES (?, ?, ?)", (bestTime, map, user))
47     except:
48         cursor.execute("UPDATE results SET time = ? WHERE map = ? AND user = ?", (bestTime, map, user))
49     conn.commit()
50     conn.close()
51     return jsonify({"success": True})
```

1. Now the server allows the best score to be recorded because the method is POST instead of GET.
 - POST method is used when the data is sent to the server.
 - The sever must be mentioned in methods to work.

TESTING

- **Objective:** The mission completed screen is rendered and the best score recorded locally and globally.
- **Blackbox Testing Data:** N/A
- **Video:**
 - Mission-completed-screen-success
- **Syntax Errors:** N/A
- **Status: PASS**

PROTOTYPE

- **Stakeholders' feedback:** Approved
- **Video:**
 - Prototype-week-6
- **Images:** N/A

WEEK 7 - MINIGAMES, LASER, GUARD AND GAME OVER

- **Objective:** Develop the minigames and the sprites related to them. In addition, make the game over screen be rendered after player lose the mission.
- **Success Criteria:**
 - 17 - E is used to start the game directly from gameplay screen.
 - 19 - Player loses one life.
 - 20 - It moves constantly, but dogs are no longer used.
 - 21 - Feature was abandoned
 - 22 – The idea about cameras was abandoned
 - 23 - It blocks the statue.
 - 24 - Lasers are disabled, but cameras aren't present in the map.
 - 25 - Lock is used to block a room.
 - 26 - It's successfully completed.
 - 27 - Now the E is used to open the game directly.
 - 32 - It's rendered correctly.
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
- **Progress Management:**
 - 3 - Completed
 - 4 - Completed
 - 7 - Completed
 - 8 - Completed
 - 9 - Completed
 - 10 - Completed
 - 11 - Completed
 - 14 - Completed
 - 35 - Completed

- **Videos:**
 - Laser
 - Doors-fail
 - Doors-success
 - Game-over-screen
- **Tests Results: PASS**
- **Stakeholders' Approval: Approved**

INTERACTION 1.1 – LASER AND DOORS SPRITES

- **Objective:** The laser and doors sprites are printed on the screen and player can collide if them. In case of the laser, player will lose one life and return to the starter position.
- **Success Criteria:**
 - **19 - Player loses one life.**
 - **23 - It blocks the statue.**
 - **25 - Lock is used to block a room.**
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
- **Videos:**
 - Laser
 - Doors-fail
- **Tests Results: FAIL**
- **Stakeholders: N/A**

CODE

```

257     case "baker":
258         self.__minigame = None # Reset the minigame
259         if self.__gameplay is None: # Only creates a new gameplay if there isn't already one
260             laser = Sprite(image=config.bakerLaserPath, coordinate=(385, 22)) # Create the laser sprite
261             timeNow = datetime.datetime.now() # Get the current time
262             server = pygame.Rect(0, 5, 216, 94) # Create the server hitbox
263             guard = Guard(image=config.guardSpritePath, coordinate=(330, 600), system=self) # Create the guard sprite
264
265             codeDoor = Sprite(image=config.codeDoorPath, coordinate=(73, 230)) # Create the sprite of the door with code lock
266             lockDoor = Sprite(image=config.lockDoorPath, coordinate=(657, 303)) # Create the sprite of the door with normal lock
267             # Store and control the information about the current gameplay session
268             self.__gameplay = Gameplay(map="baker", startTime=timeNow, walls=config.bakerWalls, searchablePlaces=config.bakerSearchablePlaces,
269                                     player=player, guard=guard, laser=laser, system=self, statueCoordinate=(425, 60),
270                                     door=config.bakerDoor, codeDoor=codeDoor, lockDoor=lockDoor, server=server)
271
272             # Load the map 1 background, Baker Avenue
273             self.__background = pygame.image.load(config.bakerScreenPath).convert_alpha()
274
275     case "usbourne":
276         self.__minigame = None # Reset the minigame
277         if self.__gameplay is None: # Only creates a new gameplay if there isn't already one
278             laser = Sprite(image=config.usbourneLaserPath, coordinate=(601, 22))
279             timeNow = datetime.datetime.now() # Get the current time
280             server = pygame.Rect(5, 703, 214, 78) # Create the server hitbox
281             player = Sprite(image=config.playerPath, coordinate=config.usbourneInitialPosition) # Create the guard sprite
282             guard = Guard(image=config.guardSpritePath, coordinate=(380, 600), system=self) # Create the player sprite
283             codeDoor = Sprite(image=config.codeDoorPath, coordinate=(322, 22)) # Create the sprite of the door with code lock
284             lockDoor = Sprite(image=config.lockDoorPath, coordinate=(322, 670)) # Create the sprite of the door with normal lock
285             # Store and control the information about the current gameplay session
286             self.__gameplay = Gameplay(map="usbourne", startTime=timeNow, walls=config.usbourneWalls,
287                                     searchablePlaces=config.usbourneSearchablePlaces, player=player, guard=guard,
288                                     laser=laser, system=self, statueCoordinate=(690, 80), door=config.usbourneDoor,
289                                     codeDoor=codeDoor, lockDoor=lockDoor, server=server)

```

The `render_screen()` method in `System` class was modified to allow the creation of the laser and doors sprites.

1. Code and lock doors sprites are created in the start of the gameplay. They have a hitbox a little bigger than the wall used to block it because it allows the user to collide and interact to start the minigame.
2. No extra code to print because they are sprites printed by `display_sprites()`.

```

670 def check_collision(self) -> pygame.Rect:
671     for wall in self.__system.gameplay.walls: # Access each wall in the list of the map walls
672         if self.rect.colliderect(wall): # Check if collided with some wall in the list
673             return wall # Return the wall collided
674     if self.__system.gameplay.statue is not None: # Check if statue wasn't taken yet
675         if self.rect.colliderect(self.__system.gameplay.statue.rect): # Check if player collided with statue
676             self.__inventory.append("Statue") # Add statue to the inventory if collision happened
677             self.__system.gameplay.statue = None # Reset the statue sprite
678             return
679     if self.rect.colliderect(self.__system.gameplay.door) and "Statue" in self.__inventory: # Check if the player touched the main door
680         # With the statue in its inventory
681         currentTime = datetime.datetime.now() # Get the time that mission finished
682         timeUsed = str(currentTime - self.__system.gameplay.startTime) # Find the time used in the mission by subtracting the final time
683         # from the start time
684         self.__system.preferences.lastTime = timeUsed[:-4] # Transform the result in the string himiss.msms limiting the time decimal
685         # to 2
686         self.__system.preferences.check_best_score(map=self.__system.gameplay.map) # Check if it was the best time
687         self.__system.render_screen(nextScreen="mission-completed") # Render the mission completed screen
688         return
689     if self.__system.gameplay.laser is not None: # Check if the laser is active
690         if self.rect.colliderect(self.__system.gameplay.laser.rect): # Check the collision with the laser
691             self.__lives -= 1 # Player lose 1 life if collided with the laser
692             # The player will return to the initial position according to the map.
693             if self.__system.gameplay.map == "baker":
694                 self.rect.topleft = config.bakerInitialPosition
695             else:
696                 self.rect.topleft = config.usbourneInitialPosition
697             return
698     if self.rect.colliderect(self.__system.gameplay.guard.rect): # Check the collision with the guard
699         self.__lives -= 1 # Player lose 1 life if collided with the guard
700         # The player will return to the initial position according to the map.
701         if self.__system.gameplay.map == "baker":
702             self.rect.topleft = config.bakerInitialPosition
703         else:
704             self.rect.topleft = config.usbourneInitialPosition
705     return

```

1. Added the collision with the laser.
 - o If it exists, player collision is checked.
 - o Lose 1 life and return to the starter position, if collided.
2. The same idea was applied for the future guard.

3. The collision with the door is checked (not mentioned in the past week).
 - If the statue is in the inventory and the collision happened, then the mission will finish.

TESTING

- **Objective:** The laser and doors sprites are printed on the screen and player can collide if them. In case of the laser, player will lose one life and return to the starter position.
- **Blackbox Testing Data:**
 - **56 – Player loses a life.**
- **Video:**
 - Laser
 - Doors-fail
- **Syntax Errors:**
 - **'Player' object has no attribute '_Player__rect'.**
 - The attribute `rect` is inherited, so I can't access directly without a getter method.
 - I solved by adding a getter method to Minigame using the `@property` decorator.
- **Status: FAIL, the doors are printed in the horizontal instead of vertical in the Usbourne Way screen.**

INTERACTION 1.2 – LASER AND DOORS SPRITES

- **Objective:** The laser and doors sprites are printed on the screen and player can collide if them. In case of the laser, player will lose one life and return to the starter position.
- **Success Criteria:**
 - **19 - Player loses one life.**
 - **23 - It blocks the statue.**
 - **25 - Lock is used to block a room.**
- **Directories and files:**
 - **Distribution-code**

- Main.py
- Functions.py
- Config.py

➤ **Videos:**

- Doors-success

➤ **Tests Results: PASS**

➤ **Stakeholders: Approved**

CODE

```

415 def display_sprites(self) -> None:
416     if self.__screenName in ["baker", "usbourne"]: # Only displays sprites during gameplay screens
417         if not self.__gameplay.player.rect.collidect(config.inventoryArea): # Won't display the main character above the inventory
418             self.__screenObj.blit(self.__gameplay.player.image, self.__gameplay.player.rect) # Print the main character
419         if not self.__gameplay.guard.rect.collidect(config.inventoryArea): # Won't display the guard above the inventory
420             self.__screenObj.blit(self.__gameplay.guard.image, self.__gameplay.guard.rect) # Display the guard
421         if self.__gameplay.statue is not None: # Only display if the statue wasn't stolen yet
422             self.__screenObj.blit(self.__gameplay.statue.image, self.__gameplay.statue.rect) # Print the statue
423         if self.__gameplay.codeDoor is not None: # Only display if the fingerprint minigame isn't completed and door still in its place
424             # It prints the door in the original direction if it's the baker map
425             if self.__gameplay.map == "baker":
426                 self.__screenObj.blit(self.__gameplay.codeDoor.image, self.__gameplay.codeDoor.rect)
427             # It rotates the door before print if it's the usbourne map
428             else:
429                 rotatedImg = pygame.transform.rotate(self.__gameplay.codeDoor.image, 90) # Rotate in the top left vertex
430                 rotateRect = rotatedImg.get_rect(topleft=self.__gameplay.codeDoor.rect.topleft) # Create the new hitbox
431                 self.__screenObj.blit(rotatedImg, rotateRect)
432         if self.__gameplay.lockDoor is not None: # Only display if the lockpick minigame isn't completed and door still in its place
433             # It prints the door in the original direction if it's the baker map
434             if self.__gameplay.map == "baker":
435                 self.__screenObj.blit(self.__gameplay.lockDoor.image, self.__gameplay.lockDoor.rect)
436             # It rotates the door before print if it's the usbourne map
437             else:
438                 rotatedImg = pygame.transform.rotate(self.__gameplay.lockDoor.image, 90) # Rotate in the top left vertex
439                 rotateRect = rotatedImg.get_rect(topleft=self.__gameplay.lockDoor.rect.topleft) # Create the new hitbox
440                 self.__screenObj.blit(rotatedImg, rotateRect)
441         if self.__gameplay.laser is not None: # Display laser if the hacking minigame isn't completed
442             self.__screenObj.blit(self.__gameplay.laser.image, self.__gameplay.laser.rect)

```

1. Both lock and code door are rotated if the map is the Usbourne Way.
 - I can do it by using an if statement to check the map name.
 - I rotated it using the Pygame function `pygame.transform.rotate`.
 - I need to create the hitbox again.
2. The laser still the same.

TESTING

- **Objective:** The laser and doors sprites are printed on the screen and player can collide if them. In case of the laser, player will lose one life and return to the starter position.

➤ **Blackbox Testing Data:**

- **56 – Player loses a life.**

➤ **Video:**

- Laser-doors-success

➤ **Syntax Errors:** N/A

➤ **Status:** PASS

INTERACTION 2 – GAME OVER SCREEN

➤ **Objective:** Game over screen is rendered when the player has no lives left.

➤ **Success Criteria:**

- **32 - It's rendered correctly.**

➤ **Directories and files:**

- **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py

➤ **Videos:**

- Game-over-screen

➤ **Tests Results:** PASS

➤ **Stakeholders:** Approved

CODE

```
314     case "game-over":
315         self.__minigame = None
316         self.__background = pygame.image.load(config.gameOverScreenPath).convert_alpha()
```

1. I added the game over screen to the `render_screen()` method.

- It sets the minigame to 0 to avoid that its actions to interfere in the other screens.
- Background is loaded.

```
47 def game_loop(system: System, clock: pygame.time.Clock) -> None:
48     while True:
49         # Update screen elements, i.e. background, buttons and textbox.
50         system.update_screen()
51         if system.screenName in ["baker", "usbourne", "hacking", "lockpick", "fingerprint", "pause"]: # Only happen in gameplay screens
52             system.gameplay.update_status() # Update and print the time and lives
53             system.gameplay.player.display_inventory() # Display the inventory
54             system.gameplay.guard.move() # Update guard position
55             system.gameplay.player.check_collision() # Check any collisions with the player
56             if system.gameplay.player.lives == 0: # Finish the mission if player lose all lives
57                 system.render_screen(nextScreen="game-over") # Fail screen
58         input(system.system) # Where the input will be handled
59         pygame.display.flip() # Change the frame
60         clock.tick(60) # Limit the fps to 60
```

1. If statement define when the life is 0.
 - After collision check to avoid negative lives.
 - Render the game over screen if live is 0.

TESTING

- **Objective:** The laser and doors sprites are printed on the screen and player can collide if them. In case of the laser, player will lose one life and return to the starter position.
- **Blackbox Testing Data:**
 - **32 - It's rendered correctly.**
- **Video:**
 - Gamer-over-screen
- **Syntax Errors:** N/A
- **Status:** **PASS**

INTERACTION 3 – MINIGAMES

- **Objective:** The minigames are opened and the minigame start. Afterwards, the sequence is correctly check, and when correctly finished one of the sprites of doors or laser is removed from the map.
- **Success Criteria:**
 - **17 - E is used to start the game directly from gameplay screen.**
 - **24 - Lasers are disabled, but cameras aren't present in the map.**
 - **26 - It's successfully completed.**
 - **27 - Now the E is used to open the game directly.**
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
- **Videos:**
 - Minigames

➤ Tests Results: PASS

➤ Stakeholders: Approved

CODE

```
195 # Enable the access of the private attribute minigame
196 @property
197 def minigame(self):
198     return self.__minigame

299     case "fingerprint":
300         sequence = random.sample(range(0, 10), 3)
301         self.__minigame = Fingerprint(correctSequence=sequence, system=self)
302         # The background of fingerprint minigame screen
303         self.__background = pygame.image.load(config.fingerprintScreenPath).convert_alpha()
304     case "lockpick":
305         sequence = config.lockpickSequence.copy()
306         random.shuffle(sequence)
307         self.__minigame = Lockpick(correctSequence=sequence, system=self)
308         # The background of lockpick minigame screen
309         self.__background = pygame.image.load(config.lockpickScreenPath).convert_alpha()
310     case "hacking":
311         elements = random.choice(config.hackingSequences)
312         self.__minigame = Hacking(correctSequence=elements[1], system=self, word=elements[0])
313         self.__background = pygame.image.load(config.hackingScreenPath)
```

1. In the `System` class, a new getter method is used to allow other methods to access the private attribute `minigame`.
2. The `render_screen()` method in `System` class was modified to allow the creation of the minigame places and guard sprites, as well as the game over screen when the player loses.
 - The guard sprites are now created when the map is initialized. It uses the `Guard` class and has different coordinates but same place to start, that is the bottom right of their paths.
 - The fingerprint minigame is created using a random sequence chosen by `random.sample()`, only numbers from 0 to 9, and the `Fingerprint` class that has all methods necessities to handle it.
 - The lockpick minigame is created using a sequence randomized by `random.shuffle()`, the sequence is a list of 4 different arrows, and the `Lockpick` class that has all methods necessities to handle it. The method `copy()` is used because if use the actual list directly, the changes will be saved.
 - The hacking minigame is created using a choosing a sequence sequence chosen by `random.choice()`, the `config.hackingSequence` is a list of lists that contain words and their missing characters, and the `Hacking` class that has all methods necessities to handle it.
 - The game over screen sets the minigame to `None` before render the screen, avoiding the minigame to affect the next gameplay session.

```

326 def update_screen(self) -> None:
327     self.__screenObj.blit(self.__background, (0,0)) # Print background
328     self.display_buttons() # Display buttons
329     self.display_textbox() # Display textbox
330     self.display_rank() # Display users in ranking
331     self.display_options() # Display the current options
332     self.display_sprites() # Display the sprites
333     self.display_result() # Display the final time (mission completed only)
334     self.display_sequence() # Starts the minigame displaying the sequence

```

The `display_sequence()` method is called only in the `update_screen()`, inside `System` class, to avoid the arrows, from lockpick minigame, being skipped.

```

448 def display_sequence(self) -> None:
449     if self.__minigame is not None:
450         self.__minigame.start()

```

The `display_sequence()` will start the minigames by calling the method `start()`, but only if any minigame was initialized.

```

452 class Minigame:
453     def __init__(self, correctSequence: list, system: System):
454         self.__correctSequence = correctSequence
455         self.__userSequence = []
456         self.__system = system
457
458     # Getter method for self.__correctSequence
459     @property
460     def correctSequence(self):
461         return self.__correctSequence
462
463     # Getter method for self.__system
464     @property
465     def system(self):
466         return self.__system
467
468     # Getter method for self.__userSequence
469     @property
470     def userSequence(self):
471         return self.__userSequence
472
473     # Setter method for self.__correctSequence
474     @correctSequence.setter
475     def correctSequence(self, value: list):
476         if type(value) == list:
477             self.__correctSequence = value
478
479     # Setter method for self.__userSequence
480     @userSequence.setter
481     def userSequence(self, value: list):
482         if type(value) == list:
483             self.__userSequence = value

```

The `Minigame` class has all the attributes and methods used in all the minigames. To enable the inherited classes to access and modify its private attributes, getter and setter methods are used to maintain the code secure.

1. `self.__correctSequence` stores the user input sequence needed to finish the minigame successfully, used to compare with the user one.
2. `self.__userSequence` stores the user input during the minigame, used to compare with the correct one.
3. `self.__system` stores the system object allowing its methods to use the `System` class methods and attributes.

4. The setter methods only allow lists because the sequences must be a list to compare them.

```
485     def check(self) -> None:
486         if self.__userSequence != self.__correctSequence:
487             self.__system.gameplay.player.lives -= 1
488             if self.__system.gameplay.map == "baker":
489                 self.__system.gameplay.player.rect.topleft = config.bakerInitialPosition
490             else:
491                 self.__system.gameplay.player.rect.topleft = config.usbourneInitialPosition
492             self.__system.render_screen(nextScreen=self.__system.gameplay.map)
```

The `check()` method inside the `Minigame` class is used for the cases in which user wrong the sequence, so the output will be the same for all minigames.

1. If the sequence is incorrect, the player loses one live.
2. The player will return to the main door after it, so a selection is needed to send it to the correct coordinate according to the map.
3. The current map screen is rendered using the map name stored.

```
494 class Fingerprint(Minigame):
495     def start(self) -> None:
496         num1 = self.correctSequence[0]
497         weakFinger = pygame.image.load(config.weakFingerprintPath).convert_alpha()
498         self.system.screenObj.blit(weakFinger, config.numbersPositions[num1])
499         num2 = self.correctSequence[1]
500         midFinger = pygame.image.load(config.midFingerprintPath).convert_alpha()
501         self.system.screenObj.blit(midFinger, config.numbersPositions[num2])
502         num3 = self.correctSequence[2]
503         strongFinger = pygame.image.load(config.strongFingerprintPath)
504         self.system.screenObj.blit(strongFinger, config.numbersPositions[num3])
```

The `start()` method in `Fingerprint` class will start the fingerprint minigame by printing the circles of different tonalities in the number corresponding it. The correct sequence is from the lightest to the darkest circle.

1. The lightest circle is printed in the coordinate of the number in the index 0 of the sequence, `config.numbersPositions` stores the position of the numbers in the fingerprint screen.
2. The medium intensity circle is printed in the coordinate of the number in the index 1 of the sequence, `config.numbersPositions` stores the position of the numbers in the fingerprint screen.
3. The darkest circle is printed in the coordinate of the number in the index 2 of the sequence, `config.numbersPositions` stores the position of the numbers in the fingerprint screen.

```
506     def check(self) -> None:
507         super().check()
508         if self.userSequence == self.correctSequence:
509             if self.system.gameplay.map == "baker":
510                 self.system.gameplay.walls.remove(config.bakerCodeDoor)
511             else:
512                 self.system.gameplay.walls.remove(config.usbourneCodeDoor)
513             self.system.gameplay.codeDoor = None
514             self.system.render_screen(nextScreen=self.system.gameplay.map)
```

It's the extension of the `check()` method inherited from `Minigame` class. It removes the code door hitbox from the walls, so the passage becomes free, and reset its sprite making user unable to interact with it after.

1. It removes different hitbox according to the map, because each map hitbox has a different position in the map.
2. It resets the code door by setting the `self.system.gameplay.codeDoor` to `None`.
3. It renders the map screen again

```
516 class Hacking(Minigame):
517     def __init__(self, correctSequence: list, system: System, word: str):
518         super().__init__(correctSequence=correctSequence, system=system)
519         self.__word = word
```

The `Hacking` class is used to handle the hacking minigame, so it will extend the constructor method to add the word, it has missing character, in this way allowing it to be printed on the screen.

```
521     def start(self) -> None:
522         font = pygame.font.Font(None, 48)
523         text = font.render(self.__word, True, (0, 255, 0))
524         rect = text.get_rect(center=(400, 400))
525         self.system.screenObj.blit(text, rect)
```

The `Hacking` class starts the minigame by printing the word centralized on the screen. It uses a green colored text with standard font of size 48.

```
527     def check(self) -> None:
528         super().check()
529         if self.userSequence == self.correctSequence:
530             self.system.gameplay.laser = None
531             self.system.render_screen(nextScreen=self.system.gameplay.map)
```

The `check()` method extension in the `Hacking` class will only check if the sequence is correct, if correct, it reset the laser, so it is no longer printed, it's set to `None`.

```
533 class Lockpick(Minigame):
534     def __init__(self, correctSequence: list, system: System):
535         super().__init__(correctSequence=correctSequence, system=system)
536         self.__printableSequence = correctSequence.copy()
```

The `Lockpick` class handles the lockpick minigame. As it has a sequence of arrows to be printed, it extends the constructor method by adding the attribute to store the arrows to be printed.

1. `self.__printableSequence` copies the correct sequence, it's used instead of the original one because the item is deleted from the sequence when it's printed. It was the easiest solution to print the arrows.

```

538     def start(self) -> None:
539         try:
540             time.sleep(0.5)
541             item = self.__printableSequence[0]
542             match item:
543                 case pygame.K_LEFT:
544                     img = pygame.image.load(config.lockpickBackgrounds[pygame.K_LEFT])
545                     self.system.screenObj.blit(img, (334, 519))
546                 case pygame.K_RIGHT:
547                     img = pygame.image.load(config.lockpickBackgrounds[pygame.K_RIGHT])
548                     self.system.screenObj.blit(img, (334, 519))
549                 case pygame.K_UP:
550                     img = pygame.image.load(config.lockpickBackgrounds[pygame.K_UP])
551                     self.system.screenObj.blit(img, (334, 519))
552                 case pygame.K_DOWN:
553                     img = pygame.image.load(config.lockpickBackgrounds[pygame.K_DOWN])
554                     self.system.screenObj.blit(img, (334, 519))
555             self.__printableSequence.remove(item)
556         except:
557             return

```

The `start()` method for `Lockpick` class uses the attribute of printable arrows and use its items as indexes for a background list to print it in order.

1. The try statement is used because when the sequence finish an error will arise when the code try to access the index 0.
2. A selection is used to choose the correct background based on the element from the sequence.
3. The integer constant of the arrow is used as index for the arrow background.
4. The item is removed so the index 0 will be the next item in the sequence, it repeats until all the elements be printed and removed in the correct order.

```

559     def check(self) -> None:
560         super().check()
561         if self.userSequence == self.correctSequence:
562             if self.system.gameplay.map == "baker":
563                 self.system.gameplay.walls.remove(config.bakerLockDoor)
564             else:
565                 self.system.gameplay.walls.remove(config.usbourneLockDoor)
566             self.system.gameplay.lockDoor = None
567             self.system.render_screen(nextScreen=self.system.gameplay.map)

```

The `check()` method is extended in the `Lockpick` class by removing the hitbox and resetting the sprite of the lock door.

1. The removed wall is based on the current map because they use different hitboxes and coordinates for the door.
2. The lock door is set to `None`, so it is no longer printed.

TESTING

- **Objective:** The minigames are opened and the minigame start. Afterwards, the sequence is correctly check, and when correctly finished one of the sprites of doors or laser is removed from the map.
- **Blackbox Testing Data:**

- 20 - E is no longer used in the minigame screen.
- 21 - E is no longer used in the minigame screen.
- 22 - It allows 3 numbers instead of 4 because it would be too difficult to recognise all the numbers quickly.
- 23 - Extra numbers are ignored
- 24 - Correct sequence conclude the mini game successfully.
- 25 - Player lose one live and the minigame isn't completed.
- 26 - Return to the gameplay.
- 27 - Other keys are ignored.
- 28 - E is no longer used in the minigame screen.
- 29 - E is no longer used in the minigame screen.
- 30 - It allows 3 letters instead of 4 because the it would be too difficult to recognise the word.
- 31 - Extra letters are ignored
- 32 - Correct sequence conclude the mini game successfully.
- 33 - Player lose one live and the minigame isn't completed.
- 34 - Return to the gameplay.
- 35 - Other keys are ignored.
- 36 - E is no longer used in the minigame screen.
- 37 - E is no longer used in the minigame screen.
- 38 - It allows me to input 4 arrows.
- 39 - Extra arrows are ignored
- 40 - Correct sequence conclude the mini game successfully.
- 41 - Player lose one live and the minigame isn't completed.
- 42 - Return to the gameplay.
- 43 - Other keys are ignored.

➤ **Video:**

- Minigames

➤ **Syntax Errors:**

- **AttributeError: 'Lockpick' object has no attribute '_Lockpick__correctSequence'.**
 - The correct sequence is inherited from Minigame class, so I can't access directly.
 - It's solved by adding a getter method in Minigame class using @property decorator.
- **TypeError: 'NoneType' object is not iterable.**
 - Minigame still existing after the game finishes.
 - I solved it by setting the attribute storing minigame object to null before return to the menu screen.

➤ **Status: PASS**

INTERACTION 4 – GUARD

- **Objective:** Guard moves around the map, and user loses a live when collide to it.
- **Success Criteria:**
 - **19 - Player loses one life.**
 - **20 - It moves constantly, but dogs are no longer used.**
- **Directories and files:**
 - **Distribution-code**
 - Main.py
 - Functions.py
 - Config.py
- **Videos:**
 - Guard
- **Tests Results: PASS**
- **Stakeholders: Approved**

CODE

```
584 class Guard(Sprite):
585     def __init__(self, image: str, coordinate: tuple, system: System):
586         super().__init__(image=image, coordinate=coordinate)
587         self.__moveX = 5
588         self.__moveY = 0
589         self.__system = system
```

The Guard class is used to create a guard to walk in the map. It extends the `Sprite` class constructor method by adding extra attributes.

1. `self.__moveX` is the number of pixels that guard moves in the x direction at certain point. It's 5 in the start because the guard initialize in the bottom left of the screen, so it must move to the right. The Pygame counts the pixels from the top left.
2. `self.__moveY` is the number of pixels that guard moves in the y direction at certain point. It's 0 in the start because the guard initialize in the bottom left of the screen, so it must not move up or down. The Pygame counts the pixels from the top left.
3. `self.__system` stores the system object, so it allows the Guard methods to use the methods and attributes of the `System` class.

```

591 def move(self) -> None:
592     if self.__system.screenName in ["baker", "usbourne"]:
593         self.rect.x += self.__moveX
594         self.rect.y += self.__moveY
595     if self.__system.screenName == "baker":
596         if self.rect.topleft == (330, 330):
597             self.__moveX = 0
598             self.__moveY = 5
599         elif self.rect.topleft == (330, 600):
600             self.__moveX = 5
601             self.__moveY = 0
602         elif self.rect.topleft == (615, 600):
603             self.__moveX = 0
604             self.__moveY = -5
605         elif self.rect.topleft == (615, 330):
606             self.__moveX = -5
607             self.__moveY = 0
608     else:
609         if self.rect.topleft == (380, 400):
610             self.__moveX = 0
611             self.__moveY = 5
612         elif self.rect.topleft == (380, 600):
613             self.__moveX = 5
614             self.__moveY = 0
615         elif self.rect.topleft == (705, 600):
616             self.__moveX = 0
617             self.__moveY = -5
618         elif self.rect.topleft == (705, 400):
619             self.__moveX = -5
620             self.__moveY = 0

```

The `move()` method in the `Guard` class is the responsible for the pre-defined movement of the guard sprite.

1. If it's the one of the maps screens, it will move the guard based on the movement stored in the x and y movement attributes.
2. Each time that it finds a vertex of the rectangular path, it will change the movement of x and y directions.
 1. If it's the top left vertex it will moves down, so x and y movement will be 0 and 5, respectively, because Pygame count from the top left of the screen.
 2. If it's the bottom left vertex it will moves right, so x and y movement will be 5 and 0, respectively, because Pygame count from the top left of the screen.
 3. If it's the bottom right vertex it will moves up, so x and y movement will be 0 and -5, respectively, because Pygame count from the top left of the screen.
 4. If it's the top right vertex it will moves left, so x and y movement will be -5 and 0, respectively, because Pygame count from the top left of the screen.

TESTING

- **Objective:** Guard moves around the map, and user loses a live when collide to it.

- **Blackbox Testing Data:**
 - **56 – Player loses a live.**
- **Video:**
 - Guard
- **Syntax Errors:** N/A
- **Status:** **PASS**

PROTOTYPE

- **Stakeholders' feedback:** **Approved**
- **Video:**
 - Prototype-week-7
- **Images:** N/A

EVALUATION

TESTING

WEEK 8 ALPHA AND BETA TESTINGS

- **Objective:** The alpha testing checks how all the features of the game work together. I played it and identified which of the parts of the alpha testing table from design were met. Moreover, getting the stakeholders feedback about the final result, which allow me to expand the game in the future.
- **Success Criteria:** N/A
- **Directories and files:** N/A
- **Videos:**
 - Alpha-testing
 - Beta-testing-Enzo

- Beta-testing-Alex
- Beta-testing-Zach
- **Progress Management:**
 - **36 - Completed**
 - **37 - Completed**
- **Tests Results: PASS**
- **Stakeholders' Approval: N/A**

INTERACTION 1 - ALPHA TESTING

- **Objective:** The alpha testing checks how all the features of the game work together. I played it and identified which of the parts of the testing table from design were met.
- **Success Criteria:** N/A
- **Directories and files:** N/A
- **Videos:**
 - Alpha-testing
- **Tests Results: ALMOST FULL PASS**
- **Stakeholders' Approval: N/A**

TESTING

- **Objective:** The alpha testing checks how all the features of the game work together. I played it and identified which of the parts of the testing table from design were met.
- **Alpha Testing Data:**
 - **1 - It's opened, and it doesn't appear anymore after the register.**
 - **2 - A 10 alphanumeric characters username can be registered.**
 - **3 - It's rendered correctly**
 - **4 - The buttons can be selected and act in loop.**

- 5 - The screen is rendered and usernames are printed.
- 6 - The screen is rendered, and preferences are printed.
- 7 - The screen is rendered, and buttons are printed.
- 8 - The game closes.
- 9 - The buttons can be selected and act in loop.
- 10 - The Baker Avenue has the “Baker”, but the Usbourne Way is only “Map 2”.
- 11 - The menu screen is rendered.
- 12 - The usernames are printed properly.
- 13 - The menu screen is rendered.
- 14 - The preferences are printed.
- 15 - It no longer uses buttons, but an arrow instead, and it doesn't act as a loop.
- 16 - The volume can be adjusted using A and D. Moreover, these buttons can be altered.
- 17 - The buttons preferences can be changed using SHIFT + alphabetic key.
- 18 - The menu screen is rendered.
- 19 - The initial position is the main door.
- 20 - The player collides correctly with the walls.
- 21 - Guard is moving in loops.
- 22 - The player loses a live.
- 23 - The item is appended to the inventory.
- 24 - The minigame starts.
- 25 - A sprite is removed.
- 26 - The player loses a live.
- 27 - The screen is rendered.
- 28 - The time is printed correctly updated.
- 29 - The statue goes to the inventory.
- 30 - The mission completed screen is rendered.
- 31 - The menu screen is rendered.

➤ **Video:**

- Alpha-testing

➤ **Syntax Errors:** N/A

- **Status: ALMOST FULL PASS**, the background removes software didn't allow me to create a "Usbourne" buttons. Furthermore, the idea of buttons for preferences became unreliable because of the number of possibilities.

INTERACTION 2 - BETA TESTING

- **Objective:** Show the result to stakeholders to get their final feedback, which allows me to create future expansions.
- **Success Criteria:** N/A
- **Directories and files:** N/A
- **Videos:**
 - Beta-testing-Enzo
 - Beta-testing-Alex
 - Beta-testing-Zach
- **Tests Results: PASS**
- **Stakeholders' Approval: General approve, but with some weak points that can be changed in the future.**

TESTING

- **Objective:** Show the result to stakeholders to get their final feedback, which allows me to create future expansions.
- **Beta Testing Data:**

TUTORIAL

STANDARD COMMANDS	OPTIONS
W - Move upward and up selection A - Move leftwards and decrease the volume S - Move downwards and down selection D - Move rightwards and increase the volume E - Use the item to start the minigame F - Search the searchable place. ENTER - Confirm the button selection ESC - Return to the menu or gameplay	Volume - Increase with D and decrease with A as default SHIFT + Key - Change the preference to the key
GAMEPLAY	
• Searchable places are marked with F, press F touching it to search • Press E in the doors or server to start the minigame, items must be found before • You can return to the gameplay pressing ESC during the minigame • You lose one life if you touch the guard or laser, and if you fall the minigame.	

It must be at maximum 10 alphanumeric characters of length

 Video:

- **Video:**
 - Beta-testing-Enzo
 - Beta-testing-Alex
 - Beta-testing-Zach

➤ **Syntax Errors:** N/A

➤ **Status: PASS**

STAKEHOLDERS' FEEDBACK

- Enzo Garofani (translated from Portuguese):

Strong:

- Strong:
- Original idea, I had never seen a competitive stealth game before.
 - Simple but beautiful graphics, the geometry makes them good.
 - Competitvity, it enhances the engagement with the mission.

Weak:

- Weak:
- No animations, the game is too static.
 - I need to open the server myself before playing it.

- Zach Ward:

Strong:

- Strong:
- Fluid gameplay
 - Creative elements, such as the change of # to \$ in the ranking.
 - Accessibility, there is many instructions for the user.

Weak:

- No sound effects, only the music.
- Some collisions lead with teleports.
- The button for Usbourne map is called map 2

2. Alex Michalski:

Strong:

- The game screens are connected well; you can access everything intuitively.
- The sprites and screen have a pretty good design.

Weak:

- Few varieties of maps.
- The items in the searching aren't random.

FINAL PROGRESS MANAGEMENT

Number	Design part	Status
Distribution code		
1	main() function	Completed
2	game_loop() function	Completed
3	input() function	Completed
4	System class	Completed
5	Button class	Completed
6	Preferences class	Completed
7	Gameplay class	Completed
8	Minigame class	Completed
9	Hacking class	Completed
10	Fingerprint class	Completed
11	Lockpick class	Completed
12	Sprites class	Completed
13	Player class	Completed
14	Guards class	Completed
15	Local database	Completed
Server code		

16	new_user()	Completed with different name
17	set_score()	Completed with different name
18	best_score()	Completed with different name
19	Complete server database	Completed
20	Put the code in some free Internet server	Not completed
Graphic design		
21	Account screen	Completed
22	Menu screen	Completed
23	Rank screen	Completed
24	Option screen	Completed
25	Start screen	Completed
26	Map 1 gameplay screen	Completed
27	Map 2 gameplay screen	Completed
28	Lockpick screen	Completed
29	Hacking screen	Completed
30	Fingerprint screen	Completed
31	Pause screen	Completed
32	Mission completed screen	Completed
33	Game over screen	Completed
Testing		
34	White box testing	Not completed
35	Black box testing	Completed
36	Alpha testing	Completed
37	Beta testing	Completed

NOT COMPLETED

20 – The use of an Internet server would be too resource-cost because I would need to frequently maintain it. Therefore, a paid host could be needed to automate this process. Moreover, I might need to buy a domain name for it.

34 – It was not tested because it would take too much time of the development. Furthermore, the code is simple, consequently it's structure won't be too affect by it.

GAME IDEA AND IMPACT

The initial goal for the Richouse was a 2D top-down stealth game divided in phases. However, the new emerging ideas originated a new aspect for the game. Therefore, the standard structure was replaced by a completely original idea of a competitive stealth game with fewer maps.

The new approach allows the introduction of an idea like the arcades in the past. The users can register their usernames and try to get the best result, but now against the whole world instead of locally. It can impact people to become better in the videogames to be rewarded with the world recognition.

The introduction of minigames demanding memory and perception is now one of its key aspects. I believe that with the time persons will start to improve and it will be noticeable even in their daily life.

In summary, it became not only one more stealth game, but a competitive experience that induces the pressure of the stealth games with the feeling of reward. Moreover, I developed it to impact the user's life instead of just give fun for them. Consequently, it can be amazing for self-development, mainly for the young persons.

SUCCESS CRITERIA

After developing the project, it's important to check which of the success criteria was met. I returned to the table designed in the analysis section for it. The competition of this criterion was showed during the alpha testing, including a video of the full game working.

Number	Requirement	Justification	Result	Evidence
1	Menu screen	We need a screen to access the game resources such as maps selection and ranking table.	Achieved.	Videos: Username-screen-input, Menu-screen-input-success
2	We can selected buttons using W to select up and S to select down	Buttons are aligned in a column, so the game code can be reused, and the keys serve as general selection. Besides it, these	Achieved. In addition, the buttons can now be changed.	Video: Menu-screen-input-success

		keys are intuitive because are commonly used for it in other games.		
3	The option button opens the option screen when pressed	In this way user can access the preferences of controls and change it.	Achieved.	Video: Menu-screen-input-success
4	The ranking button opens the ranking when pressed	It allows the user opens the ranking screen and sees the highest scores	Achieved.	Video: Menu-screen-input-success
5	The start game button starts the mission when pressed	It will enable the user to choose one of the two maps and start it.	Not Achieved	N/A
6	The quit button stops the program when pressed	It enables the user to leave the game.	Achieved.	Video: Menu-screen-input-success
7	Ranking screen display two tables with top 10 scores in each map	It makes the best 10 results in each map available for every user can see.	Achieved.	Video: Rank-screen-success
8	The players life after mission is the status with highest impact during ranking. Time used to break the tie between users.	The ranking should have a main criterion to be ranked, and the player's life is the most important in a game. Time will break the tie, so player will try to finish as quick as possible playing more times the same map.	Not achieved, the life is no longer considered for the ranking. The time taken is the only status ranked.	N/A
9	Return to menu when ESC is pressed	Allows the user to return to menu screen, it's intuitive because	Achieved.	Video: Rank-screen-success

		it's used in multiple games.		
10	Configurations screen	Show all game preferences, allow user to see and change it	Achieved, but the name is now options screen.	Video: Options-screen-success
11	Configuration selection with W to select up and S to selected down	It allows the code of button selection be reused. Furthermore, it makes intuitive to the user.	Achieved. In addition, the buttons for selection can now be changed.	Video: Options-screen-success
12	Volume bar increase when selected and D is pressed, and decrease when selected and A is pressed	User is allowed to change the volume intuitively using buttons close to general selection, W and S that are commonly used in games.	Achieved. In addition, the buttons to change it can now be changed.	Video: Options-screen-success
13	Selected preferences are changed when SHIFT + (alphabetic/numeric key) is pressed, the key is defined for that action.	It enables the selection of different buttons from keyboard for the gameplay.	Partially achieved, the numbers are no longer available to be chosen.	Video: Options-screen-success
14	Return to menu screen when ESC is pressed.	Allows user to return to menu screen using an intuitive key, because it's commonly used for it in other games.	Achieved.	Video: Options-screen-success
15	Use only keyboard keys for any game action.	It makes easier to adapt the game for consoles in future given that they don't have mouse	Achieved.	Video: Alpha-testing

		but a multiple static buttons.		
16	Press W character moves forward, S it moves backward, A it moves left, and D it moves right.	It's the standard button for movement in many games. It can be personalized at configurations	Achieved.	Video: Player-movement
17	Press F searches the searchable places, and E uses the item during minigame interaction.	Makes easier to press because they are close to the movement buttons.	Partially achieved, now the E is used to interact with the minigame sprite to start it directly.	Videos: Searching-success-1, Searching-success-2, Minigames
18	Get 3 items in total after search all searchable places	Drawers and cabinets will hide the items necessary to start minigames. It will be 3 because they're 3 minigames. Some of the places will be empty.	Achieved.	Videos: Searching-success-1, Searching-success-2
19	Security system decreases player's life by 1 when find them.	It's used to make the game harder, so user will take more time.	Achieved.	Videos: Laser, Guard, Minigames
20	Sprites of guards and dogs constantly moves across the map during gameplay screen.	Forces the user to move around the map, if they are caught, they lose 1 live	Partially achieved, the dogs are no longer used and there is only one guard.	Video: Guard
21	Guards and dogs disable temporarily after kicking them	It disables the sprite for few seconds, so player	Not achieved, this feature	N/A

	from behind, using the key F	can pass without care about them.	was abandoned.	
22	Sprites of cameras rotating in same place.	It covers the maximum angle possible, so it makes harder to pass.	Not achieved, no cameras are used.	N/A
23	Lasers blocking some places.	Forces user to play the hacking minigame to disable it.	Achieved	Video: Laser
24	After player finish the hacking minigame, the sprites of cameras and lasers can't get them no more.	Turns the game dynamic because the user should find an item to hack it with the tablet, need to be found.	Partially, cameras are no longer used.	Video: Minigames
25	Locks blocking some rooms.	Forces user to find the lockpick and play the minigame to enter.	Achieved.	Video: Doors-success
26	Users complete the lockpick minigame if press the 3 arrow keys correctly	It's a challenge that demand attention, but it isn't that hard.	Achieved.	Video: Minigames
27	When user press E in fingerprint minigame while the UV light is in inventory, circles with different intensities shows the correct order, less intense to more intense.	Common feature shown in movies. Turn harder because the user should find the UV light.	Partially, the circles appear, but the E is now used to start directly from gameplay.	Video: Minigames
28	Player statues bar at top.	Allows user to keep track of their performance.	Achieved.	Images: Baker-avenure-screen.png, Usbourne-way-screen.png, Video: Mission-completed-success

29	Player's life	Shows how many times that user can be caught before losing. Limit of tries.	Achieved.	Images: Baker-avenue-screen.png, Usbourne-way-screen.png, Video: Mission-completed-success
30	Shown the time taken.	Allows the user to know how long they're tanking.	Achieved.	Images: Baker-avenue-screen.png, Usbourne-way-screen.png, Video: Mission-completed-success
31	Game finish when the golden statue arrives in the door	It gives a clear and predefined objective; money amount couldn't be clear enough about how much is the maximum.	Achieved.	Video: Mission-completed-success
32	Game over screen appear when user lost all lives.	It makes the game finish; users can't play the same mission infinitely.	Achieved.	Video: Game-over-screen
33	Cartoon graphics	Common graphics style for 2D games. Easier to develop than realistic or pixelated.	Achieved.	Images: Username-screen.png, Menu-screen.png, Start-button.png, Rank-button.png, Options-button.png, Quit-button.png, Options-screen.png, Rank-screen.png, Baker-avenue-screen.png, Usbourne-way-screen.png, Main-character.png, Guard-sprite.png,

				Lock-door.png, Code-door.png, Golden- statue.png, Hacking- screen.png, Lockpick- screen.png, Up- arrow.png, Down- arrow.png, Left- arrow.png, Right- arrow.png, Fingerprint- screen.png, Weak- finger.png, Mid- finger.png, Strong- finger.png, Pause- screen.png, Continue- button.png, Menu- pause.button.png, Mission- completed- screen.png, Game- over-screen.png
--	--	--	--	--

The game was developed meeting many of the criteria. On the other hand, some features were adapted or completely abandoned. These changes were essential to turn the game clear and competitive.

PARTIALLY ACHIEVED

13 - The next change is in the preferences choices. Users can't select numbers as preferences because given the actions used in the game, the use of numbers would be so unusual. Therefore, the implementation of a code to allow it would take too much time for something that is unpopular.

17 - The keys usage during the gameplay had a little adjustment as well. I changed the key to interact with the minigame sprites and start them, as a result the key E directly starts them when clicked touching one of their sprites with the needed item in the inventory.

20 - Already in the gameplay, I reduced the number of sprites, consequently there is only one guard and no dogs. I decided to do it because it makes the map clear, so user can focus more on minigames improving their immersion and motivating them to finish the game faster.

24 - The laser sprite is disable, but there is no camera to disable. The cameras were removed because the inclusion of that many sprites was making the game screen to polluted, so I focus in the most essential sprites.

27 - The circles still being printed but directly after open the screen. Minigame screens the key E is no longer used for anything. It happens because I decided to use the key to interact with the minigame sprite starting it immediately, as a result no extra code is needed, and implementation is easier.

NOT ACHIEVED

5 - The first noticeable change was the start button. It no longer opens a panel to select a map and directly start it. Its function is now open a map selection screen, consequently the interface becomes clear and more structured to user. Moreover, it allows the use of buttons, so I can reuse the selection code used for the other screens.

8 - The first feature to be abandoned was the ranking organization by number of lives. It now uses only the time taken because multiple users can get the same live but probably not the same time taken. Therefore, if I make the user focus on time, so they can develop strategies to be faster even losing a live, making the game more competitive.

21 - The security systems also suffered changes, including the remove of the feature to disable the guard. The guard disabling was abandoned to make the game harder because it will be constantly moving. **22** -

22 - The cameras don't rotate because they don't exist anymore. The cameras were removed because the inclusion of that many sprites was making the game screen to polluted, so I focus in the most essential sprites.

STAKEHOLDERS NEEDS

In the beginning of the project, several ideas and requirements were provided by the stakeholders. As the persons chosen to represent the users view about the project, it's essential to meet as many of their wishes as possible. Therefore, I returned to the interviews results in the analysis section and checked which parts were implemented.

Feature idea	From	Implementation	Evidence
Gameplay			
Buttons			
Movement: W, A, S and D	All stakeholders	Implemented. In addition, it can be changed.	Video: Player-movement
Interactions: E and F	All stakeholders	Implemented. In addition, it can be changed.	Videos: Searching-success-2, Minigames
Statuses			
Player's life	Alex and Enzo	Implemented.	Video: Prototype-weel-6
Stealthiness	Enzo and Alex	Not implemented.	N/A
Amount stolen	Alex and Zach	Not implemented.	N/A
Time taken	Zach	Implemented.	Video: Prototype-weel-6
Most valuable items	Zach	Not implemented.	N/A
Audiovisual effects			
Notify what kind of object was got	Zach	Not implemented.	N/A
Interactions have effects and affect sprites	Zach	Not implemented.	N/A
Ghosting, hide in places	Alex	Not implemented.	N/A
NPC bar turn yellow when start to see you	Alex	Not implemented.	N/A
Configurations			
Option to change graphics	Zach	Not implemented.	N/A
Option to change controls	Zach	Implemented.	Video: Options-screen-success
Option to change sound volume	Alex	Implemented.	Video: Options-screen-success
Ranking			
Should be included	Alex and Zach	Implemented.	Video: Rank-screen-success
Status to be ranked			

Amount stolen	Alex	Not implemented.	N/A
Time taken	Alex	Implemented.	Video: Rank-screen-success
Stealthiness	Alex	Not implemented.	N/A
Design			
Camera			
Top-down	Enzo, Alex and Zach	Implemented	Video: Prototype-week-7
Horizontal	Enzo	Not implemented.	N/A
Art style			
Realistic	Alex and Zach	Not implemented.	N/A
Cartoon	Enzo and Alex	Implemented.	
Pixel	Zach	Not implemented.	N/A
Menu screen			
Dynamic	Enzo	Implemented.	Video: Menu-screen-input-success
Start button	Alex	Implemented.	Video: Menu-screen-input-success
Quit button	Alex	Implemented.	Video: Menu-screen-input-success
Options button	Alex	Implemented.	Video: Menu-screen-input-success
Credits button	Alex	Not implemented.	N/A
Change your character	Alex	Not implemented.	N/A
Attractive art	Enzo	Implemented.	Video: Menu-screen-input-success

Even with the competition of many of the requirements, a half of them wasn't completed because of complexity, change in goals or time needed. On the other hand, the game also implemented unexpected features. Consequently, the final feedback, as showed during the beta testing, showed that users was satisfied with the final code result.

NOT IMPLEMENTED FEATURES

Some statuses ideas weren't followed because of the change in the goals of the game. The amount stolen and most valuable item would need a gameplay structure in which player objective is steal as many items as possible. However, the game act as a global arcade with a single item to be stolen, it makes the results less dependent of the luck, given the different values, and more focused in the competition.

In addition, the stealthiness was also abandoned because it is a too vague measurement, so it won't fit in the competitive. As a result, the hiding feature and guard perception bar were abandoned. They are both rely on idea of don't be noticed by systems, but now the guard work with collisions.

I didn't add some of the requirements because of their complexity and time that would take. The graphics changing, for instance, would be hard to introduce given that everything is an external image and Pygame can't handle their resolution. The same difficulty is found in the realistic and pixel graphics, they are too detailed and complex so I would use existent ones, therefore the game would become less original.

I decided to remove some features as well. The credits button is unnecessary for this style of game, because there is only one developer. The character change feature experiences the same, as it's not online, there is no reason to differentiate the character. Furthermore, they use mask that hides most of them.

SYSTEM PERFORMANCE

One of the strongest points of the game is the fast rendering of the screens. It was possible given the selection used in the `render_screen()` function that only try to execute the code linked to the screen name. Consequently, the delay is only few milliseconds long, imperceptible for users, and it makes this part of the code has a performance of $O(1)$, given the Python match system.

The game remains smooth because of the limitation of 60 frames per second. It allows the computer to process the frames at same rate, so movements in the map, such as the guard and player, remain at same velocity and time update at constant rate. On the other hand, the lockpick screens doesn't follow the same pattern, it must wait 1s per arrow, consequently it updates the frame each one second, so the time doesn't update evenly.

The storage methods include a local JSON file and a server SQLite database. The file is usually less efficient because it doesn't have an index, however in this case it doesn't affect the execution time given that it stores only one record. Already the queries in the database are used to access many users' data, but it uses an index system, so they are

executed quickly. Therefore, they are both imperceptible by users given that the data access time is lower than 0.1s.

Finally, there is the efficiency of the API to response the request of the many distributed games. Currently, I don't have a final test for it given that it wasn't added to an actual server. It runs locally so the requests don't pass through internet, but for the future we can expect a delay of about 1s to 2s, depending in the internet connection and hardware reliability. On the other hand, I can test the handling of many requests at same time, it's basically imperceptible for a small number of users, but for a big amount the machine will need to work more. Therefore, for API response we can expect a final delay of less than 10s.

➤ **Videos:** Alpha-testing, Beta-testing-Enzo

In this video I have identified the robustness of the game about inputs, time taken for responses, as well as bugs.

- **Positive aspects:**
 - The code showed to be input resistant in many cases. It ignores the keys without any function.
 - The screens are quickly rendered without any loading time.
 - Server didn't demonstrate problems and responded fast (However it couldn't be tested when not running locally).
 - Guard still following the path independent of the time taken in the mission.
- **Negative aspects:**
 - The code isn't handling well the username screen inputs. The SPACE bar and arrow, for instance, are printed as a word.
 - The options screen allows you to use the same key in multiple actions, which sometimes causes a loop that break the game. For example, when the up and down button are the same and you return to the menu.
 - Some of the walls are teleporting the player, mainly the toilets ones.
 - The lockpick minigame is updating the time each one second, so it isn't fluid.
 - The game sometimes becomes less fluid when the fps drops a little.

TESTING RESULTS

The testing was divided in four steps, including the black box, alpha and beta testing. This approach allows multiple games aspects to be tested more specifically, for example the functions inputs can be tested alone or everything together.

The only difference from design is the white box testing, which didn't happen because of the lack of time. It requires the understanding of testing tools that would take too long to learn and after implement, therefore it wasn't included as the code already have a good structure because of the OOP architecture.

BLACK BOX TESTING

In the code development period, week 3 to 7, the code was tested using different inputs and checking their reliability, consequently executing the black box testing. The following table contain all the results in the different screens.

Number	Menu	Expected result	Expected	Test result	Evidence
1	W	Up button selected, return to last button if top arrived	Valid	passed	Video: Menu-screen-input-success
2	S	Down button selected, return to first button if bottom arrived	Valid	passed	Video: Menu-screen-input-success
3	ENTER	Confirm the selected button, open the screen	Valid	passed	Video: Menu-screen-input-success
4	Any other key	The key doesn't do anything	Invalid	passed	Video: Menu-screen-input-success
Number	Account	Expected result	Expected	Test result	Evidence
5	Enter a username smaller than 10 characters	The username is sent to API to check if it's available	Valid	Passed	Video: Username-screen-input
6	Enter a username bigger than 10 characters	The keys after 10 th character are ignored	Invalid	Passed	Video: Username-screen-input

7	Number 0-9	It enters in the username	Valid	passed	Video: Username-screen-input
8	Letters A-Z	It enters in the username	Valid	passed	Video: Username-screen-input
9	_ key	It enters in the username	Valid	No longer used	N/A
10	Any other key	It won't enter in the username	Invalid	passed	Video: Username-screen-input
Number	Rank	Expected result	Expected	Test result	Evidence
11	ESC	Return to the menu screen	Valid	passed	Video: Rank-screen-success
12	Any other key	The key doesn't do anything	Invalid	passed	Video: Rank-screen-success
Number	Options	Expected result	Expected	Test result	Evidence
13	W	Up button selected, return to last button if top arrived	Valid	Partially, it doesn't return to the last. No buttons.	Video: Options-screen-success
14	S	Down button selected, return to first button if bottom arrived	Valid	Partially, it doesn't return to the first. No buttons	Video: Options-screen-success
15	A	It's valid if volume button selected, decrease volume by 5 if higher than 5, else doesn't do anything	Valid	passed	Video: Options-screen-success
16	D	It's valid if volume button selected, increase volume by 5 if lower than 100, else doesn't do anything	Valid	passed	Video: Options-screen-success

17	SHIFT + Letters A - Z	It's valid if the button selected is used to update preferences, it will be the button for the action chose	Valid	passed	Video: Options-screen-success
18	SHIFT + Numbers 0 - 9	It's valid if the button selected is used to update preferences, it will be the button for the action chose	Valid	No longer used	N/A
19	ESC	Return to the menu screen	Valid	passed	Video: Options-screen-success
20	Any other key	The key doesn't do anything	Invalid	passed	Video: Options-screen-success
Number	Start	Expected result	Expected	Test result	Evidence
21	W	Up button selected, return to last button if top arrived	Valid	Passed	Video: Prototype-weel-6
22	S	Down button selected, return to first button if bottom arrived	Valid	Passed	Video: Prototype-weel-6
23	ENTER	Confirm the selected button, open the screen	Valid	Passed	Video: Prototype-weel-6
24	ESC	Return to the menu screen	Valid	Passed	Video: Prototype-weel-6
25	Any other key	The key doesn't do anything	Invalid	Passed	Video: Prototype-weel-6
Number	Pause	Expected result	Expected	Test Result	Evidence
26	W	Up button selected, return to last button if top arrived	Valid	Passed	Video: Prototype-weel-6
27	S	Down button selected, return to first button if bottom arrived	Valid	Passed	Video: Prototype-weel-6

28	ENTER	Confirm the selected button and render the screen	Valid	Passed	Video: Prototype-weel-6
29	Any other key	The key doesn't do anything	Invalid	Passed	Video: Prototype-weel-6
Number	Gameplay	Expected result	Expected	Test result	Evidence
30	W	Move character five pixels up if there is no wall, else it is move to one pixel before the wall.	Valid	Partially, now it moves 4 pixels to decrease the velocity.	Video: Player-movement
31	A	Move character five pixels left if there is no wall, else it is move to one pixel before the wall.	Valid	Partially, now it moves 4 pixels to decrease the velocity.	Video: Player-movement
32	S	Move character five pixels down if there is no wall, else it is move to one pixel before the wall.	Valid	Partially, now it moves 4 pixels to decrease the velocity.	Video: Player-movement
33	D	Move character five pixels right if there is no wall, else it is move to one pixel before the wall.	Valid	Partially, now it moves 4 pixels to decrease the velocity.	Video: Player-movement
34	F	It will interact with minigame or searchable place if next to it, else do nothing	Valid	Partially, it's now used only for searching.	Video: Seraching-success-2
35	ESC	Open the pause screen	Valid	Passed	Video: Prototype-week-6
36	Open game over screen if lost all lives	Print the game over screen and return to menu	Valid	Passed	Video: Game-over-screen

37	Open mission completed screen if arrive at door with statue	Print the mission completed screen and return to the menu	Valid	Passed	Video: mission-completed-success
38	Lost a live if detected by security system	Lost one live and teleport user to front door	Valid	Passed	Videos: Laser, Guard, Minigames
39	Any other input	Do nothing	Invalid	Passed	Video: Prototype-week-7
Number	Fingerprint	Expected result	Expected	Test result	Evidence
40	E pressed once	Start the minigame by changing the background screen	Valid	No longer used, the minigame is used right after opened.	N/A
41	E pressed more than once	The key E pressed more than once should do anything more	Invalid	No longer used, the minigame is used right after opened.	N/A
42	Input of 4 numbers 0-9	The result will be checked by comparing with correct sequence	Valid	Partially, the sequence now has 3 numbers.	Video: Minigames
43	Input of more than 4 numbers 0-9	The extra keys inputted will be ignored.	Invalid	Partially, the sequence now has 3 numbers.	Video: Minigames
44	Correct sequence of numbers	The minigame is completed and return to gameplay changing some sprite state	Valid	Passed	Video: Minigames
45	Incorrect sequence of numbers	The minigame is completed and return to gameplay without change sprites state	Valid	Passed	Video: Minigames

46	ESC	Return to the gameplay screen without change any sprite state	Valid	Passed	Video: Minigames
47	Any other inputs	The key doesn't do anything	Invalid	Passed	Video: Minigames
Number	Hacking	Expected result	Expected	Test result	Evidence
48	E pressed once	Start the minigame by printing the word with missing characters	Valid	No longer used, the minigame is used right after opened.	N/A
49	E pressed more than once	The key E pressed more than once should do anything more	Invalid	No longer used, the minigame is used right after opened.	N/A
50	Input of 4 letters A-Z	The result will be checked by comparing with correct sequence	Valid	Partially, now the sequence uses 3 letters.	Video: Minigames
51	Input of more than 4 letters A-Z	The extra keys inputted will be ignored.	Invalid	Partially, now the sequence uses 3 letters.	Video: Minigames
52	Correct sequence of letters	The minigame is completed and return to gameplay changing some sprite state	Valid	Passed	Video: Minigames
53	Incorrect sequence of letters	The minigame is completed and return to gameplay without change sprites state	Valid	Passed	Video: Minigames
54	ESC	Return to the gameplay screen without change any sprite state	Valid	Passed	Video: Minigames
55	Any other inputs	The key doesn't do anything	Invalid	Passed	Video: Minigames
Number	Lockpick	Expected result	Expected	Test result	Evidence

56	E pressed once	Start the minigame by printing the arrows sequence	Valid	No longer used, the minigame is used right after opened.	N/A
57	E pressed more than once	The key E pressed more than once should do anything more	Invalid	No longer used, the minigame is used right after opened.	N/A
58	Input of 4 arrows	The result will be checked by comparing with correct sequence	Valid	Passed	Video: Minigames
59	Input of more than 4 arrows	The extra keys inputted will be ignored.	Invalid	Passed	Video: Minigames
60	Correct sequence of arrows	The minigame is completed and return to gameplay changing some sprite state	Valid	Passed	Video: Minigames
61	Incorrect sequence of arrows	The minigame is completed and return to gameplay without change sprites state	Valid	Passed	Video: Minigames
62	ESC	Return to the gameplay screen without change any sprite state	Valid	Passed	Video: Minigames
63	Any other inputs	The key doesn't do anything	Invalid	Passed	Video: Minigames

Most of the tests passed in the end. Returning to each week, 3 to 7, we can see the videos of the result and the testing logs. On the other hand, they were tested more than once, because of it the errors were registered in logs.

PARTIALLY OR NOT PASSED

Starting with the username screen, I changed the rules for the username be valid. There is a limitation of 10 alphanumeric characters, it is half of the initial 20 characters. The reason is to fit into the ranking easily without the need of a too small font, and it's more than enough for a big variety of names. Moreover, the use of “_” is now forbidden

because it would be harder to validate the username and it already has a good number of possible combinations.

I also changed the options screen layout. The idea of buttons was abandoned given that create a specific layout for each preference is unreliable. As a solution, I implemented an arrow that is an adaptation of the Button class. On the other hand, the cost of this change was that the button loop is no longer available.

The keys functionality in the gameplay screen also experienced changes. The velocity of the player decreased from 5 to 4 pixels because it's more adequate for the size of the maps. The F key now isn't responsible to open the minigame screen given that now the key E is used to directly start it when interact with its sprite.

Because of the change in keys, the minigame screens no longer use the key E to start because it already starts automatically after interacting with its sprite and its screen rendered. Other adaptation was the length of the sequence, every minigame now uses 3 elements, it allows me to standardize the code for the checking.

ALPHA TESTING

At this stage, the game features were tested together, so I have experienced what a user will see when open it. The functionalities were checked using the testing table from design. The following table contain the results:

Number	Feature	Validity	Result	Evidence
1	The username register screen is opened right after game start.	Only if it's the first time opening the game, or no username is registered.	Passed	Video: Alpha-testing
2	A username can be registered and confirmed pressing ENTER.	Only if the username has a maximum of 10 characters (alphanumeric only).	Passed	Video: Alpha-testing
3	Menu screen is rendered after register username.	Only if the username isn't in use. Otherwise, it will return "Unavailable".	Passed	Video: Alpha-testing
Menu screen				
4	Menu allows to selected buttons in a column. It acts as a loop when the top or bottom arrive.	Only if the buttons are the ones used for the selection. It's defined in the options screen.	Passed	Video: Alpha-testing

5	The rank screen is rendered when the rank button is confirmed.	Only if ENTER is pressed and the server is opened.	Passed	Video: Alpha-testing
6	The options screen is rendered when the buttons is confirmed.	Only if the ENTER is pressed.	Passed	Video: Alpha-testing
7	The maps selection screen is rendered when the buttons is confirmed.	Only if the ENTER is pressed.	Passed	Video: Alpha-testing
8	The game is stopped if the quit button is confirmed.	Only If the ENTER is pressed.	Passed	Video: Alpha-testing
Maps selection screen				
9	It allows to selected buttons in a column. It acts as a loop when the top or bottom arrive.	Only if the buttons are the ones used for the selection. It's defined in the options screen.	Passed	Video: Alpha-testing
10	The buttons have name of abbreviate the map.	Only an abbreviation of the name.	Partially, I couldn't implement the "Usbourne".	Video: Alpha-testing
11	User can return to the menu if ESC is pressed.	The buttons in the screen changes, but the background stays the same.	Passed	Video: Alpha-testing
Rank screen				
12	The top 10 results in each map are printed in different tables.	The order is from the higher amount of life and lowest time. Only one result per user.	Partially, the number of lives is now not counted as result.	Video: Alpha-testing
13	User can return to the menu if ESC is pressed.	The background changes and buttons appear.	Passed	Video: Alpha-testing
Options screen				
14	The current preferences are printed.	They are printed in the correct places.	Passed	Video: Alpha-testing
15	It allows to selected buttons in a column. It	Only if the buttons are the ones used for the	Partially, the buttons were	Video: Alpha-testing

	acts as a loop when the top or bottom arrive.	selection. It's defined in the options screen.	substituted by an arrow, and it doesn't act as a loop.	
16	The volume can be adjusted by using D to increase and A to decrease.	Only if the buttons are the ones defined for it as preference.	Passed	Video: Alpha-testing
17	The button preference can be changed by pressing SHIF + wanted key.	Only if the key is an alphanumeric character.	Partially, now only alphabetic characters are allowed.	Video: Alpha-testing
18	User can return to the menu if ESC is pressed.	The background changes and buttons appear.	Passed	Video: Alpha-testing
Baker Avenue and Usbourne Way screens				
19	Player stats in the main door.	It will depend on the map.	Passed	Video: Alpha-testing
20	Player collisions with the walls are handled correctly.	The player can't pass through the wall or be teleported.	Bugs found, some wall collisions are teleporting the player.	Video: Alpha-testing
21	Guard moving in loops.	It's constantly moving around something.	Passed	Video: Alpha-testing
22	Player loses one live when collide with the guard.	Even if it's below the inventory.	Passed	Video: Alpha-testing
23	An item is appended to the inventory after searching.	Fridge – Lockpick. TV cabinet – UV light. Desk – Chip. Sink - Nothing	Passed	Video: Alpha-testing
24	A minigame starts if player interact with its sprite with the correct item.	Fingerprint – code door with the UV light. Lockpick – lock door with the lockpick	Passed	Video: Alpha-testing

		Hacking – server with the chip.		
25	A sprite is removed when the minigame is completed successfully.	Lockpick – remove the lock door. Fingerprint – remove the code door. Hacking – remove the laser.	Passed	Video: Alpha-testing
26	Player loses one live when collide with the guard or fail in the minigame.	Only if its lives are higher than 0.	Passed	Video: Alpha-testing
27	Game over screen is rendered when lives become 0.	It will print no final time taken.	Passed	Video: Alpha-testing
28	The time is updated in every gameplay related screen.	It includes the maps, minigames and pause screens.	Passed	Video: Alpha-testing
29	The statue is appended to the inventory when the player collides with its sprite.	The sprite is no longer printed after collision and only one statue is appended.	Passed	Video: Alpha-testing
30	The mission is successfully completed when player arrive in the main door with the statue.	The mission completed screen is rendered and the time taken is showed.	Passed	Video: Alpha-testing
End game screens				
31	The menu screen is rendered when the ENTER is pressed.	The screen shows the user what to do.	Passed	Video: Alpha-testing

Most of the original feature expectations were met, it because of the integration of the code using the design pseudo codes. On the other hand, new ideas caused adaptations resulting in some of them being partially met.

The only actual bug found was the collisions with the walls. Some of them are teleporting the player instead of blocking its passage. It can be solved by polishing the code to find the possible errors in the hitboxes of the map.

PARTIALLY MET

Firstly, in the start screen the button for the Usbourne Way map has the name “map 2” instead of “Usbourne”. It’s consequence of the limitation in the software used to

remove the background, which is the MacBook Finder. As it wasn't removing the white from the image, I preferred to use a generic button than the one with the background.

Other change was the functionality of the options screen, which now uses an arrow instead of buttons. It's unreliable to create a button background for each possible preference, therefore I used an arrow. I introduced it by adapting the `Button` class, consequently it can't act as a loop as the buttons do.

BETA TESTING

The final testing is related to the stakeholders and their feedback, as a result it can be used for a final evaluation of their expectations. They are divided in strong and weak points:

- **Videos:** Beta-testing-Alex, Beta-testing-Zach, Beta-testing-Enzo
 - **Strong points:**
 - Original idea, I had never seen a competitive stealth game before.
 - Simple but beautiful graphics, the geometry makes them good.
 - Competitiveness, it enhances the engagement with the mission.
 - Fluid gameplay
 - Creative elements, such as the change of # to \$ in the ranking.
 - Accessibility, there is many instructions for the user.
 - The game screens are connected well; you can access everything intuitively.
 - The sprites and screen have a pretty good design.
 - **Weak points:**
 - No animations, the game is too static.
 - I need to open the server myself before playing it.
 - No sound effects, only the music.
 - Some collisions lead with teleports.
 - The button for Usbourne map is called map 2
 - Few varieties of maps.
 - The items in the searching aren't random.

What impressed them the most were the UI, the innovation and fluidness of the game. It was possible given the different ideas catch on the analysis, which were adapted in an original way. Furthermore, the fps limitation makes the code smooth because it gives time to CPU to process everything in a fixed amount of time.

WEAK POINTS

The audiovisual features were one of the weakest points given the amount of time needed to make them. To improve it, I can add the animations by creating more sprite states. Meanwhile, I can find more sound effects without copyright to integrate with the game actions, such as searching. Finally, I can find a new software to remove the backgrounds with superior quality, consequently enabling me to introduce the “Usbourne” button.

Some features are directly connected to the gameplay. One of the biggest limitations is the number of maps, only 2. It can be expanded by creating more given that the modular structure of the code allows the easier implementation, with some adaptations needed. In addition, the randomization of the items being searched can also enhance the playability.

Finally, some technical issues must be fixed. One of them is the already mentioned collision handling, which sometimes teleport the player. The other problem is the fact that user should open the server locally by itself because it isn't in the web. It can be fixed by adding its code in a free web server and changing the URL in the distribution code.

RELIABILITY

At first, I should consider how the inputs are being handled by the code, including the invalid ones. What I saw during the black box testing, weeks 3 to 7 of development, was the code ignoring the invalid inputs as the design expect. For one side it's good, but for another it can make difficult to understand what is going wrong because I don't have the track of the input. It can be solved my adding more returns and exceptions, therefore the code can be ready for any possible issue related to inputs. Mainly with future expansions.

Talking about the errors that could arise, it's essential to use the exceptions in the correct way. These exceptions handling aren't too present in the code, they were only used in some parts of the server code, it can be a problem in future implementation given that unexpected errors can arise in some already implemented piece of code. In addition, I used an exception without a specific error, so the code can have problems with different errors arising. Consequently, it must be redesigned before any further introductions to avoid crashes or code slowness.

The game is now working smoothly, so I didn't experience any major performance bug or crashes. On the other hand, a functional bug was found, the teleports during collisions. It can be stressful for users given that the game is competitive. It can be solved by changing the hitboxes of the walls to avoid wrong collisions. The other problem is about time handling when it exceeds the limit, less than 10 hours of the

mission. It can cause an overflow and returning to 0, so it would be unfair with the other users, or completely crash the game.

Finally, it's important to mention the slowness in the lockpick screen because of the 1 second of wait in each arrow. After some time, it can lead with some crash given that the performance drastically drops from 60 to 1 fps per second during this screen time. Unfortunately, I don't have a solution for this issue so far.

USER EXPERIENCE

Using the alpha testing in the week 8 of development I could see how the interfaces look like as a user. Moreover, the stakeholders feedback of the Beta gives me a non-developer vision, so the evaluation becomes less biased.

One of the strongest points about the general experience was the ease navigation with instructions to guide user. Furthermore, the remotion of mouse cursor and limitation of keys to alphabetic makes easier for user remember the commands and to possibly adapt to consoles. On the other hand, the gameplay doesn't have sound effects to guide the user, so it eliminates completely the blind public. In addition, there's no features for color blindness, therefore it may cause difficulties for them during the game. It's important to remember the existence of the tutorial in the week 8, it's distributed to guide users about standard controls

In the username register screen, there is good user guidance through a message that explain how to confirm the username using the ENTER key. Moreover, it's checked to confirm its availability, if it already exists, it returns "Unavailable" in the textbox. The font used for typing is big enough for everyone see, size 48, and its black color contrast with the white background making it more readable. To verify it, you can return to week 4 of the development, or see directly in the alpha testing, week 8.

The menu screen doesn't contain many items, but the present ones are enough to show user what are their functions. The buttons have their names directly linked to the screen that it renders making its access intuitive. There is no button to return to username register screen, given that it's one time only. It can be checked in the weeks 4 and 8.

The first menu button opens the start screen containing the map selection. It's well-structured and includes the two different maps selection. However, the Usbourne Way map button doesn't contain its name, so it can confuse the user. Furthermore, there is no guide about how to return to menu screen. It can be checked in the weeks 5 and 8.

The second button opens the rank screen. It contains big text, with background contrast, showing the maps names to identify the correct rank. In addition, there is a

guidance, in the top-right of the screen, about how to return to menu screen using the ESC key. On the other hand, names inside the tables uses a smaller font, size 24, with less contrast with the background, consequently it may be a problem for colorblind persons. It can be checked in the weeks 5 and 8.

The third button opens the options screen. It has a good accessibility with a guidance of how to return to the menu and an arrow to select the desired preference. Furthermore, there is big title with the screen name to identify the screen. However, the preferences and the arrow use a color with low contrast with the background, so the colorblind people may struggle with it. It can be checked in the weeks 5 and 8.

Already during the gameplay, multiple elements were included for guidance. Starting with the statuses and inventory that are rounded by a yellow box highlighting them from the other elements. Moreover, there is many “?” showing which of the places are searchable, so user don’t need to check everywhere, and there is a guidance of how to access the pause screen using the ESC key. On the other hand, it may be difficult to identify the server minigame, sometimes people can’t link the idea. It can be checked in the weeks 6, 7 and 8.

Finally, I checked the endgame and pause screens. The mission completed and game over screens are clear for everyone with the necessary guidance to return to menu. However, the few contrasts of the game over one can confuse colorblind users. Already the pause screen, it has good guidance with the buttons with intuitive actions and a statuses box on top. On the other hand, it has the same color problem of the endgame ones. It can be checked in the weeks 7 and 8.

There is also the user feedback divided into strong and weak points:

USABILITY FEATURES

- **Videos:** Beta-testing-Alex, Beta-testing-Zach, Beta-testing-Enzo

SCREENS



- Buttons are centralized because it fixes the user attention to the middle of the screen while the side spaces can be fill by a beautiful art, in this way user can feel more immersion.
- The game title is the bigger text because in this way user can have sure that it's the title furthermore it's more important to identify the game than the buttons.
- Buttons are in vertical; it was a decision for all screens to allow me to reuse the general selection code in all screens with buttons.
- The box around the selected button is used to show the user what they are currently selecting, connection between computer logic and user understand.
 - I didn't use boxes because it wouldn't fit on the cartoon style.
 - I used a different technique instead, which is making the selected button slightly bigger than the other. Therefore, the design is preserved and it still usable for the users.

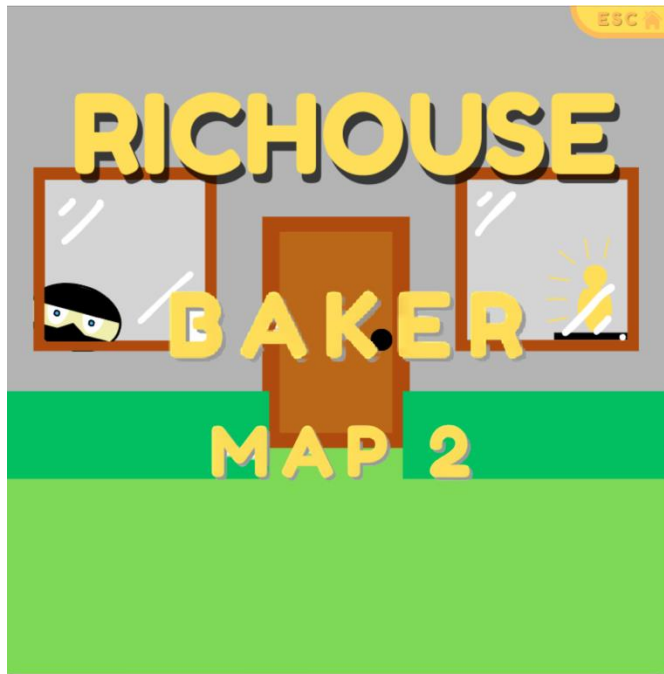


- There are 2 rankings because there are 2 different maps that take different amount of time, it's unfair to compare the time using only 1 rank.
- The top 10 was chosen because it's more attractive than top 50 for example, the exclusivity can make the users play and effort more to participate in the ranking.

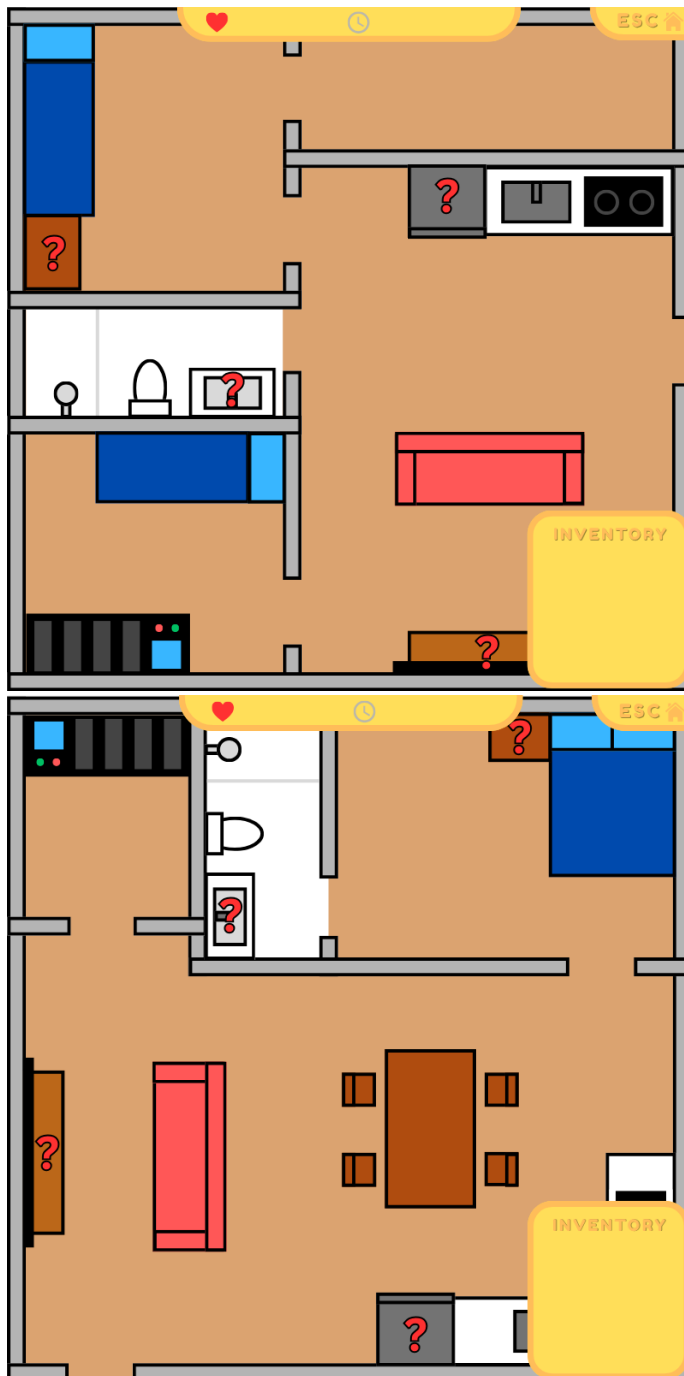


- The screen used to change the buttons and volume preferences.
- The volume is shown by a bar because in this way we can visualize the audio intensity.

- The other options are the button preferences; it show what button it refers and in the right is the current button registered.
- The screen name is always print on top; it supports the user identify where they entered.

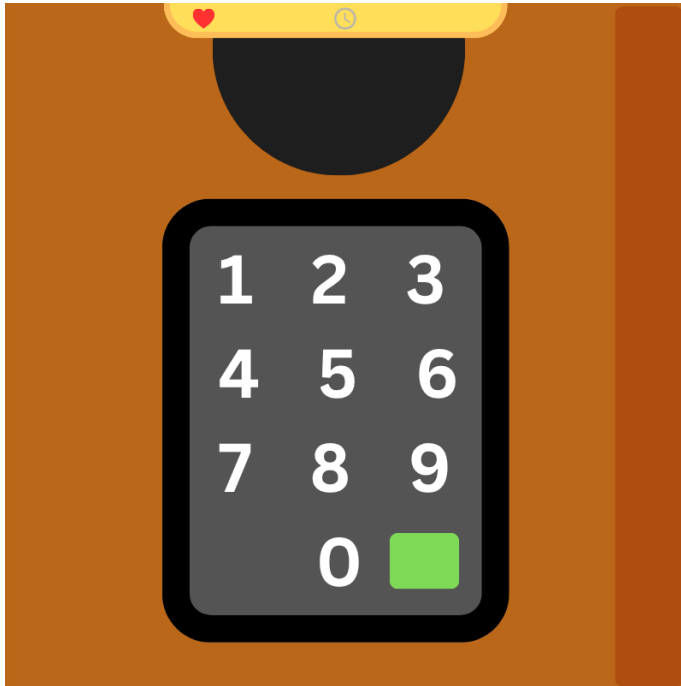


- The screen used to choose the map before starting the gameplay.
- It will have 2 maps, and the background will be divided in 2, one showing a part of each map, in this way the user can see and choose the map.
- Again, the button selected has a box and the title showing the screen.

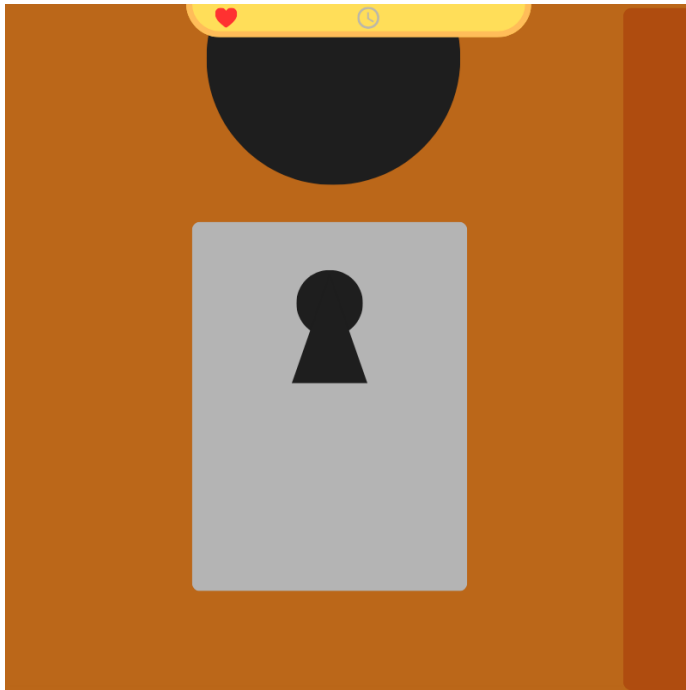


- It's the structure of the gameplay screen; the maps will be different.
- The screen has the statuses on top, it shows the number of lives and the time taken, it supports the user to see its performance in the mission.
- The map has lines showing the walls, it connects the logic of walls in code with users' vision.
- **The map is square to support the implementation of Dijkstra algorithm.**
 - The map continues using the squares because they are simple to handle the collisions.

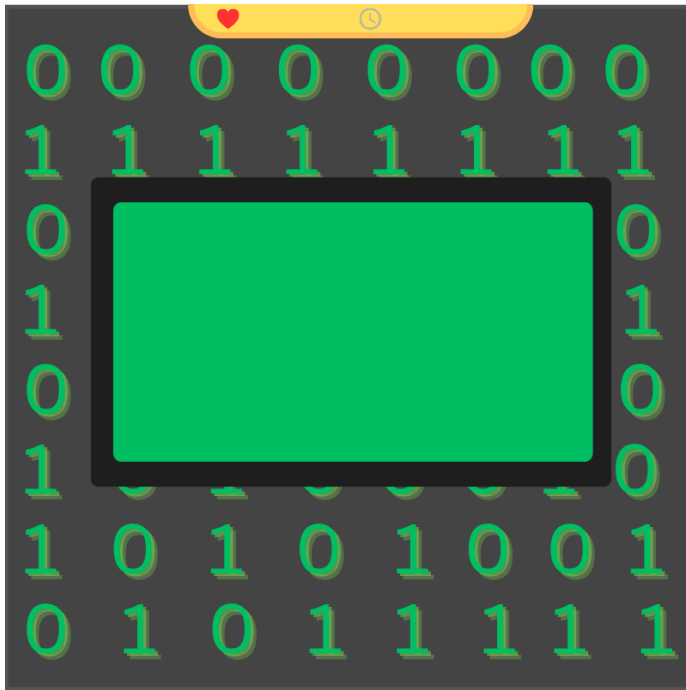
- The reason changed because the Dijkstra algorithm idea was abandoned because it would make the game too difficult given that character would be constantly chased.
- The algorithm can be implemented in future in case of bigger maps are introduced, therefore giving more space for the guard movement.



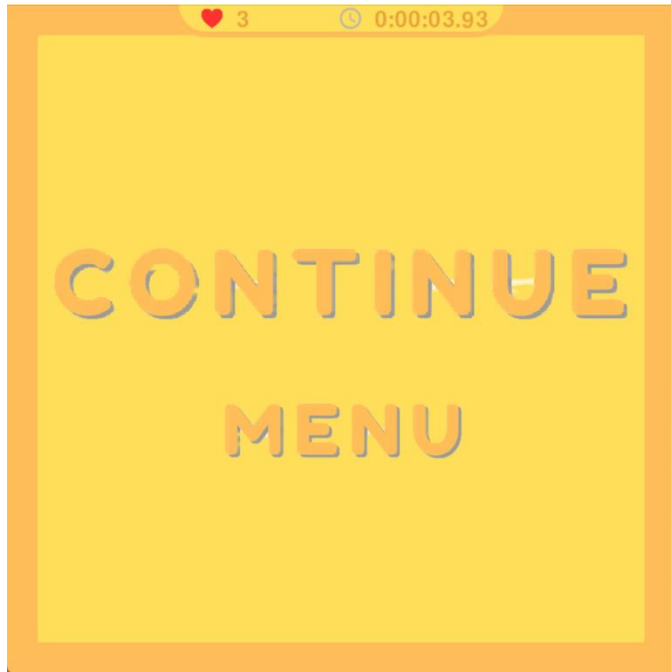
- The first screen is when you enter in the minigame, it gives a background to user understand what's going on.
- The statuses still being printed because the guards will continue to look for player and the time still running.
- The second is when you start it with the necessary tool.
 - There's no second screen because now the minigame starts as soon as the screen opens.
 - It makes easier for the user because they can just access it with the item, consequently decreasing the chance to be stuck because of usability features.
- The screen shows the sequence by increasing order of intensity, it looks like a fingerprint.



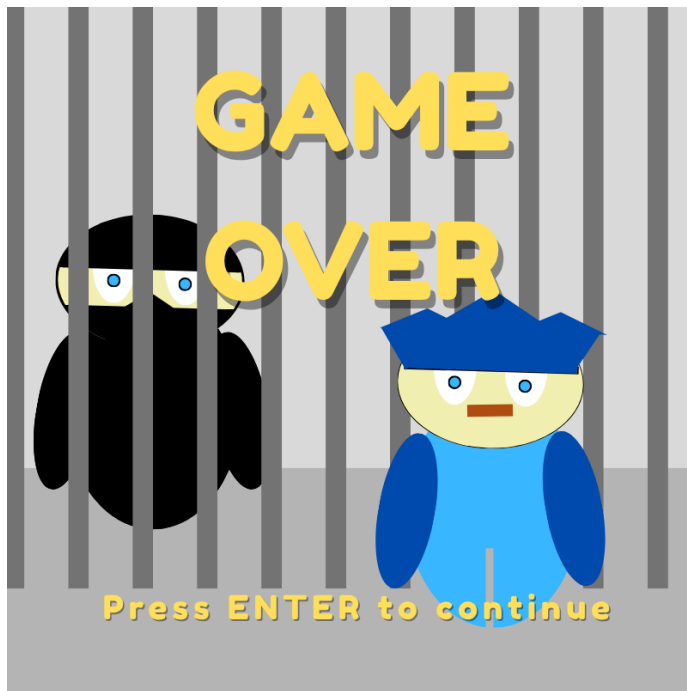
- It's the screen opened when the lockpick game is entered.
- In beginning the lock doesn't have anything inside, after starts it show a sequence of 4 arrow.
 - There's no start because it will start to print as soon as the screen is rendered.
 - It makes easier for the user because they can just access it with the item, consequently decreasing the chance to be stuck because of usability features.
- The user should input the correct arrow sequence.
- The statuses continue to be printed because the gameplay is happening.



- The screen when you enter in the hacking minigame.
- The screen when user enter in the minigame there is no word in it, after starting a word with messing characters is printed.
 - There's no start because it will start to print as soon as the screen is rendered.
 - It makes easier for the user because they can just access it with the item, consequently decreasing the chance to be stuck because of usability features.
- User should input the missing characters in order. The statuses continue to be printed.



- When you press ESC during the gameplay.
- There are 2 options, the return to continue gameplay or menu to return to menu screen.
- **Big title to know what's happening.**
 - Given the buttons and the statuses, the screen is already intuitive about what it is.
 - In the future, I can change the screen adding a title, but it would require a better organization of buttons and statuses bar
- **The selected button is marked by a box.**
 - I didn't use boxes because it wouldn't fit on the cartoon style.
 - I used a different technique instead, which is making the selected button slightly bigger than the other. Therefore, the design is preserved and it still usable for the users.



- The screen opened when you lost all the lives, and the mission fail.
- It's just a screen with an art as background and a big “Game over” to tell user what happened.

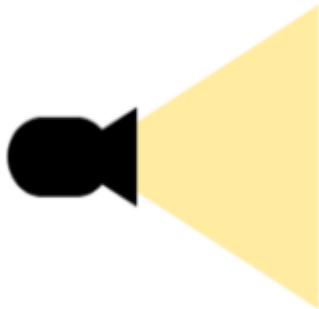


- Appear when the player arrives in the front door with the golden statue.
- It will have an art showing the glory of success and a big mission completed to give a feel of success for the user.

SPRITES



- A simple character that will be used as base of player and guards' sprites.
- It has a cartoon and doesn't have complex details, it makes the user focus on gameplay instead of graphics, furthermore it's a good minimalist design.
- There's no mouth because the game doesn't have dialogues.



- It's a simple sprite that will rotate during the game.
- The yellow part shows the range that camera can see the user, a way to connect the code logic and the user game view.
- It's simpler to combine with another map elements.

The cameras were completely removed from the game because there were too many elements in the map. Although it's a good feature, maintain the guard instead is much more essential in this case to make the game dynamic. In the future, it can be implemented if bigger maps, or maps without guard, originate.

STAKEHOLDERS' FEEDBACK

- **Strong points:**

- Simple but beautiful graphics, the geometry makes them good.
- Competitiveness, it enhances the engagement with the mission.
- Fluid gameplay
- Creative elements, such as the change of # to \$ in the ranking.
- Accessibility, there are many instructions for the user.
- The game screens are connected well; you can access everything intuitively.
- The sprites and screen have a pretty good design.
- **Weak points:**
 - No animations, the game is too static.
 - I need to open the server myself before playing it.
 - No sound effects, only the music.
 - Some collisions lead with teleports.
 - The button for Usbourne map is called map 2
 - Few varieties of maps.
 - The items in the searching aren't random.

MAINTAINABILITY

Started talking about code readability we can find multiple elements that support other developers to understand the code. The final solution has a very similar logic with the algorithms design, so at any time someone can access the pseudocode of the section to support on the understanding (it's why I used pseudocode). Furthermore, I introduced multiple comments in the code to explain its structure, which is combined with intuitive names, such as "guard". It's enhanced by the OOP architecture used, which divide the code into different classes and methods, consequently we can check only the parts related to that certain feature.

As I said, the code is divided in classes containing attributes and methods. This architecture allowed me to divide the features into the classes. It was useful for the minigames, which inherit the `Minigame` class, and to control the screen-related parts using the `System` class. On the other hand, I believe that parts of the code must be changed to prepare the code for future updates or just make it more efficient. The `display_inventory()` method for example would be better if located inside the `System` class because it handles all the other display methods. In addition, a creation of a `Map` class would be ideal to adapt the code to be more general in the way that it handles the gameplay. For example, the way that it handles the player teleport after loses one life, it's currently using a selection according to map name, which is inefficient for a big number of options.

The creation of new maps is the biggest issue. It will be difficult with these map name selections, therefore even with a good modular structure I believe that the code still needs more modules. An `Arrow` class, for instance, is the ideal to handle the arrow in the options instead of adapting the `Button` class, consequently allowing the loop structure to return to work and making the code more readable.

There is also the problem related with external libraries. As these pieces of code aren't developed by me, I can't be sure about their future, therefore if any of them stop to be updated in the future, it can affect the game. However, I used famous libraries and tools that have a big community to update, so it won't be an issue for a while. Furthermore, some tools can be replaced for more complex ones, such as the. Flask by the Django because it contains more functionalities and more efficient for larger data sets.

The local database won't be a problem because it used only a flat file with only one record, so if the user doesn't change anything, it will work as long as needed. On the other hand, the maintenance of the global storage should be considered. The actual SQLite database works good for large sets as well because it was normalized, so new users, results and maps can be easily added. However, if I upload it to a server, it can be a problem if not maintained. The duration of hosting and memory usage are limited in this service, so somebody should be responsible to check and update the code to avoid the loss of user data and performance. Depending on the complexity, a professional would be needed.

LIMITATIONS

The game showed many innovative and well-structured features, but as every software it has limitations. The main reasons for these limitations were missing elements in design, complexity and time needed to solve them.

Firstly, I would like to talk about the main limitation, the software isn't compiled nor connected to an actual server. The code is distributed in the original files, so users can change it causing problems or misusing it. Furthermore, the server code must be initialized by the user locally before the game starts because a properly server wasn't used. As a result, any of the global ranking are online, so the users can't update them once the game is installed, the only way to compare with others is the database being sent from user to another. It can be solved by compiling the code and using a server, but there is some maintenance planning to do first.

After the last limitation be solved, a new one arises because of user validation. The code compares the last result with the local best result and after send to the server to avoid overwhelming the server. However, there is no user authentication before adding

the result to the global database, so they may forge bad results for the other players. For instance, someone can change the local database username, adding first place in ranking username, and best time, increasing it to the maximum, so when it finishes the game play with a bad result, it will be saved as the first-place best result. It can be solved in the future by adding an authentication method, such as a digital certificate, but it will require a broader knowledge in cybersecurity applications.

There are also limitations on user interface; the screen is limited to 800x800. Therefore, it may be too small or too big according to the platform. Furthermore, it doesn't enable the full screen. Unfortunately, I haven't had any solution to this issue so far.

The game is also limited to accessibility. It doesn't contain any sound effect other than the music, so the blind persons aren't able to play. Moreover, there are no changes in graphics, so there is no colorblind adaptation, and some text may lack contrast. It may be fixed by adding new sound effects that allows the blind persons to play the game, but it will require a lot of effort and time given that a completely new option of play would be developed. I don't have a solution for colorblindness, but I believe that Pygame may have some filters.

There is a platform limit. The game isn't adapted for consoles, mobiles, or even other operational systems. As I didn't test Linux machines, I'm not sure about the game's behavior in these systems, given that it considers everything a file. Furthermore, it was developed on a computer, and it wasn't adapted to mobile and console. To fix it, it will require the use of virtual machines to test the code into different systems, in case of console I will probably need to convert the code to another specific computer language.

Finally, I would like to talk about the graphics development limitations. As I want an original layout, I developed everything by myself using free software. It limited the result because the time didn't allow the development of complex elements, so no animations and effects are present. In addition, the application used to remove the backgrounds of the elements, the Apple Finder, didn't work as expected, deleting some parts of the design and leaving some white spaces. It didn't also work with the "Usbourne" button, which was replaced by a generic "map 2". It can be improved by spending more time creating new sprite states and using better applications.

POSSIBLE EXTENSIONS

There are two main extensions that I'm planning for this game. They are related to the increase in the variety of maps and distribution for multiple platforms.

Firstly, I'm aiming to solve the problem about maps variety. Currently, there are only two maps, so users can't see many new things. The solution would be the implementation of a Map class and adaptation of the selections based on map names. It will allow us to easily implement a map using a predeveloped object. Therefore, it would allow even the users to create their own maps to be approved to be introduced in the game. It allows the users' contributions to the game. Furthermore, the rank must be adapted for it. Instead of a screen with all rankings, it would be present in the map selection when you select a map. I got this idea from Super Mario Maker, in which users create personalized maps with rankings.

The second major change is platform accessibility. The game can be adapted and translated to another languages or game engines to allow the support for consoles and mobiles. Moreover, the server code must be in an actual server at the time allowing the global rank to work properly. The main problem with this extension is the different requirements and prices to release it in certain places, such as Xbox and Apple stores that contain strict rules and fees.

REFLECTION

The idea of the game was the aspect that I liked the most. The combination of interactive minigames and competitiveness allows the user to engage with it and develop themselves. Moreover, the game is accessible for all ages, but the main goal is the young user because they are the most propense to take advantage of mental development given that they are in the growing age.

There are some parts that I would like to change if planned again. It includes the modular structure for the map and the implementation of ranking together with map selection, consequently allowing other maps to be introduced easily in the future.

It was an amazing opportunity to explore many new tools. The experience using the Visual Studio Code brings any problems, and it's easy to install the needed tools, so I would maintain it for the future projects. Furthermore, Pygame allowed me to develop a game with no prior experience; it's intuitive and well documented. On the other hand, it has some limitations of graphics, 2D only, and platform accessibility. Therefore, I would use Unity for future games because it contains many built-in functionalities and it can be easily distributed to other systems, mainly mobile ones. The server currently uses a Flask framework that is beginner friendly. Although it works well for small scale, other frameworks, such as Django, will be needed, if the game expands and more users arrive. In the databases I wouldn't change anything locally because there is only one record to be accessed, whereas in the global SQLite one I would create indexes to make the frequent queries faster.

The game was developed using an agile methodology allowing you to follow the steps in a dynamic way. I would continue using this methodology because it allows me to adapt the game according to the client's needs, differently from the waterfall one. However, I would probably change the structure to the XP life cycle because it would provide quality software and frequent updates to the game, so making the process more professionally structured.

As it was my first game, I had some issues understanding and introducing some features. At first what worried me the most was the collisions handling. I spent a lot of time searching how it works, and as a result I discovered that Pygame can handle it by itself if I use rectangles. Consequently, the game hitboxes are all rectangles that work well for now, but for more complex maps it can be a limitation. Moreover, I implemented a server for the game. It was the first time I used a Flask server with the `request` library in Python, but as I had prior experience, it wasn't a big problem after some time.

In summary, the game has a big potential to be scaled and distributed to as many platforms as possible. However, it will require the use of more professionally used tools, such as Django or Node.js for the server and Unity for the distribution code. The game idea was adapted to benefit the cognitive skills of the users while they have fun with competitiveness, which amazed me.

PROJECT APPENDIXES

DISTRIBUTION-CODE

CONFIG.PY

```
config.py x
Distribution-code > config.py > ...
1 import os
2 import pygame
3
4
5
6 # Path of this file
7 basePath = os.path.dirname(__file__)
8
9 # Path for local database
10 localDatabasePath = os.path.join(basePath, "LocalUserInfo.json")
11 localDatabasePath = os.path.abspath(localDatabasePath)
12
13 # Screens backgrounds paths
14 menuScreenPath = os.path.join(basePath, "..", "Media", "menu-screen.png")
15 menuScreenPath = os.path.abspath(menuScreenPath)
16 usernameScreenPath = os.path.join(basePath, "..", "Media", "username-screen.png")
17 usernameScreenPath = os.path.abspath(usernameScreenPath)
18 rankScreenPath = os.path.join(basePath, "..", "Media", "rank-screen.png")
19 rankScreenPath = os.path.abspath(rankScreenPath)
20 optionsScreenPath = os.path.join(basePath, "..", "Media", "options-screen.png")
21 optionsScreenPath = os.path.abspath(optionsScreenPath)
22 bakerScreenPath = os.path.join(basePath, "..", "Media", "baker-avenue-screen.png")
23 bakerScreenPath = os.path.abspath(bakerScreenPath)
24 usbourneScreenPath = os.path.join(basePath, "..", "Media", "usbourne-way-screen.png")
25 usbourneScreenPath = os.path.abspath(usbourneScreenPath)
26 pauseScreenPath = os.path.join(basePath, "..", "Media", "pause-screen.png")
27 pauseScreenPath = os.path.abspath(pauseScreenPath)
28 missionCompletedScreenPath = os.path.join(basePath, "..", "Media", "mission-completed-screen.png")
29 missionCompletedScreenPath = os.path.abspath(missionCompletedScreenPath)
30 fingerprintScreenPath = os.path.join(basePath, "..", "Media", "fingerprint-screen.png")
31 fingerprintScreenPath = os.path.abspath(fingerprintScreenPath)
32 lockpickScreenPath = os.path.join(basePath, "..", "Media", "lockpick-screen.png")
33 lockpickScreenPath = os.path.abspath(lockpickScreenPath)
34 gameOverScreenPath = os.path.join(basePath, "..", "Media", "game-over-screen.png")
35 gameOverScreenPath = os.path.abspath(gameOverScreenPath)
36 hackingScreenPath = os.path.join(basePath, "..", "Media", "hacking-screen.png")
37 hackingScreenPath = os.path.abspath(hackingScreenPath)
38
39 # Button backgrounds paths
40 startButtonPath = os.path.join(basePath, "..", "Media", "start-button.png")
41 startButtonPath = os.path.abspath(startButtonPath)
42 rankButtonPath = os.path.join(basePath, "..", "Media", "rank-button.png")
43 rankButtonPath = os.path.abspath(rankButtonPath)
44
45 optionsButtonPath = os.path.join(basePath, "..", "Media", "options-button.png")
46 optionsButtonPath = os.path.abspath(optionsButtonPath)
47 quitButtonPath = os.path.join(basePath, "..", "Media", "quit-button.png")
48 quitButtonPath = os.path.abspath(quitButtonPath)
49 againButtonPath = os.path.join(basePath, "..", "Media", "again-button.png")
50 againButtonPath = os.path.abspath(againButtonPath)
51 menuPauseButtonPath = os.path.join(basePath, "..", "Media", "menu-pause-button.png")
52 menuPauseButtonPath = os.path.abspath(menuPauseButtonPath)
53 menuMenuButtonPath = os.path.join(basePath, "..", "Media", "menu-menu-button.png")
54 menuMenuButtonPath = os.path.abspath(menuMenuButtonPath)
55 continueButtonPath = os.path.join(basePath, "..", "Media", "continue-button.png")
56 continueButtonPath = os.path.abspath(continueButtonPath)
57 bakerButtonPath = os.path.join(basePath, "..", "Media", "baker-button.png")
58 bakerButtonPath = os.path.abspath(bakerButtonPath)
59 map2ButtonPath = os.path.join(basePath, "..", "Media", "map-2-button.png")
60 map2ButtonPath = os.path.abspath(map2ButtonPath)
61 arrowPath = os.path.join(basePath, "..", "Media", "arrow.png")
62 arrowPath = os.path.abspath(arrowPath)
63
64 # Elements for Baker Avenue map
65 bakerLaserPath = os.path.join(basePath, "..", "Media", "laser-baker-sprite.png")
66 bakerLaserPath = os.path.abspath(bakerLaserPath)
67 bakerInitialPosition = (100, 710)
68 bakerCodeDoor = pygame.Rect(73, 256, 75, 17)
69 bakerLockDoor = pygame.Rect(657, 303, 75, 17)
70 bakerDoor = pygame.Rect(70, 775, 80, 25)
71 bakerServer = pygame.Rect(0, 0, 216, 94)
72
73 > bakerSearchable = {"desk": pygame.Rect(562, 5, 69, 77),
74
75 bakerWalls = [pygame.Rect(778, 0, 22, 800), pygame.Rect(0, 0, 22, 800), pygame.Rect(0, 0, 800, 22), pygame.Rect(0, 778, 800, 22),
76               pygame.Rect(211, 0, 22, 325), pygame.Rect(211, 308, 446, 22), pygame.Rect(733, 306, 68, 22), pygame.Rect(0, 257, 75, 22),
77               pygame.Rect(147, 257, 80, 22), pygame.Rect(364, 0, 22, 213), pygame.Rect(364, 279, 22, 47), pygame.Rect(170, 427, 83, 222),
78               pygame.Rect(390, 413, 207, 185), pygame.Rect(520, 700, 281, 100), pygame.Rect(698, 533, 101, 268), pygame.Rect(632, 0, 168, 211),
79               pygame.Rect(233, 135, 59, 51), pygame.Rect(562, 0, 69, 77), pygame.Rect(233, 206, 60, 102), pygame.Rect(0, 437, 66, 192),
80               pygame.Rect(431, 700, 89, 85), bakerServer, bakerCodeDoor, bakerLockDoor]
81
82 # Elements for Usbourne Way map
83 usbourneLaserPath = os.path.join(basePath, "..", "Media", "laser-usbourne-sprite.png")
84 usbourneLaserPath = os.path.abspath(usbourneLaserPath)
85 usbourneInitialPosition = (710, 365)
86 usbourneCodeDoor = pygame.Rect(322, 223, 17, 75)
87 usbourneLockDoor = pygame.Rect(322, 670, 17, 75)
88 usbourneDoor = pygame.Rect(770, 365, 30, 85)
89 usbourneServer = pygame.Rect(5, 708, 214, 78)
```

```

88 usbourneDoor = pygame.Rect(770, 365, 30, 85)
89 usbourneServer = pygame.Rect(5, 708, 214, 78)
90 > usbourneSearchable = {"desk": pygame.Rect(5, 241, 85, 94), -
94 > usbourneWalls = pygame.Rect(770, 0, 22, 800), pygame.Rect(0, 0, 800, 22), pygame.Rect(0, 778, 800, 22), -
100
101 # Inventory rectangle
102 inventoryArea = pygame.Rect(608, 588, 192, 208)
103
104 # Sprites images
105 mainSpritePath = os.path.join(basePath, "..", "Media", "main-character.png")
106 mainSpritePath = os.path.abspath(mainSpritePath)
107 statueSpritePath = os.path.join(basePath, "..", "Media", "golden-statue.png")
108 statueSpritePath = os.path.abspath(statueSpritePath)
109 codeDoorPath = os.path.join(basePath, "..", "Media", "code-door.png")
110 codeDoorPath = os.path.abspath(codeDoorPath)
111 lockDoorPath = os.path.join(basePath, "..", "Media", "lock-door.png")
112 lockDoorPath = os.path.abspath(lockDoorPath)
113 guardSpritePath = os.path.join(basePath, "..", "Media", "guard-sprite.png")
114 guardSpritePath = os.path.abspath(guardSpritePath)
115
116 # Lockpick elements
117 lockpickSequence = [pygame.K_LEFT, pygame.K_RIGHT, pygame.K_DOWN, pygame.K_UP]
118 leftArrow = os.path.join(basePath, "..", "Media", "left-arrow.png")
119 rightArrow = os.path.join(basePath, "..", "Media", "right-arrow.png")
120 upArrow = os.path.join(basePath, "..", "Media", "up-arrow.png")
121 downArrow = os.path.join(basePath, "..", "Media", "down-arrow.png")
122 > lockpickBackgrounds = {pygame.K_LEFT: os.path.abspath(leftArrow), pygame.K_RIGHT: os.path.abspath(rightArrow), -
124
125 # Fingerprint elements
126 weakFingerprintPath = os.path.join(basePath, "..", "Media", "weak-finger.png")
127 weakFingerprintPath = os.path.abspath(weakFingerprintPath)
128 midFingerprintPath = os.path.join(basePath, "..", "Media", "mid-finger.png")
129 midFingerprintPath = os.path.abspath(midFingerprintPath)
130 strongFingerprintPath = os.path.join(basePath, "..", "Media", "strong-finger.png")
131 strongFingerprintPath = os.path.abspath(strongFingerprintPath)
132 > numbersPositions = ((348, 614), (227, 282), (336, 282), (447, 282), (227, 392), -
134
135 # Hacking elements
136 > hackingSequences = [{"R__er", ['o', 'b', 'b']}, -

```

FUNCTIONS.PY

```
Richouse

functions.py X
Distribution-code > functions.py > System > render_screen

1 import config
2 import datetime
3 import json
4 import pygame
5 import random
6 import requests
7 import time
8
9
10
11 class Preference:
12     def __init__(self, username: str="Guest", up: int=pygame.K_w, down: int=pygame.K_s, left: int=pygame.K_a,
13                 right: int=pygame.K_d, interact: int=pygame.K_f, use: int=pygame.K_e, volume: int=0.5):
14         self.__buttonUp = up
15         self.__buttonDown = down
16         self.__buttonLeft = left
17         self.__buttonRight = right
18         self.__buttonInteract = interact
19         self.__buttonUse = use
20         self.__volume = volume
21         self.__username = username
22         self.__lastTime = "0:00:00.00"
23
24     def __dir__(self):
25         return [pygame.key.name(self.__buttonUp), pygame.key.name(self.__buttonDown),
26                 pygame.key.name(self.__buttonRight), pygame.key.name(self.__buttonLeft),
27                 pygame.key.name(self.__buttonUse), pygame.key.name(self.__buttonInteract)]
28
29     # Getter method for self.__volume
30     @property
31     def volume(self):
32         return self.__volume
33
34     # Getter method for self.__buttonInteract
35     @property
36     def buttonInteract(self):
37         return self.__buttonInteract
38
39     # Getter method for self.__buttonUse
40     @property
41     def buttonUse(self):
42         return self.__buttonUse
43
44     # Getter method for self.__buttonUp
45     @property
46     def buttonUp(self):
47         return self.__buttonUp
48
49     # Getter method for self.__buttonDown
50     @property
51     def buttonDown(self):
52         return self.__buttonDown
53
54     # Getter method for self.__buttonLeft
55     @property
56     def buttonLeft(self):
57         return self.__buttonLeft
58
59     # Getter method for self.__buttonRight
60     @property
61     def buttonRight(self):
62         return self.__buttonRight
63
64     # Getter method for self.__username
65     @property
66     def username(self):
67         return self.__username
68
69     # Getter method for self.__lastTime
70     @property
71     def lastTime(self):
72         return self.__lastTime
73
74     # Setter method for self.__lastTime
75     @lastTime.setter
76     def lastTime(self, lastTime: str):
77         if lastTime is not None:
78             self.__lastTime = lastTime
79
80     # Setter method for self.__buttonUp
81     @buttonUp.setter
82     def buttonUp(self, buttonUp: int):
83         if buttonUp is not None:
84             self.__buttonUp = buttonUp
85
86     # Setter method for self.__buttonDown
87     @buttonDown.setter
88     def buttonDown(self, buttonDown: int):
89         if buttonDown is not None:
90             self.__buttonDown = buttonDown
91
92     # Setter method for self.__buttonLeft
93     @buttonLeft.setter
94     def buttonLeft(self, buttonLeft: int):
95         if buttonLeft is not None:
96             self.__buttonLeft = buttonLeft
97
98     # Setter method for self.__buttonRight
99     @buttonRight.setter
100    def buttonRight(self, buttonRight: int):
101        if buttonRight is not None:
102            self.__buttonRight = buttonRight
103
104     # Setter method for self.__username
105     @username.setter
106     def username(self, username: str):
107         if username is not None:
108             self.__username = username
109
110     # Setter method for self.__lastTime
111     @lastTime.setter
112     def lastTime(self, lastTime: str):
113         if lastTime is not None:
114             self.__lastTime = lastTime
```

```

86 # Setter method for self.__buttonUp
87 @buttonUp.setter
88 def buttonUp(self, key: int):
89     if key is not None:
90         if pygame.key.name(key).isalpha():
91             self.__buttonUp = key
92
93 # Setter method for self.__buttonDown
94 @buttonDown.setter
95 def buttonDown(self, key: int):
96     if key is not None:
97         if pygame.key.name(key).isalpha():
98             self.__buttonDown = key
99
100 # Setter method for self.__buttonRight
101 @buttonRight.setter
102 def buttonRight(self, key: int):
103     if key is not None:
104         if pygame.key.name(key).isalpha():
105             self.__buttonRight = key
106
107 # Setter method for self.__buttonLeft
108 @buttonLeft.setter
109 def buttonLeft(self, key: int):
110     if key is not None:
111         if pygame.key.name(key).isalpha():
112             self.__buttonLeft = key
113
114 # Setter method for self.__buttonUse
115 @buttonUse.setter
116 def buttonUse(self, key: int):
117     if key is not None:
118         if pygame.key.name(key).isalpha():
119             self.__buttonUse = key
120
121 # Setter method for self.__buttonInteract
122 @buttonInteract.setter
123 def buttonInteract(self, key: int):
124     if key is not None:
125         if pygame.key.name(key).isalpha():
126             self.__buttonInteract = key
127
128 # Setter method for self.__volume
129 @volume.setter
130 def volume(self, value: int):
131     if value is not None:
132         if value >= 0 and value <= 1:
133             self.__volume = value
134
135 def save(self) -> None:
136     # Get the user data from database
137     with open(config.LocalDatabasePath, "r") as f:
138         userData = json.load(f)
139
140     # Modify database with preferences data
141     userData[0]["username"] = self.__username
142     userData[0]["controls"]["up"] = self.__buttonUp
143     userData[0]["controls"]["down"] = self.__buttonDown
144     userData[0]["controls"]["left"] = self.__buttonLeft
145     userData[0]["controls"]["right"] = self.__buttonRight
146     userData[0]["controls"]["use"] = self.__buttonUse
147     userData[0]["controls"]["interact"] = self.__buttonInteract
148     userData[0]["volume"] = self.__volume
149
150     # Write the changes in database
151     with open(config.LocalDatabasePath, "w") as f:
152         json.dump(userData, f)
153
154 def check_best_score(self, map: str) -> None:
155     # Open local database and get the information stored in JSON format
156     with open(config.LocalDatabasePath, "r") as f:
157         localData = json.load(f)
158
159     bestTime = localData[0]["bestTime"][map] # Access the best time registered in the database
160     lastTime = self.__lastTime # Access the last time
161     # Get the number of seconds based on the string h:mm:ss.msms
162     lastTimeSeconds = int(lastTime[0]) * 3600 + int(lastTime[2:4]) * 60 + int(lastTime[5:7]) + int(lastTime[8:]) / 100
163
164     bestTimeSeconds = int(bestTime[0]) * 3600 + int(bestTime[2:4]) * 60 + int(bestTime[5:7]) + int(bestTime[8:]) / 100
165     if lastTimeSeconds < bestTimeSeconds: # Compare the amonuts of seconds
166         bestTime = self.__lastTime # If it's lower time it store as the best time
167         # Send the best result to the server
168         request = requests.post("http://127.0.0.1:5000/bestTime", json={"bestTime": bestTime, "map": map, "username": self.__username})
169         localData[0]["bestTime"][map] = bestTime # Update the local data
170
171     # Open and write the updated data
172     with open(config.LocalDatabasePath, "w") as f:
173         json.dump(localData, f)
174
175 class System:
176     def __init__(self, preferences: Preference):
177         self.__screenObj = pygame.display.set_mode((800, 800))
178         self.__screenName = ""
179         self.__background = None
180         self.__button = None
181         self.__gameplay = None
182         self.__minigame = None
183         self.__textbox = ""
184         self.__preferences = preferences
185
186     # Getter method for self.__screenObj
187     @property
188     def screenObj(self):
189         return self.__screenObj
190
191     # Enable the access of the private attribute screenName
192     @property
193     def screenName(self):
194         return self.__screenName

```

```

195 # Enable the access of the private attribute minigame
196 @property
197 def minigame(self):
198     return self.__minigame
199
200 # Getter method for self.__button
201 @property
202 def button(self):
203     return self.__button
204
205 # Getter method for self.__preferences
206 @property
207 def preferences(self):
208     return self.__preferences
209
210 # Getter method for self.__gameplay
211 @property
212 def gameplay(self):
213     return self.__gameplay
214
215 # Getter method for self.__textbox
216 @property
217 def textbox(self):
218     return self.__textbox
219
220 # Setter method for self.__textbox
221 @textbox.setter
222 def textbox(self, newText: str):
223     if newText is not None:
224         self.__textbox = newText
225
226 def render_screen(self, nextScreen: str) -> None:
227     #Reset the textbox and buttons
228     self.__button = None
229     self.__textbox = None
230
231     # Uses the next screen actions
232     match nextScreen:
233         case "menu":
234             self.__gameplay = None
235             # Create the buttons: start, rank, option and quit, the format is [name, (x-coordinate, y-coordinate)]
236             self.__button = Button(buttons=[["start", (239, 275)], ["rank", (267, 400)], ["options", (175, 525)], ["quit", (276, 650)]],

```

```

237                                     system=self)
238             # Load the menu background, menu-screen.png
239             self.__background = pygame.image.load(config.menuScreenPath).convert_alpha()
240         case "username":
241             # Load the username-screen that is the username registered screen, textbox starts empty
242             self.__background = pygame.image.load(config.usernameScreenPath).convert_alpha()
243             self.__textbox = ""
244         case "rank":
245             # Load the rank background, rank-screen.png
246             self.__background = pygame.image.load(config.rankScreenPath)
247         case "options":
248             # Create the arrow for options selection, the format is [name, [x-coordinate, y-coordinate]]
249             self.__button = Button(buttons=[["arrow", (590, 275)]], system=self)
250             # Load the username-screen that is the options screen, here user can change the predefined keys and volume
251             self.__background = pygame.image.load(config.optionsScreenPath).convert_alpha()
252         case "start":
253             # Create the buttons: map 1 and map 2, the format is [name, (x-coordinate, y-coordinate)]
254             self.__button = Button(buttons=[["baker", (250, 350)], ["usbourne", (250, 500)]], system=self)
255             # Loads the map selection background, it's the same of menu but with different buttons
256             self.__background = pygame.image.load(config.menuScreenPath).convert_alpha()
257         case "baker":
258             self.__minigame = None # Reset the minigame
259             if self.__gameplay is None: # Only creates a new gameplay if there isn't already one
260                 laser = Sprite(image=config.bakerLaserPath, coordinate=(385, 22)) # Create the laser sprite
261                 timeNow = datetime.datetime.now() # Get the current time
262                 server = pygame.Rect(0, 5, 216, 94) # Create the server hitbox
263                 guard = Guard(image=config.guardSpritePath, coordinate=(330, 600), system=self) # Create the guard sprite
264                 player = Player(system=self, coordinate=config.bakerInitialPosition) # Create the player sprite
265                 codeDoor = Sprite(image=config.codeDoorPath, coordinate=(73, 256)) # Create the sprite of the door with code lock
266                 lockDoor = Sprite(image=config.lockDoorPath, coordinate=(657, 303)) # Create the sprite of the door with normal lock

```

```

267             # Store and control the information about the current gameplay session
268             self.__gameplay = Gameplay(map="baker", startTime=timeNow, walls=config.bakerWalls, searchablePlaces=config.bakerSearchable,
269                                     player=player, guard=guard, laser=laser, system=self, statueCoordinate=(425, 60),
270                                     door=config.bakerDoor, codeDoor=codeDoor, lockDoor=lockDoor, server=server)
271
272             # Load the map 1 background, Baker Avenue
273             self.__background = pygame.image.load(config.bakerScreenPath).convert_alpha()
274         case "usbourne":
275             self.__minigame = None # Reset the minigame
276             if self.__gameplay is None: # Only creates a new gameplay if there isn't already one
277                 laser = Sprite(image=config.usbourneLaserPath, coordinate=(601, 22))
278                 timeNow = datetime.datetime.now() # Get the current time
279                 server = pygame.Rect(5, 783, 214, 78) # Create the server hitbox
280                 player = Player(system=self, coordinate=config.usbourneInitialPosition) # Create the guard sprite
281                 guard = Guard(image=config.guardSpritePath, coordinate=(380, 600), system=self) # Create the player sprite
282                 codeDoor = Sprite(image=config.codeDoorPath, coordinate=(322, 223)) # Create the sprite of the door with code lock
283                 lockDoor = Sprite(image=config.lockDoorPath, coordinate=(322, 670)) # Create the sprite of the door with normal lock
284                 # Store and control the information about the current gameplay session
285                 self.__gameplay = Gameplay(map="usbourne", startTime=timeNow, walls=config.usbourneWalls,
286                                     searchablePlaces=config.usbourneSearchable, player=player, guard=guard,
287                                     laser=laser, system=self, statueCoordinate=(690, 80), door=config.usbourneDoor,
288                                     codeDoor=codeDoor, lockDoor=lockDoor, server=server)
289             # Load the map 2 background, Usbourne Way
290             self.__background = pygame.image.load(config.usbourneScreenPath).convert_alpha()
291         case "pause":
292             # Create the buttons: continue and menu, the format is [name, (x-coordinate, y-coordinate)]
293             self.__button = Button(buttons=[["continue", (135, 300)], ["menu-pause", (260, 475)]], system=self)
294             # Load pause screen, here player choose to return to menu or continue the mission
295             self.__background = pygame.image.load(config.pauseScreenPath).convert_alpha()
296         case "mission-completed":
297             self.__minigame = None # Reset the minigame
298             # Load the menu background, menu-screen.png

```

```

298 self.__background = pygame.image.load(config.missionCompletedScreenPath).convert_alpha()
299 case "fingerprint":
300     sequence = random.sample(range(0, 10), 3)
301     self.__minigame = Fingerprint(correctSequence=sequence, system=self)
302     # The background of fingerprint minigame screen
303     self.__background = pygame.image.load(config.fingerprintScreenPath).convert_alpha()
304 case "lockpick":
305     sequence = config.lockpickSequence.copy()
306     random.shuffle(sequence)
307     self.__minigame = Lockpick(correctSequence=sequence, system=self)
308     # The background of lockpick minigame screen
309     self.__background = pygame.image.load(config.lockpickScreenPath).convert_alpha()
310 case "hacking":
311     elements = random.choice(config.hackingSequences)
312     self.__minigame = Hacking(correctSequence=elements[1], system=self, word=elements[0])
313     self.__background = pygame.image.load(config.hackingScreenPath)
314 case "game-over":
315     self.__minigame = None
316     self.__background = pygame.image.load(config.gameOverScreenPath).convert_alpha()
317 self.__screenName = nextScreen
318 self.__screenObj.blit(self.__background, (0, 0))
319 self.display_buttons() # Display the buttons (screens with buttons)
320 self.display_textbox() # Display the textbox (username register screen only)
321 self.display_rank() # Display the rank (rank screen only)
322 self.display_options() # Display the preferences (options screen only)
323 self.display_sprites() # Display the sprites (Baker Avenue and Usbourne Way screens)
324 self.display_result() # Display the final time (mission completed only)
325
326 def update_screen(self) -> None:
327     self.__screenObj.blit(self.__background, (0, 0)) # Print background
328     self.display_buttons() # Display buttons
329     self.display_textbox() # Display textbox
330     self.display_rank() # Display users in ranking
331     self.display_options() # Display the current options
332     self.display_sprites() # Display the sprites
333     self.display_result() # Display the final time (mission completed only)
334     self.display_sequence() # Starts the minigame displaying the sequence
335
336 def display_buttons(self) -> None:
337     # Check if there is some button
338     if self.__button is not None:
339         buttons = self.__button.buttons # List of all buttons
340         selectedButtonIndex = self.__button.selectedButtonIndex # The position in the list of the selected button
341         backgrounds = self.__button.backgrounds # Buttons backgrounds
342
343         # Select the buttons in list, each one has a format [name, (x-coordinate, y-coordinate)]
344         for button in buttons:
345             img = pygame.image.load(backgrounds[button[0]]).convert_alpha() # Load button background
346
347             # Check if it's the selected button
348             if button == buttons[selectedButtonIndex]:
349                 width = img.get_width()
350                 height = img.get_height()
351                 # Change the sizes by a factor 1.3
352                 newWidth = width * 1.3
353                 newHeight = height * 1.3
354                 img = pygame.transform.scale(img, (newWidth, newHeight)) # Change the button scale
355                 x = button[1][0]
356                 y = button[1][1]
357                 self.__screenObj.blit(img, (x - ((newWidth - width)/2), y - ((newHeight - height)/2))) # Recalculate the position
358                                                         # in screen and print it
359                 continue
360             self.__screenObj.blit(img, button[1]) # Print button in normal scale and position
361
362 def display_textbox(self) -> None:
363     # Check if there is textbox
364     if self.__textbox is not None:
365         font = pygame.font.Font(None, 48) # Uses the standard font and size 48
366         text_surface = font.render(self.__textbox, True, (0, 0, 0)) # Render the font with black colour
367         text_rect = text_surface.get_rect(center=(400, 400)) # Create a surface for textbox
368         self.__screenObj.blit(text_surface, text_rect) # Print the surface
369
370 def display_rank(self) -> None:
371     if self.__screenName == "rank":
372         # Request the JSON file to the server and extract the content from response
373         response = requests.get("http://127.0.0.1:5000/rank")
374         rank = response.json()
375
376         font = pygame.font.Font(None, 24) # Uses the standard font and size 48
377
378         # It will print the Baker Avenue ranking
379         baker = rank["Baker"] # Get the list for Baker Avenue rank
380         for x in range(len(baker)): # It will pass through all possible indexes
381             userB = baker[x] # Get element in that index, it has the format [username, time in seconds]
382             text_surface = font.render(f"{userB[0]} -> {str(userB[1])}", True, (235, 165, 59)) # Render the font with black colour
383             text_rect = text_surface.get_rect(topleft=(150, 265 + 50 * x)) # Create a surface for textbox, we can adjust the y-coordinate
384                                                         # of each y-coordinate of each buttons based on its index
385             self.__screenObj.blit(text_surface, text_rect) # Print the surface
386
387         # It will print the Usbourne Way ranking
388         usbourne = rank["Usbourne"] # Get the list for Usbourne Way rank
389         for x in range(len(usbourne)): # It will pass through all possible indexes
390             userU = usbourne[x] # Get element in that index, it has the format [username, time in seconds]
391             text_surface = font.render(f"{userU[0]} -> {str(userU[1])}", True, (235, 165, 59)) # Render the font with black colour
392             text_rect = text_surface.get_rect(topleft=(535, 265 + 50 * x)) # Create a surface for textbox, we can adjust the y-coordinate
393                                                         # of each y-coordinate of each buttons based on its index
394             self.__screenObj.blit(text_surface, text_rect) # Print the surface
395
396 def display_options(self) -> None:
397     if self.__screenName == "options":
398         # Create a rectangle 10x20 and print it in the middle of volume bar
399         rect = pygame.Rect(242 + 300 * self.__preferences.volume, 275, 10, 20) # The position in bar is defined by volume,
400                                                         # that is a factor of original volume
401         pygame.draw.rect(self.__screenObj, (235, 165, 59), rect) # Print with peach colour
402
403         # Print each button current preference in the correct place
404         keys = self.__preferences.__dir__() # Return the list of button preferences [up, down, right, left, use, interaction]
405         font = pygame.font.Font(None, 48) # Uses the standard text and size 48
406         for x in range(6): # Pass through the index of all buttons preferences in the list
407             key = keys[x] # Get the button preference in index x
408             text_surface = font.render(key, True, (235, 165, 59)) # Render the font with peach colour
409             text_rect = text_surface.get_rect(center=(570, 350 + 70 * x)) # Create a surface for textbox
410             self.__screenObj.blit(text_surface, text_rect) # Print the text
411

```

```

412 def display_sprites(self) -> None:
413     if self.__screenName in ["baker", "usbourne"]: # Only displays sprites during gameplay screens
414         if not self.__gameplay.player.rect.collidect(config.inventoryArea): # Won't display the main character above the inventory
415             self.__screenObj.blit(self.__gameplay.player.image, self.__gameplay.player.rect) # Print the main character
416         if not self.__gameplay.guard.rect.collidect(config.inventoryArea): # Won't display the guard above the inventory
417             self.__screenObj.blit(self.__gameplay.guard.image, self.__gameplay.guard.rect) # Display the guard
418         if self.__gameplay.statue is not None: # Only display if the statue wasn't stolen yet
419             self.__screenObj.blit(self.__gameplay.statue.image, self.__gameplay.statue.rect) # Print the statue
420         if self.__gameplay.codeDoor is not None: # Only display if the fingerprint minigame isn't completed and door still in its place
421             # It prints the door in the original direction if it's the baker map
422             if self.__gameplay.map == "baker":
423                 self.__screenObj.blit(self.__gameplay.codeDoor.image, self.__gameplay.codeDoor.rect)
424             # It rotates the door before print if it's the usbourne map
425             else:
426                 rotatedImg = pygame.transform.rotate(self.__gameplay.codeDoor.image, 90) # Rotate in the top left vertex
427                 rotateRect = rotatedImg.get_rect(topleft=self.__gameplay.codeDoor.rect.topleft) # Create the new hitbox
428                 self.__screenObj.blit(rotatedImg, rotateRect)
429         if self.__gameplay.lockDoor is not None: # Only display if the lockpick minigame isn't completed and door still in its place
430             # It prints the door in the original direction if it's the baker map
431             if self.__gameplay.map == "baker":
432                 self.__screenObj.blit(self.__gameplay.lockDoor.image, self.__gameplay.lockDoor.rect)
433             # It rotates the door before print if it's the usbourne map
434             else:
435                 rotatedImg = pygame.transform.rotate(self.__gameplay.lockDoor.image, 90) # Rotate in the top left vertex
436                 rotateRect = rotatedImg.get_rect(topleft=self.__gameplay.lockDoor.rect.topleft) # Create the new hitbox
437                 self.__screenObj.blit(rotatedImg, rotateRect)
438         if self.__gameplay.laser is not None: # Display laser if the hacking minigame isn't completed
439             self.__screenObj.blit(self.__gameplay.laser.image, self.__gameplay.laser.rect)
440
441 def display_result(self) -> None:
442     if self.__screenName == "mission-completed": # Display the final result only on the mission completed screen
443         font = pygame.font.Font(None, 80) # Uses the standar font and size 48
444         text_surface = font.render(self.__preferences.lastTime, True, (235, 165, 59)) # Render the font with peach colour
445         text_rect = text_surface.get_rect(topleft=(95, 640)) # Create a surface for textbox
446         self.__screenObj.blit(text_surface, text_rect) # Print the result
447
448 def display_sequence(self) -> None:
449     if self.__minigame is not None:
450         self.__minigame.start()
451

```

```

452 class Minigame:
453     def __init__(self, correctSequence: list, system: System):
454         self.__correctSequence = correctSequence
455         self.__userSequence = []
456         self.__system = system
457
458     # Getter method for self.__correctSequence
459     @property
460     def correctSequence(self):
461         return self.__correctSequence
462
463     # Getter method for self.__system
464     @property
465     def system(self):
466         return self.__system
467
468     # Getter method for self.__userSequence
469     @property
470     def userSequence(self):
471         return self.__userSequence
472
473     # Setter method for self.__correctSequence
474     @correctSequence.setter
475     def correctSequence(self, value: list):
476         if type(value) == list:
477             self.__correctSequence = value
478
479     # Setter method for self.__userSequence
480     @userSequence.setter
481     def userSequence(self, value: list):
482         if type(value) == list:
483             self.__userSequence = value
484
485     def check(self) -> None:
486         if self.__userSequence != self.__correctSequence:
487             self.__system.gameplay.player.lives -= 1
488             if self.__system.gameplay.map == "baker":
489                 self.__system.gameplay.player.rect.topleft = config.bakerInitialPosition
490             else:
491                 self.__system.gameplay.player.rect.topleft = config.usbourneInitialPosition
492         self.__system.render_screen(nextScreen=self.__system.gameplay.map)
493

```

```

494 class Fingerprint(Minigame):
495     def start(self) -> None:
496         num1 = self.correctSequence[0]
497         weakFinger = pygame.image.load(config.weakFingerprintPath).convert_alpha()
498         self.system.screenObj.blit(weakFinger, config.numbersPositions[num1])
499         num2 = self.correctSequence[1]
500         midFinger = pygame.image.load(config.midFingerprintPath).convert_alpha()
501         self.system.screenObj.blit(midFinger, config.numbersPositions[num2])
502         num3 = self.correctSequence[2]
503         strongFinger = pygame.image.load(config.strongFingerprintPath)
504         self.system.screenObj.blit(strongFinger, config.numbersPositions[num3])
505
506     def check(self) -> None:
507         super().check()
508         if self.userSequence == self.correctSequence:
509             if self.system.gameplay.map == "baker":
510                 self.system.gameplay.walls.remove(config.bakerCodeDoor)
511             else:
512                 self.system.gameplay.walls.remove(config.usbourneCodeDoor)
513             self.system.gameplay.codeDoor = None
514             self.system.render_screen(nextScreen=self.system.gameplay.map)
515
516 class Hacking(Minigame):
517     def __init__(self, correctSequence: list, system: System, word: str):
518         super().__init__(correctSequence=correctSequence, system=system)
519         self.__word = word
520
521     def start(self) -> None:
522         font = pygame.font.Font(None, 48)
523         text = font.render(self.__word, True, (0, 255, 0))
524         rect = text.get_rect(center=(400, 400))
525         self.system.screenObj.blit(text, rect)
526
527     def check(self) -> None:
528         super().check()
529         if self.userSequence == self.correctSequence:
530             self.system.gameplay.laser = None
531             self.system.render_screen(nextScreen=self.system.gameplay.map)
532

```

```

533 class Lockpick(Minigame):
534     def __init__(self, correctSequence: list, system: System):
535         super().__init__(correctSequence=correctSequence, system=system)
536         self.__printableSequence = correctSequence.copy()
537
538     def start(self) -> None:
539         try:
540             time.sleep(0.5)
541             item = self.__printableSequence[0]
542             match item:
543                 case pygame.K_LEFT:
544                     img = pygame.image.load(config.lockpickBackgrounds[pygame.K_LEFT])
545                     self.system.screenObj.blit(img, (334, 519))
546                 case pygame.K_RIGHT:
547                     img = pygame.image.load(config.lockpickBackgrounds[pygame.K_RIGHT])
548                     self.system.screenObj.blit(img, (334, 519))
549                 case pygame.K_UP:
550                     img = pygame.image.load(config.lockpickBackgrounds[pygame.K_UP])
551                     self.system.screenObj.blit(img, (334, 519))
552                 case pygame.K_DOWN:
553                     img = pygame.image.load(config.lockpickBackgrounds[pygame.K_DOWN])
554                     self.system.screenObj.blit(img, (334, 519))
555             self.__printableSequence.remove(item)
556         except:
557             return
558
559     def check(self) -> None:
560         super().check()
561         if self.userSequence == self.correctSequence:
562             if self.system.gameplay.map == "baker":
563                 self.system.gameplay.walls.remove(config.bakerLockDoor)
564             else:
565                 self.system.gameplay.walls.remove(config.usbourneLockDoor)
566             self.system.gameplay.lockDoor = None
567             self.system.render_screen(nextScreen=self.system.gameplay.map)
568

```

```

569 class Sprite:
570     def __init__(self, image: str, coordinate: tuple):
571         self.__image = pygame.image.load(image).convert_alpha()
572         self.__rect = self.__image.get_rect(topleft=coordinate)
573
574     # Getter method for self.__image
575     @property
576     def image(self):
577         return self.__image
578
579     # Getter method for self.__rect
580     @property
581     def rect(self):
582         return self.__rect
583
584 class Guard(Sprite):
585     def __init__(self, image: str, coordinate: tuple, system: System):
586         super().__init__(image=image, coordinate=coordinate)
587         self.__moveX = 5
588         self.__moveY = 0
589         self.__system = system
590
591     def move(self) -> None:
592         if self.__system.screenName in ["baker", "usbourne"]:
593             self.rect.x += self.__moveX
594             self.rect.y += self.__moveY
595         if self.__system.screenName == "baker":
596             if self.rect.topleft == (330, 330):
597                 self.__moveX = 0
598                 self.__moveY = 5
599             elif self.rect.topleft == (330, 600):
600                 self.__moveX = 5
601                 self.__moveY = 0
602             elif self.rect.topleft == (615, 600):
603                 self.__moveX = 0
604                 self.__moveY = -5
605             elif self.rect.topleft == (615, 330):
606                 self.__moveX = -5
607                 self.__moveY = 0
608         else:
609             if self.rect.topleft == (380, 400):
610                 self.__moveX = 0
611                 self.__moveY = 5
612
613             elif self.rect.topleft == (380, 600):
614                 self.__moveX = 5
615                 self.__moveY = 0
616             elif self.rect.topleft == (705, 600):
617                 self.__moveX = 0
618                 self.__moveY = -5
619             elif self.rect.topleft == (705, 400):
620                 self.__moveX = -5
621                 self.__moveY = 0
622
623 class Player(Sprite):
624     def __init__(self, system: System, coordinate: tuple):
625         super().__init__(image=config.mainSpritePath, coordinate=coordinate)
626         self.__lives = 3
627         self.__inventory = []
628         self.__system = system
629
630     # Getter method for self.__inventory
631     @property
632     def inventory(self):

```

```

632         return self.__inventory
633
634     # Getter method for self.__lives
635     @property
636     def lives(self):
637         return self.__lives
638
639     @lives.setter
640     def lives(self, value: int):
641         if value is not None:
642             self.__lives = value
643
644     def movement(self) -> None:
645         keys = pygame.key.get_pressed() # Provide a dict that says if key is pressed or not, uses True and False values
646         if keys[self.__system.preferences.buttonUp]: # If up button is being pressed
647             self.rect.y -= 4 # Velocity of 4 pixels
648             wall = self.check_collision() # Check if collided with something
649             if wall is not None: # Only if collided
650                 self.rect.top = wall.bottom # The player will be blocked to pass through the wall
651         if keys[self.__system.preferences.buttonDown]: # If down button is being pressed
652             self.rect.y += 4 # Velocity of 4 pixels
653             wall = self.check_collision() # Check if collided with something
654             if wall is not None: # Only if collided
655                 self.rect.bottom = wall.top # The player will be blocked to pass through the wall
656         if keys[self.__system.preferences.buttonRight]: # If right button is being pressed
657             self.rect.x += 4 # Velocity of 4 pixels
658             wall = self.check_collision() # Check if collided with something
659             if wall is not None: # Only if collided
660                 self.rect.right = wall.left # The player will be blocked to pass through the wall
661         if keys[self.__system.preferences.buttonLeft]: # If left button is being pressed
662             self.rect.x -= 4 # Velocity of 4 pixels
663             wall = self.check_collision() # Check if collided with something
664             if wall is not None: # Only if collided
665                 self.rect.left = wall.right # The player will be blocked to pass through the wall
666
667     def check_collision(self) -> pygame.Rect:
668         for wall in self.__system.gameplay.walls: # Access each wall in the list of the map walls
669             if self.rect.colliderect(wall): # Check if collided with some wall in the list
670                 return wall # Return the wall collided
671         if self.__system.gameplay.statue is not None: # Check if statue wasn't taken yet
672             if self.rect.colliderect(self.__system.gameplay.statue.rect): # Check if player collided with statue
673                 self.__inventory.append("Statue") # Add statue to the inventory if collision happened
674                 self.__system.gameplay.statue = None # Reset the statue sprite
675                 return
676         if self.rect.colliderect(self.__system.gameplay.door) and "Statue" in self.__inventory: # Check if the player touched the main door
677             # with the statue in its inventory
678             currentTime = datetime.datetime.now() # Get the time that mission finished
679             timeUsed = str(currentTime - self.__system.gameplay.startTime) # Find the time used in the mission by subtracting the final time
680             # from the start time
681             self.__system.preferences.lastTime = timeUsed[:-4] # Transform the result in the string himiss.msms limiting the time decimal
682             # to 2
683             self.__system.preferences.check_best_score(map=self.__system.gameplay.map) # Check if it was the best time
684             self.__system.render_screen(nextScreen="mission-completed") # Render the mission completed screen
685             return
686         if self.__system.gameplay.laser is not None: # Check if the laser is active
687             if self.rect.colliderect(self.__system.gameplay.laser.rect): # Check the collision with the laser
688                 self.__lives -= 1 # Player lose 1 live if collided with the laser
689                 # The player will return to the initial position according to the map.
690                 if self.__system.gameplay.map == "baker":
691                     self.rect.topleft = config.bakerInitialPosition
692                 else:
693                     self.rect.topleft = config.usbourneInitialPosition
694             return
695         if self.rect.colliderect(self.__system.gameplay.guard.rect): # Check the collision with the guard
696             self.__lives -= 1 # Player lose 1 live if collided with the guard
697             # The player will return to the initial position according to the map.
698             if self.__system.gameplay.map == "baker":
699                 self.rect.topleft = config.bakerInitialPosition
700             else:
701                 self.rect.topleft = config.usbourneInitialPosition
702             return
703
704     def interact(self) -> None:
705         searchablePlaces = self.__system.gameplay.searchablePlaces # The places where there are items
706
707         for searchable in searchablePlaces:
708             if self.rect.colliderect(searchablePlaces[searchable]): # Check collision with the searchable place
709                 match searchable: # The item to be added in the inventory depends on the place
710                     # The lockpick is inside the fridge
711                     case "fridge":
712                         if "Lockpick" not in self.__inventory:
713                             self.__inventory.append("Lockpick")
714                             return
715                     # The UV light is inside the tv support
716                     case "tv":
717                         if "UV light" not in self.__inventory:
718                             self.__inventory.append("UV light")
719                             return
720                     # The chip is inside the desk
721                     case "desk":
722                         if "Chip" not in self.__inventory:
723                             self.__inventory.append("Chip")
724
725     def use(self) -> None:
726         if self.__system.gameplay.codeDoor is not None:
727             if self.rect.colliderect(self.__system.gameplay.codeDoor.rect) and "UV light" in self.__inventory:
728                 self.__system.render_screen(nextScreen="fingerprint")
729         if self.__system.gameplay.lockDoor is not None:
730             if self.rect.colliderect(self.__system.gameplay.lockDoor.rect) and "Lockpick" in self.__inventory:
731                 self.__system.render_screen(nextScreen="lockpick")
732         if self.__system.gameplay.server is not None:
733             if self.rect.colliderect(self.__system.gameplay.server.rect) and "Chip" in self.__inventory:
734                 self.__system.render_screen(nextScreen="hacking")
735
736     def display_inventory(self) -> None:
737         if self.__system.screenName in ["baker", "usbourne"]: # Only prints on gameplay screens
738             font = pygame.font.Font(None, 25)
739             for index in range(len(self.__inventory)): # Range of all indexes of the inventory
740                 text = font.render(self.__inventory[index], True, (235, 165, 59))
741                 rect = text.get_rect(topleft=(633, 644 + 35 * index)) # The index is used to create a spacing between printed items
742                 self.__system.screenObj.blit(text, rect)
743
744 class Gameplay:
745     def __init__(self, map: str, startTime: datetime.timedelta, walls: list, searchablePlaces: list, player: Player,

```

```

745         guard: Guard, laser: Sprite, system: System, statueCoordinate: tuple, door: pygame.Rect,
746         codeDoor: Sprite, lockDoor: Sprite, server: pygame.Rect):
747         self.__map = map
748         self.__startTime = startTime
749         self.__walls = walls.copy()
750         self.__searchablePlaces = searchablePlaces
751         self.__player = player
752         self.__guard = guard
753         self.__laser = laser
754         self.__door = door
755         self.__server = server
756         self.__statue = Sprite(image=config.statueSpritePath, coordinate=statueCoordinate)
757         self.__system = system
758         self.__codeDoor = codeDoor
759         self.__lockDoor = lockDoor
760
761         # Getter method for self.__guard
762         @property
763         def guard(self):
764             return self.__guard
765
766         # Getter method for self.__server
767         @property
768         def server(self):
769             return self.__server
770
771         # Getter method for self.__walls
772         @property
773         def walls(self):
774             return self.__walls
775
776         # Getter method for self.__player
777         @property
778         def player(self):
779             return self.__player
780
781         # Getter method for self.__map
782         @property
783         def map(self):
784             return self.__map
785
786         # Getter method for self.__searchablePlaces
787         @property
788         def searchablePlaces(self):
789             return self.__searchablePlaces
790
791         # Getter method for self.__codeDoor
792         @property
793         def codeDoor(self):
794             return self.__codeDoor
795
796         # Getter method for self.__lockDoor
797         @property
798         def lockDoor(self):
799             return self.__lockDoor
800
801         # Getter method for self.__door
802         @property
803         def door(self):
804             return self.__door
805
806         # Getter method for self.__startTime
807         @property
808         def startTime(self):
809             return self.__startTime
810
811         # Getter method for self.__statue
812         @property
813         def statue(self):
814             return self.__statue
815
816         # Getter method for self.__laser
817         @property
818         def laser(self):
819             return self.__laser
820
821         # Setter method for self.__laser
822         @laser.setter
823         def laser(self, value):
824             self.__laser = value
825
826         # Setter method for self.__statue
827         @statue.setter
828         def statue(self, value):
829             self.__statue = value
830
831         # Setter method for self.__codeDoor
832         @codeDoor.setter
833         def codeDoor(self, value):
834             self.__codeDoor = value
835
836         # Setter method for self.__lockDoor
837         @lockDoor.setter
838         def lockDoor(self, value):
839             self.__lockDoor = value
840
841         def update_status(self) -> None:
842             currentTime = datetime.datetime.now() # Get the current time
843             timeUsed = str(currentTime - self.__startTime) # Find the time used so far by subtracting the current time from the start one
844             timeUsed = timeUsed[:-4] # Limit the seconds to 2 decimal places
845             font = pygame.font.Font(None, 40)
846             # Print the time
847             text = font.render(timeUsed, True, (235, 165, 59))
848             rect = text.get_rect(topleft=(440, 5))
849             self.__system.screenObj.blit(text, rect)
850             # Print the lives
851             text = font.render(str(self.__player.lives), True, (235, 165, 59))
852             rect = text.get_rect(topleft=(275, 5))
853             self.__system.screenObj.blit(text, rect)
854
855         class Button:
856             def __init__(self, buttons: list, system: System):
857                 self.__buttons = buttons
858                 self.__selectedButtonIndex = 0
859                 self.__system = system
860                 self.__backgrounds = {
861                     "start": config.startButtonPath,
862                     "rank": config.rankButtonPath,
863                     "options": config.optionsButtonPath,
864                     "quit": config.quitButtonPath,
865                     "again": config.againButtonPath,
866                     "menu-pause": config.menuPauseButtonPath,

```

```

867         "menu-menu": config.menuMenuButtonPath,
868         "continue": config.continueButtonPath,
869         "baker": config.bakerButtonPath,
870         "usbourne": config.map2ButtonPath,
871         "arrow": config.arrowPath
872     }
873
874     # buttons getter method
875     @property
876     def buttons(self):
877         return self.__buttons
878
879     # selectedButtonIndex getter method
880     @property
881     def selectedButtonIndex(self):
882         return self.__selectedButtonIndex
883
884     # backgrounds getter method
885     @property
886     def backgrounds(self):
887         return self.__backgrounds
888
889     def selection_general(self, key: int) -> None:
890         # Selection based on constant value of the key pressed
891         match key:
892             # Access the system object to access the preferences and after get the constant of up button
893             case self._system.preferences.buttonUp:
894                 self.__selectedButtonIndex = (self.__selectedButtonIndex - 1) % len(self.__buttons) # Decrease the index
895                                                         # without exceed the index
896             # Access the system object to access the preferences and after get the constant of down button
897             case self._system.preferences.buttonDown:
898                 self.__selectedButtonIndex = (self.__selectedButtonIndex + 1) % len(self.__buttons) # Increase the index
899                                                         # without exceed the index
900
901     def selection_menu(self, key: int) -> None:
902         # Uses the general selection for M and S buttons
903         self.selection_general(key=key)
904         # The RETURN key is responsible to confirm the selection
905         if key == pygame.K_RETURN:
906             # Selection based on the button selected
907             match self.__buttons[self.__selectedButtonIndex][0]:
908                 case "start":
909                     self._system.render_screen(nextScreen="start") # Render start screen
910                 case "rank":
911                     self._system.render_screen(nextScreen="rank") # Render rank screen
912                 case "options":
913                     self._system.render_screen(nextScreen="options") # Render rank screen
914                 case "quit":
915                     pygame.quit() # Stop the game
916
917     def selection_start(self, key: int) -> None:
918         # Uses the general selection for M and S buttons
919         self.selection_general(key=key)
920         match key:
921             case pygame.K_RETURN:
922                 match self.__buttons[self.__selectedButtonIndex][0]:
923                     case "baker":
924                         self._system.render_screen(nextScreen="baker")
925                     case "usbourne":
926                         self._system.render_screen(nextScreen="usbourne")
927             case pygame.K_ESCAPE:
928                 self._system.render_screen(nextScreen="menu")
929

```

```
930 def selection_pause(self, key: int) -> None:
931     self.selection_general(key=key)
932     if key == pygame.K_RETURN:
933         match self.__buttons[self.__selectedButtonIndex][0]:
934             case "continue":
935                 self.__system.render_screen(nextScreen=self.__system.gameplay.map)
936             case "menu-pause":
937                 self.__system.render_screen(nextScreen="menu")
938
939 def selection_options(self, event: pygame.event.Event) -> None:
940     # Selection case the SHIFT is held
941     if event.mod & pygame.KMOD_SHIFT:
942         # Selection with the y coordinate, buttons has the format [[button name, [x, y]]] in this case
943         match self.__buttons[0][1][1]:
944             case 345: # y-position of up button preference
945                 self.__system.preferences.buttonUp = event.key # Change the up button preference
946             case 415: # y-position of down button preference
947                 self.__system.preferences.buttonDown = event.key # Change the down button preference
948             case 485: # y-position of right button preference
949                 self.__system.preferences.buttonRight = event.key # Change the right button preference
950             case 555: # y-position of left button preference
951                 self.__system.preferences.buttonLeft = event.key # Change the left button preference
952             case 625: # y-position of use button preference
953                 self.__system.preferences.buttonUse = event.key # Change the use button preference
954             case 695: # y-position of interaction button preference
955                 self.__system.preferences.buttonInteract = event.key # Change the interact button preference
956         else:
957             match event.key: # Use the integer constant of the key to select
958                 case self.__system.preferences.buttonUp: # up selection
959                     new = self.__buttons[0][1][1] - 70 # find the new y position for the arrow
960                     if new >= 275: # check if it isn't below the y position of the first preference
961                         self.__buttons[0][1][1] = new # set the new preference
962                 case self.__system.preferences.buttonDown: # down selection
963                     new = self.__buttons[0][1][1] + 70 # find the new y position for the arrow
964                     if new <= 695: # check if it isn't above the y position of the last preference
965                         self.__buttons[0][1][1] = new # set the new preference
966                 case self.__system.preferences.buttonLeft: # left selection
967                     if self.__buttons[0][1][1] == 275: # It will restrict it to volume preference
968                         self.__system.preferences.volume -= 0.05 # decrease the volume by 0.05, min of 0
969                 case self.__system.preferences.buttonRight: # right selection
970                     if self.__buttons[0][1][1] == 275: # restrict it to volume preference
971                         self.__system.preferences.volume += 0.05 # increase the volume by 0.05, max of 1
972
973             case pygame.K_ESCAPE: # ESC selection
974                 self.__system.preferences.save() # Method to save the current preferences in the local database
975                 pygame.mixer.music.set_volume(self.__system.preferences.volume) # Use the new volume
976                 self.__system.render_screen(nextScreen="menu") # Return to menu
977
978 def textbox_handling(system: System, key: int) -> None:
979     # Use the key int constant to choose the correct action
980     match key:
981         # RETURN key confirm the username
982         case pygame.K_RETURN:
983             if system.textbox != "":
984                 request = requests.post("http://127.0.0.1:5000/register", json={"username": system.textbox})
985                 status = request.json()["success"]
986                 if status:
987                     system.preferences.username = system.textbox # store username into preferences attributes
988                     system.preferences.save() # Save current preferences attributes into local database
989                     # Render the menu screen
990                     system.render_screen(nextScreen="menu")
991             else:
992                 system.textbox = "Unavailable"
993         # BACKSPACE delete one character
994         case pygame.K_BACKSPACE:
995             system.textbox = system.textbox[:-1] # Create a new string cutting off the last charactex of textbox
996         # Any other character
997         case _:
998             if pygame.key.name(key).isalnum() and len(system.textbox) < 10: # Limit the username to 10 alphanumeric characters
999                 system.textbox = system.textbox + pygame.key.name(key) # append the new character to the textbox
```

MAIN.PY

```
Richouse

LocalUserInfo.json  main.py x
Distribution-code > main.py > game_loop
1 from functions import System, Preference, textbox_handling
2 import json
3 import pygame
4 import sys
5
6
7
8 # Main module excuted when game starts
9 def main() -> None:
10     # Initialization of pygame and its features
11     pygame.init()
12     pygame.mixer.init()
13
14     clock = pygame.time.Clock()
15
16     # Create the title
17     pygame.display.set_caption("Richouse")
18
19     # Start the music
20     pygame.mixer.music.load("/Users/viniciusferrari/Documents/Richouse/Media/stealth-battle.mp3")
21     pygame.mixer.music.play(loops=-1)
22
23     #Set the mouse cursor invisible in screen
24     pygame.mouse.set_visible(False)
25
26     # Get the user data from local flatfile database, JSON type
27     with open("/Users/viniciusferrari/Documents/Richouse/Distribution-code/LocalUserInfo.json", "r") as f:
28         userData = json.load(f)
29
30     # Check for registered username
31     username = userData[0]["username"]
32     if username == "":
33         preferences = Preference() # Give default preferences
34         system = System(preferences=preferences) # Create the system object, it will be responsible for screens
35         system.render_screen(nextScreen="username") # Render the screen to register an username
36     else:
37         controls = userData[0]["controls"]
38         preferences = Preference(username=username, up=controls["up"], down=controls["down"],
39                                 left=controls["left"], right=controls["right"], interact=controls["interact"],
40                                 use=controls["use"], volume=userData[0]["volume"]) # Uses the preferences
41         # from database
42         system = System(preferences=preferences) # Create the system object, it will be responsible for screens
43         system.render_screen(nextScreen="menu") # Render the menu screen
44
45     pygame.mixer.music.set_volume(preferences.volume)
46     game_loop(system=system, clock=clock) # Redirect to the function with the game loop
47
48 def game_loop(system: System, clock: pygame.time.Clock) -> None:
49     while True:
50         # Update screen elements, i.e. background, buttons and textbox.
51         system.update_screen()
52         if system.screenName in ["baker", "usbourne", "hacking", "lockpick", "fingerprint", "pause"]: # Only happen in gameplay screens
53             system.gameplay.update_status() # Update and print the time and lives
54             system.gameplay.player.display_inventory() # Display the inventory
55             system.gameplay.guard.move() # Update guard position
56             system.gameplay.player.check_collision() # Check any collisions with the player
57             if system.gameplay.player.lives == 0: # Finish the mission if player lose all lives
58                 system.render_screen(nextScreen="game-over") # Fail screen
59             input(system=system) # Where the input will be handled
60             pygame.display.flip() # Change the frame
61             clock.tick(60) # Limit the fps to 60
62
63 def input(system: System) -> None:
64     # Get the events to be handled
65     for event in pygame.event.get():
66         # Essential to maintain pygame working
67         if event.type == pygame.QUIT:
68             pygame.quit()
69             sys.exit()
70         # Handle any key pressed, no matters if key still pressed, it will hanlde once per press
71         elif event.type == pygame.KEYDOWN:
72             # Different handling accordin to the screen
73             match system.screenName:
74                 case "menu":
75                     system.button.selection_menu(key=event.key) # Uses a method to handle the buttons
76                 case "username":
77                     textbox_handling(system=system, key=event.key) # Uses a function to handle the textbox
78                     # Just to enable me to return to menu screen at this stage
79                 case "start":
80                     system.button.selection_start(key=event.key) # Use a method to handle the buttons
81                 case "rank":
82                     # Return to menu in case of ESC be pressed
83                     if event.key == pygame.K_ESCAPE:
84                         system.render_screen(nextScreen="menu")
85                 case "options":
86                     system.button.selection_options(event=event) # Uses Button method to do the selection
```

```
86     case "baker" | "usbourne":
87         match event.key:
88             case system.preferences.buttonInteract: # Button used to search
89                 system.gameplay.player.interact() # Player method
90             case system.preferences.buttonUse: # Button used to start the minigames
91                 system.gameplay.player.use() # Player method
92             case pygame.K_ESCAPE: # Button to open the pause screen
93                 system.render_screen(nextScreen="pause") # System method
94
95     case "pause":
96         system.button.selection_pause(key=event.key) # Button method
97     case "mission-completed":
98         if event.key == pygame.K_RETURN: # Button used to continue to menu
99             system.render_screen(nextScreen="menu") # System method
100
101     case "lockpick":
102         if event.key in [pygame.K_UP, pygame.K_DOWN, pygame.K_LEFT, pygame.K_RIGHT] and len(system.minigame.userSequence) < 4:
103             system.minigame.userSequence.append(event.key)
104             if len(system.minigame.userSequence) == 4:
105                 system.minigame.check()
106             if event.key == pygame.K_ESCAPE:
107                 system.render_screen(nextScreen=system.gameplay.map)
108
109     case "fingerprint":
110         if pygame.key.name(event.key).isnumeric() and len(system.minigame.userSequence) < 3:
111             system.minigame.userSequence.append(int(pygame.key.name(event.key)))
112             if len(system.minigame.userSequence) == 3:
113                 system.minigame.check()
114             if event.key == pygame.K_ESCAPE:
115                 system.render_screen(nextScreen=system.gameplay.map)
116
117     case "game-over":
118         if event.key == pygame.K_RETURN:
119             system.render_screen(nextScreen="menu")
120
121     case "hacking":
122         if pygame.key.name(event.key).isalpha() and len(system.minigame.userSequence) < 3:
123             system.minigame.userSequence.append(pygame.key.name(event.key))
124             if len(system.minigame.userSequence) == 3:
125                 system.minigame.check()
126             if event.key == pygame.K_ESCAPE:
127                 system.render_screen(nextScreen=system.gameplay.map)
128
129 if system.screenName in ["baker", "usbourne"]: # Limit the action to gameplay screens
130     system.gameplay.player.movement() # Player Method
131
132 if __name__ == '__main__':
133     main()
134
```

LOCALUSERINFO.JSON

```
1 [{"username": ""},
2  {"controls": {"up": 119, "down": 115, "left": 97, "right": 100, "interact": 102, "use": 101},
3   "volume": 0.5,
4   "bestTime": {"baker": "9:59:59.99", "usbourne": "9:59:59.99"}}]
```

SERVER-CODE

APP.PY

```
LocalUserInfo.json  app.py x
Richouse

Server-code > app.py > ...
1 from flask import Flask, request, jsonify, Response
2 import sqlite3
3
4
5
6 app = Flask(__name__)
7
8 @app.route("/register", methods=["POST"])
9 def new_user() -> Response:
10     # Get the username sent inside a dictionary, json style
11     data = request.get_json()
12     username = data["username"] # Access the username given that data is a dictionary
13     conn = sqlite3.connect("/Users/viniciusferrari/Documents/Richouse/Server-code/GlobalDatabase.db") # Connect with database
14     cursor = conn.cursor()
15     try:
16         cursor.execute('INSERT INTO user (username) VALUES (?, ?)', (username,)) # Check with username already exist, as username is primary
17                                         # key it will return error if already exist
18     except:
19         conn.close()
20     return jsonify(success=True) # Return successful status
21 except:
22     conn.close()
23     return jsonify(success=False) # Return unsuccessful status
24
25 @app.route("/rank")
26 def ranking() -> Response:
27     conn = sqlite3.connect("/Users/viniciusferrari/Documents/Richouse/Server-code/GlobalDatabase.db") # connect with database
28     cursor = conn.cursor()
29     cursor.execute("SELECT user, time FROM results WHERE map = 'baker' ORDER BY time LIMIT 10") # Get the best 10 scorers in
30                                                         # Baker Avenue
31     resultsBaker = cursor.fetchall()
32     cursor.execute("SELECT user, time FROM results WHERE map = 'usbourne' ORDER BY time LIMIT 10") # Get the best 10 scorers in
33                                                         # Usbourne Way
34     resultsUsbourne = cursor.fetchall()
35     return jsonify({"Baker": resultsBaker, "Usbourne": resultsUsbourne}) # return both ranks in a json format
36
37 @app.route("/bestTime", methods=["POST"])
38 def best_time() -> None:
39     data = request.get_json()
40     bestTime = data["bestTime"]
41     map = data["map"]
42     user = data["username"]
43     conn = sqlite3.connect("/Users/viniciusferrari/Documents/Richouse/Server-code/GlobalDatabase.db") # Connect with database
44     cursor = conn.cursor()
45     try:
46         cursor.execute("INSERT INTO results(time, map, user) VALUES (?, ?, ?)", (bestTime, map, user))
47     except:
48         cursor.execute("UPDATE results SET time = ? WHERE map = ? AND user = ?", (bestTime, map, user))
49     conn.commit()
50     conn.close()
51     return jsonify({"success": True})
52
53 if __name__ == "__main__":
54     app.run(debug=True)
```

GLOBALDATABASE.DB

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
sqlite3 - Server-code + - - - - - x

sqlite> .schema
CREATE TABLE user (
  username TEXT NOT NULL PRIMARY KEY
);
CREATE TABLE map (
  name TEXT NOT NULL PRIMARY KEY
);
CREATE TABLE results (
  time TEXT NOT NULL,
  map TEXT NOT NULL,
  user TEXT NOT NULL,
  FOREIGN KEY (map) REFERENCES map (name),
  FOREIGN KEY (user) REFERENCES user (username),
  PRIMARY KEY (map, user)
);
sqlite> 
```

MEDIA

